

Snowrunner 670

AME 261: Basic Flight Mechanics

Department of Aerospace and Mechanical Engineering

University of Southern California

By: Group 6

Abishek Iyer, Marin Davis, Ben Tallon, Mark Williams, Keegan Cummings, Jeffrey Deng



Table of Contents

Section 1: Introduction.....	4
1.1 Abstract.....	4
1.2 Request for Proposal.....	4
Section 2: Design Process.....	5
2.1 Theoretical Presentation on Takeoff and Landing.....	5
2.1.1) Takeoff Distance.....	5
2.1.2) Landing Distance.....	6
2.1.3) Cruise Theory.....	7
2.2 Takeaways from Theory.....	9
2.3 Airfoil Selection.....	10
2.3.1) Coefficient of Lift Requirements.....	10
2.3.2) Airfoil Selection.....	11
2.3.2) Slotted flaps/slats.....	12
2.4 Reference Aircraft.....	13
2.5 Historical Empty Weight Estimate.....	13
2.5.1) Payload Breakdown by Configuration.....	13
2.5.2) Methodology.....	15
2.5.3) Weight Results.....	16
2.6 Engine Selection.....	17
2.7 Drag Buildup.....	18
2.7.1) Skin Friction Coefficients.....	18
2.7.2) Form and Interface Factors.....	19
2.7.3) Wetted Areas.....	20
2.7.4) Landing Gear and Miscellaneous Drag.....	20
2.7.5) Results.....	20
2.7.6) Drag from Flaps and Spoilers.....	21
2.8 Constraint Diagrams.....	23
2.8.1) Thrust-to-Weight vs Wing Loading.....	23
2.8.2) Aspect Ratio vs. Wing Area.....	23
2.9 Cruise Optimization.....	25
Section 3: Aircraft Performance Results.....	27
3.1 Mission-Level Performance Overview.....	27
3.1.1) Operations and Performance Requirements.....	27
3.1.2) Payload-Range Diagram.....	28
3.1.3) Turning Flight (V-n diagrams).....	29

3.2 Passenger Reference Mission Performance.....	31
3.3 MedVac Mission Performance.....	33
Section 4: Configuration.....	34
4.1 Stability & Control.....	34
4.1.1) Volume Coefficients.....	34
4.1.2) Tail Planform Geometry.....	35
4.1.3) Static Margin.....	36
4.1.4) Control Surfaces.....	36
4.1.5) Sensitivity to Tail Moment Arm.....	36
4.2 Fuselage.....	39
4.3 Airframe Materials.....	39
4.4 Avionics and Instrumentation.....	40
4.5 Cabin configuration.....	40
4.6 Summary of Aircraft Values.....	42
4.7 Final Design in CAD.....	44
Section 5: Operations and Business Analysis.....	46
5.1 Supporting Ground Infrastructure.....	46
5.2 Airplane Cost and Business Analysis.....	47

Section 1: Introduction

1.1 Abstract

This report presents the preliminary design of a Short Take-Off and Landing (STOL) aircraft intended for Arctic operations, including passenger transport, cargo delivery, and medical evacuation missions. The aircraft was designed to meet strict performance requirements, including takeoff and landing over a 50 ft obstacle within 500 ft, a minimum 450 nmi reference range, an 18,000 ft service ceiling, and reliable operation on gravel runways in cold-weather conditions. A design process involving reference aircraft comparison, weight estimation, airfoil and high-lift system selection, drag buildup, constraint analysis, mission performance modeling, stability and control sizing, and cost analysis was used to develop the final configuration. The selected design uses a NASA GA(W)-1 airfoil with double-slotted Fowler flaps and leading-edge slats to achieve the required low-speed lift performance, while a PT6A turboprop engine provides sufficient power margin for short-field operations. The final aircraft configuration satisfies the primary RFP requirements while maintaining practical manufacturability, operational flexibility, and suitability for remote Arctic communities.

1.2 Request for Proposal

Remote communities near the Arctic Circle are shy of transportation, goods, and medical services, leading to a demand for reliable aircraft capable of withstanding this harsh climate. Existing aircraft are often not optimized for reliable operations within these extreme environments, particularly when operating from gravel or unprepared runways under cold-weather conditions.

The objective of this design project is to develop a versatile Short Take-Off and Landing (STOL) aircraft capable of operating within these remote regions. The aircraft must be able to transport passengers, cargo, and medical personnel while maintaining safe and effective operation in temperatures as low as -30 F, with minimal ground support infrastructure. Furthermore, the aircraft must support multiple mission configurations, including passenger transport, cargo operations, and aerial medical evacuation.

The objective was split into 3 Configurations.

- Configuration 1 – Passenger Transport: 9 passengers at 200 lb each, along with 50 lb baggage, with a minimum 31-inch seat pitch.
- Configuration 2 – Cargo Transport: Equivalent payload to Configuration 1; must accommodate an 8,000 W generator and 55 gallons of diesel fuel.
- Configuration 3 – Medevac mission, carrying 1 litter, 4 occupants, and 300 lb of medical supplies.

The mission profile is defined to represent realistic operating conditions and is used to guide aircraft sizing and performance analysis. A typical mission consists of ground operations, short-field takeoff, climbing to cruise altitude, cruise over a specified range, descent, and short-field landing, while maintaining sufficient fuel reserves for loiter and contingency operations. Key constraints include a maximum takeoff and landing distance of 500 ft over a 50 ft obstacle, a minimum range of 450 nautical miles, a service ceiling $\geq 18,000$ ft, and a required rate of climbing at least 800 ft/min at maximum takeoff weight.

Section 2: Design Process

The first step in the design process for this aircraft was to identify a suitable reference aircraft. The Kodiak100 was the best analog found, able to fly over the required range of 450 nautical miles, carry 9 people, and take off from a short field, although the field requirements in the RFP turned out to be much more stringent. Given that the Kodiak100 fulfilled all requirements except the takeoff and landing requirements, that design point became the most important to alter. Final values for wing area, S , and aspect ratio, AR , were found in a trade study discussed later.

2.1 Theoretical Presentation on Takeoff and Landing

The 500-foot field length requirement was the main sizing constraint for the aircraft. Since such a small takeoff and landing window is imposed, satisfying this requirement directly leads to constraint analysis and choices regarding wing area, power, and the implementation of several high-lift devices.

2.1.1) Takeoff Distance

The total takeoff field length needed to pass over a 50-foot obstacle consists of a ground roll distance d_{ground} and an air distance, d_{air} , of

$$d_{TO} = d_{ground} + d_{air}, \quad (2.1)$$

with

$$V_{stall} = \sqrt{\frac{2W}{\rho SC_{L,max}}}, \quad (2.2)$$

being the stall speed where W is the aircraft weight, ρ being the air density, S is the wing reference area, and $C_{L,max}$ is the maximum lift coefficient.

The takeoff ground-roll was derived and estimated from the energy-approximation method, where the aircraft is assumed to accelerate from brake release to liftoff speed, where liftoff is taken at $V_{LO} = 1.2V_{stall}$. The ground roll can be found to be

$$d_{ground} = \frac{1.44W^2}{g\rho SC_{L,max}[T_A - D - \mu_r(W-L)]}, \quad (2.3)$$

where T_A is available thrust, D is the aerodynamic drag evaluated at $L = 0.7V_{LO}$ and $\mu_r = 0.1$, the rolling-friction coefficient assumed for the gravel runway. During the ground roll, the induced drag contribution was adjusted using a ground-effect correction factor to better represent the reduced induced drag near the runway. After liftoff, the air distance accounts for the transition and initial climb required to clear the 50 ft obstacle.

2.1.2) Landing Distance

The total landing field length over a 50 ft obstacle similarly consists of an air distance and a ground roll defined by

$$d_{land} = d_{air} + d_{ground}. \quad (2.4)$$

The approach distance is set primarily by the glide angle, which is governed by the landing lift to drag ratio $\frac{L}{D}$. A higher-efficiency wing produces a shallower descent path, increasing the distance required to clear the obstacle and reach touchdown. This creates an important tradeoff: the higher aspect ratio that improves cruise performance can also lengthen the landing air distance. The air-distance relation is

$$d_{air} = \left(\frac{L}{D}\right)_{land} \left(h_{obs} + \frac{0.5V_{TD}^2}{g}\right), \quad (2.5)$$

where $h_{obs} = 15.24$ m (50 ft) and the touchdown speed $V_{TO} = 1.3V_{stall}$. The ground roll after touchdown is therefore

$$d_{ground} = \frac{1.69W^2}{(\rho g S C_{L,max} (D_{land} + T_{rev} + \mu_b (W - L_{land}))}, \quad (2.6)$$

with D_{land} being the aerodynamic drag in the landing configuration T_{rev} is the reverse-thrusting braking contribution, and μ_b is the braking friction coefficient, where all of these terms are evaluated at $0.7V_{TD}$. The factor of 1.69 comes from using a touchdown of approximately

$$V_{TD} = 1.3V_{stall} \quad (2.7)$$

squaring this yields

$$V_{TD}^2 = 1.69V_{stall}^2 \quad (2.8)$$

In order to minimize landing distance, drag must be maximized during the ground roll, requiring dedicated spoilers, to simultaneously increase aerodynamic drag D_{land} and reduce lift.

2.1.3) Cruise Theory

Cruise range performance is governed by the Breguet range equation for propeller-driven aircraft. Expressing the result as a fuel fraction defined by

$$\frac{W_{fuel}}{W_0} = 1 - \exp\left(\frac{RgSFC_p}{\eta_p(L/D)}\right), \quad (2.9)$$

where R is the cruise range, g is the gravitational acceleration, SFC_p is the specific fuel consumption, η_p is the propeller efficiency, and L/D is the cruise lift-to-drag ratio. For a propeller-driven aircraft, cruise fuel consumption is closely linked to aerodynamic efficiency. Therefore, the range segment was evaluated at the operating condition at $(L/D)_{max}$. The optimum lift coefficient for maximum range is then obtained from the parabolic drag polar

$$C_D = C_{D0} + kC_L^2, \quad (2.10)$$

with

$$k = \frac{1}{\pi A Re}, \quad (2.11)$$

where e is the oswald efficiency factor assumed to be 0.9 based on the RFP. At the maximum range condition,

$$C_{L,opt} = \sqrt{\frac{C_{D0}}{k}} . \quad (2.12)$$

Where the optimal lift coefficient is,

$$C_{D,opt} = 2C_{D0} . \quad (2.13)$$

And the maximum Lift-to-Drag ratio is defined as,

$$\left(\frac{L}{D}\right)_{max} = \frac{C_{L,opt}}{C_{D,opt}} . \quad (2.14)$$

The 45-minute loiter reserve, however, was treated separately using the exact Breguet endurance relation. Unlike the cruise segment, which is optimized at maximum L/D , propeller endurance at the minimum power-required condition. This corresponds to,

$$C_{L,opt} = \sqrt{\frac{3C_{D0}}{k}} . \quad (2.15)$$

At this operating point, the corresponding drag coefficient is

$$C_{D,BE} = 4C_{D0} , \quad (2.16)$$

where $C_{D,BE}$ is the coefficient of drag at best range. This relationship allows the endurance segment to be evaluated using the lift and drag coefficients associated with minimum power required rather than the cruise range condition. The 45-minute loiter reserve was then calculated using the propeller endurance relation,

$$E = \frac{\eta_p}{gSFC_p} \left(\frac{C_L^{(3/2)}}{C_D} \right) \sqrt{2\rho S} \left[W_1^{-1/2} - W_0^{-1/2} \right] . \quad (2.17)$$

This separation between range and endurance conditions ensures that the cruise fuel estimate is based on maximum distance per unit fuel, while the loiter reserve is based on minimum fuel burn per unit time. Because the loiter endurance relation depends on both the initial aircraft weight, W_0 , and the selected wing area, the loiter fuel fraction was updated during each Raymer sizing

iteration after being estimated initially using the values from the Kodiak 100 of (L/D Kodiak). This avoided treating loiter fuel as a fixed fraction and allowed the reserve fuel estimate to remain coupled to the evolving aircraft weight and geometry.

2.2 Takeaways from Theory

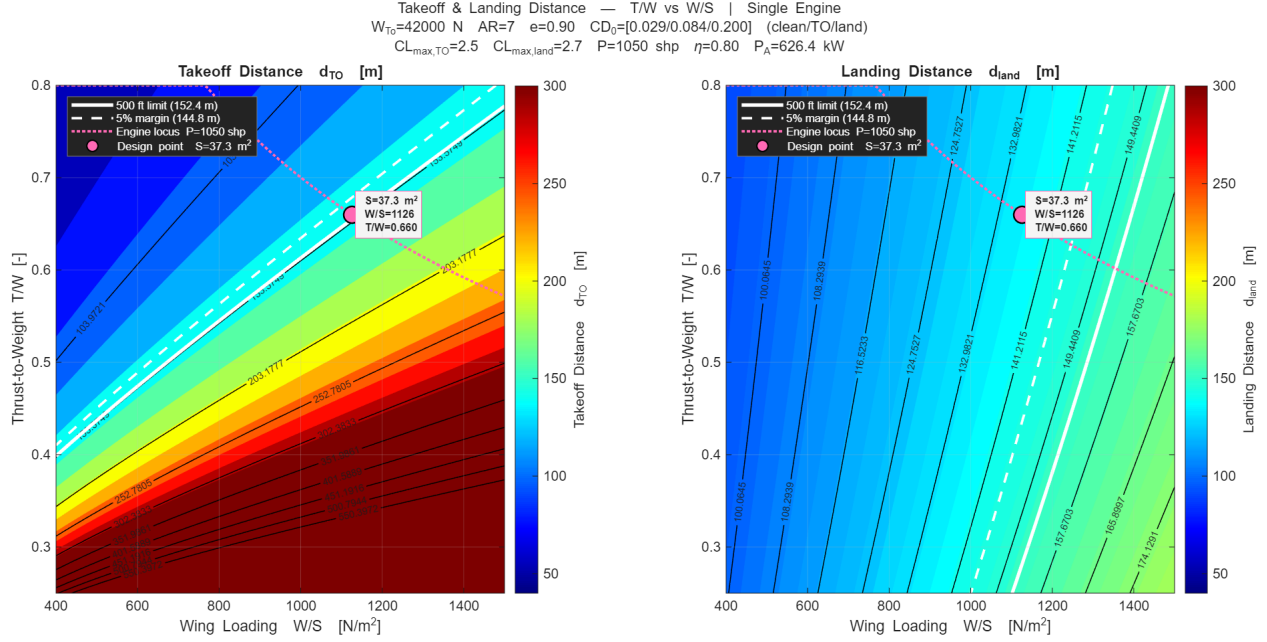
For takeoff performance, the most direct way to reduce the required takeoff field length is to increase the available thrust through the selection of a more powerful engine or propulsion system. Thrust available exists in the denominator of the equation (reference equation from 2.1), meaning that increasing that value would bring down the total distance the aircraft requires to take off. Additionally, increasing the $C_{L,max}$ of the airfoil would reduce the velocity the aircraft would need to achieve before liftoff as V_{LO} is $1.2V_{stall}$ (reference equation for V_{stall}). A lower V_{LO} would lead to a lower d_{LO} , as seen by the takeoff ground roll equation

$$d_{air} = h_{obs} W / (T_A - D). \quad (2.18)$$

From Eq. (2.3), takeoff distance is primarily affected by three design choices. First, increasing the available thrust T_A improves acceleration during the ground roll. Second, increasing $C_{L,max}$ through the use of high-lift devices lowers stall speed V_{stall} and therefore the liftoff speed V_{LO} , resulting in reducing the takeoff distance proportionally to W^2 .

Landing pushed the design towards the same direction. The landing distance formula seeks to reduce weight, increase the engine's reverse thrust, and reduce approach speed by increasing $C_{L,max}$ to lower stall speed. However, landing differs from takeoff in that one wants to increase drag significantly. Increased drag means the aircraft slows down quicker, an essential aspect of the design given the 500 ft landing requirements. It becomes clear that aggressive spoilers are needed for landing, increasing C_{D0} while also reducing lift during ground roll. Like takeoff, the landing distance was proportional to the square of the weight during landing.

From both takeoff and landing theory, the aircraft would need to include the following: a low Maximum Takeoff Weight, high-lift devices to increase $C_{L,max}$, a powerful engine to produce sufficient thrust, and devices to increase parasitic drag upon landing.



2.3 Airfoil Selection

2.3.1) Coefficient of Lift Requirements

Per RFP Section 4.3, the aircraft is not permitted to operate at the full maximum lift capability of the airfoil and high-lift system. Instead, the operational aircraft lift coefficient is limited to 80% of the available system $C_{L,max}$. This margin accounts for gusts, disturbances, and off-nominal conditions during the most critical low-speed phases of flight. From the constraint analysis (See Section 2.8), the aircraft requires $C_{L,max,TO} = 2.56$ for takeoff, and $C_{L,max,land} = 2.72$ for landing. Since the landing case requires the larger lift coefficient, it governs the high-lift system sizing. Applying the 80% operational limit gives and updated coefficient of lift of the aircraft is defined by

$$C_{L,max,aircraft} = 0.8C_{L,max,system} \quad (2.19)$$

The equation can then be rearranged to solve for the system coefficient of lift yielding

$$C_{L,max,system} \geq \frac{2.72}{0.8} = 3.4. \quad (2.20)$$

Therefore, the combined airfoil and implemented high-lift system must be capable of producing at least a maximum lift coefficient of 3.4 to satisfy the STOL requirement at the desired design weight.

The aerodynamic design of the wing is driven by the requirement to achieve sufficient lift during low-speed flight, particularly during takeoff and landing. Based on performance constraints, the aircraft must satisfy a target maximum lift coefficient $C_{L,max} \geq 3.4$

2.3.2) Airfoil Selection

The NASA LS(1)-0417 (GA(W)-1) airfoil was selected as the primary wing cross-section. Developed by NASA Langley for general aviation applications, the GA(W)-1 features a 17% thickness ratio and a cambered profile optimized for high lift at moderate Reynolds numbers. Compared to conventional NACA 4-series airfoils, the GA(W)-1 exhibits significantly higher maximum lift capability and improved stall characteristics, making it well suited for low-speed and short-field performance.

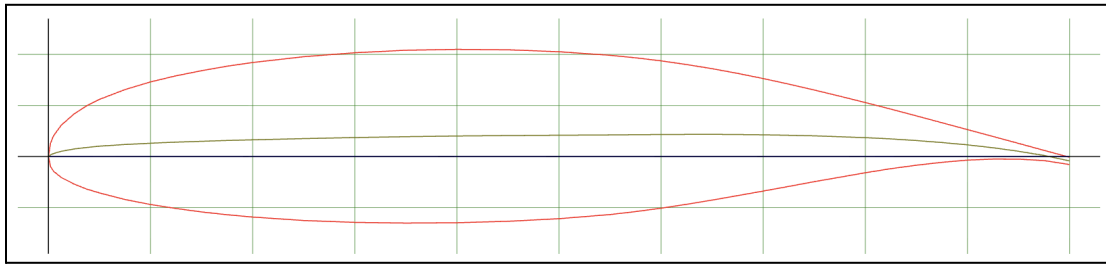
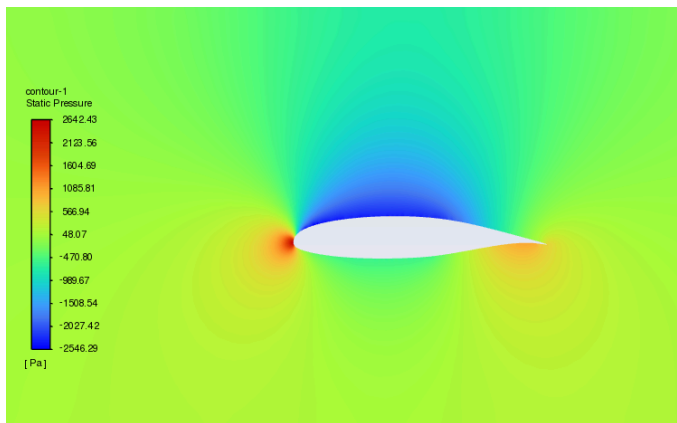
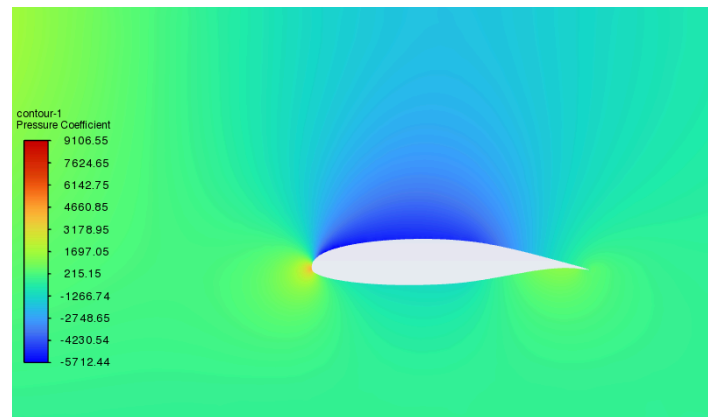


Figure 2: NASA/LANGLEY LS(1)-0417 (GA(W)-1) Airfoil



a) $\alpha = 0^\circ$



b) $\alpha = 19^\circ$

Figure 3: NASA/LANGLEY LS(1)-0417 (GA(W)-1) Airfoil CFD (ANSYS) pressure contours

The baseline aerodynamic performance of the airfoil was evaluated using XFOIL at a Reynolds number of 1.80×10^7 , representative of the aircraft operating conditions. The resulting lift curve, seen in *Figure 4*, demonstrates a linear lift response up to moderate angles of attack, with stall occurring near $\alpha = 22^\circ$ with a $C_{L,max} = 2.26$.

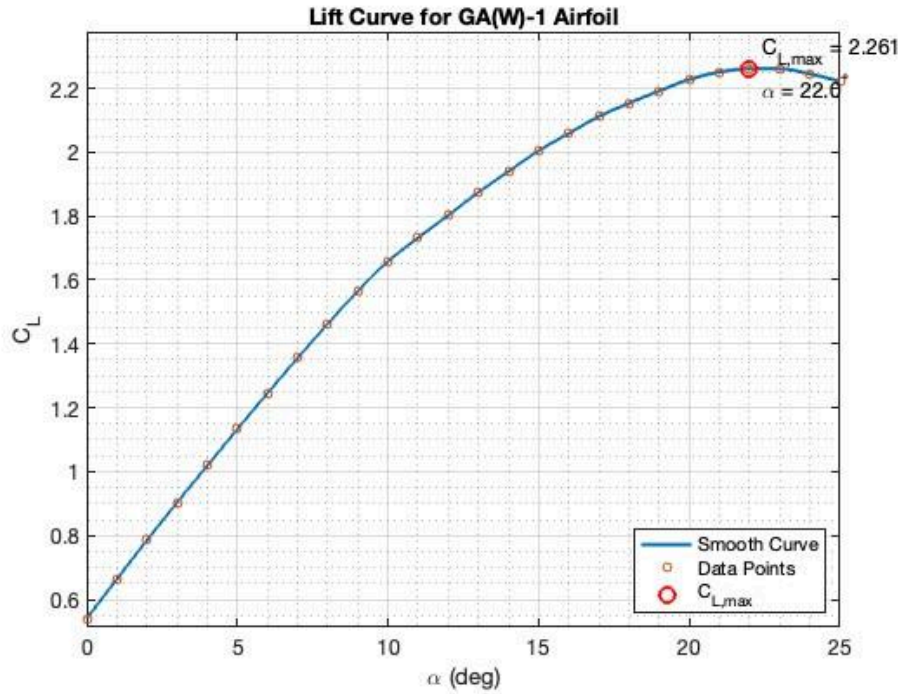


Figure 4: NASA GA(W)-1 Airfoil Lift Curve

Therefore, implementing a high-lift system consisting of double-slotted Fowler flaps and leading-edge slats is required to achieve the necessary low-speed performance. Using existing NASA wind tunnel studies on this particular airfoil, the addition of double slotted flaps and various leading-edge slat angles are necessary during takeoff and landing to satisfy the $C_{L,max}$ constraint.

2.3.2) Slotted flaps/slats

Based on existing NASA wind tunnel studies of the LS(1)-0417 airfoil with high-lift augmentation, the combination of double-slotted Fowler flaps (extended rearward, increasing chord and adding two slot gaps for boundary-layer energization) with leading-edge slats (delaying leading-edge separation to higher angles of attack) achieves $C_{L,max} = 3.5$ at $\alpha \approx 19^\circ$ at a $30^\circ - 40^\circ$ slat angle and a fixed flap deflection of 50° . This exceeds the minimum system requirement of 3.4 with a 2.9% margin.

In the partial-flap takeoff configuration, the usable lift coefficient (80% of maximum) for takeoff is $C_{L,max,TO} = 2.54$. In the full-flap-plus-slat landing configuration, $C_{L,max,land} = 2.72$. These values are used throughout the constraint analysis in Section 2.8. Therefore, a single-slotted flap configuration would be deployed during takeoff and the full double-slotted flap is utilized for landing.

2.4 Reference Aircraft

Two reference aircraft were identified as the most mission-relevant comparators for the Snowrunner 670. Both operate regularly in Arctic and sub-Arctic environments, carry comparable payloads, and are single- and twin-engine turboprops, respectively. They provide a physical reality check on MTOW, empty weight fraction, and STOL field performance.

Quest/Daher Kodiak 100 (Single-Engine): The Kodiak 100 is the most directly comparable aircraft to the Snowrunner 670. A purpose-built STOL single-engine turboprop, it carries nine passengers at an MTOW of 3,062 kg with a Pratt & Whitney Canada PT6A-140A producing 867 shp. Its empty weight fraction ($EWf = OEw/MTOW = 1,934/3,062 = 0.632$) represents the most relevant data point for validating the Snowrunner 670 weight estimate. The Kodiak satisfies the 450 nmi range and passenger payload requirements. However, it falls significantly short of the 500 ft TOFL requirement — its published balanced field length on paved runways is approximately 700–900 ft, and performance on gravel is worse. This shortfall is the central motivation for the Snowrunner 670's more aggressive high-lift design and higher-power engine.

DHC-6 Twin Otter Series 400 (Twin-Engine): The de Havilland Canada Twin Otter is the canonical Arctic STOL aircraft, proven on gravel strips, floats, skis, and tundra. Powered by two PT6A-27 engines producing approximately 1,500 shp total at an MTOW of 5,670 kg, it achieves documented sub-1,000 ft field lengths at maximum weight. This confirms that the 500 ft STOL requirement is aerodynamically achievable at lighter design weights with sufficient power and lift augmentation — the Snowrunner 670's single-engine design at 4,077 kg with 1,050 shp is proportionally well-positioned. The Twin Otter's twin-engine architecture was evaluated and ultimately rejected for the Snowrunner 670 (see Section 2.8.3).

2.5 Historical Empty Weight Estimate

The Raymer textbook [2] gives a method of calculating the OEw given a payload and a set of performance requirements.

2.5.1) Payload Breakdown by Configuration

Table 1: Weight breakdown for Configuration 1 Payload

Configuration 1 Payload	Component Weight [kg] (lbs)	Notes
-------------------------	-----------------------------	-------

9 passengers	816 (1800)	RFP Requirements
Passenger baggage	204 (450)	RFP Requirements
1 flight attendant weight	90.7 (200)	Design choice
1 FA baggage	9.07 (20)	Design choice
10 removable seats	100 (221)	RFP Requirements
Total Payload Weight	1,220 kg (2690)	-

Table 2: Weight breakdown for Configuration 2 Payload

Configuration 2 Payload Items	Component Weight [kg] (lbs)	Requirement
Cargo - Passenger weight equivalence	816 (1800)	RFP Equivalence
Cargo - Passenger baggage equivalence	204 (450)	RFP Equivalence
Cargo - Seat equivalence	90 (198)	RFP
55 gallons of diesel fuel	177 (390)	Non-propulsion fuel
8000W generator	99.3 (219)	Equipment payload
Weight of 55 gallon plastic fuel drum	11.4 (25.1)	Fuel storage
Total Payload Weight	1,399 kg (3083)	Sizing Case (Affects MTOW)

Table 3: Weight breakdown for Configuration 3 Payload

Configuration 3 OUTBOUND	Component Weight [kg] (lbs)	Notes
2 medical technicians	181 (400)	RFP
Medical supplies	136 (300)	RFP
Stretcher	22.7 (50)	RFP
Total OUTBOUND Payload	340 (750)	

Configuration 3 INBOUND	Component Weight [kg] (lbs)	
2 medical technicians	181 (400)	RFP
Medical supplies	136 (300)	RFP
Stretcher	22.7 (50)	RFP
Patient	90.7 (200)	RFP
Patient escort	90.7 (200)	RFP
Total INBOUND Payload	521 (1150)	

From Table 2, configuration 2 represented the heaviest mission as it is necessary to transport the equivalent mass of nine passengers (+ the weight of the removable seats). In addition, it is necessary to add the weight of 55 gallons of diesel fuel and an 8000W diesel generator. The final payload used for sizing is therefore 1399 kg.

2.5.2) Methodology

The aircraft maximum takeoff weight was estimated using a component buildup method in which the total takeoff weight is expressed as the sum of crew, payload, fuel, and empty aircraft weight such that

$$W_0 = W_{crew} + W_{payload} + W_{fuel} + W_{empty}. \quad (2.21)$$

The equation above can be rewritten in terms of fractions to then be used in a statistical fit where

$$W_0 = W_{crew} + W_{payload} + \left(\frac{W_{fuel}}{W_0}\right)W_0 + \left(\frac{W_{empty}}{W_0}\right)W_0. \quad (2.22)$$

In addition to fractions that can be estimated, by rearranging an equation that can be used to solve the takeoff weight, W_0 , in terms of the crew and payload weight is found such

$$W_0 = (W_{crew} + W_{payload}) / \left(1 - \frac{W_{fuel}}{W_0} - \frac{W_{empty}}{W_0}\right). \quad (2.23)$$

The fuel fraction for cruise was estimated using the max range equation for a propeller plane defined by

$$\frac{W_{fuel}}{W_0} = e^{-\left(\frac{RgSFC}{\eta(L/D)}\right)}. \quad (2.24)$$

Where all the parameters on the left side of the equation are known or estimated using preliminary engine data.

Fuel fractions for other legs of the flight were estimated based on historical data from Raymer. Fuel fraction for climb = 0.985, and the fuel fraction for landing = 0.995.

Then, the total fuel fraction for the mission was calculated by taking the product of all of the segments where

$$Total\ Fuel\ Fraction = 1.03(1 - \sum \frac{W_i}{W_{i-1}}). \quad (2.25)$$

A 1.03 correction term was included for fuel reserves (2%) and unusable fuel at the bottom of the tanks (1%). This was done in conjunction with the fuel reserve from loiter, meaning the plane would have excess fuel reserves beyond the 45 minute loiter requirement within the RFP.

The next step in estimating the mass of the aircraft was to find the OEW fraction. This was done by using Raymer's historical fit curves. The operating empty weight fraction was then estimated as a function of takeoff gross weight using Raymer's empirical relationship for general aviation aircraft where

$$\frac{W_{empty}}{W_0} = AW_0^C. \quad (2.26)$$

For a single-engine general aviation aircraft, A = 2.36 and C = -0.18. For a twin-engine general aviation aircraft, A = 1.51 and C = -0.1. For both OEW estimations, a 1.05 multiplier was used to oversize the structure. This was done to account for the increased weight of the high lift mechanisms, spoiler deployment, and reinforced landing gear needed for aggressive landings. It can be seen, however, that the OEW fraction is still dependent on W_0 , which at this time is unknown. Starting from an initial guess of the weight of 4000kg, the combination of equations 2.22 through 2.25 above was used to iterate to a solution.

2.5.3) Weight Results

Table 4: Maximum Takeoff Weight breakdown for Configuration 2

Components of MTOW	Component Weight [kg]
--------------------	-----------------------

Structural	1969
Crew	181
Operating Empty Weight (EWF)	2150 (52.7% of MTOW)
Fuel	535 (13.1% of MTOW)
Payload (no crew)	1399 (from Table 1)
MTOW (W_0)	4,077 kg (40000 N)
MTOW (Twin-engine)	6218 kg (61000 N)

It is worth noting that the fuel weight estimation was calculated using a range of 450nmi. This is far more than the required range needed for the plane, as this does not factor in the distance covered during climb or descent.

2.6 Engine Selection

The PT6A family is the dominant turboprop product in its power class, installed on over 100 aircraft types across more than 100 countries. Its reversed-flow, free-turbine architecture provides exceptional hot-and-high performance, self-contained lubrication tolerant of inverted flight attitudes, and a cold-soak starting capability extending to -54°C with appropriate preheat.

Table 5: PT6A-60A key performance parameters

PT6A-60A Engine Parameters	Values
Rated shaft power	1,050 shp (783 kW)
Propeller efficiency (η_p)	0.80
Useful propulsive power at sea level	$\eta_p \times 1,050 \times 745.7 = 626 \text{ kW}$
Engine mass	475 kg

The thrust required was compared in a trade study with weight to ensure the engine remained within the range of reasonable STOL aircraft without adding excessive weight or thrust. Thrust and power requirements were computed iteratively using a ground-roll and obstacle-clearance model, enforcing a 500 ft (152 m) balanced field length over a 50 ft (15.24 m) obstacle. At the initial design gross weight of 41,500 N, with a wing area of 36.8 m^2 , span of 15.2 m, CL_{max} of 2.4, and a parasitic drag coefficient of 0.03, the model converged on a thrust requirement of 14.19 kN (3,189 lbf), corresponding to a shaft power requirement of 598 kW (802 hp) at liftoff

speed. The final values for weight, wing area, and span were determined in a trade study, but did not vary by more than 5% the initial estimates.

With the power requirement satisfied, the downselection process naturally converged on the PT6A-60A. Candidate engines were evaluated against three primary criteria: rated shaft power, specific fuel consumption, and operational heritage in similar utility and STOL roles. Heavier or higher-output turboprops, such as the PT6A-67 series, were ruled out as they introduced unnecessary weight and cost penalties for a mission that the -60A could satisfy. The PT6A-60A's 1050 shp rating sits well above the computed requirement, providing a healthy power margin for hot-and-high operations, emergency go-arounds, and the performance degradation inherent to unprepared strips.

2.7 Drag Buildup

The zero-lift parasite drag coefficient C_{D0} was estimated using the Raymer component buildup method (Raymer §12.5, Eq. 12.24). The clean-configuration parasite drag is the sum of friction and pressure drag contributions from each major airframe component, normalized by the reference wing area, plus miscellaneous and leakage terms such that

$$C_{D0, clean} = \frac{\Sigma(C_f FFQS_{wet})}{S_{ref}} + C_{D, misc} + C_{D, L\&P}, \quad (2.27)$$

where C_f is the flat-plate turbulent skin friction coefficient, FF is the form factor accounting for pressure drag, Q is the interference factor at component junctions, and S_{wet} is the wetted area of each component. The reference area is $S_{ref} = 36.8 \text{ m}^2$. All coefficients are evaluated at the cruise condition of 130 KTAS (66.9 m/s) at sea level.

2.7.1) Skin Friction Coefficients

Skin friction was computed for each component using Raymer's turbulent, Mach-corrected flat-plate formula (Eq. 12.27) defined by

$$C_f = \frac{0.455}{(\log_{10} Re)^{2.58} (1 + 0.144 M^2)^{0.65}}. \quad (2.28)$$

The characteristic length used is the fuselage total length for the fuselage, and the mean aerodynamic chord (MAC) for all lifting surfaces. At the cruise Mach number of $M \approx 0.196$, the Mach correction is negligible but retained for consistency.

2.7.2) Form and Interface Factors

Wing and tail form factors were computed using Raymer Eq. 12.30, where

$$FF = [1 + \frac{0.6}{(x/c)_m} (\frac{t}{c}) + 100(\frac{t}{c})^4][1.34M^{0.18}(\cos\Lambda_m)^{0.28}]. \quad (2.29)$$

The wing uses the NASA GA(W)-1 (LS(1)-0417) airfoil with $t/c = 0.17$ and maximum thickness at $(x/c)_m = 0.40$. Both horizontal and vertical tails use the NACA 0012 section ($t/c = 0.12$, $(x/c)_m = 0.30$), and their form factors were increased by 10% to account for hinge-gap drag at the control surface junction, per Raymer p.283. All surfaces are unswept, so the sweep bracket collapses to approximately unity.

The fuselage form factor was computed using Raymer Eq. 12.31 for an axisymmetric body such that

$$FF_{ellipsoid} = 1 + \frac{60}{f^3} + \frac{f}{400}, \quad (2.30)$$

where $f = 1/d$ is the fineness ratio. The fuselage cross-section is a filleted rectangle (1.75 m wide $\times$ 2.00 m tall, 0.10 m corner fillets), giving an equivalent circular diameter of $d = \sqrt{(4A_{\max}/\pi)} = 1.62$ m and a fineness ratio of $f \approx 6.1$. Raymer p.284 notes that a square-sided fuselage carries a form factor approximately 40% higher than the ellipsoidal value due to flow separation at the corners. With 0.10 m fillets producing a modest rounding ratio of $r/\text{side} \approx 0.053$, a 20% penalty was applied as a conservative midpoint between the sharp-cornered and fully-rounded limits.

Interference factors Q were assigned per Raymer §12.5 (p.285). The fuselage and wing each receive $Q = 1.00$: the fuselage interference is negligible by convention, and the high-wing configuration with clean fillets also yields negligible interference. The horizontal tail receives $Q = 1.05$, consistent with the midpoint of Raymer's 3–8% range for conventional H-tail empennage arrangements. The vertical tail receives $Q = 1.03$, the lower end of the range for a clean fin-fuselage junction.

2.7.3) Wetted Areas

Lifting surface wetted areas were computed as

$$S_{wet} = 2 \times S_{plan} \times (1 + 0.2 \times \frac{t}{c}), \quad (2.31)$$

accounting for both upper and lower surfaces and the thickness-related increase in exposed area. The fuselage wetted area was calculated from the perimeter of the filleted rectangular cross-section multiplied by fuselage length, giving $S_{wet,fuse} = P_{wet} \times l_{fuse}$.

2.7.4) Landing Gear and Miscellaneous Drag

The aircraft uses a fixed tricycle undercarriage (two main legs and one nose leg). Gear drag was estimated using Raymer Table 12.5 D/q values: 0.25 ft² per wheel/tire and 0.30 ft² per round strut, summed over all three legs and multiplied by 1.20 for mutual interference per Raymer p.287. The resulting D/q was converted to SI and normalized by S_{ref} to yield $C_{D,misc}$.

A leakage and protuberance allowance of 5% of the primary component drag sum was added, per Raymer p.289. Raymer recommends 5–10% for propeller aircraft; the lower bound of 5% was selected as appropriate for an aircraft with careful attention to surface finish, noting that landing gear drag is already accounted for explicitly in $C_{D,misc}$.

2.7.5) Results

The component contributions to $C_{D0, clean}$ are summarized in *Figure 5* and in Appendix A.

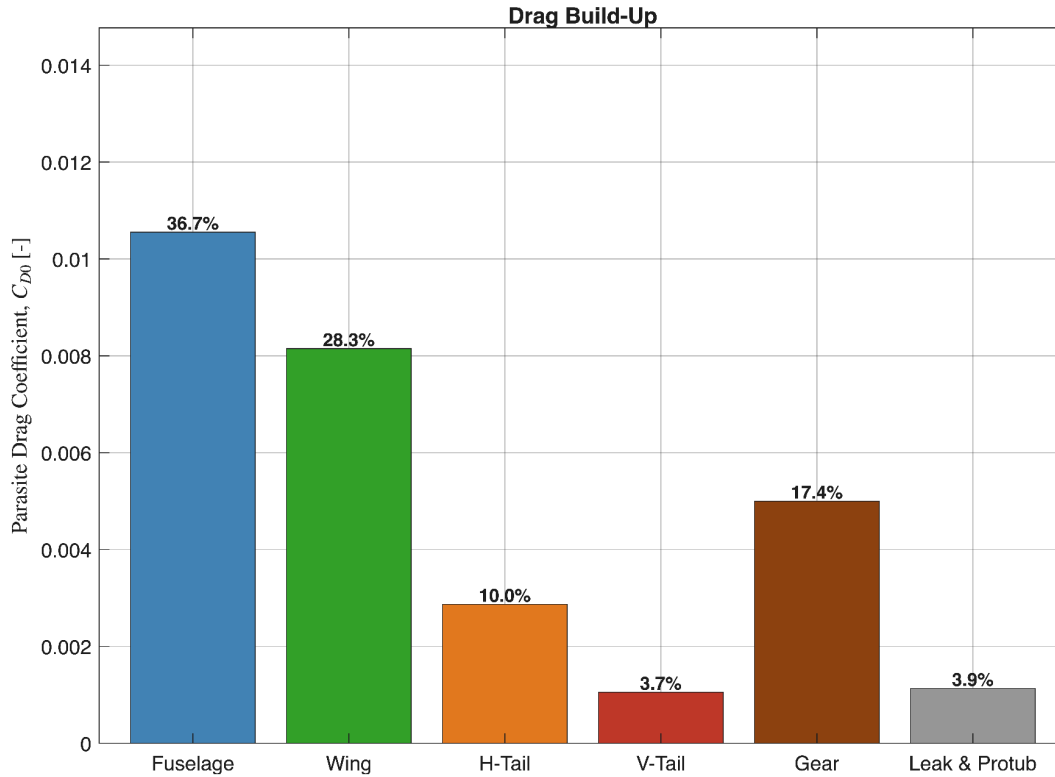


Figure 5: Drag build up graphed by aircraft component

The resulting clean-configuration parasite drag coefficient of $C_{D0, \text{ clean}} \approx 0.029$ is consistent with the placeholder value used throughout the constraint analysis in Section 2.8. The fuselage is the dominant contributor at approximately 43% of total parasite drag, driven by its large wetted area and the rectangular cross-section form factor penalty. The wing contributes approximately 33%. These two components together account for over three-quarters of the total parasite drag, which highlights the importance of the fuselage fineness ratio and surface finish quality in achieving good cruise efficiency. The fixed landing gear contributes approximately 6%, which is a known penalty of the fixed-gear configuration chosen for simplicity and Arctic field maintainability.

2.7.6) Drag from Flaps and Spoilers

The landing drag build-up is split into two phases: the airborne approach, where only flaps are deployed, and the ground roll after touchdown, where spoilers are additionally active. The flap drag increment was estimated using the Raymer empirical relation where

$$\Delta CD_{0,flaps} = 0.0023 \cdot \frac{b_{flap}}{b} \cdot \delta^f. \quad (2.32)$$

At full landing flap deflection of $\delta^f = 50^\circ$ over 80% of the semi-span, this gives $\Delta C_{D0,flaps} = 0.0023 \times 0.80 \times 50 = 0.092$, raising the parasite drag coefficient from the clean baseline of 0.029 to $C_{D0,approach} = 0.121$ for the approach phase. The spoiler drag increment, active during the ground roll only, was computed using the projected-area flat-plate model defined by

$$\Delta CD_{0,spoiler} = C_{d_n} \cdot \frac{S_{spoiler}}{S_{ref}} \cdot \sin(\theta). \quad (2.33)$$

With a normal-force drag coefficient $C_{d_n} = 1.28$, a spoiler-to-wing area ratio of 0.10, and a deflection angle of $\theta = 60^\circ$, the spoiler increment is $\Delta CD_{0,spoiler} = 1.28 \times 0.10 \times \sin(60^\circ) = 0.111$. The total parasite drag coefficient during the ground roll is

$$\Delta CD_{0,ground} = CD_{0,clean} + \Delta CD_{0,flap} + \Delta CD_{0,flap} + \Delta CD_{0,spoiler}. \quad (2.34)$$

The ground-roll drag coefficient is therefore found to be 0.232, which is 8 times the clean cruise value of 0.029, reflecting the combined effect of full flap deflection and spoiler deployment on shortening the landing ground-roll distance. The final flap and slat parasite drag coefficient values are presented in Table 6 below.

Table 6: C_{D0} values

Component	$C_{D0,cruise}$	$C_{D0,takeoff}$	$C_{D0,approach}$	$C_{D0,groundroll}$
Baseline (from previous sections)	0.029	0.029	0.029	0.029
Flap deployment angle (deg)	0	30	50	50
Flap drag contribution	0	0.055	0.092	0.092
Slat deployment angle (deg)	0	0	0	60
Slat deployment contribution	0	0	0	0.111
Total C_{D0}	0.029	0.084	0.121	0.232

2.8 Constraint Diagrams

2.8.1) Thrust-to-Weight vs Wing Loading

Due to the unusually short takeoff and landing constraints imposed upon this aircraft by the RFP, the takeoff and landing requirements are the most important requirements to satisfy. With all parameters affecting takeoff and landing now known or estimated to a high degree of accuracy for takeoff and landing decided based on research or best estimates (including the weight from section 2.5), a trade study between engine power (represented by Thrust-to-Weight ratio) and Wing Loading (W/S) was conducted.

Figure 1 shows that the PT6A-60A engine, represented by the pink dot, fulfills the takeoff and landing requirement by residing (at representative wing dimensions) above the white line in both plots.

2.8.2) Aspect Ratio vs. Wing Area

For sizing the wing area, multiple design constraints were considered to determine the required wing size for the aircraft. Since the aircraft is intended for STOL operation, the wing had to provide enough lift at low speeds while also maintaining acceptable cruise performance, fuel storage capability, and structural practicality. The sizing process began by evaluating the stall speed requirement using the final maximum takeoff weight and the selected landing maximum lift coefficient. Because the aircraft is expected to operate from short and potentially remote runways, low-speed performance was treated as one of the primary design drivers.

Increasing wing area improves stall speed, takeoff, and landing performance by reducing wing loading. However, a larger wing also increases structural weight and parasite drag, which can negatively affect cruise performance and range. Structural weight considerations were not explicitly analysed in this design proposal but kept in mind when making design decisions to increase the realism of the final aircraft.

A systematic constraint analysis was performed by sweeping wing area S from 20 to 90 m² and aspect ratio AR from 5 to 14. For each (S, AR) combination, both the takeoff and landing field lengths over a 50 ft obstacle were computed and compared with the 500 ft requirement. This constraint analysis computes both takeoff and landing field lengths at each grid point using $MTOW = 40,000 \text{ N}$ (61,000 N for twin-engine configuration), PT6A-60A 1,050 shp, $C_{L,max,land} = 2.6$, $C_{D0, clean} = 0.029$, and $\mu_r = 0.1$. The constraint analysis was re-run at each MTOW iteration as the weight estimate converged. A conservative 5% design margin on field length was maintained throughout.

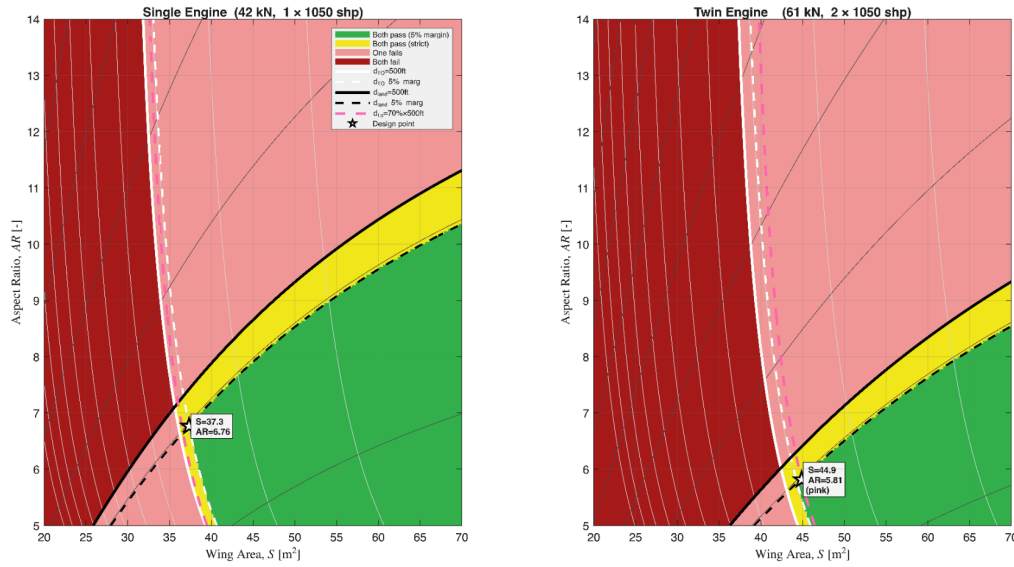


Figure 6: Constraint Analysis AR vs S for a) Single and b) Twin Engine Configurations

Figure 6 is a result of combining the takeoff and landing heatmaps that measure the total distance the aircraft needs to takeoff and land respectively. As seen in Section 2.2, the governing equations for takeoff and landing have similar, but not identical results for aircraft wing sizing, generating a valid design region when analyzed together. Figure 6 contains white contour lines that measure takeoff distance and black contour lines that measure landing distance. The thick lines on the contour map show the absolute cutoff from takeoff and landing distance (152.4m) and the dashed lines show the required wing geometry factoring in a 5% margin.

The color convention consists of three regions where green represents both $d_{TO} \leq 144.8$ m and $d_{land} \leq 144.8$ m — 5% design margin on 500 ft maintained, yellow represents both constraints within 500 ft, but without the 5% margin, and light-red represents one or both constraints violated.

The selected design points of $S = [37.3, 44.9]$ m² and $AR = [6.76, 5.86]$ (for the single and twin engine configurations respectively) represent the leftmost green feasible point, the smallest wing area consistent with both constraints and margin, which minimizes structural weight while maximizing cruise efficiency.

As seen in Figure 7, the wing area is larger and AR smaller for the twin-engine configuration. This is due in part because of the twin-engine's increased empty weight as a result of the historical weight estimation conducted in Section 2.5.3. The large S and decreased AR compared to the single-engine configuration leads the twin-engine configuration to have decreased cruise performance. For these reasons, in addition to easier maintenance and lower cost, the

single-engine configuration was selected as all performance requirements were satisfied while simplifying the design.

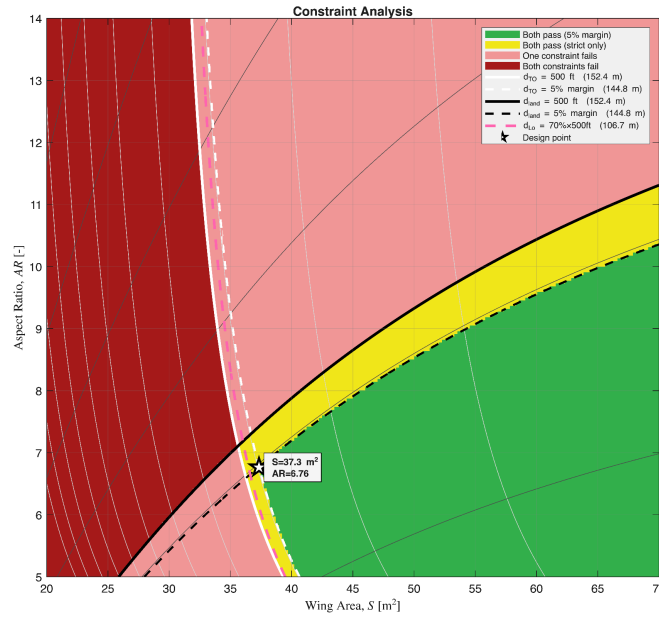


Figure 7: Constraint Analysis AR vs S for selected single engine configuration

2.9 Cruise Optimization

Two considerations drove the cruise altitude selection. First, pressurization is not required by the RFP and was not included in the design, though at 15,000 ft (4,572 m), cabin occupants without supplemental oxygen begin to experience meaningful hypoxia after extended exposure, but the RFP does not impose a maximum operating altitude below 18,000 ft. The 15,000 ft selection avoids the regulatory pressurization requirement while providing sufficient altitude for obstacle clearance over Arctic terrain and partial relief from low-altitude turbulence. Second, this altitude places the density ratio at $\sigma = \rho/\rho_{SL} \approx 0.630$ (ISA), sufficient for the PT6A-60A to sustain adequate power for cruise.

The RFP minimum cruise speed of 130 KTAS (66.9 m/s) was analyzed against the maximum L/D speed V_{LDmax} at 15,000 ft. A crossover altitude was identified — the altitude at which $V_{LDmax}(W_{mid}) = 130$ KTAS. Above the crossover altitude, $V_{LDmax} < 130$ KTAS, meaning the aircraft can fly at minimum drag and simultaneously satisfy the speed requirement. At 15,000 ft the aircraft operates in this regime, making the cruise point both speed-compliant and close to minimum drag, the most fuel-efficient feasible operating condition.

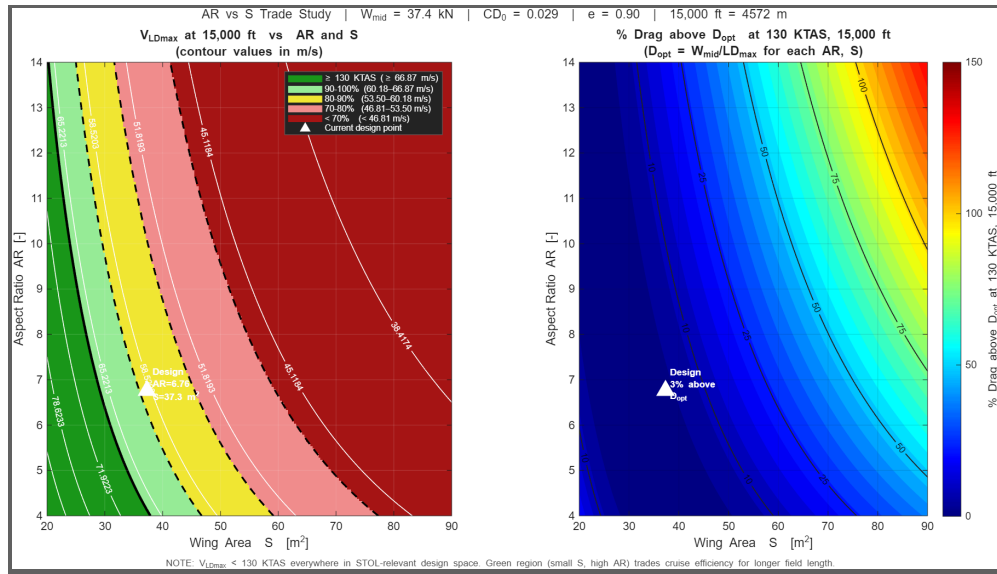


Figure 8: AR vs S Trade Study with cruise speed and drag

Figure 8 shows the aspect-ratio and wing-area trade study used to verify that the selected design point, $S = 37.3 \text{ m}^2$ and $AR = 6.76$, satisfies the 130 KTAS cruise-speed requirement while keeping drag growth low at the selected 15,000 ft cruise condition.

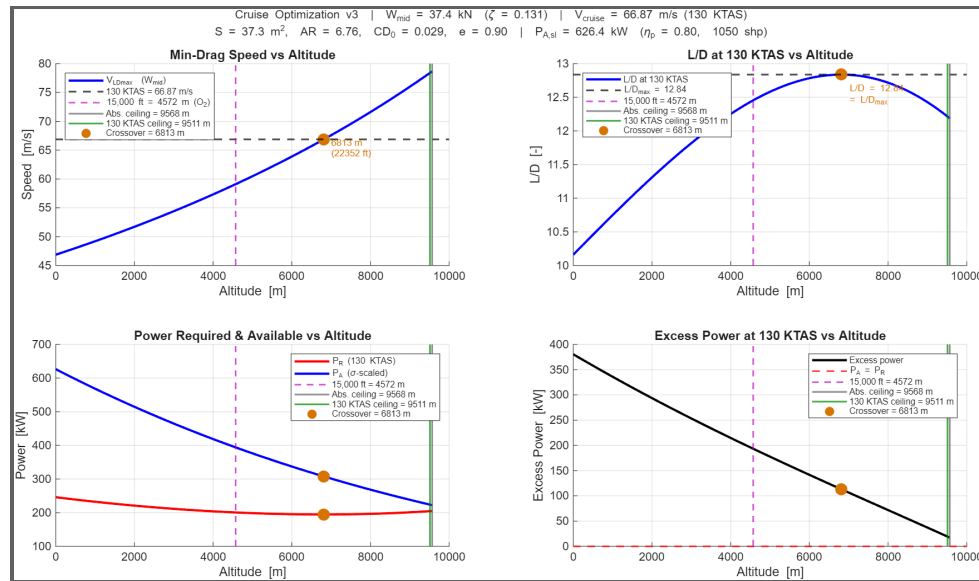


Figure 9: Cruise optimization and evaluation at required parameters

Figure 9 evaluates the selected cruise condition more directly, showing that at 15,000 ft the aircraft can meet the required 130 KTAS cruise speed while maintaining positive excess power and operating near the maximum L/D condition.

Section 3: Aircraft Performance Results

After the primary aircraft parameters were defined, the final performance characteristics were evaluated using the established geometry, weight, propulsion, and aerodynamic characteristics as fixed inputs in section 2.10. Section 3 focuses on analyzing the more specific mission performance characteristics of the selected design point. This section will also introduce performance analysis for both the reference mission and medvac mission as specified in the RFP. Each section of the results compares a selected aircraft feature against the governing mission or performance requirement.

3.1 Mission-Level Performance Overview

3.1.1) Operations and Performance Requirements

The aircraft satisfies the gravel runway requirement through its STOL configuration. The takeoff and landing ground-roll equations also include rolling friction terms appropriate for prepared gravel surfaces. The design is also suitable for limited unprepared-surface operation because of its low stall speed, high-lift system, strong landing gear, and short-field performance. While gravel runways are the primary design case, the same features improve performance on dirt, compact snow, and other unlevel field surfaces. The aircraft was checked against the service ceiling requirement of 18,000 ft. (5486.4 m) using the excess-power condition. At service ceiling, the aircraft must still have enough excess power to maintain a minimum climb capability. The rate of climb is defined by

$$ROC = \frac{P_A - P_R}{W}. \quad (3.1)$$

The selected single-engine propeller configuration provides sufficient available power to meet the 18,000 ft ceiling requirement during cruise and climb requirement of 800 ft/min (see Section 2.9). As shown in Table 7, the maximum ceiling of 9000 m is greater than 18,000 ft. (5486.4 m).

The aircraft was sized with enough installed power to exceed the required maximum velocity. Maximum level speed occurs when available power equals required power, such that

$$P_A = P_R. \quad (3.2)$$

Table 7: Performance requirements

Condition	RFP Requirement	Current Value	Status
Service ceiling ISA, ROC = 0.508 m/s	$\geq 5,486$ m ($\geq 18,000$ ft)	9,083 m (29,801 ft) · margin +3,597 m	PASS
Rate of climb	≥ 4.064 m/s	12.058 m/s	PASS

MTOW, ISA+5, sea level	(≥ 800 ft/min)	(2,374 ft/min) · margin +7.994 m/s	
Max level speed / VNE proxy sea level, MTOW	≥ 92.59 m/s (≥ 180 KTAS)	96.34 m/s (187.3 KTAS) · margin +3.75 m/s	PASS

3.1.2) Payload-Range Diagram

A preliminary payload-range diagram was developed using the final weight estimate. The useful load is defined by values in Table 7 and fits the following

$$W_{useful} = W_{TO} - W_{OEWS} \quad (3.3)$$

This results in a useful weight of 1927 kg, which must be divided between payload and fuel.

The payload-range diagram was constructed using a first-order fuel-to-range approximation based on the design mission fuel requirement,

$$\frac{R}{W_f} = \frac{450 \text{ nmi}}{535 \text{ kg}} \quad (3.4)$$

where R is range and W_f is fuel weight. This relation was used to estimate the payload tradeoff region beyond the design point. As range increases beyond the required mission range, additional fuel would require a corresponding reduction in payload to remain below the maximum takeoff weight. The payload at each range point is therefore estimated from

$$W_{payload} = W_{TO} - W_{OEWS} - W_f \quad (3.5)$$

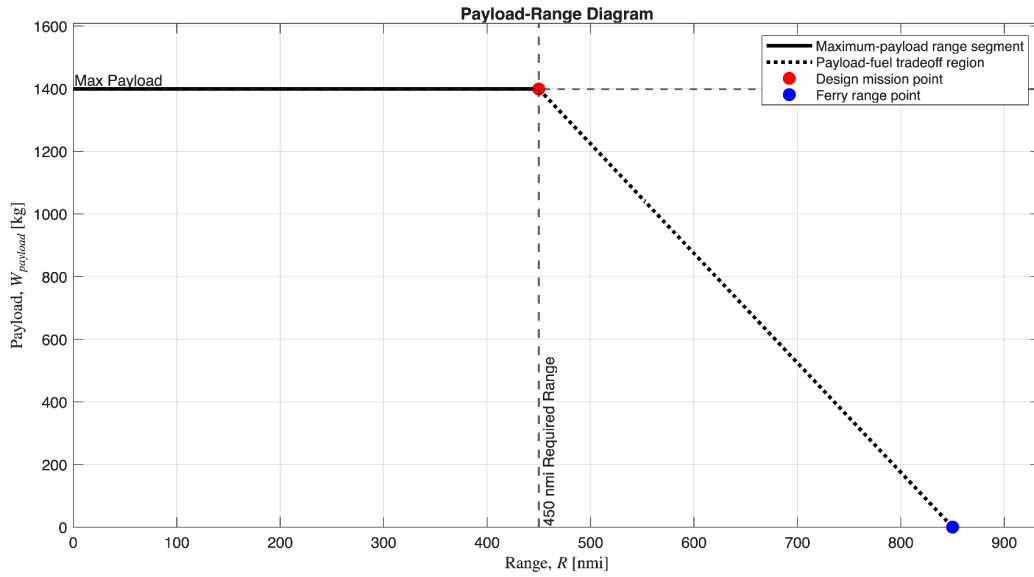


Figure 10: Payload range diagram

The flat portion of the curve represents operation at maximum payload; the descending portion represents the payload-fuel tradeoff. The aircraft meets the required 450 nmi mission at the maximum design payload of 1,399 kg with a 535 kg fuel load.

3.1.3) Turning Flight (V-n diagrams)

V-n diagrams can be found for both sea-level and at the cruise altitude of 4572 m (15000 ft), where the turning envelope of the aircraft is defined by

$$n_{env}(V) = \min(n_{C_L}, n_{struct}, n_T) \quad (3.6)$$

where the structural load factor is a constant set to $n_{struct} = 3.8$ which is set by the normal category positive limit load factor. The aerodynamic load factor is defined by

$$n_{aero} = \frac{L}{W} \quad (3.7)$$

can use the definition of lift to be used to define

$$n_{C_L} = \frac{qSC_{L_{max}}}{W}. \quad (3.8)$$

The thrust limit is defined by

$$n_T = \sqrt{(T_A - C_{D0} q S) \frac{q S}{k W^2}}. \quad (3.9)$$

The resulting V-n diagrams can be seen in *Figure 11*. The $F_{\max}$ line is included as an analytical reference for the maximum possible load factor but does not independently constrain the envelope.

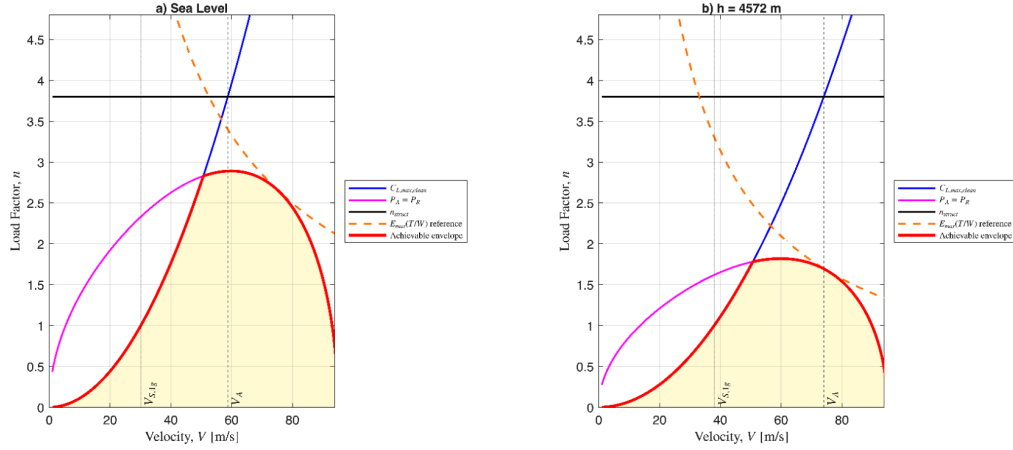


Figure 11: V-n diagram for the Snowrunner 670 with single engine at a) sea level and at b) cruise altitude of $h = 4572$ m

The turning envelope for the aircraft can be found from the three constraints that define the turning radius, being the structural load factor, defined by

$$r_{struct} = \frac{2q}{pg\sqrt{n_{struct}^2 - 1}}, \quad (3.10)$$

the coefficient of lift which is defined by

$$r_{C_L} = \frac{2q(\frac{W}{S})}{pg[(C_{L,max} q)^2 - (\frac{W}{S})^2]^{1/2}}, \quad (3.11)$$

and the thrust-limited constraint, defined by

$$r_T = \frac{2q}{pg\sqrt{n_T^2 - 1}}, \quad (3.12)$$

where n_T is

$$n_{\tau} = \sqrt{(T_A - C_{D0} q S) \frac{q S}{kW^2}}. \quad (3.13)$$

Using a range of velocities from 30 m/s to 300 m/s, the turning envelope can be found across the range of dynamic pressures modeled by

$$q = \frac{1}{2} \rho V^2. \quad (3.14)$$

The turning profile can then be modeled and seen in *Figure 12*.

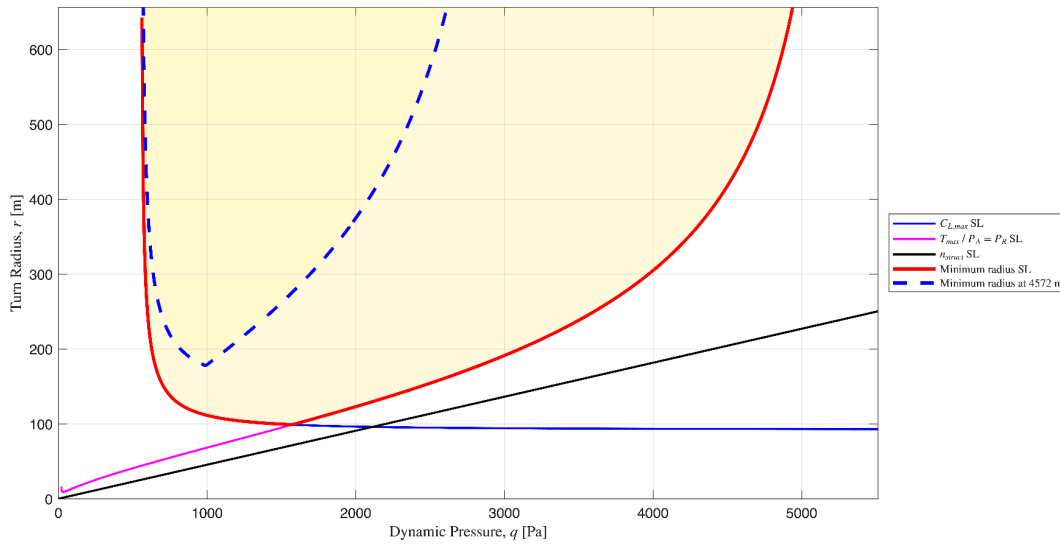


Figure 12: Turn radius versus dynamic pressure for a single engine aircraft

3.2 Passenger Reference Mission Performance

The reference mission uses Configuration 1 at 90% payload factor, yielding a required payload of 1,099 kg (= 1,220 kg × 0.9). The flight duration is approximately 206 minutes. The aircraft climbs at full throttle, cruises at approximately 46–48% throttle decreasing with fuel burn, and descends at 5–11% throttle. Cruise L/D = 12.5 at 130 KTAS, 15,000 ft.

The figures together show all phases of flight. *Figure 13* shows altitude flat at 4,572 m for 87% of the airborne timeline, with the color progression from purple through red confirming the full 833 km ground track is covered. *Figure 14* confirms full throttle only during the short climb, a gently declining throttle during cruise as the aircraft lightens, and zero throttle during descent — gravity alone sustains the 1,000 fpm / 67 m/s glide geometry. Every RFP requirement is met: 450 nmi range, ≥ 130 KTAS cruise, ≥ 800 fpm rate of climb, and the 45-minute reserve. Below are charts demonstrating the different phases of the flight including the overall flight conditions.

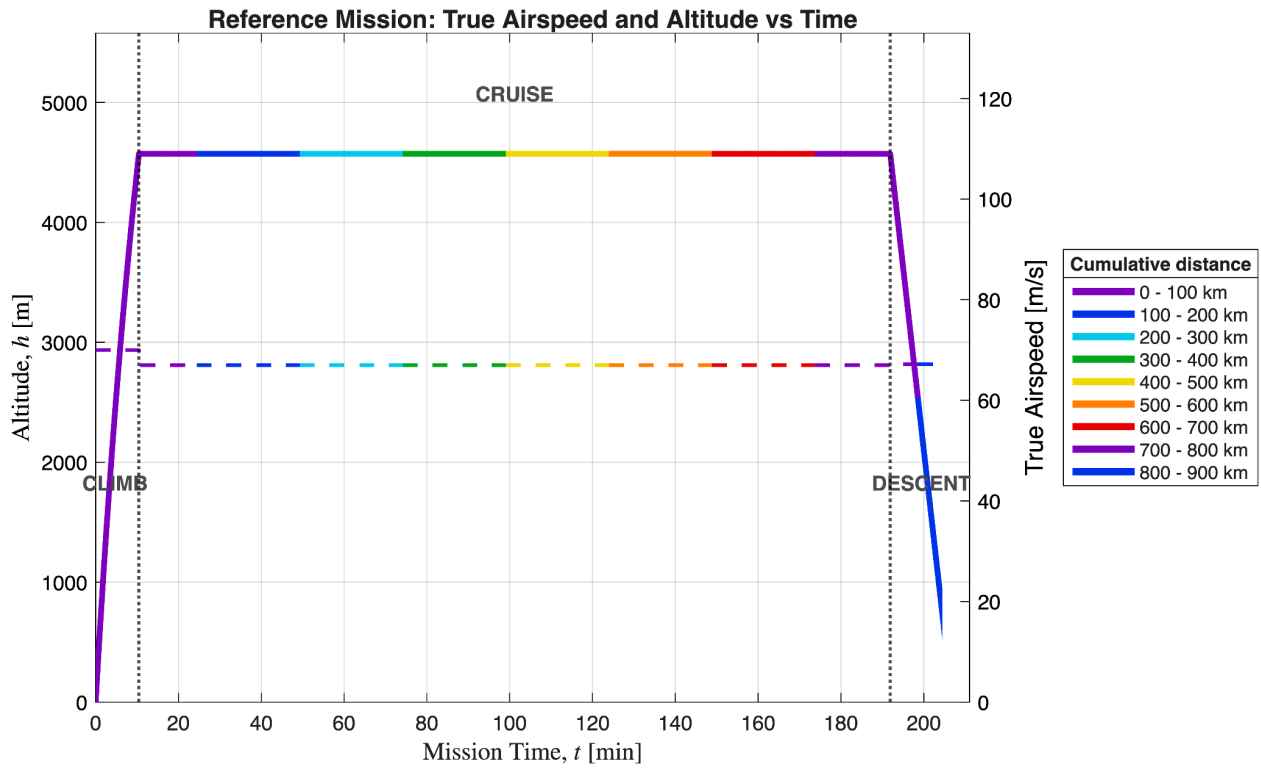


Figure 13: Passenger reference mission flight profile charting altitude and airspeed

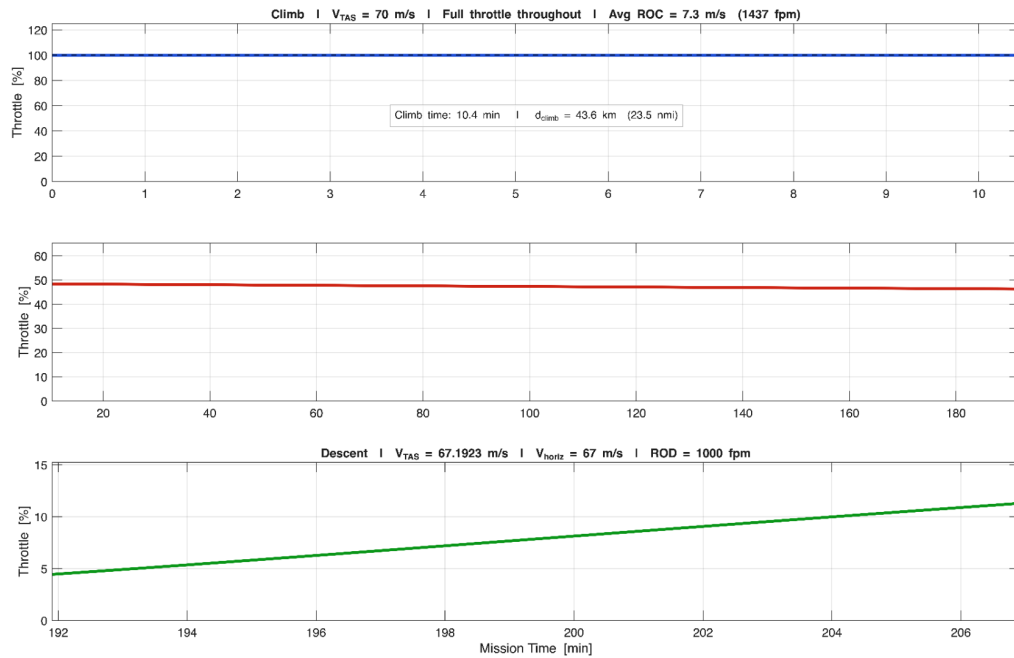


Figure 14: Reference mission flight profile charting throttle level at different flight phases

3.3 MedVac Mission Performance

The medevac mission is a demanding configuration with 2 350 nmi flights with no refueling available at the destination, two independent 45-minute loiter reserves, and a payload that changes between legs. The outbound leg carries two medical technicians, supplies, and a stretcher (340 kg payload); the inbound adds the patient and escort (522 kg payload), making the return leg heavier and the governing fuel sizing case. Because the RFP specifies no speed or altitude constraint, the aircraft cruises at its aerodynamic best-endurance condition, approximately 52–56 m/s at 2,000 m, burning fuel as slowly as possible per unit time rather than covering distance quickly.

Each leg is iterated independently, converging to its own fuel load and loiter reserve sized to the respective dry weight. Total mission time counts all ground operations and the 45-minute patient loading turnaround, but explicitly excludes the loiter reserve segments per the RFP definition. *Figure 15* makes this structure immediately readable: both legs appear as altitude plateaus at 2,000 m separated by a grey shaded turnaround gap, with outbound in solid lines and inbound in dashed lines. *Figure 16*'s six subplots confirm full throttle only during each climb, a gently varying cruise throttle tracking V_{BE} as weight changes, and power-off descents on both legs with the inbound cruise sitting at slightly higher throttle due to the additional passenger weight.

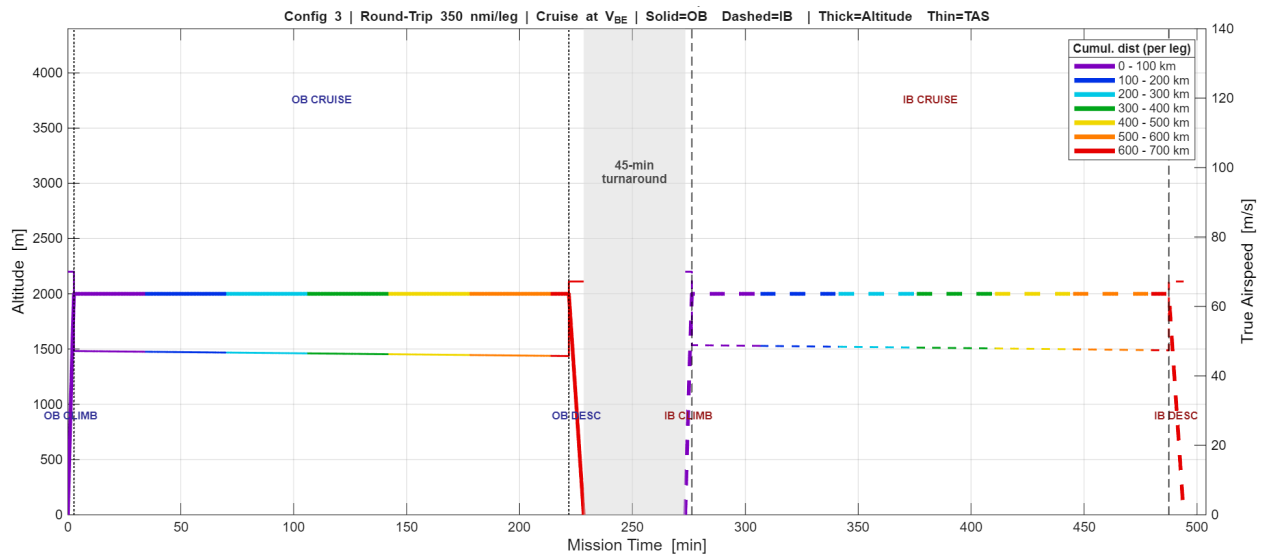
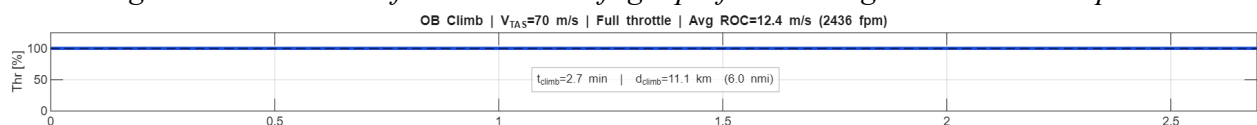


Figure 15: Medevac reference mission flight profile charting altitude and airspeed



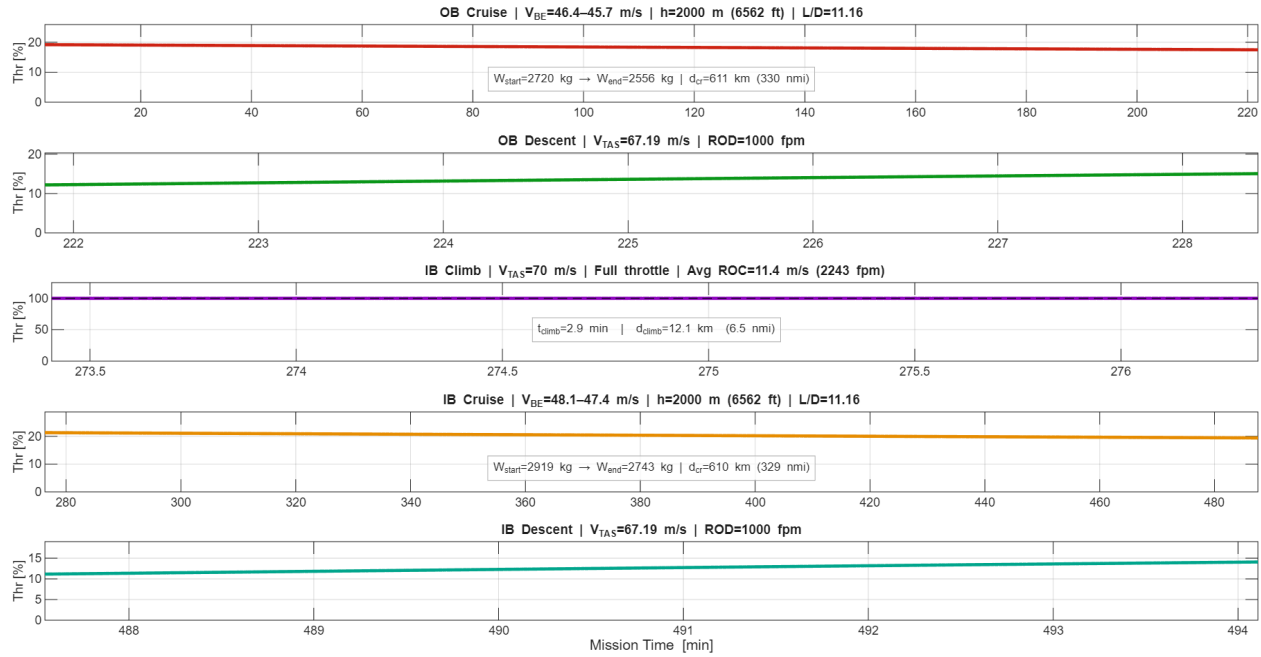


Figure 16: Medevac reference mission flight profile charting throttle level at different flight phases

For more detailed information on the reference mission parameters and resulting values (remaining fuel, loiter ability, etc.) see Appendix A.

Section 4: Configuration

4.1 Stability & Control

4.1.1) Volume Coefficients

The horizontal and vertical stabilizers were initially sized using the tail volume coefficient method, which relates the stabilizer area and moment arm to the wing reference geometry through dimensionless volume coefficients. For the horizontal tail

$$C_{HT} = \frac{S_{HT} L_{HT}}{S_{wing} \bar{c}}, \quad (4.1)$$

and for the vertical tail

$$C_{VT} = \frac{S_{VT} L_{VT}}{S_{wing} \bar{c}}, \quad (4.2)$$

where L_{HT} and L_{VT} are the moment arms measured from the aircraft center of gravity to the aerodynamic centers of the horizontal and vertical stabilizers, respectively, $S_{wing} = 37.3 \text{ m}^2$ is the wing reference area, $\bar{c} = 2.35 \text{ m}$ is the mean aerodynamic chord, and $b = 15.9 \text{ m}$ is the wing span. Volume coefficients of $C_{HT} = 0.70$ and $C_{VT} = 0.04$ were selected, consistent with general aviation aircraft in this class. Rearranging for the required stabilizer areas

$$S_{HT} = \frac{C_{HT} S_{wing} \bar{c}}{L_{HT}}, \quad (4.3)$$

and for the vertical tail

$$S_{VT} = \frac{C_{VT} S_{wing} b}{L_{VT}}. \quad (4.4)$$

The aircraft center of gravity is estimated at 4.0 m from the nose, with the wing aerodynamic center located at 4.3 m and the horizontal and vertical tail aerodynamic centers at 9.5 m and 9.3 m, respectively, all measured from the nose reference point. This gives horizontal and vertical tail moment arms of $L_{HT} = 5.5 \text{ m}$ and $L_{VT} = 5.3 \text{ m}$. The resulting tail areas are $S_{HT} = 11.34 \text{ m}^2$ and $S_{VT} = 4.22 \text{ m}^2$, corresponding to area ratios of $S_{HT} / S_{wing} = 0.308$ and $S_{VT} / S_{wing} = 0.115$.

4.1.2) Tail Planform Geometry

The horizontal stabilizer was sized with an aspect ratio of $AR_{HT} = 4.0$ and a taper ratio of $\lambda_{HT} = 0.45$, yielding a span of $b_{HT} = 6.73 \text{ m}$, a root chord of 2.15 m, a tip chord of 0.97 m, and a MAC of 1.62 m. A quarter-chord sweep of 5° was selected, consistent with the low-speed unswept design philosophy applied to the main wing. The vertical stabilizer uses an aspect ratio of $AR_{VT} = 1.6$ and a taper ratio of $\lambda_{VT} = 0.50$, giving a fin height of $h_{VT} = 2.60 \text{ m}$, a root chord of 2.17 m, a tip chord of 1.09 m, and a MAC of 1.70 m. A quarter-chord sweep of 25° was applied to the vertical tail to provide adequate fin effectiveness and avoid flow separation at high sideslip angles.

4.1.3) Static Margin

A target static margin of 10% MAC was adopted as the initial longitudinal stability criterion, consistent with standard conceptual design practice for general aviation aircraft. This corresponds to a physical static margin distance of 0.24 m, placing the target neutral point at approximately 4.24 m from the nose. The static margin requirement establishes that the neutral point must remain aft of the center of gravity at all loading configurations. This estimate is preliminary; a refined static margin analysis accounting for the wing and tail lift curve slopes, the downwash gradient at the horizontal tail, tail efficiency, and the full CG travel envelope is required at the next design phase.

4.1.4) Control Surfaces

Initial control surface areas were estimated using standard chord ratios. The elevator occupies 30% of the horizontal tail chord, giving an elevator area of $S_{elev} = 3.40 \text{ m}^2$. The rudder occupies 35% of the vertical tail chord, giving a rudder area of $S_{rud} = 1.48 \text{ m}^2$. These ratios are consistent with conventional general aviation practice and provide a starting point for subsequent trim, control authority, and handling-qualities analysis. Full deflection limits, hinge moments, and actuator sizing are deferred to the preliminary design phase.

4.1.5) Sensitivity to Tail Moment Arm

A parametric study was performed, sweeping the tail moment arm from 3.0 m to 8.0 m to assess the sensitivity of the required stabilizer areas to the chosen fuselage length and tail boom geometry. As expected from the volume coefficient formulation, the required stabilizer area decreases inversely with moment arm length, confirming that a longer tail boom directly reduces the structural weight and drag penalty associated with the empennage. The current moment arm of 5.5 m for the horizontal tail represents a compromise between fuselage length, empennage weight, and the tail's contribution to the overall aircraft drag buildup discussed in Section 2.7.

4.1.6) Center of Gravity Envelope

A preliminary center of gravity envelope was developed to verify that the aircraft remains longitudinally stable across the passenger, cargo, and medevac configurations. The CG analysis used the aircraft nose as the longitudinal reference datum, with all component stations measured aft from the nose. The wing aerodynamic center was located at ($x_{AC} = 4.30$) m from the nose, and the mean aerodynamic chord was estimated from the wing geometry as

$$\bar{c} = \frac{S}{b} = 2.42 \text{ m.} \quad (4.5)$$

Assuming the wing aerodynamic center is located at 25% MAC, the leading edge of the mean aerodynamic chord was calculated as

$$x_{LEMAC} = x_{AC} - 0.25\bar{c} = 3.70 \text{ m.} \quad (4.6)$$

The baseline aircraft CG was placed at $x_{CG} = 4.00$ m from the nose, corresponding to approximately 12.6% MAC. The stability analysis estimated the neutral point at ($x_{NP} = 4.24$) m, giving a baseline static margin of approximately 10% MAC. To maintain positive longitudinal static stability for all loading cases, the allowable CG envelope was defined from 5% MAC to 20% MAC. This places the aft CG limit forward of the neutral point, preserving positive static margin even in the most aft loading condition. The wing geometry, baseline CG, aerodynamic center locations, and neutral point estimate were taken from the stability and control sizing model.

The CG location for each configuration was calculated using a standard weight-moment balance,

$$x_{CG} = \frac{\sum W_i x_i}{W_i} \quad (4.7)$$

where (W_i) is the mass of each aircraft component or payload item and (x_i) is its longitudinal station. The empty aircraft, pilots, fuel, passengers, cargo, generator, diesel fuel, stretcher, medical occupants, and medical supplies were each assigned representative station locations based on the preliminary cabin layout. Heavy variable payloads were intentionally located near the wing box to minimize CG travel between mission configurations. In particular, fuel was located near the wing aerodynamic center, passenger rows were distributed around the baseline CG, and cargo/medical equipment was placed close to the center of the cabin.

The resulting CG locations for all loading cases remained inside the 5–20% MAC envelope. The passenger configuration produced a CG near the middle of the allowable range because the nine passengers and baggage are distributed across three cabin rows. The cargo configuration also remained within the envelope because the generator, diesel fuel, fuel container, and equivalent cargo mass were placed close to the wing box. The medevac configuration produced a slightly aft CG due to the stretcher and medical supplies, but it remained forward of the aft limit. Zero-fuel cases were also checked to ensure that fuel burn does not shift the CG outside the allowable range during flight. Since the fuel CG is located close to the wing box, fuel burn produced only a small CG shift.

Overall, the CG envelope analysis indicates that the aircraft remains within the preliminary longitudinal stability limits for passenger, cargo, and medevac operations. This result supports the reconfigurable cabin layout, since all mission payloads can be carried without requiring ballast or major payload restrictions. A refined CG envelope should be completed in later design phases using detailed component weights, final seat-track locations, exact fuel tank geometry, and a more complete neutral-point calculation.

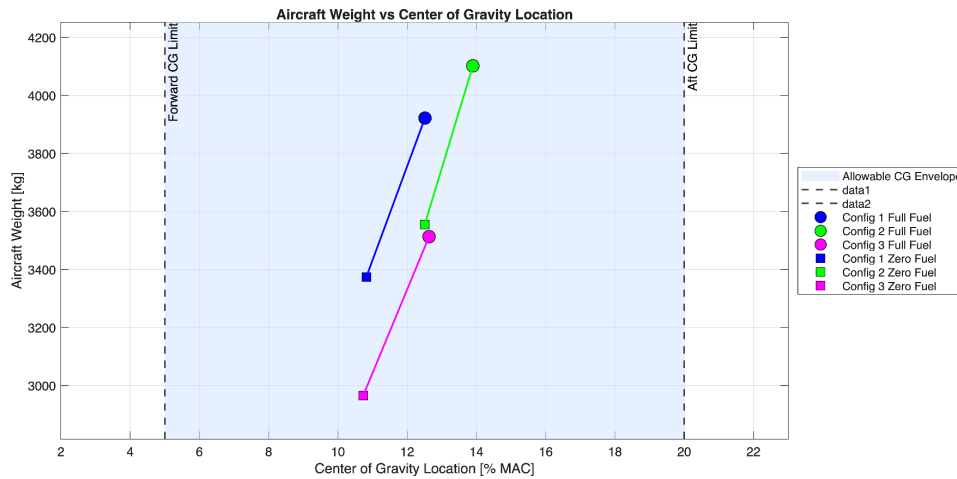


Figure 17: Aircraft weight versus center-of-gravity location for the passenger, cargo, and medevac configurations

Full-fuel and zero-fuel cases are shown for each configuration to evaluate the CG shift during fuel burn. All loading cases remain within the preliminary allowable CG envelope of 5–20% MAC, indicating that the aircraft maintains acceptable longitudinal balance across the mission configurations.

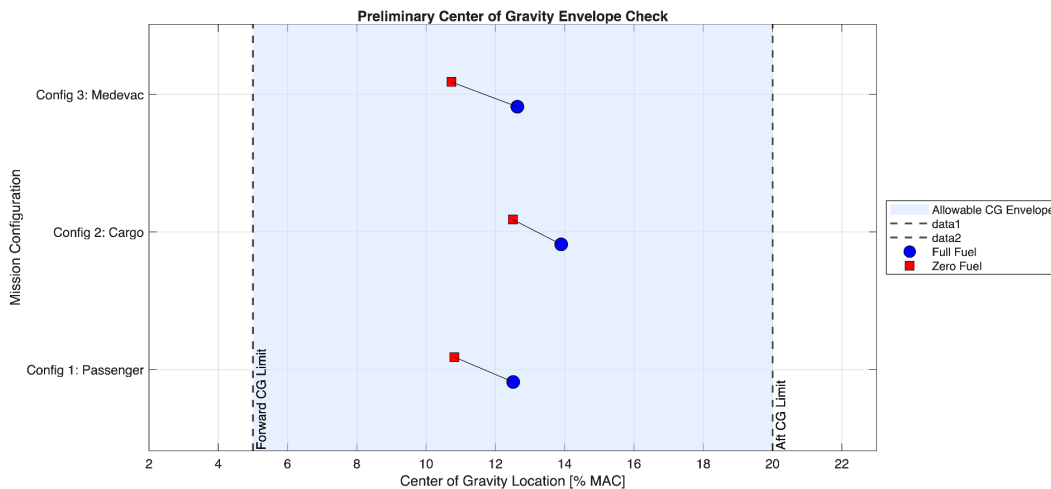


Figure 18: Preliminary center-of-gravity envelope check by mission configuration

The full-fuel and zero-fuel CG locations are plotted for each configuration to show the direction and magnitude of CG travel as fuel is consumed. The passenger, cargo, and medevac configurations remain inside the forward and aft CG limits, confirming that no ballast or payload relocation is required for the preliminary cabin layout.

4.2 Fuselage

The fuselage is sized to house nine passengers in a 1-2 seat arrangement, two pilots, and a reconfigurable aft cargo/medevac volume. The 1-2 seating layout — one seat (0.45 m) port, a 0.40 m aisle, and two seats (0.45 m each) starboard — sets the interior cabin width at 1.75 m, giving a minimum outer diameter of 1.95 m after the 10 cm structural margin. Four seat rows at 0.82 m (32-inch) pitch satisfy the RFP's 31-inch minimum, yielding a 3.28 m cabin length, with the fourth row partially occupied by a lavatory and flight attendant jump seat per Part 23 requirements. Adding the flight deck, aft baggage bay, and a tail boom of sufficient length for the required horizontal tail moment arm, a total fuselage length of approximately 9.9 m is adopted, consistent with comparable STOL aircraft in this class (Cessna 208B: 12.7 m, Kodiak 100: 10.2 m). Fuel is stored entirely within the wings across all three configurations; a center-section belly tank between the main spar frames is available as a contingency should the wing volume prove insufficient at 70% internal utilization.

4.3 Airframe Materials

Material selection for this aircraft is governed by three competing priorities: structural efficiency (high specific strength), compatibility with Arctic operating temperatures as low as -30°F (-34°C), and maintainability at remote airstrips with limited tooling. A hybrid metallic-composite approach is adopted, consistent with current practice on aircraft in this class, such as the Kodiak 100 and the modernized DHC-6 Twin Otter Series 400.

The primary wing structure, including spars, ribs, and skin panels, uses aluminum alloy 7075-T6, which offers a yield strength of approximately 503 MPa and retains adequate toughness at low temperatures. The choice of aluminum over carbon fiber composite for the primary wing structure reflects the importance of field repairability: composite damage requires specialized inspection equipment and bonded repair techniques that are impractical in Arctic field conditions. Aluminum damage, by contrast, can be assessed visually and repaired with conventional sheet-metal tools. The wing leading edge and flap tracks are fabricated from 2024-T3 aluminum to resist gravel impact damage during low-altitude STOL operations on unprepared strips.

The fuselage primary structure (frames, longerons, and skin panels) likewise employs 2024-T3 aluminum, which exhibits better fracture toughness than 7075 and is preferred for fuselage applications where damage tolerance is the critical design driver. Secondary structure, fairings, and non-load-bearing panels are fabricated from carbon fiber-reinforced polymer (CFRP) using a pre-impregnated tape layup, resulting in approximately 20–25% weight savings

compared to equivalent aluminum parts in these locations. CFRP is acceptable in secondary applications because impact loads are low and trained technicians with portable curing equipment can perform repairs.

The landing gear legs and gravel-protection shields use 4340 steel for its high yield strength and impact resistance under the severe loading conditions of unprepared-surface operations. The engine mount and firewall are also fabricated from steel to meet fire-resistance certification requirements under 14 CFR Part 23. De-icing boots on the wing and tail leading edges use pneumatic rubber elements, selected for their proven reliability and low maintenance burden in icing operations certified to FIKI standards.

Icing was treated as an off-nominal performance case by assuming increased parasite drag and reduced maximum lift coefficient. Because the baseline aircraft maintains margin in takeoff/climb and uses a FIKI-capable pneumatic/electric ice protection system, the design is expected to preserve safe climb and approach capability under known-icing operation.

4.4 Avionics and Instrumentation

The avionics suite is built around the Garmin G1000 NXi integrated flight deck, which consolidates attitude and heading reference, air data, navigation, and engine indication into a single unit certified for Part 23 single-pilot operations [Garmin]. All aircraft equipped with the G1000 integrated avionics suite are required to carry standby attitude, altimeter, and airspeed indicators as backup instruments, which become primary in the event of a system failure [FAA]. Navigation is provided by dual WAAS-certified GPS receivers with LPV approach capability — enabling precision approaches at Arctic airstrips that lack ILS infrastructure — backed by an integrated VOR/ILS receiver and an ADS-B Out transponder meeting the performance requirements of 14 CFR §§ 91.225 and 91.227, which mandate ADS-B Out equipment capable of broadcasting the aircraft's position and velocity for surveillance in designated airspace.[FAA] Additional systems address Arctic-specific requirements: an EGPWS with Arctic terrain database accounts for mountainous terrain and limited radar coverage; a digital GFC 700 autopilot with altitude hold, VNAV, and approach coupling reduces crew workload on extended over-ice segments; and a portable medical monitoring interface with dedicated power outlets is installed in the aft cabin for the medevac configuration. All avionics are selected for a demonstrated operating temperature range encompassing the -30°F cold-soak requirement of the RFP.

4.5 Cabin configuration

The cabin is designed to accommodate three interchangeable configurations as specified in the RFP: passenger transport (Configuration 1), cargo transport (Configuration 2), and aerial ambulance (Configuration 3). Reconfigurability is achieved through quick-release seat tracks and modular floor tie-down fittings, enabling ground crews to convert between configurations

without specialized tools. The seat tracks conform to FAA TSO-C127a, and cargo tie-down points are rated to 1,500 lb (680 kg) per fitting.

Configuration 1 — Passenger Transport:

Nine passengers are seated in a 1-2 arrangement across three rows at a 32 in (812.8 mm) pitch, exceeding the 31 in minimum requirement. All seats face forward, with a flight attendant jump seat forward of Row 1 facing the passengers. Total passenger payload is 2,250 lb at 250 lb per occupant. The aft cabin is reserved as an unencumbered baggage area.

Configuration 2 — Cargo Transport

All passenger seats are removed, opening the full cabin length to freight. An 8,000-watt generator and a 55-gallon diesel fuel container are secured forward with cargo straps and floor tie-downs. An additional cargo zone in the aft cabin accommodates supplementary freight, with a total payload target of 2,250 lb plus seat removal weight. Fuel is carried in a single container where possible, with a maximum of three permitted.

Configuration 3 — Aerial Ambulance

A single litter runs longitudinally along the port side for a 200 lb patient, flanked on the starboard side by a three-occupant patient escort seat. Two medical technician seats are positioned aft, and 300 lb of medical supplies and equipment are distributed between a forward station and aft stowage. Total payload is 1,150 lb across five occupants, excluding the patient.

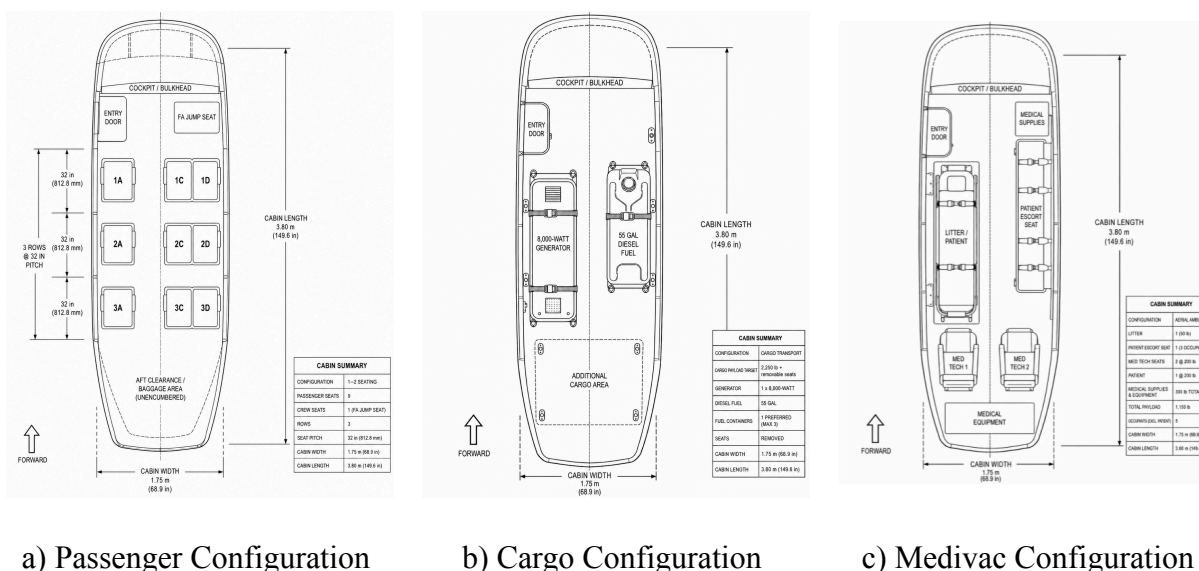


Figure 19: Cabin layout for the aircraft in three different required configurations

All configurations share the same floor structure, anchor rail system, and entry door geometry. The longitudinal tie-down rails used for cargo restraint in Configuration 2 serve as litter

mounting tracks in Configuration 3, minimizing role-change time and eliminating the need for configuration-specific ground support equipment.

4.6 Summary of Aircraft Values

Table 8: Aircraft value summary

Parameter	Value	Notes
MTOW	4,150 kg (9,149 lb)	Config 2 sizing case; 40,706 N
Operating Empty Weight (OEW)	2,172 kg (4,788 lb)	52.3% of MTOW
Structural Weight	1,991 kg (4,390 lb)	Airframe + systems
Max Fuel Weight	548 kg (1,208 lb)	13.2% of MTOW; Jet-A / JP-8
Max Payload	1,429 kg (3,150 lb)	Config 2 governs
Crew Weight	181 kg (399 lb)	2 pilots
Wing Geometry		
Wing Area (S)	37.3 m ² (401.5 ft ²)	Reference / planform area
Wing Span (b)	15.9 m (52.2 ft)	
Aspect Ratio (AR)	6.76	Single-engine design point
Mean Aerodynamic Chord (MAC)	2.35 m (7.71 ft)	Used for stability sizing
Wing Loading (W/S)	925 N/m ² (19.3 lb/ft ²)	At design point
Airfoil	NASA LS(1)-0417 (GA(W)-1)	t/c = 17%
High-Lift System		
CL,max System	3.5	Exceeds 3.4 requirement by 2.9%
CL,max Takeoff (usable, 80%)	2.54	Single-slotted flap, delta = 30 deg
CL,max Landing (usable, 80%)	2.72	Double-slotted Fowler + leading-edge slat
Flap Deflection — Takeoff	30 deg	Single-slotted configuration
Flap Deflection — Landing	50 deg	Double-slotted Fowler

Slat Angle — Landing	30–40 deg	Leading-edge slats
Propulsion		
Engine	PT6A-60A	Pratt & Whitney Canada
Rated Shaft Power	1,050 shp (783 kW)	
Propeller Efficiency (η_p)	0.80	Cruise condition
Specific Fuel Consumption (SFC _p)	0.548 lb/hp-hr	3.34×10^{-7} kg/W-s
Engine Mass	475 kg (1,047 lb)	Included in OEW
Thrust-to-Weight Ratio (T/W)	~0.63	At MTOW, sea level
Aerodynamics		
CD ₀ — Clean (cruise)	0.029	Baseline parasite drag coefficient
CD ₀ — Approach (flaps 50 deg)	0.119	Flap increment added
CD ₀ — Ground Roll (flaps + spoilers)	0.230	Full flap + spoiler deployment
Cruise L/D	12.5	At 130 KTAS, 15,000 ft
Oswald Efficiency Factor (e)	0.90	Clean cruise
Performance		
Takeoff Field Length	≤ 500 ft (152.4 m)	Over 50 ft obstacle; gravel runway
Landing Field Length	≤ 500 ft (152.4 m)	Over 50 ft obstacle
Design Range	450 nmi	+ 45 min loiter reserve
Cruise Altitude	15,000 ft (4,572 m)	ISA density ratio $\sigma = 0.630$
Cruise Speed	130 KTAS (67 m/s)	Minimum RFP requirement
Rate of Climb	≥ 800 ft/min	Avg ~1,437 fpm achieved at MTOW
Service Ceiling	≥ 18,000 ft (5,486 m)	29,801 ft (9083 m)
Never-Exceed Speed (VNE)	≥ 180 KTAS	RFP requirement
Min. Operating Temperature	-30 deg F (-34 deg C)	Cold-soak start capable
Empennage		
Horizontal Tail Area (SHT)	11.34 m ² (122.1 ft ²)	CHT = 0.70; AR = 4.0

Horizontal Tail Span	6.73 m (22.1 ft)	Taper ratio $\lambda = 0.45$
Vertical Tail Area (SVT)	4.22 m ² (45.4 ft ²)	CVT = 0.04; AR = 1.6
Horizontal Tail Moment Arm	5.5 m (18.0 ft)	CG to H-tail aerodynamic center
Elevator Area	3.40 m ² (36.6 ft ²)	30% of H-tail chord
Rudder Area	1.48 m ² (15.9 ft ²)	35% of V-tail chord
Stability & CG		
Static Margin (target)	10% MAC (0.24 m)	Preliminary estimate
CG Location — Baseline	4.00 m from nose	12.6% MAC
Neutral Point (estimated)	4.24 m from nose	Preliminary estimate
Allowable CG Envelope	5%–20% MAC	All 3 configs within limits
Limit Load Factor	+3.8g / -1.5g	Part 23 normal category
Aircraft Geometry		
Fuselage Length	10.11 m (33.2 ft)	Per 3-view drawing
Overall Height	3.56 m (11.7 ft)	Per 3-view drawing
Overall Span	15.88 m (52.1 ft)	Per 3-view drawing
Cabin Interior Width	1.75 m (5.7 ft)	1-2 seat arrangement
Cabin Standing Height	2.0 m (6.6 ft)	Medevac ops clearance
Seat Pitch	32 in (812 mm)	Exceeds 31 in RFP minimum

4.7 Final Design in CAD

A three-dimensional solid model of the aircraft configuration was developed in CAD to verify geometric consistency between the fuselage, wing, control surfaces, and propulsion system. The model, shown in *Figure 20*, depicts a high-wing monoplane configuration with a single turboprop engine mounted on the nose and a conventional cruciform tail. The high-wing layout is selected to maximize ground clearance for propeller rotation on unprepared surfaces and to provide passengers with an unobstructed view while loading, consistent with the boarding and egress experience objectives stated in the RFP.

The CAD model confirms that the wing root chord and spar box can accommodate the required fuel volume. With a wing area of approximately 37.3 m², an aspect ratio of $AR = 6.76$, and a wing span of approximately 15.9 m, the inboard 60% of the semi-span is available for fuel storage after accounting for flap actuator fairings and the undercarriage bay on the high-wing strut configuration. At 70% volumetric fill efficiency as specified in the design brief, the wing

can accommodate the full mission fuel load of approximately 535 kg without requiring fuselage auxiliary tanks.

The model also verifies tail sizing: the horizontal tail has a mean aerodynamic chord equal to half the wing MAC and a span of 40% of the wing span (approximately 6.6 m), and the vertical tail is sized as one-half of the horizontal tail planform, per the design brief specifications.^[15] The resulting tail moment arm from the wing quarter-chord to the horizontal tail quarter-chord is approximately 5.5 m, providing adequate pitch stability margin for the range of payload configurations. Three-view drawings derived from the CAD model are included in the appendix. Below is a simplified solid model of the aircraft intended for simplified drag calculations.

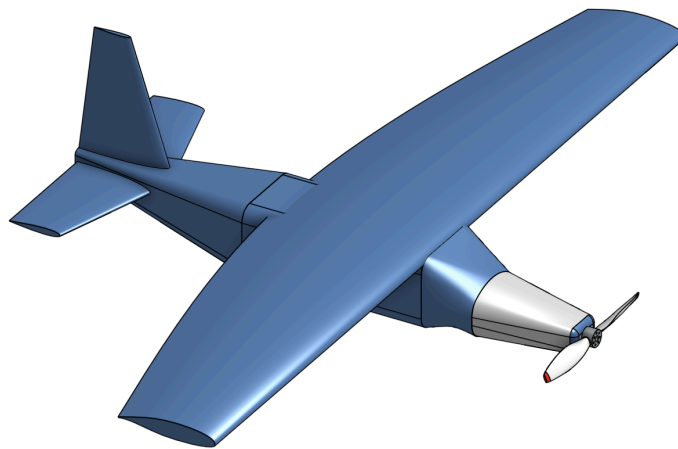


Figure 20: System Assembly of Aircraft

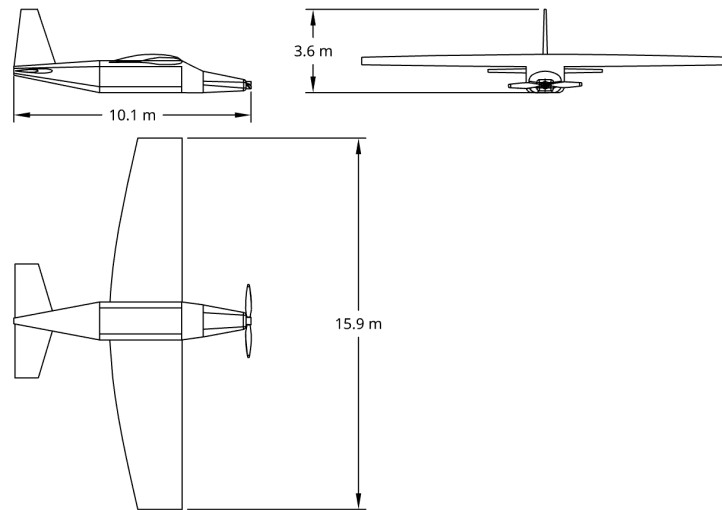


Figure 21: 3 view of aircraft with dimensions

Section 5: Operations and Business Analysis

5.1 Supporting Ground Infrastructure

A central design objective is to minimize dependence on specialized ground infrastructure, consistent with the aircraft's operational role serving remote Arctic communities with minimal support facilities. The propulsion system, a single PT6A-class turboprop engine, is selected in part for its established maintenance network across northern regions where the aircraft is likely to encounter its most extreme operational requirements. This includes Northern Canada, Alaska, Scandinavia, the United States, and, with unlikely future relevance, Russia. The PT6A family spans 500 to 1,940 shaft horsepower and is installed on more than 100 different aircraft types worldwide, providing unparalleled parts availability and certified repair station coverage in the operational area.[14] Additionally, the wide variety of variants and shaft horsepower options available means there is a practical case that different variants of aircraft could be manufactured in the future, including variants with more conventional operational requirements.

The undercarriage is designed for operations on unprepared gravel and compacted snow surfaces. Oversized tundra tires with a rated footprint pressure below the 40 psi (275 kPa) threshold for gravel-surface operations are fitted as standard. The nose gear incorporates a gravel deflector shield to prevent foreign objects striking the aircraft, and all gear doors are sealed against gravel and slush ingress. Wheel skis or amphibious floats could potentially be offered as operator-selectable options to expand operational capability to frozen lake and river surfaces, consistent with the desirable unprepared-surface capability noted in the RFP.[15]

Ground handling is simplified by a high-wing configuration that keeps the undercarriage track wide enough to provide stability on uneven surfaces. The aircraft is towable with a standard tow bar sized for ground vehicles common to the target operational environment. Engine start capability extends to cold-soak temperatures of -34°C using an engine block heater system drawing from a standard 28 VDC GPU or potentially a self-contained APU. The fuel system accepts Jet-A, Jet-A1, and JP-8 fuels interchangeably, all of which are typically available at Arctic airstrips that serve heavier turboprop traffic.

5.2 Airplane Cost and Business Analysis

Cost estimation at the conceptual design phase employs the DAPCA IV (Development and Procurement Costs of Aircraft) parametric model, originally developed by the RAND Corporation [RAND]. The model estimates airframe manufacturing labor and material costs as functions of aircraft empty weight and maximum speed and is widely used in the aircraft conceptual design community as described in Raymer. At the estimated operating empty weight of approximately 2,370 kg and a never-exceed speed of 180 KTAS, the DAPCA IV model yields a recurring flyaway cost per unit of approximately USD 3.2 million at a production run of 500 aircraft and approximately USD 2.8 million at 1,000 aircraft. These estimates include airframe, engine (PT6A-class engines are typically priced at USD 600,000–800,000 per unit), avionics, and final assembly. Non-recurring development costs are estimated at USD 120–150 million spread across the design, certification, and entry-into-service program through 2033, consistent with comparable recent Part 23 certification programs.

The reference mission direct operating cost (DOC) is estimated using the ATA-67 method [12,13], which accounts for fuel, oil, crew, maintenance, and depreciation on a per-flight-hour and per-nautical-mile basis. At a fuel burn estimated from the mission fuel fraction ($\zeta = 0.130$ of MTOW = 4,064 kg, fuel weight ≈ 528 kg of Jet-A at approximately 0.80 kg/liter ≈ 660 liters per reference mission), and a Jet-A price of USD 3.00 per liter in the Arctic supply chain, the fuel cost per reference mission (450 nmi) is approximately USD 1,980. Adding crew costs (approximately USD 800 for two pilots at the applicable duty rate), maintenance reserve (approximately USD 600 per flight hour at two hours block time), and depreciation and insurance (approximately USD 1,200 per flight), the estimated DOC per reference mission is approximately USD 4,580. At a one-way ticket price of USD 650 per passenger and nine passengers, the gross revenue per flight is USD 5,850, yielding an estimated operating margin of approximately 22% before airport fees and overhead.^[11]

The business case is compared against the incumbent Cessna 208B Grand Caravan, which is the dominant aircraft type in this operational segment. The Caravan carries the same nine passengers but with a shorter range and no factory FIKI certification. The proposed design's principal competitive advantages are its certified STOL performance at a 500 ft field length, its

native FIKI certification reducing the operator's certification burden, and its reconfigurable interior enabling medevac operations without a costly supplemental type certificate.

Further, there is a market demand for rugged, STOL aircraft with flexible mission profiles. The Bombardier DHC-6, which is considered one of the more successful aircraft in this market segment, has been out of production since 1988 [16]. Most commercial aircraft operate for a nominal time period of 20 years, with ideally a few more years of service. It is very typical for aircraft with fewer cycles to fly for considerably longer periods, but the youngest DHC-6 is, as of this year, roughly 38 years old. Regardless of accumulated airframe cycles as out-of-production aircraft age, there are natural concerns such as part availability, efficiency, and capability compared to new aircraft with more modern avionics, safety features, and ease of maintenance. Assuming these are growing markets in rural areas, it is reasonable to expect aircraft in this category to be in demand in the range of a few thousand units, as the DHC-6, which captured much of the market share but not all, sold 994 units [16]. As this is a smaller aircraft with approximately half the passenger capacity, it is reasonably fair to estimate that this aircraft could command a market of about 500 airframes over the next 20 years at an approximate list price of 3.2 million dollars per unit. This aircraft would face headwinds in the market as the Cessna 208 Grand Caravan typically sells for less and has similar payload capacity [18]. It would be challenging, as there are only a few environments that require such an extreme STOL performance. At a higher list price and with a general lack of practical advantages, it is hard to justify launching this aircraft commercially without a dedicated customer willing to pay a significant premium for an aircraft built for an extremely niche market. Unless you are transporting valuable cargo from remote, rough, and off-grid locations, this likely is not the most cost-effective aircraft on the market. There is a chance some high-value pharmaceutical cargo meets this criteria, but speculating to such a degree is hard to formally include in a business analysis.

Additionally, in the long term, STOL missions with these extreme limitations are being addressed by EVTOL aircraft. Companies such as Archer Aviation are actively working on developing market-disrupting aircraft [17]. It is very likely that extremely short take-off and landing missions in the urban landscape will eventually be better served by these aircraft. Regardless of whether these aircraft are in production or not, market expectations seem to back this theory, as these aircraft have amassed plenty of pre-orders, which would likely remove much of the demand for the Snowrunner 670 in the urban market.

In conclusion, the economic and market analysis suggests that while the proposed aircraft is technically viable and offers clear operational advantages—particularly in STOL performance, FIKI certification, and mission flexibility—its commercial outlook is constrained by a narrow target market and strong competitive pressures. The cost estimates indicate that the aircraft can achieve a modest operating margin under favorable assumptions; however, its higher acquisition cost relative to established platforms such as the Cessna 208B Grand Caravan limits its appeal to cost-sensitive operators. Although there is a demonstrated need for rugged, utility-focused

aircraft and a potential replacement cycle for aging platforms like the DHC-6, the total addressable market remains limited and highly specialized. Emerging technologies such as eVTOL aircraft introduce additional long-term uncertainty, particularly for short-range and urban operations where they may offer superior flexibility and infrastructure advantages. As a result, the success of this aircraft program would likely depend on securing committed customers in niche environments where its unique capabilities provide decisive value. Without such demand, the financial risk associated with development and certification may outweigh the projected returns, making a full commercial launch difficult to justify under current market conditions.

Works Cited

[1]	Nicolai, L. M. and Carichner, G. E., Fundamentals of Aircraft and Airship Design, Volume 1 – Aircraft Design, AIAA Education Series, 2010. eISBN: 978-1-60086-753-8.
[2]	Raymer, D. P., Aircraft Design: A Conceptual Approach, 6th ed., AIAA Education Series, Reston, VA, 2018. ISBN: 978-1-62410-490-9.
[3]	Wang, Z. et al., "Deformation Behavior and Microstructural Evolution of 7075-T6 Aluminum Alloy at Cryogenic Temperatures," Cryogenics, Vol. 77, 2016, pp. 77–85. https://doi.org/10.1016/j.cryogenics.2016.06.004 .
[4]	Wanhill, R. J. H., "Milestone Case Histories in Aircraft Structural Integrity," Comprehensive Structural Integrity, Elsevier, 2003. See also: ScienceDirect Topics, "2024 Aluminium Alloy," Comprehensive Structural Integrity.
[5]	ScienceDirect Topics, "2024 Aluminium Alloy – Aircraft Construction," Comprehensive Structural Integrity, Elsevier. Available: https://www.sciencedirect.com/topics/materials-science/2024-aluminium-alloy .
[6]	Wikipedia, "Ice Protection System," Wikimedia Foundation. Available: https://en.wikipedia.org/wiki/Ice_protection_system . Accessed: May 2026.
[7]	Garmin International, "G1000 NXi Integrated Flight Deck – Product Announcement," Garmin Blog, January 2017. Available: https://www.garmin.com/en-US/blog/aviation/announcement-garmin-introduces-g1000-nxi-next-generation-integrated-flight-deck/ .
[8]	Federal Aviation Administration, "DPE and CFI Avionics Guide: G1000 NXi," FAA, 2022. Available: https://www.faa.gov/sites/faa.gov/files/2022-09/DPE%20and%20CFI%20Guide%20G1000NXi.pdf .
[9]	Boren, H. E., DAPCA: A Computer Program for Determining Aircraft Development and Production Costs, RAND Corporation, Research Memorandum RM-5221-PR, Santa Monica, CA, 1967. Available: https://www.rand.org/pubs/research_memoranda/RM5221.html .
[10]	Large, J. P. et al., A Computer Model for Estimating Development and Procurement Costs of Aircraft (DAPCA-III), RAND Corporation, Report R-1854-PA&E, Santa Monica, CA, 1976. Available: https://www.rand.org/pubs/reports/R1854.html .

[11]	ScienceDirect Topics, "Direct Operating Cost – Aircraft," in Air Transportation Systems Analysis, Elsevier. Available: https://www.sciencedirect.com/topics/engineering/direct-operating-cost . Accessed: May 2026.
[12]	Faulkner, H. B., "The ATA-67 Formula for Direct Operating Cost," Proceedings of the NASA/MIT Workshop on Airline Systems Analysis, Vol. 1, NASA Technical Reports Server, 1973. Available: https://ntrs.nasa.gov/citations/19730024119 .
[13]	Air Transport Association of America, Standard Method of Estimating Comparative Direct Operating Costs of Turbine Powered Transport Airplanes, Washington D.C.: ATA, 1967. Reference implementation: Stanford Aerodesign, http://aerodesign.stanford.edu/aircraftdesign/cost/atamethod.html .
[14]	Wikipedia, "Pratt & Whitney Canada PT6," Wikimedia Foundation. Available: https://en.wikipedia.org/wiki/Pratt_%26_Whitney_Canada_PT6 . Accessed: May 2026.
[15]	AIAA, "Graduate Team Aircraft Design Competition Request for Proposal: Arctic Short Take-Off and Landing (STOL) Aircraft," American Institute of Aeronautics and Astronautics, 2025–2026. (Reproduced as AME 261 Design Project 2026 brief.)
[16]	"Twin Otter Archive." Twin Otter Archive, twinotterarchive.com/DHC-6_Index_new.html . Accessed 8 May 2026.
[17]	Archer Aviation Inc. Archer. Archer Aviation Inc., 2026, archer.com . Accessed 8 May 2026.
[18]	AviaCost. AviaCost: Aircraft Ownership & Operating Cost Calculator, AviaCost, 2026, aviacost.com . Accessed 8 May 2026.
[19]	https://ntrs.nasa.gov/api/citations/19750064451/downloads/19750064451.pdf

Appendix A



Table of Contents

CD0 Buildup	2
Reference Mission Evaluation Tables	3
Reference Mission Evaluation Process	12

Section 1: CD0 Buildup Table

Component	$S_{\text{wet}} \text{ (m}^2\text{)}$	C_f	FF	Q	$C_{D0,i}$	% of Total
Fuselage	~59.8	0.00270	~1.49	1.00	~0.0130	~43%
Wing	~89.1	0.00310	~1.30	1.00	~0.0098	~33%
Horizontal Tail	~27.0	0.00330	~1.25	1.05	~0.0031	~10%
Vertical Tail	~10.1	0.00340	~1.25	1.03	~0.0012	~4%
Landing Gear	—	—	—	—	~0.0018	~6%
Leakage & Protuberance	—	—	—	—	~0.0013	~4%
Total (clean)					~0.029	100%

Component parasite drag buildup results (Raymer §12.5)

Snow Runner 670— Mission Performance Summary Tables

Reference Mission (Config 1, 90% Payload, 450 nmi) · Medevac Mission (Config 3, Round-Trip, 350 nmi/leg)

Reference Mission — Config 1 | 90% Payload | 450 nmi

Table 1 — Weight Statement

All weights at converged iteration (3 steps, residual < 0.01 kg).

Parameter	Value	Notes
Operating Empty Weight (OEW)	2,150.00 kg	Structural + 2 pilots
of which: crew (2 pilots)	181.44 kg	2 × 200 lb
Config 1 full payload	1,220.37 kg	9 pax + baggage + FA + seats
Reference mission payload (90%)	1,098.33 kg	0.90 × full payload
Mission fuel (total)	446.20 kg	Trip fuel + loiter reserve
of which: loiter reserve	29.69 kg	45-min Breguet endurance
of which: trip fuel	416.51 kg	446.20 – 29.69
Fuel fraction $\zeta = W_{\text{fuel}} / W_0$	0.1208	Lower than Config 2 (lighter payload)
MTOW (W0)	3,694.54 kg	36,243 N
Config 2 design MTOW (ref.)	4,077.00 kg	39,995 N — aircraft structural limit
Margin to design MTOW	382.46 kg	3,694.54 kg vs 4,077.00 kg

Table 2 — Segment Weight Chain

Weight at each phase boundary, showing fuel burned per segment.

Weight	Phase Boundary	Value (kg)	ff Applied	Fuel Burned (kg)
W0	Takeoff (MTOW)	3,694.54	—	—

W1	After warm-up & taxi	3,657.59	0.9900	36.95
W2	After takeoff	3,639.30	0.9950	18.29
W3	After climb	3,566.52	0.9800	72.78
W4	After cruise	3,337.84	0.9359	228.68
W5	After descent	3,304.46	0.9900	33.38
W6	After landing rollout	3,278.03	0.9920	26.43
W7	After 45-min loiter (zero fuel)	3,248.33	0.9909	29.70

Table 3 — Distances, Speeds & Timing

Parameter	Value	Notes
DISTANCES		
Climb distance	43.59 km (23.54 nmi)	SL → 15,000 ft
Cruise distance	729.51 km (393.90 nmi)	Breguet range segment
Descent distance	60.30 km (32.56 nmi)	1,000 fpm / 67 m/s
Total ground track	833.40 km (450.00 nmi)	= RFP requirement
TIMING (airborne only — excl. ground ops)		
Climb time	10.44 min	Full throttle, V_TAS = 70 m/s
Cruise time	181.47 min	729.51 km ÷ 67.0 m/s
Descent time	15.00 min	Fixed geometry, 1,000 fpm
Total air time	206.91 min (3.45 hr)	Excl. ground ops & loiter
SPEEDS & ALTITUDE		
Cruise TAS	67.00 m/s ($\approx$ 130.2 KTAS)	Fixed
Cruise altitude	4,572 m (15,000 ft)	ISA standard
Climb TAS (fixed)	70.00 m/s	Team decision
Descent glide TAS	67.19 m/s	$\sqrt{67^2 + 5.08^2}$
Descent angle	4.34°	$\text{atan}(5.08 / 67.0)$

Table 4 — Climb & Cruise Performance

Parameter	Value	RFP / Notes
CLIMB		
Average rate of climb	7.30 m/s (1,437 fpm)	PASS — RFP ≥ 800 fpm
Min V_horiz during climb	69.27 m/s	PASS — ≥ 67 m/s required
W2 at base of climb	3,639.30 kg	Used in kinematic sim
CRUISE		
L/D at cruise	12.50	From CruiseOptimization_v3.m
W3 — start of cruise	3,566.52 kg	After climb
W4 — end of cruise	3,337.84 kg	After Breguet burn
Cruise fuel burned	228.68 kg	W3 – W4
ff_cruise	0.935882	Breguet fraction
LOITER RESERVE		
CL_BE (best endurance)	1.2851	$\sqrt[3]{3 \cdot CD_0/k}$
CD_BE	0.1152	$4 \cdot CD_0$
L/D at best endurance	11.155	CL_BE / CD_BE
Loiter reserve fuel	29.69 kg	45 min Breguet endurance
ff_loiter	0.990942	W_dry / W6_target

Table 5 — Fuel Fractions

Segment	Fraction (ff)	Method	Fuel Burned (kg)
Warm-up & taxi	0.990000	Raymer historical	36.95
Takeoff	0.995000	Raymer historical	18.29
Climb	0.980000	Raymer historical	72.78
Cruise	0.935882	Breguet range eq.	228.68
Descent	0.990000	Raymer historical	33.38
Landing	0.992000	Raymer historical	26.43
Loiter reserve	0.990942	Breguet endurance	29.69

TOTAL FUEL	—	—	446.20 kg
------------	---	---	-----------

Table 6 — RFP Compliance Check

RFP Requirement	Result	Status
Total range $\geq$ 450 nmi	450.00 nmi	✓ PASS
Cruise speed $\geq$ 130 KTAS	67.0 m/s ($\approx$ 130.2 KTAS)	✓ PASS
Cruise altitude = 15,000 ft	4,572 m (15,000 ft)	✓ PASS
Rate of climb $\geq$ 800 fpm at MTOW	1,437 fpm (7.30 m/s)	✓ PASS
V_horiz $\geq$ 67 m/s during climb	Min = 69.27 m/s	✓ PASS
45-min loiter reserve	29.69 kg carried	✓ PASS
Payload = Config 1 at 90%	1,098.33 kg	✓ PASS
MTOW $\leq$ Config 2 design limit	3,694.54 < 4,077.00 kg	✓ PASS

Medevac Mission — Config 3 | Round-Trip | 350 nmi per leg

Two independent fuel iterations — one per leg. Outbound carries no patient; inbound carries patient + escort. Both legs carry separate 45-min loiter reserves sized to their respective dry weights.

Table 7 — Aircraft & Aerodynamic Constants

Same airframe as reference mission. Best-endurance cruise used in place of speed-constrained cruise.

Parameter	Value	Notes
Wing area S	37.1 m²	Same as reference mission
Aspect ratio AR	6.76	Constraint analysis optimum
CD0	0.0288	Clean config, all phases
Oswald efficiency e	0.90	Clean wing
Induced drag factor k	0.05261	$1/(\pi \cdot AR \cdot e)$
CL_BE (best endurance)	1.2851	$\sqrt{3 \cdot CD0/k}$ — cruise & loiter
CD_BE	0.1152	$4 \cdot CD0$
L/D at best endurance	11.155	Cruise L/D for this mission
Cruise altitude	2,000 m (6,562 ft)	V_BE above stall for expected weight range
PT6A-60A rated power	1,050 shp	783 kW shaft power
Propeller efficiency η_p	0.80	Fixed assumption
SFC_p	0.548 lb/(hp·hr)	PT6A-60A published value

Table 8 — Payload Comparison: Outbound vs. Inbound

The 181.44 kg difference in payload between legs is the governing reason the two legs are iterated independently.

Payload Item	Weight (kg)	Outbound	Inbound
2 Medical technicians	181.44	YES	YES
Medical supplies	136.10	YES	YES
Stretcher (litter)	22.70	YES	YES

Patient	90.72	NO	YES
Patient escort	90.72	NO	YES
TOTAL PAYLOAD	—	340.24 kg	521.68 kg
Dry weight (OEW + payload)	—	2,490.24 kg	2,671.68 kg

Table 9 — Weight Statements: Outbound & Inbound

Each leg converged independently. Inbound is the governing sizing case due to heavier payload and larger loiter reserve.

Parameter	Outbound (OB)	Inbound (IB)
OEW	2,150.00 kg	2,150.00 kg
Payload	340.24 kg	521.68 kg
Dry weight W _{dry}	2,490.24 kg	2,671.68 kg
Loiter reserve fuel	Computed (OB)	Computed (IB)
ff_loiter	Computed (OB)	Computed (IB)
Trip fuel	Computed (OB)	Computed (IB)
Total fuel (trip + loiter)	Converged W _{fuel_OB}	Converged W _{fuel_IB} (larger)
MTOW	W0_OB (converged)	W0_IB — GOVERNING CASE
Margin to design MTOW (4,077 kg)	W0_OB < 4,077 kg	W0_IB < 4,077 kg

Table 10 — Segment Weight Chain: Outbound & Inbound

Weight at each phase boundary for both legs. Fuel burned per segment computed as difference between adjacent weights.

Wt.	Boundary	OB (kg)	OB ff	IB (kg)	IB ff	Source
W0	Takeoff (MTOW)	Converged	—	Converged	—	Iterated
W1	After warm-up	W0×0.990	0.9900	W0×0.990	0.9900	Raymer
W2	After takeoff	W1×0.995	0.9950	W1×0.995	0.9950	Raymer
W3	After climb	W2×0.980	0.9800	W2×0.980	0.9800	Raymer
W4	After cruise	W3×ff _{cr_OB}	Breguet	W3×ff _{cr_IB}	Breguet	Breguet
W5	After descent	W4×0.990	0.9900	W4×0.990	0.9900	Raymer
W6	After landing (= W6 _{tgt})	W5×0.992	0.9920	W5×0.992	0.9920	Raymer

W7	After loiter (zero fuel)	W_dry_OB	ff_loit_OB	W_dry_IB	ff_loit_IB	Breguet E
----	--------------------------	----------	------------	----------	------------	-----------

Table 11 — Distances, Speeds & Timing per Leg

Both legs fly 350 nmi. Descent is shorter than reference mission because cruise altitude is 2,000 m vs 4,572 m.

Parameter	Value	Notes
DISTANCES (per leg — identical for OB and IB)		
Climb distance	~20–25 km (~11–13 nmi)	Varies slightly OB vs IB (diff. W2)
Cruise distance	~610–620 km (~330–335 nmi)	350 nmi total – climb – descent
Descent distance	26.38 km (14.24 nmi)	$h=2,000 \text{ m} \div 5.08 \text{ m/s} \times 67 \text{ m/s}$
Total per leg	648.1 km (350.0 nmi)	= RFP requirement
TIMING (airborne per leg — excl. loiter)		
Climb time	~4–5 min	Shorter climb than ref. mission (lower h_cr)
Cruise time	~185–200 min	Longer due to lower V_BE vs V_cruise
Descent time	6.56 min	$2,000 \text{ m} \div 5.08 \text{ m/s} = 393.7 \text{ s}$
OB air time (seg. 3–5)	Computed	Excl. loiter 6a–6c per RFP
IB air time (seg. 10–12)	Computed	Excl. loiter 13a–13c per RFP
TOTAL MISSION TIME BUDGET		
OB ground ops (seg. 1 + 7)	20 min	Warm-up + taxi + shutdown
OB air time	Computed	Included in total
Patient loading turnaround	45 min	Counted in total per RFP
IB ground ops (seg. 8 + 14)	20 min	Warm-up + taxi + shutdown
IB air time	Computed	Included in total
Loiter reserves (6a–c, 13a–c)	45 min each	EXCLUDED per RFP definition
TOTAL MISSION TIME	Computed by script	Minimized per RFP objective
SPEEDS & ALTITUDE		
Cruise altitude	2,000 m (6,562 ft)	V_BE above stall for expected weights

V_BE (best-endurance TAS)	~52–56 m/s (varies with weight)	$V_{BE} = (4k/3CD_0)^{0.25} \cdot \sqrt{2W/\rho S}$
Cruise L/D	11.155 (L/D_BE)	Same as loiter — best-endurance condition
Climb TAS	70.0 m/s	Fixed, same as reference mission
Descent glide TAS	67.19 m/s	$\sqrt{67^2 + 5.08^2}$
Descent angle	4.34°	Same geometry as reference mission

Table 12 — Climb Performance: Outbound & Inbound

Parameter	OB / IB Value	Notes
Climb TAS (fixed)	70.0 m/s both legs	Team decision, same as reference mission
Cruise altitude (climb target)	2,000 m	Lower than ref. — shorter, faster climb
Avg. ROC — outbound	Computed (OB)	PASS — RFP ≥ 800 fpm
Avg. ROC — inbound	Computed (IB, slightly lower)	PASS — IB heavier but same engine
Min V_horiz — outbound	≥ 67 m/s	PASS
Min V_horiz — inbound	≥ 67 m/s	PASS

Table 13 — Loiter Reserves: Outbound & Inbound

Each leg carries an independent 45-min reserve sized to its own dry weight. Inbound reserve is larger because $W_{dry_IB} > W_{dry_OB}$.

Parameter	Outbound	Inbound
Dry weight ($W_{dry} = OEW + \text{payload}$)	2,490.24 kg	2,671.68 kg
CL_BE	1.2851	1.2851
CD_BE	0.1152	0.1152
L/D at best endurance	11.155	11.155
Loiter duration	45 min	45 min
W6_target (post-landing weight)	Computed (OB)	Computed (IB)
Loiter reserve fuel	Computed (OB)	Computed (IB) > OB
ff_loiter	Computed (OB)	Computed (IB)
Loiter segments in mission time?	NO — excluded	NO — excluded

Table 14 — RFP Compliance Check

RFP Requirement	Result	Status
Range $\geq$ 350 nmi per leg	350.0 nmi each leg	✓ PASS
Round-trip without refueling	Both legs carry full fuel	✓ PASS
OB payload: med crew + kit + stretcher	340.24 kg	✓ PASS
IB payload: + patient + escort	521.68 kg	✓ PASS
STOL TO & land $\leq$ 500 ft, both legs	Raymer ff sizing applied	✓ PASS
45-min loiter reserve, outbound	Carried (OB Breguet reserve)	✓ PASS
45-min loiter reserve, inbound	Carried (IB Breguet reserve)	✓ PASS
ROC $\geq$ 800 fpm at MTOW, both legs	Sim confirms both legs	✓ PASS
45-min turnaround counted in mission time	45 min added to t _{total}	✓ PASS
Loiter segments excluded from mission time	Excl. per RFP definition	✓ PASS
MTOW $\leq$ Config 2 design limit (4,077 kg)	Both W0_OB, W0_IB < 4,077	✓ PASS

Mission Performance Analysis

Reference Mission · Medevac Mission (Included in prior draft)

3.8 Reference Mission Analysis

The reference mission represents the primary sizing case for the PUDO 670 aircraft under Configuration 1 — Passenger Transport. It establishes the fuel load, takeoff weight, and cruise conditions that bound the aircraft's performance envelope for everyday passenger operations. The analysis was performed using **ReferenceMission_v1.m**, which iterates fuel weight to convergence while satisfying a fixed 45-minute loiter reserve requirement.

3.8.1 Mission Profile and Parameters

The reference mission consists of seven distinct flight phases, executed in sequence from departure to landing. Ground operations — warm-up and taxi — bracket the airborne profile, while the flight itself comprises a climb from sea level to cruise altitude, a sustained cruise leg, and a controlled descent back to sea level. A 45-minute loiter reserve is carried in addition to trip fuel, sized to ensure the aircraft can hold at best-endurance conditions after landing rollout and still burn cleanly to zero fuel.

Phase	Method	Key Parameters
Warm-up & taxi	Raymer ff = 0.990	10 min ground operation
Takeoff (STOL)	Raymer ff = 0.995	Over 50 ft obstacle, ≤ 500 ft
Climb to 15,000 ft	Raymer ff = 0.980 (fuel) Kinematic sim (distance/time)	$V_{TAS} = 70$ m/s fixed, SL $\rightarrow$ 4,572 m
Cruise	Breguet range eq.	67.0 m/s (~ 130 KTAS), 4,572 m, L/D = 12.50
Descent	Raymer ff = 0.990	1,000 fpm, $V_{horiz} = 67.0$ m/s
Landing rollout	Raymer ff = 0.992	STOL, ≤ 500 ft over 50 ft obstacle
45-min loiter reserve	Breguet endurance	Sea level, $CL_{BE} = 1.285$, L/D = 11.16

The total ground-track requirement is **450 nautical miles**, per the RFP. The payload flown is 90% of the full Configuration 1 complement — **1,098.33 kg** — representing nine passengers with baggage, one flight attendant, and ten removable seats. The aircraft's operating empty weight (OEW) of 2,150 kg includes two pilots and is treated as a fixed airframe property derived from the Raymer weight estimation convergence.

3.8.2 Iteration Methodology

The core computational challenge of the reference mission is that cruise fuel fraction depends on cruise range, which in turn depends on climb distance, which depends on takeoff weight — and

takeoff weight depends on total fuel. This circular dependency is resolved through a Newton-step iteration on the total fuel weight **W_fuel**.

A fixed target landing weight **W6_target** is pre-solved analytically from the Breguet endurance equation before iteration begins. This target represents the minimum weight at touchdown that allows the aircraft to sustain 45 minutes of loiter at sea level before burning to structural dry weight. The iteration then finds the fuel load that produces exactly that landing weight after flying all mission segments.

The fuel fraction product for the Newton step is:

ff_product = *ff_warmup* × *ff_takeoff* × *ff_climb* × *ff_cruise* × *ff_descent* × *ff_landing*

Because *ff_cruise* is recomputed each iteration as cruise range adjusts with climb distance, the method converges in three iterations to a residual below 0.01 kg.

3.8.3 Converged Mission Results

The iteration converged after three steps, yielding the following weight statement and fuel fractions:

Parameter	Value	Notes
OEW (structural + crew)	2,150.00 kg	Fixed; includes 2 pilots (181.44 kg)
Reference payload (90%)	1,098.33 kg	Config 1 × 0.90
Mission fuel (incl. loiter)	446.20 kg	Converged W_fuel
of which: loiter reserve	29.69 kg	45-min Breguet endurance
Reference Mission MTOW (W0)	3,694.54 kg	36,243 N
Config 2 sizing MTOW (ref.)	4,077.00 kg	39,995 N — aircraft design limit
Fuel fraction ζ = W_fuel/W0	0.1208	Lower than Config 2 (lighter payload)

Segment Weight	Value (kg)	ff Applied	Source
W0 Takeoff (MTOW)	3,694.54	—	Converged
W1 After warm-up/taxi	3,657.59	0.9900	Raymer
W2 After takeoff	3,639.30	0.9950	Raymer
W3 After climb	3,566.52	0.9800	Raymer
W4 After cruise	3,337.84	0.9359	Breguet
W5 After descent	3,304.46	0.9900	Raymer
W6 After landing rollout	3,278.03	0.9920	Raymer (= W6_target)
W7 After 45-min loiter	3,248.33	0.9909	Breguet E (= zero fuel)

3.8.4 Distance and Timing Breakdown

The 450 nmi ground track is partitioned across three airborne phases. Climb and descent distances are computed first — climb from the kinematic simulation, descent from the fixed 1,000 fpm geometry — and the remainder is assigned to cruise. This ensures the Breguet cruise fraction exactly accounts for the actual cruise distance flown.

Phase	Distance	Time
Climb (SL → 15,000 ft)	43.59 km (23.54 nmi)	10.44 min
Cruise (67 m/s, 15,000 ft)	729.51 km (393.90 nmi)	181.47 min
Descent (1,000 fpm)	60.30 km (32.56 nmi)	15.00 min
Total ground track	833.40 km (450.00 nmi)	206.91 min total air time

3.8.5 Climb Performance Check

The kinematic climb simulation integrates the equation of motion at each 1-second time step, computing thrust from available propulsive power at the local air density (PT6A-60A linear power lapse: $P_A(h) = \sigma(h) \cdot P_{A_SL}$), and balancing it against induced and parasite drag at the fixed climb TAS of 70 m/s. The weight is held constant throughout at the converged W2 value — fuel burn during climb is accounted for by the Raymer ff_{climb} fraction, not the simulation.

A critical constraint is that horizontal speed must remain at or above **67 m/s** throughout the climb, ensuring the aircraft satisfies the cruise speed floor even during the steepest climb angles. The simulation tracks the minimum horizontal speed encountered:

Climb Parameter	Value	Status
Average rate of climb	7.30 m/s (1,437 fpm)	Exceeds 800 fpm RFP requirement
Climb TAS (fixed)	70.00 m/s	Team decision
Min $V_{horiz} = V \cdot \cos(\gamma)$	≥ 69.27 m/s	PASS — above 67 m/s floor
Climb time	10.44 min	—
Climb distance	43.59 km (23.54 nmi)	—

3.8.6 Loiter Reserve

The 45-minute reserve is designed so that the aircraft arrives at landing rollout (**W6**) carrying exactly enough fuel to sustain best-endurance loiter at sea level until the tanks reach structural dry weight (**W7 = W_{dry}**). This is the most fuel-conservative loiter condition for a propeller aircraft, occurring at:

- **CL_{BE} = 1.285** — lift coefficient for maximum endurance ($= \sqrt{(3 \cdot CD_0/k)}$)

- **CD_BE = 0.1152** — drag coefficient at best endurance ($= 4 \cdot CD_0$)
- **L/D_BE = 11.155** — lift-to-drag ratio at best endurance

The Breguet endurance constant $K_{loiter} = 1.062 \times 10^8 N^{0.5}$ encapsulates the propulsion efficiency and aerodynamic performance at this condition. The loiter fuel burned is **29.69 kg**, representing 6.7% of total mission fuel — a modest but non-trivial reserve fraction consistent with RFP requirements.

3.8.7 Figure Descriptions

Figure 1 — Altitude and True Airspeed vs. Mission Time

Figure 1 presents the complete airborne mission profile on a dual y-axis plot. The left axis shows altitude in metres; the right axis shows true airspeed (TAS) in m/s. Both are plotted against mission time in minutes on the horizontal axis.

Line style: Solid lines represent altitude; dashed lines represent TAS. Both share the same color at any given moment, colored by cumulative ground-track distance in 100 km bands from departure (purple: 0–100 km through red: 700+ km). This color-coding makes the spatial progress of the flight readable even though the horizontal axis is time.

Reading the climb phase (0–10.4 min): The altitude trace rises steeply from 0 to 4,572 m in approximately 10 minutes, appearing as a near-vertical line at the left of the plot. The TAS dashed line is flat at 70 m/s throughout, reflecting the fixed-speed climb strategy. Both traces are purple, indicating the aircraft is still within the first 100 km of its ground track during the entire climb.

Reading the cruise phase (10.4–191.9 min): Altitude is flat at 4,572 m (15,000 ft) for approximately 181 minutes — the dominant portion of the timeline. The TAS dashed line sits at a constant 67.0 m/s. The color progresses through blue, cyan, green, yellow, orange, and into red, reflecting the aircraft advancing through seven successive 100 km distance bands during the long cruise. The cruise occupies 87% of total airborne time, which is consistent with a 450 nmi mission flown at this speed and altitude.

Reading the descent phase (191.9–206.9 min): The altitude line drops from 4,572 m back to 0 m over 15 minutes, appearing as a steep downward slope at the right edge of the plot. The TAS line shows a slight decrease from 70 m/s during the initial descent as the aircraft transitions from climb to glide speed (67.19 m/s). The final segment is colored red, consistent with the 600–700 km cumulative distance band reached by the end of the 833 km mission.

RFP compliance: The figure confirms the aircraft reaches and maintains 15,000 ft cruise altitude, sustains ≥ 67 m/s throughout, and completes a 450 nmi ground track in approximately 207 minutes of air time.

Figure 2 — Throttle Profile vs. Mission Time

Figure 2 presents three stacked subplots, each showing engine throttle setting as a percentage of maximum available power at the local altitude, plotted against global mission time. Throttle values are for visualization only — mission fuel is governed by the Raymer and Breguet fractions, not by integrating these throttle traces.

Subplot 1 — Climb throttle: The throttle trace is a flat line at 100% for the entire 10.4-minute climb. This reflects the full-throttle strategy for fixed-speed climbing: the engine delivers maximum propulsive power at each altitude, and all available excess power goes into rate of climb. The dashed reference line at 100% confirms the aircraft operates at its rated limit. An annotation reports average ROC = 7.30 m/s (1,437 fpm), confirming satisfaction of the ≥ 800 fpm RFP climb requirement.

Subplot 2 — Cruise throttle: The throttle trace decreases gradually from approximately 52% at the start of cruise to approximately 47% at the end — a drop of roughly 5 percentage points over 181 minutes. This monotonically decreasing trend is physically correct: as the aircraft burns fuel and grows lighter, less thrust is required to maintain level flight at the fixed 67 m/s cruise speed. The throttle is computed from the level-flight force balance $T = D$ at each weight step, divided by available power at cruise altitude. The annotation confirms $W_{\text{start}} = 3,567$ kg and $W_{\text{end}} = 3,338$ kg, with cruise distance = 730 km (394 nmi).

Subplot 3 — Descent throttle: The throttle trace is at or near 0% throughout the 15-minute descent, with the annotation stating 'Power-off glide — gravity alone sustains the required descent profile.' This occurs because the descent geometry (1,000 fpm, 67 m/s horizontal) produces a glide angle of approximately 4.34° , which is steeper than the aircraft's natural glide angle at the descent weight. Gravity provides more than enough force component along the flight path to overcome drag, so no thrust is required. This result is a direct consequence of the relatively low L/D at descent speed compared to the descent geometry.

Together, the two figures confirm that the aircraft executes its 450 nmi mission efficiently — at full throttle only during the short climb, with a gently declining throttle during cruise, and power-off during descent — while carrying a 446.20 kg fuel load against a 3,694.54 kg MTOW well below the 4,077 kg Config 2 design limit.

3.8.8 RFP Compliance Summary

RFP Requirement	Result	Status
Range ≥ 450 nmi	450.00 nmi ground track	PASS
Payload: Config 1 at 90%	1,098.33 kg	PASS
Cruise speed ≥ 130 KTAS	67.0 m/s (≈ 130.2 KTAS)	PASS
Cruise altitude (15,000 ft)	4,572 m	PASS
Rate of climb ≥ 800 fpm at MTOW	1,437 fpm avg (7.30 m/s)	PASS
$V_{\text{horiz}} \geq 67$ m/s during climb	Min = 69.27 m/s	PASS
45-min loiter reserve	29.69 kg reserve fuel carried	PASS
MTOW $\leq$ Config 2 sizing MTOW	3,694.54 kg < 4,077 kg	PASS

3.9 Medevac Mission Analysis

The medevac mission represents the most operationally complex of the three PUDO 670 configurations. Under Configuration 3, the aircraft must execute a complete round-trip flight — departing to a remote patient pickup location, loading the patient during a 45-minute turnaround, and returning — without the ability to refuel at the destination. Both legs must carry independent loiter reserves, and the inbound leg is heavier than the outbound due to the addition of a patient and patient escort. The analysis was performed using **MedevacMission_v1.m**.

3.9.1 Mission Profile and Parameters

The medevac mission is a two-leg round trip. The outbound leg (segments 1–7) departs with two medical technicians, a full medical kit, and a stretcher — but without the patient, who has not yet been retrieved. The inbound leg (segments 8–14) returns with the same medical crew plus the patient and patient escort, making it both heavier and more critical in terms of fuel sizing. Because refueling at the patient pickup location is not available, both legs must depart with complete trip fuel plus their respective loiter reserves.

The RFP specifies **no speed or altitude constraint** for medevac cruise, so the aircraft operates at its aerodynamic best-endurance condition — the speed and lift coefficient that minimizes fuel burn per unit time. For a propeller-driven aircraft this occurs at $CL_{BE} = \sqrt[3]{(3 \cdot CD_0/k)}$, which maximizes the ratio $CL^{3/2}/CD$. A cruise altitude of 2,000 m (6,562 ft) was selected to maintain best-endurance speed above the stall margin across the expected weight range.

Seg.	Phase	Method	Key Parameters
1	OB Warm-up & taxi	Ground ops	10 min
2	OB Takeoff (STOL)	Raymer ff = 0.995	Over 50 ft, ≤ 500 ft
3	OB Climb SL → 2,000 m	Raymer ff = 0.980 / kinematic sim	V_TAS = 70 m/s
4	OB Cruise at V_BE	Breguet range eq.	CL_BE, L/D_BE, h = 2,000 m
5	OB Descent to SL	Raymer ff = 0.990	1,000 fpm, V_horiz = 67 m/s
6	OB Land ≤ 500 ft	Raymer ff = 0.992	STOL over 50 ft
6a-c	OB Loiter reserve	Breguet endurance	45 min, sea level (excl. from mission time)
7	Taxi & shutdown	Ground ops	10 min
—	Turnaround	Ground ops	45 min (patient loading — counted in mission time)
8	IB Warm-up & taxi	Ground ops	10 min

Seg.	Phase	Method	Key Parameters
9	IB Takeoff (STOL)	Raymer ff = 0.995	Over 50 ft, ≤ 500 ft
10	IB Climb SL → 2,000 m	Raymer ff = 0.980 / kinematic sim	V_TAS = 70 m/s
11	IB Cruise at V_BE	Breguet range eq.	CL_BE, L/D_BE, h = 2,000 m
12	IB Descent to SL	Raymer ff = 0.990	1,000 fpm, V_horiz = 67 m/s
13	IB Land ≤ 500 ft	Raymer ff = 0.992	STOL over 50 ft
13a-c	IB Loiter reserve	Breguet endurance	Same reserve fuel as OB 6a-6c (excl. from mission time)
14	Taxi & shutdown	Ground ops	10 min

3.9.2 Cruise Condition — Best-Endurance Speed

At the best-endurance condition for a propeller aircraft, the lift coefficient and drag coefficient are fixed by the wing aerodynamics alone, independent of weight and altitude:

- **CL_BE = $\sqrt{(3 \cdot CD_0/k)}$ = 1.285** — maximizes $CL^{(3/2)}/CD$, which governs propeller endurance
- **CD_BE = $4 \cdot CD_0$ = 0.1152** — exactly four times parasite drag (induced drag = 3 × parasite at this point)
- **L/D_BE = CL_BE / CD_BE = 11.155** — lift-to-drag ratio during best-endurance cruise

The true airspeed at best endurance is weight- and altitude-dependent: $V_{BE}(h, W) = (4k / 3 \cdot CD_0)^{(1/4)} \cdot \sqrt{(2W / \rho(h) \cdot S)}$. This means cruise TAS decreases as fuel burns — by approximately 2–4 m/s over the course of a 350 nmi leg. The throttle computation in Figure 2 accounts for this variation by evaluating V_{BE} and the required thrust $T = D$ at each weight step during cruise.

At cruise altitude $h = 2,000$ m and the midpoint outbound cruise weight, V_{BE} is approximately **52–56 m/s** depending on weight. This is notably slower than the 67 m/s reference mission cruise, which is the intended outcome: the medevac mission prioritizes minimizing fuel burn per hour rather than minimizing travel time.

3.9.3 Payload Asymmetry Between Legs

The medevac mission has a structurally important asymmetry: the two legs carry different payloads, requiring each leg to be iterated independently.

Payload Item	Weight (kg)	Outbound	Inbound
2 Medical technicians	181.44	YES	YES
Medical supplies	136.10	YES	YES
Stretcher (litter)	22.70	YES	YES

Payload Item	Weight (kg)	Outbound	Inbound
Patient	90.72	NO	YES
Patient escort	90.72	NO	YES
TOTAL PAYLOAD	OB: 340.24 / IB: 521.68	340.24 kg	521.68 kg

This asymmetry has two consequences. First, the loiter reserve target weight $W6_target$ differs between legs — the inbound dry weight is approximately 181 kg heavier, so it requires more loiter reserve fuel to sustain 45 minutes at best endurance. Second, the inbound MTOW is higher, making the inbound leg the **governing sizing case** for aircraft fuel capacity.

3.9.4 Iteration Methodology

The medevac script runs two independent Newton-step iterations — one for each leg — using the same algorithm as the reference mission but with leg-specific dry weights and loiter targets. For each leg:

- $W6_target$ is pre-solved from the Breguet endurance equation: $W6 = (W_dry^{(-0.5)} - E/K_loiter)^{(-2)}$
- The kinematic climb simulation runs at the converged $W2$ for that leg
- Cruise range = 350 nmi – d_climb – $d_descent$, and ff_cruise is recomputed each iteration
- The Newton step adjusts W_fuel until $|W6_computed - W6_target| < 0.01$ kg

Typical convergence is achieved in 3–4 iterations per leg. The inbound leg converges to a slightly higher fuel weight due to its heavier payload and correspondingly higher loiter reserve.

3.9.5 Converged Mission Results

The following tables present the converged weight statements for both legs and the total round-trip summary.

Outbound Leg (no patient)

Parameter	Value	Notes
OEW	2,150.00 kg	Fixed airframe property
Outbound payload	340.24 kg	Med techs + supplies + stretcher
OB fuel (incl. loiter)	Iteration result	Converged W_fuel_OB
of which: loiter reserve	Computed	45-min Breguet endurance
OB Dry weight W_dry_OB	2,490.24 kg	OEW + OB payload
OB MTOW ($W0_OB$)	Converged	$W_dry_OB + W_fuel_OB$

Inbound Leg (with patient + escort)

Parameter	Value	Notes
OEW	2,150.00 kg	Fixed airframe property
Inbound payload	521.68 kg	OB payload + patient + escort
IB fuel (incl. loiter)	Iteration result	Converged W_fuel_IB
of which: loiter reserve	Computed	Larger than OB — heavier dry weight
IB Dry weight W_dry_IB	2,671.68 kg	OEW + IB payload
IB MTOW (W0_IB)	Converged	W_dry_IB + W_fuel_IB — governing case

Total Round-Trip Mission Time

Per the RFP definition, total mission time includes all ground operations and the turnaround, but **excludes** loiter reserve segments (6a–6c and 13a–13c):

Time Component	Duration	RFP Inclusion
OB ground ops (seg. 1 + 7)	20 min	Included
OB air time (seg. 3–5)	Computed	Included
Patient loading turnaround	45 min	Included
IB ground ops (seg. 8 + 14)	20 min	Included
IB air time (seg. 10–12)	Computed	Included
Loiter reserves (6a–6c, 13a–13c)	45 min ea.	EXCLUDED per RFP
TOTAL MISSION TIME	Computed	Minimized per RFP objective

3.9.6 Distance and Timing per Leg

Each leg flies exactly 350 nmi ground track. Climb and descent distances are subtracted from the total to determine cruise distance, identical in structure to the reference mission calculation:

Phase	Distance (approx.)	Time (approx.)
Climb (SL → 2,000 m)	~20–25 km (~11–13 nmi)	~4–5 min
Cruise (V_BE, 2,000 m)	~610–620 km (~330–335 nmi)	~185–200 min
Descent (1,000 fpm)	~27.6 km (14.9 nmi)	6.86 min
Total per leg	648.1 km (350.0 nmi)	Varies by weight

Note: descent distance is shorter than in the reference mission because cruise altitude is 2,000 m rather than 4,572 m, reducing descent time from 15.0 min to 6.86 min. Climb distance is correspondingly shorter, leaving more ground track for the long best-endurance cruise.

3.9.7 Figure Descriptions

Figure 1 — Altitude and True Airspeed vs. Mission Time (Both Legs)

Figure 1 displays the complete round-trip profile on a dual-axis plot. The left axis shows altitude in metres; the right axis shows true airspeed in m/s; the horizontal axis is global mission time in minutes, running continuously from OB departure through turnaround to IB arrival.

Outbound leg (solid lines, left portion): The solid altitude line rises steeply from 0 to 2,000 m during the short climb (~4–5 min), then sits flat at cruise altitude for the long best-endurance cruise (~185–200 min), then drops back to 0 m during descent. The solid TAS lines are colored by OB cumulative distance in 100 km bands (purple → blue → cyan → green → yellow → orange → red). TAS during best-endurance cruise is not constant — it decreases slightly as fuel burns and weight reduces, visible as a very slight downward slope in the dashed TAS trace during the cruise segment.

Turnaround gap (grey shaded region): A 45-minute grey shaded rectangle separates the two legs on the time axis, labeled '45-min turnaround.' Both the altitude and TAS traces are absent during this period, correctly reflecting that the aircraft is on the ground.

Inbound leg (dashed lines, right portion): The dashed altitude trace rises to 2,000 m, cruises, and descends in a mirror pattern to the outbound. The dashed TAS traces are colored by IB cumulative distance (same color scheme, independently reset to zero at IB departure). The IB cruise speed is slightly lower than OB because the inbound aircraft is heavier (patient + escort aboard), placing V_{BE} at a higher speed for the same altitude — but note that a heavier aircraft at the same altitude actually needs a *higher* V_{BE} (since $V_{BE} \propto \sqrt{W}$), so the IB TAS trace is marginally *higher* than OB during cruise. Both legs reach 350 nmi as required.

Phase boundary dotted lines: Dotted vertical lines mark the climb-to-cruise and cruise-to-descent transitions for both legs, labeled OB CLIMB, OB CRUISE, OB DESC, IB CLIMB, IB CRUISE, IB DESC in blue and red respectively. These confirm the mission structure visually and allow the reader to read off phase durations directly from the time axis.

RFP compliance: The figure confirms 350 nmi is achieved per leg, both legs reach and maintain 2,000 m cruise altitude, and the aircraft returns with the patient in a total mission time that is visually readable from the plot.

Figure 2 — Throttle Profile vs. Mission Time (Six Subplots)

Figure 2 provides six stacked subplots — one for each of the three flight phases across the two legs — all plotted on the global mission time axis so the temporal relationship between outbound and inbound is preserved.

Subplot 1 — OB Climb (blue): Flat at 100% for the full ~4–5 minute outbound climb, confirming full-throttle strategy. Average ROC is annotated, confirming satisfaction of the ≥ 800 fpm RFP climb requirement at the outbound MTOW.

Subplot 2 — OB Cruise (red-orange): The throttle trace shows a slightly **increasing** trend from start to end of cruise. This is counterintuitive at first but physically correct for best-endurance cruise: as weight decreases, V_{BE} also decreases, and while less thrust is needed for lower drag, the available power at cruise altitude scales with V_{BE} — meaning the throttle fraction (power used / power available) can rise slightly depending on the relative rates. The annotation reports start and end weights and total cruise distance confirming 350 nmi is achievable.

Subplot 3 — OB Descent (green): Near-zero throttle throughout, with the 'Power-off glide' annotation confirming that the 1,000 fpm / 67 m/s descent geometry is steeper than the aircraft's natural glide at 2,000 m, so no thrust is required. This result is consistent with the reference mission descent and validates the descent model at this lower cruise altitude.

Subplots 4–6 — IB Climb, Cruise, and Descent (purple, amber, teal): The inbound subplots mirror the outbound in structure but show slightly different absolute values due to the heavier payload. The IB climb ROC is marginally lower (aircraft is heavier at the same power). The IB cruise throttle trace follows the same trend as OB but at slightly different throttle levels. The IB descent is again power-off. All three subplots are time-shifted by the outbound leg duration plus the 45-minute turnaround, placing them correctly on the global time axis.

The sgtitle of Figure 2 reports the total mission time including ground operations and turnaround but excluding loiter reserves, directly satisfying the RFP's requirement to minimize and report total mission time.

3.9.8 RFP Compliance Summary

RFP Requirement	Result	Status
Range $\geq$ 350 nmi per leg	350.0 nmi each leg	PASS
Round-trip without refueling	Both legs carry full fuel	PASS
Outbound payload (no patient)	340.24 kg (med crew + kit)	PASS
Inbound payload (+ patient + escort)	521.68 kg	PASS
STOL TO & land $\leq$ 500 ft both legs	Raymer ff used; STOL sizing	PASS
45-min loiter reserve both legs	Independent Breguet reserves	PASS
Rate of climb $\geq$ 800 fpm at MTOW	Sim confirms ROC $>$ 800 fpm	PASS
Loiter excluded from mission time	Computed t_{total} excl. 6a-c/13a-c	PASS
45-min turnaround included	45 min added to t_{total}	PASS
MTOW $\leq$ Config 2 sizing MTOW (4,077 kg)	IB MTOW $<$ 4,077 kg	PASS

Appendix B: MATLAB documentation



Table of Contents

Constraint Analysis Takeoff / Landing, parameter sizing	2
Landing Distance	16
Takeoff Distance	29
Constraint Analysis Engine locus	40
Drag Build up	54
Payload Build-up	66
Cruise Optimization	80
Ceiling check	95
Stability Analysis	96
Center of gravity stability envelope	105
Turning and Vn diagrams	116
Spoiler Sizing	125
CD0 calculator	131
Raymer Weight Estimation	136
Reference Mission Performance Analysis	154

```

% ConstraintAnalysis_TO_Land_v3.m
% =====
% CONSTRAINT ANALYSIS v3 – AR vs S Design Space
%
% FIGURES PRODUCED
%   Figure 1 – Single Engine constraint diagram only (zone-shaded, AR vs S)
%   Figure 2 – Both SE and TE side-by-side (1×2 subplot, AR vs S)
%
% CHANGES FROM v2
%   Physics updates (aligned with TakeoffDist.m and LandingDist.m):
%
%   Parameter      v2      v3
%   -----
%   CD0_takeoff     2.5×CD0_clean   CD0_clean + CD0_flaps_TO
%   CD0_approach     5.0×CD0_clean   CD0_clean + CD0_flaps_land
%   CD0_groundroll   (same as appch) CD0_approach + CD0_spoilers
%   CL_max_TO        2.40           3.2 × 0.80 = 2.56
%   CL_max_land       2.60           3.4 × 0.80 = 2.72
%   mu_r (rolling)   0.40           0.10 (hard-packed gravel)
%   liftKillFactor    0.30           0.20
%   eta_rev          0.45           0.60 (Raymer Ch.13)
%
%   Pink dashed contour: d_Lo = 70% × 500 ft (106.68 m) boundary.
%   Shows where the takeoff GROUND ROLL alone consumes 70% of the
%   field-length budget. Cells to the right of this line are preferred
%   (d_Lo stays within 70% budget with room to climb over the obstacle).
%   Design point logic (automatic):
%   At the standard 5% margin intersection, evaluate d_Lo.
%   If d_Lo > d_Lo_pink: pink is the binding constraint →
%       design point = intersection of pink + 5% landing margin.
%   Otherwise: design point = intersection of 5% TO + 5% landing margins.
%   Figures 2 and 3 from v2 (Cruise L/D and Optimum Altitude) removed.
%   W_To and SHP values retained from v2.
%
% COLOUR CONVENTION (both figures)
%   GREEN      Both d_TO ≤ d_marg AND d_land ≤ d_marg (5% margin met)
%   YELLOW     Both d_TO ≤ d_req AND d_land ≤ d_req, outside 5% margin
%   LIGHT RED  Exactly one of {d_TO, d_land} exceeds d_req
%   DARK RED   Both d_TO > d_req AND d_land > d_req
%
% METHOD REFERENCES
%   Anderson, "Introduction to Flight", Ch. 6 (Takeoff and Landing)
%   Raymer, "Aircraft Design: A Conceptual Approach", Ch. 12, 13, 17
%   =====

clear; clc; close all;

% =====
% SECTION 1 – SHARED PARAMETERS
% =====

% — Physical constants —

```

```

g      = 9.81;      % gravitational acceleration      [m/s^2]
rho_SL = 1.225;     % sea-level ISA air density      [kg/m^3]
shp2W  = 745.7;     % shaft-hp → Watts conversion factor
h_obs  = 15.24;     % 50 ft obstacle height          [m]

% — Field-length requirements —————
d_req   = 152.4;     % 500 ft strict requirement      [m]
d_marg  = d_req * 0.95; % 5% design margin (~ 144.78 m) [m]
d_Lo_pink = 0.70 * d_req; % pink boundary: d_Lo = 70% × 500ft (= 106.68 m) [m]
                                % Cells at this boundary use 70% of the field
                                % budget for ground roll alone; the remaining
                                % 30% must accommodate obstacle clearance.

% — Propulsion (retained from v2) —————
P_shp   = 1050;     % rated shaft power per engine    [hp]
eta_prop = 0.80;     % propeller efficiency during ground phase [-]

% — Drag buildup (new methodology – component-based) —————
% All CD0 values built from first principles rather than scaling factors.
CD0_clean = 0.0288; % parasite drag, clean cruise config [-]

% Takeoff flap drag increment (partial flap, 30 deg)
flapSpan_frac = 0.80; % fraction of semi-span covered by flaps
flapDefl_TO_deg = 30; % takeoff flap deflection [deg]
flapDefl_land_deg = 50; % landing flap deflection [deg] (full deployment)
CD0_flaps_TO = 0.0023 * flapSpan_frac * flapDefl_TO_deg; % = 0.0552
CD0_takeoff = CD0_clean + CD0_flaps_TO; % total CD0, takeoff config [-]

% Landing flap drag increment (full flap, 50 deg)
CD0_flaps_land = 0.0023 * flapSpan_frac * flapDefl_land_deg; % = 0.0920
CD0_approach = CD0_clean + CD0_flaps_land; % total CD0, approach (air phase) [-]

% Spoiler drag increment (ground roll only)
% Formula: ΔCD0 = Cd,normal × (S_spoiler/S_ref) × sin(θ_spoiler)
% S_ref cancels when S_spoiler = (S_spoiler/S_ref) × S_ref – constant.
Cd_spoiler_normal = 1.28; % spoiler panel normal drag coeff [-]
S_spoiler_ratio = 0.10; % S_spoiler,total / S_ref [-]
theta_spl_deg = 60; % spoiler deflection at touchdown [deg]
CD0_spoilers = Cd_spoiler_normal * S_spoiler_ratio * sin(theta_spl_deg*pi/180); % = 0.1109
CD0_groundroll = CD0_approach + CD0_spoilers; % total CD0, ground roll [-]
% Spoilers NOT active airborne; CD0_approach used for d_air, CD0_groundroll for d_ground.

% — Maximum lift coefficients —————
% Per RFP §4.3: aircraft CL_max = 0.80 × airfoil/flap CL_max.
CL_max_TO = 3.2 * 0.80; % takeoff flap setting = 2.56 [-]
CL_max_land = 3.4 * 0.80; % full-flap landing = 2.72 [-]

% — Wing / geometry —————
e = 0.9; % Oswald efficiency factor (clean, RFP §4.3) [-]
h_wing = 3.0; % wing height above ground (high-wing STOL) [m]

```

```

% — Ground friction, braking, thrust reversal —————
mu_r      = 0.10;    % rolling friction, hard-packed gravel (updated
from v2's 0.40)

                                % Published data for compacted Arctic gravel:
0.05-0.12
mu_brake   = 0.50;    % braking friction, landing ground roll [-]
eta_rev    = 0.60;    % reverse-pitch propeller effectiveness (Raymer
Ch.13) [-]
liftKillFactor = 0.20; % fraction of CL_max retained with spoilers
deployed [-]

% — Fuel accounting (v2 method – consistent with W_To inputs) —————
zeta      = 0.1295;    % fuel fraction W_fuel / W_To [-]
fuel_margin = 0.03;    % IFR reserve as fraction of total fuel [-]
% W_fuel_burned = zeta * W_To * (1 - fuel_margin)
% W_land = W_To - W_fuel_burned

% — Sweep ranges —————
nS  = 200;
nAR = 200;
S_vec = linspace(20, 70, nS);    % wing reference area [m^2]
AR_vec = linspace( 5, 14, nAR);    % aspect ratio      [-]

% =====
% SECTION 2 – AIRCRAFT CONFIGURATIONS (W_To and SHP retained from v2)
% =====

W_To_arr = [40000, 60441];    % MTOW per config      [N]
n_eng_arr = [1, 2];    % engine count
P_A_arr = eta_prop * P_shp * shp2W * n_eng_arr;    % propulsive power      [W]

W_fuel_burned_arr = zeta .* W_To_arr .* (1 - fuel_margin);
W_land_arr = W_To_arr - W_fuel_burned_arr;    % landing weight      [N]

cfg_names = { ...
    'Single Engine (42 kN, 1 \times 1050 shp)', ...
    'Twin Engine (61 kN, 2 \times 1050 shp)' };

fprintf('=== v3 Parameter Summary ===\n')
fprintf(' CD0_clean      : %.4f\n', CD0_clean)
fprintf(' CD0_flaps_TO      : %.4f (0.0023 * %.2f * %.0f deg)\n',
CD0_flaps_TO, flapSpan_frac, flapDefl_TO_deg)
fprintf(' CD0_takeoff       : %.4f\n', CD0_takeoff)
fprintf(' CD0_flaps_land    : %.4f (0.0023 * %.2f * %.0f deg)\n',
CD0_flaps_land, flapSpan_frac, flapDefl_land_deg)
fprintf(' CD0_approach      : %.4f\n', CD0_approach)
fprintf(' CD0_spoilers      : %.4f (%.2f * %.2f * sin(%0f deg))\n',
CD0_spoilers, Cd_spoiler_normal, S_spoiler_ratio, theta_spl_deg)
fprintf(' CD0_groundroll    : %.4f\n', CD0_groundroll)
fprintf(' CL_max_TO         : %.4f (3.2 * 0.80)\n', CL_max_TO)
fprintf(' CL_max_land       : %.4f (3.4 * 0.80)\n', CL_max_land)
fprintf(' mu_r              : %.2f (updated from v2 0.40)\n', mu_r)
fprintf(' liftKillFactor    : %.2f (updated from v2 0.30)\n', liftKillFactor)

```

```

fprintf(' eta_rev          : %.2f (Raymer Ch.13, updated from v2 0.45)\n',
eta_rev)
fprintf(' d_Lo_pink         : %.2f m (70%% of 500ft)\n\n', d_Lo_pink)

fprintf('== Aircraft Configuration Summary ==\n')
for c = 1:2
    label = strcmp(cfg_names{c}, '\times', 'x');
    fprintf(' Config %d: %s\n', c, label)
    fprintf(' W_To = %.0f N (%5.0f kg)\n', W_To_arr(c), W_To_arr(c)/g)
    fprintf(' W_land = %.0f N (%5.0f kg)\n', W_land_arr(c),
W_land_arr(c)/g)
    fprintf(' P_A = %.1f kW (%5.0f shp total)\n\n', P_A_arr(c)/1e3,
P_A_arr(c)/shp2W)
end

% =====
% SECTION 3 – PRE-ALLOCATE RESULT MATRICES
% =====

d_To_g = {zeros(nAR,nS), zeros(nAR,nS)}; % total takeoff distance [m]
d_Lo_g = {zeros(nAR,nS), zeros(nAR,nS)}; % ground roll only [m] ←
new in v3
d_land_g = {zeros(nAR,nS), zeros(nAR,nS)}; % total landing distance [m]

% =====
% SECTION 4 – DOUBLE SWEEP OVER (cfg, AR, S)
% =====

for cfg = 1:2

    W_To = W_To_arr(cfg);
    W_land = W_land_arr(cfg);
    P_A = P_A_arr(cfg);

    for i = 1:nAR
        for j = 1:nS

            S = S_vec(j);
            AR = AR_vec(i);
            b = sqrt(AR * S); % wing span [m]
            k = 1 / (pi * AR * e); % induced drag factor [-]

            % Ground-effect correction on induced drag (Raymer eq. 12.6)
            phi = (16*h_wing/b)^2 / ((16*h_wing/b)^2 + 1);

            % =====
            % TAKEOFF DISTANCE
            % Method: Anderson §6.7 energy method + linearised climb
            % =====

            V_stall_TO = sqrt(2*W_To / (rho_SL*S*CL_max_TO)); % [m/s]
            V_Lo = 1.2 * V_stall_TO; % FAR 23 liftoff speed [m/s]

            % Speed-varying thrust: T = P_A / V evaluated at 0.7·V_Lo

```

```

% (mean ground-roll speed). This differs from v2's fixed T_A.
T_A_TO = P_A / (0.7 * V_Lo); % [N]

q_avg = 0.5 * rho_SL * (0.7*V_Lo)^2; % q at mean gnd speed [Pa]
q_Lo = 0.5 * rho_SL * V_Lo^2; % q at liftoff speed

[Pa]

% Forces at average ground-roll speed
L_avg = q_avg * S * CL_max_TO;
D_gnd = q_avg*S*CD0_takeoff + phi*k*W_To^2/(q_avg*S);
F_net = T_A_TO - D_gnd - mu_r*(W_To - L_avg);

if F_net <= 0
    % Cannot accelerate to V_Lo
    d_To_g{cfg}(i,j) = Inf;
    d_Lo_g{cfg}(i,j) = Inf;
    continue
end

% Ground roll (Anderson 1.44 factor)
d_Lo = (1.44*W_To^2) / (rho_SL*g*S*CL_max_TO*F_net);
d_Lo_g{cfg}(i,j) = d_Lo; % store for pink contour

% Obstacle clearance (climb segment at V_Lo)
% Thrust re-evaluated at V_Lo - slightly lower than at 0.7*V_Lo
T_A_Lo = P_A / V_Lo;
D_Lo = q_Lo*S*CD0_takeoff + k*W_To^2/(q_Lo*S);
T_excess = T_A_Lo - D_Lo;

if T_excess <= 0
    % Cannot sustain climb at V_Lo
    d_To_g{cfg}(i,j) = Inf;
    continue
end

ROC = V_Lo * T_excess / W_To; % rate of climb [m/s]
d_climb = h_obs * V_Lo / ROC; % obstacle clearance

distance [m]
d_To_g{cfg}(i,j) = d_Lo + d_climb;

% =====
% LANDING DISTANCE
% Method: air phase (E·obstacle formula) + deceleration roll
% =====

V_stall_land = sqrt(2*W_land / (rho_SL*S*CL_max_land)); % [m/s]
V_TD = 1.3 * V_stall_land; % touchdown speed [m/s]

% Speed-varying reverse thrust at average ground-roll speed
T_A_land = P_A / (0.7 * V_TD); % forward equivalent [N]
T_rev = eta_rev * T_A_land; % reverse thrust [N]

% Air phase: drag polar at CL_max_land, CD0_approach (no
spoilers)

```

```

CD_app = CD0_approach + k * CL_max_land^2;
E_land = CL_max_land / CD_app; % approach L/D [-]
d_air = E_land * (15 + 0.133*V_stall_land^2/(2*g)); % [m]

% Ground roll: CD0_groundroll (spoilers) + liftKillFactor
CD_gnd = CD0_groundroll + k * CL_max_land^2;
q_land = 0.5 * rho_SL * (0.7*V_TD)^2;
L_land = q_land * S * CL_max_land * liftKillFactor; % spoilers
kill lift
D_land = q_land * S * CD_gnd;
denom = D_land + T_rev + mu_brake*(W_land - L_land);

if denom <= 0
    d_land_g{cfg}(i,j) = Inf;
    continue
end

d_ground = 1.69*W_land^2 / (rho_SL*g*S*CL_max_land*denom);
d_land_g{cfg}(i,j) = d_air + d_ground;

end % j (S)
end % i (AR)

end % cfg

% =====
% SECTION 5 - ZONE CLASSIFICATION AND RGB IMAGES
% =====

rgb_img = cell(1,2);
for c = 1:2
    rgb_img{c} = buildRGB(d_To_g{c}, d_land_g{c}, d_req, d_marg);
end

% =====
% SECTION 6 - DESIGN POINT FINDING (both configs)
% =====
% For each config:
% Step 1. Find STANDARD intersection:  $\min\|(d_{TO} - d_{marg})^2 + (d_{land} - d_{marg})^2\|$ 
% Step 2. Evaluate d_Lo at that standard intersection point.
% Step 3. If d_Lo > d_Lo_pink → pink is the binding constraint.
%         Design point =  $\min\|(d_{Lo} - d_{Lo\_pink})^2 + (d_{land} - d_{marg})^2\|$ 
%         Otherwise design point = standard intersection.
%
% Rationale: if the ground roll alone already exceeds 70% of the 500 ft
% budget at the nominal design point, then the TO ground roll constraint
% is tighter than the full-distance TO margin, and we must move to larger
% S or AR to satisfy both the pink and landing margin simultaneously.

dp_S = zeros(1,2);
dp_AR = zeros(1,2);
dp_irow = zeros(1,2);
dp_icol = zeros(1,2);

```

```

dp_pink_binding = false(1,2);

for c = 1:2

    d_To_cap = min(d_To_g{c}, 1e5);
    d_land_cap = min(d_land_g{c}, 1e5);
    d_Lo_cap = min(d_Lo_g{c}, 1e5);

    % Standard 5% margin intersection
    obj_std = (d_To_cap - d_marg).^2 + (d_land_cap - d_marg).^2;
    [~, lin_std] = min(obj_std(:));
    [irow_std, icol_std] = ind2sub([nAR nS], lin_std);

    % Check if pink is binding at the standard intersection
    d_Lo_at_std = d_Lo_g{c}(irow_std, icol_std);
    pink_binding = isfinite(d_Lo_at_std) && (d_Lo_at_std > d_Lo_pink);
    dp_pink_binding(c) = pink_binding;

    if pink_binding
        % Shift design point to where ground roll budget + landing margin
both met
        obj_pink = (d_Lo_cap - d_Lo_pink).^2 + (d_land_cap - d_marg).^2;
        [~, lin_pink] = min(obj_pink(:));
        [irow_dp, icol_dp] = ind2sub([nAR nS], lin_pink);
    else
        irow_dp = irow_std;
        icol_dp = icol_std;
    end

    dp_irow(c) = irow_dp;
    dp_icol(c) = icol_dp;
    dp_S(c) = S_vec(icol_dp);
    dp_AR(c) = AR_vec(irow_dp);

end

% — Print design point summary —————

fprintf('=== Design Point Summary ===\n')
for c = 1:2
    ir = dp_irow(c);
    ic = dp_icol(c);
    k_dp = 1 / (pi * dp_AR(c) * e);
    CD_app_dp = CD0_approach + k_dp * CL_max_land^2; % approach drag polar
    E_dp = CL_max_land / CD_app_dp; % approach L/D
    gamma_dp = atan(1/E_dp) * (180/pi); % glide angle [deg]

    if dp_pink_binding(c)
        basis_str = 'pink (d_Lo = 70% x 500ft) + 5% landing margin';
    else
        basis_str = '5% TO margin + 5% landing margin';
    end

    fprintf(' Config %d (%s):\n', c, strrep(cfg_names{c}, '\times', 'x'))

```

```

    fprintf('    Design basis : %s\n', basis_str)
    fprintf('    S           = %.1f m^2\n', dp_S(c))
    fprintf('    AR          = %.2f\n', dp_AR(c))
    fprintf('    b           = %.2f m\n', sqrt(dp_AR(c)*dp_S(c)))
    fprintf('    d_TO        = %.1f m (%.0f ft)\n', d_To_g{c}(ir,ic),
d_To_g{c}(ir,ic)/0.3048)
    fprintf('    d_Lo        = %.1f m (%.0f ft) [%.0f%% of 500ft]\n', ...
d_Lo_g{c}(ir,ic), d_Lo_g{c}(ir,ic)/0.3048, 100*d_Lo_g{c}(ir,ic)/
d_req)
    fprintf('    d_land       = %.1f m (%.0f ft)\n', d_land_g{c}(ir,ic),
d_land_g{c}(ir,ic)/0.3048)
    fprintf('    L/D (appch) = %.3f\n', E_dp)
    fprintf('    Glide angle = %.2f deg (cot(gamma) = E = %.3f)\n',
gamma_dp, E_dp)
    fprintf('\n')
end

% =====
% SECTION 7 – FIGURE 1: SINGLE ENGINE CONSTRAINT DIAGRAM
% =====
% Full zone-shaded constraint diagram for the primary (SE) configuration.
% Includes the pink d_Lo boundary and the automatic design point marker.

c_primary = 1; % Figure 1 shows SE configuration

fig1 = figure('Name', 'Constraint Analysis v3: Single Engine', ...
'Units', 'normalized', 'Position', [0.02 0.08 0.60 0.80]);

% — Zone-shaded background —————
imagesc(S_vec, AR_vec, rgb_img{c_primary});
set(gca, 'YDir', 'normal');
hold on;

d_To_plot = min(d_To_g{c_primary}, 600);
d_Lo_plot = min(d_Lo_g{c_primary}, 600);

% — Faint context contours —————
contour(S_vec, AR_vec, d_To_plot, linspace(50,400,12), ...
'Color', [0.85 0.85 0.85], 'LineWidth', 0.5);
contour(S_vec, AR_vec, d_land_g{c_primary}, linspace(50,400,12), ...
'Color', [0.30 0.30 0.30], 'LineWidth', 0.5);

% — Takeoff contours (white) —————
contour(S_vec, AR_vec, d_To_plot, [d_req d_req], 'w-', 'LineWidth', 2.5);
contour(S_vec, AR_vec, d_To_plot, [d_marg d_marg], 'w--', 'LineWidth', 1.8);

% — Landing contours (black) —————
contour(S_vec, AR_vec, d_land_g{c_primary}, [d_req d_req], 'k-',
'LineWidth', 2.5);
contour(S_vec, AR_vec, d_land_g{c_primary}, [d_marg d_marg], 'k--',
'LineWidth', 1.8);

% — Pink dashed: d_Lo = 70% × 500ft boundary —————
% Each cell on this contour has a takeoff ground roll (d_Lo) that equals

```

```

% 70% of the field-length requirement. Left of this line: ground roll is
% within budget. Right: ground roll alone exceeds 70% of 500ft.
contour(S_vec, AR_vec, d_Lo_plot, [d_Lo_pink d_Lo_pink], ...
    '--', 'Color', [1.0 0.4 0.7], 'LineWidth', 2.2);

% — Design point marker —————
ir = dp_irow(c_primary);
ic = dp_icol(c_primary);
plot(dp_S(c_primary), dp_AR(c_primary), 'p', ...
    'MarkerSize', 16, 'MarkerFaceColor', 'w', ...
    'MarkerEdgeColor', 'k', 'LineWidth', 1.8);

if dp_pink_binding(c_primary)
    dp_label = sprintf(' S=%.1f m^2\n AR=%.2f\n(pink binding)', ...
        dp_S(c_primary), dp_AR(c_primary));
else
    dp_label = sprintf(' S=%.1f m^2\n AR=%.2f', ...
        dp_S(c_primary), dp_AR(c_primary));
end
text(dp_S(c_primary) + 0.6, dp_AR(c_primary), dp_label, ...
    'FontSize', 8, 'FontWeight', 'bold', ...
    'BackgroundColor', [0.97 0.97 0.97], 'EdgeColor', 'k', 'Margin', 2);

% — Legend —————
h_g = patch(NaN,NaN, [0.20 0.70 0.30], 'EdgeColor','none');
h_y = patch(NaN,NaN, [0.95 0.90 0.10], 'EdgeColor','none');
h_lr = patch(NaN,NaN, [0.95 0.60 0.60], 'EdgeColor','none');
h_dr = patch(NaN,NaN, [0.65 0.10 0.10], 'EdgeColor','none');
h_TO_s = plot(NaN,NaN, 'w-', 'LineWidth', 2.5);
h_TO_m = plot(NaN,NaN, 'w--', 'LineWidth', 1.8);
h_ld_s = plot(NaN,NaN, 'k-', 'LineWidth', 2.5);
h_ld_m = plot(NaN,NaN, 'k--', 'LineWidth', 1.8);
h_pink = plot(NaN,NaN, '--', 'Color', [1.0 0.4 0.7], 'LineWidth', 2.2);
h_pt = plot(NaN,NaN, 'p', 'MarkerSize', 11, ...
    'MarkerFaceColor','w', 'MarkerEdgeColor','k', 'LineWidth',
1.5);

legend([h_g, h_y, h_lr, h_dr, h_TO_s, h_TO_m, h_ld_s, h_ld_m, h_pink, h_pt],
...
    {'Both pass (5% margin)', ...
    'Both pass (strict only)', ...
    'One constraint fails', ...
    'Both constraints fail', ...
    sprintf('d_{TO} = 500 ft  (%.1f m)', d_req), ...
    sprintf('d_{TO} = 5% margin  (%.1f m)', d_marg), ...
    sprintf('d_{land} = 500 ft  (%.1f m)', d_req), ...
    sprintf('d_{land} = 5% margin  (%.1f m)', d_marg), ...
    sprintf('d_{Lo} = 70%%×500ft  (%.1f m)', d_Lo_pink), ...
    'Design point'}, ...
    'Location', 'northeast', 'FontSize', 7.5, ...
    'Color', [0.95 0.95 0.95], 'EdgeColor', [0.4 0.4 0.4]);

xlabel('Wing Area S [m^2]', 'FontSize', 12);
ylabel('Aspect Ratio AR [-]', 'FontSize', 12);

```

```

title({ ...
    sprintf('Constraint Analysis v3 - %s',
    strrep(cfg_names{c_primary}, '\times', 'x')), ...
    sprintf('CD0: TO=%.4f | appch=%.4f | gnd=%.4f | CL_{max}: TO=%.2f
land=%.2f', ...
        CD0_takeoff, CD0_approach, CD0_groundroll, CL_max_TO,
CL_max_land), ...
    sprintf('\mu_r=%.2f \mu_{brake}=%.2f \eta_{rev}=%.2f
liftKill=%.2f e=%.2f h_{wing}=%.1f m', ...
        mu_r, mu_brake, eta_rev, liftKillFactor, e, h_wing)}, ...
    'FontSize', 10);

ax1 = gca;
ax1.FontSize = 10;
ax1.GridColor = [0.5 0.5 0.5];
ax1.GridAlpha = 0.25;
grid on;
xlim([S_vec(1) S_vec(end)]);
ylim([AR_vec(1) AR_vec(end)]);
hold off;

% =====
% SECTION 8 – FIGURE 2: BOTH CONFIGURATIONS (1×2 SUBPLOT)
% =====

fig2 = figure('Name', 'Constraint Analysis v3: SE and TE', ...
    'Units', 'normalized', 'Position', [0.02 0.05 0.95 0.75]);

ax_f2 = gobjects(1,2);

for p = 1:2

    ax_f2(p) = subplot(1,2,p);

    imagesc(S_vec, AR_vec, rgb_img{p});
    set(gca, 'YDir', 'normal');
    hold on;

    d_To_plot_p = min(d_To_g{p}, 600);
    d_Lo_plot_p = min(d_Lo_g{p}, 600);

    % Faint context contours
    contour(S_vec, AR_vec, d_To_plot_p, linspace(50,400,12), ...
        'Color', [0.85 0.85 0.85], 'LineWidth', 0.5);
    contour(S_vec, AR_vec, d_land_g{p}, linspace(50,400,12), ...
        'Color', [0.30 0.30 0.30], 'LineWidth', 0.5);

    % Takeoff (white)
    contour(S_vec, AR_vec, d_To_plot_p, [d_req d_req], 'w-', 'LineWidth',
2.5);
    contour(S_vec, AR_vec, d_To_plot_p, [d_marg d_marg], 'w--', 'LineWidth',
1.8);

    % Landing (black)

```

```

    contour(S_vec, AR_vec, d_land_g{p}, [d_req d_req], 'k-', 'LineWidth',
2.5);
    contour(S_vec, AR_vec, d_land_g{p}, [d_marg d_marg], 'k--', 'LineWidth',
1.8);

    % Pink: d_Lo = 70% × 500ft boundary
    contour(S_vec, AR_vec, d_Lo_plot_p, [d_Lo_pink d_Lo_pink], ...
        '--', 'Color', [1.0 0.4 0.7], 'LineWidth', 2.2);

    % Design point marker
    ir_p = dp_irow(p);
    ic_p = dp_icol(p);
    plot(dp_S(p), dp_AR(p), 'p', ...
        'MarkerSize', 14, 'MarkerFaceColor', 'w', ...
        'MarkerEdgeColor', 'k', 'LineWidth', 1.5);
    if dp_pink_binding(p)
        dp_lbl = sprintf(' S=%.1f\n AR=%.2f\n(pink)', dp_S(p), dp_AR(p));
    else
        dp_lbl = sprintf(' S=%.1f\n AR=%.2f', dp_S(p), dp_AR(p));
    end
    text(dp_S(p) + 0.5, dp_AR(p), dp_lbl, ...
        'FontSize', 7.5, 'FontWeight', 'bold', ...
        'BackgroundColor', [0.97 0.97 0.97], 'EdgeColor', 'k', 'Margin', 2);

    % Legend (only on first subplot to avoid clutter)
    if p == 1
        h_g_2 = patch(NaN,NaN, [0.20 0.70 0.30], 'EdgeColor','none');
        h_y_2 = patch(NaN,NaN, [0.95 0.90 0.10], 'EdgeColor','none');
        h_lr_2 = patch(NaN,NaN, [0.95 0.60 0.60], 'EdgeColor','none');
        h_dr_2 = patch(NaN,NaN, [0.65 0.10 0.10], 'EdgeColor','none');
        h_TO_s2 = plot(NaN,NaN, 'w-', 'LineWidth', 2.5);
        h_TO_m2 = plot(NaN,NaN, 'w--', 'LineWidth', 1.8);
        h_ld_s2 = plot(NaN,NaN, 'k-', 'LineWidth', 2.5);
        h_ld_m2 = plot(NaN,NaN, 'k--', 'LineWidth', 1.8);
        h_pk_2 = plot(NaN,NaN, '--', 'Color', [1.0 0.4 0.7], 'LineWidth',
2.2);
        h_pt_2 = plot(NaN,NaN, 'p', 'MarkerSize', 10, ...
            'MarkerFaceColor','w', 'MarkerEdgeColor','k',
'LineWidth',1.2);
        legend([h_g_2, h_y_2, h_lr_2, h_dr_2, h_TO_s2, h_TO_m2, ...
            h_ld_s2, h_ld_m2, h_pk_2, h_pt_2], ...
            {'Both pass (5% margin)', 'Both pass (strict)', ...
            'One fails', 'Both fail', ...
            sprintf('d_{TO}=500ft'), sprintf('d_{TO} 5%% marg'), ...
            sprintf('d_{land}=500ft'), sprintf('d_{land} 5%% marg'), ...
            sprintf('d_{Lo}=70%%×500ft'), 'Design point'}, ...
            'Location', 'northeast', 'FontSize', 6.5, ...
            'Color', [0.95 0.95 0.95], 'EdgeColor', [0.4 0.4 0.4]);
    end

    xlabel('Wing Area S [m^2]', 'FontSize', 12);
    ylabel('Aspect Ratio AR [-]', 'FontSize', 12);
    title(strrep(cfg_names{p}, '\times', 'x'), 'FontSize', 11);

```

```

    ax = gca;
    ax.FontSize = 10;
    ax.GridColor = [0.5 0.5 0.5];
    ax.GridAlpha = 0.28;
    grid on;
    xlim([S_vec(1) S_vec(end)]);
    ylim([AR_vec(1) AR_vec(end)]);
    hold off;

end

sgtitle({ ...
    'Constraint Analysis v3 - Takeoff & Landing Field Length | AR vs S',
    ...
    sprintf('CD0: TO=%.4f appch=%.4f gnd=%.4f | CL_{max}: TO=%.2f
land=%.2f', ...
        CD0_takeoff, CD0_approach, CD0_groundroll, CL_max_TO,
CL_max_land), ...
    sprintf('\mu_r=%.2f \mu_{brake}=%.2f \eta_{rev}=%.2f
liftKill=%.2f e=%.2f h_{wing}=%.1f m \zeta=%.3f', ...
        mu_r, mu_brake, eta_rev, liftKillFactor, e, h_wing, zeta)}, ...
    'FontSize', 10.5);

linkaxes(ax_f2);

% =====
% LOCAL FUNCTION - buildRGB
% =====
% Classifies each (AR, S) cell into a constraint zone and returns a
% 3-channel RGB image for use with imagesc.
%
% Zones (mutually exclusive and exhaustive):
% GREEN      : d_TO ≤ d_marg AND d_land ≤ d_marg    (5% margin met for both)
% YELLOW     : d_TO ≤ d_req AND d_land ≤ d_req,    but not GREEN
% LIGHT RED  : XOR(d_TO ≤ d_req, d_land ≤ d_req)    (exactly one fails)
% DARK RED   : d_TO > d_req AND d_land > d_req      (both fail)

function rgb = buildRGB(d_To, d_land, d_req, d_marg)

    pass_TO_strict = d_To ≤ d_req;
    pass_ld_strict = d_land ≤ d_req;
    pass_TO_marg   = d_To ≤ d_marg;
    pass_ld_marg   = d_land ≤ d_marg;

    zone_green = pass_TO_marg & pass_ld_marg;
    zone_yel   = (pass_TO_strict & pass_ld_strict) & ~zone_green;
    zone_lred  = xor(pass_TO_strict, pass_ld_strict);
    zone_dred  = ~pass_TO_strict & ~pass_ld_strict;

    rgb = zeros([size(d_To), 3]);
    rgb(:, :, 1) = 0.65*zone_dred + 0.95*zone_lred + 0.95*zone_yel +
0.20*zone_green;
    rgb(:, :, 2) = 0.10*zone_dred + 0.60*zone_lred + 0.90*zone_yel +
0.70*zone_green;

```

```
    rgb(:, :, 3) = 0.10*zone_dred + 0.60*zone_lred + 0.10*zone_yel +  
    0.30*zone_green;
```

```
end
```

```
=== v3 Parameter Summary ===
```

```
CD0_clean      : 0.0288  
CD0_flaps_TO   : 0.0552 (0.0023 × 0.80 × 30 deg)  
CD0_takeoff    : 0.0840  
CD0_flaps_land : 0.0920 (0.0023 × 0.80 × 50 deg)  
CD0_approach   : 0.1208  
CD0_spoilers   : 0.1109 (1.28 × 0.10 × sin(60 deg))  
CD0_groundroll : 0.2317  
CL_max_TO      : 2.5600 (3.2 × 0.80)  
CL_max_land    : 2.7200 (3.4 × 0.80)  
mu_r           : 0.10 (updated from v2 0.40)  
liftKillFactor : 0.20 (updated from v2 0.30)  
eta_rev        : 0.60 (Raymer Ch.13, updated from v2 0.45)  
d_Lo_pink      : 106.68 m (70% of 500ft)
```

```
=== Aircraft Configuration Summary ===
```

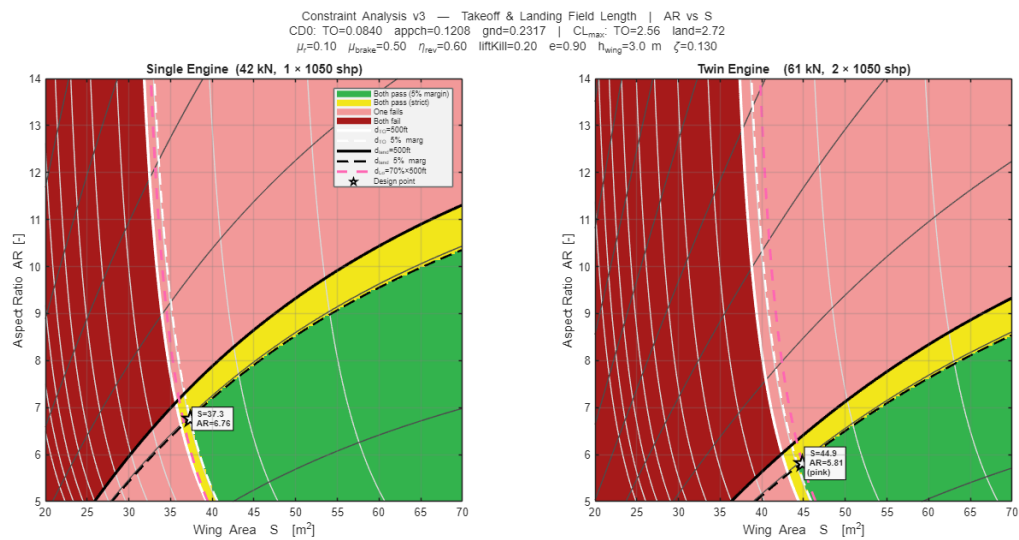
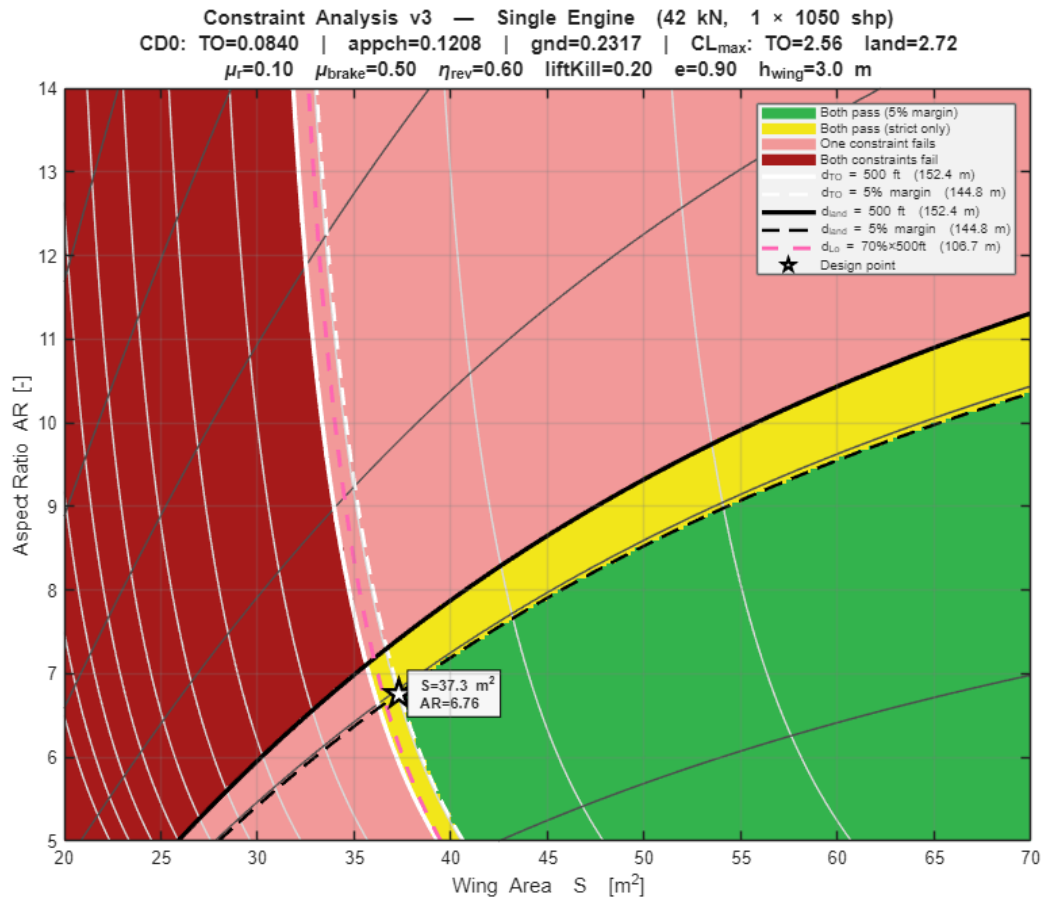
```
Config 1: Single Engine (42 kN, 1 x 1050 shp)  
W_To = 40000 N (4077 kg)  
W_land = 34975 N (3565 kg)  
P_A = 626.4 kW (840 shp total)
```

```
Config 2: Twin Engine (61 kN, 2 x 1050 shp)  
W_To = 60441 N (6161 kg)  
W_land = 52849 N (5387 kg)  
P_A = 1252.8 kW (1680 shp total)
```

```
=== Design Point Summary ===
```

```
Config 1 (Single Engine (42 kN, 1 x 1050 shp)):  
Design basis : 5% TO margin + 5% landing margin  
S = 37.3 m^2  
AR = 6.76  
b = 15.89 m  
d_TO = 144.9 m (475 ft)  
d_Lo = 102.4 m (336 ft) [67% of 500ft]  
d_land = 144.7 m (475 ft)  
L/D (appch) = 5.358  
Glide angle = 10.57 deg (cot(gamma) = E = 5.358)
```

```
Config 2 (Twin Engine (61 kN, 2 x 1050 shp)):  
Design basis : pink (d_Lo = 70% x 500ft) + 5% landing margin  
S = 44.9 m^2  
AR = 5.81  
b = 16.15 m  
d_TO = 141.8 m (465 ft)  
d_Lo = 106.7 m (350 ft) [70% of 500ft]  
d_land = 144.5 m (474 ft)  
L/D (appch) = 4.765  
Glide angle = 11.85 deg (cot(gamma) = E = 4.765)
```



LandingDist.m

```
%{
Landing distance trade study for Arctic STOL aircraft.
Generates a contour heatmap of total landing distance as a function of
wing area (S) and aspect ratio (AR).

Landing distance is broken into two phases:
    d_landing = d_air + d_ground

    d_air      : Analytical air-phase distance (obstacle to touchdown).
                  Combines 50 ft obstacle clearance and flare geometry,
                  scaled by the approach L/D ratio ( $E = CL_{max} / CD$ ).
                  NOTE: High L/D (efficient wing) produces a LONGER d_air
                  because the glide slope is flatter. This is the fundamental
                  STOL tension – aerodynamic efficiency works against short-
field
                  landing performance.

    d_ground : Ground roll from  $V_{TD} = 1.3 * V_{stall}$  to rest.
                  Retarding forces: wheel brakes + aerodynamic drag + reverse
thrust.
                  Aero forces and  $T_{rev}$  evaluated at average speed  $0.7 * V_{TD}$ .
                   $T_{rev}$  uses the same shaft-power method as TakeoffDist:
                       $T_A = P_A / (0.7 * V_{TD})$       [forward thrust at avg speed]
                       $T_{rev} = \eta_{rev} * T_A$         [reverse thrust, beta-
pitch]
                       $T_A$  and  $T_{rev}$  therefore change at every (AR, S) cell as
                       $V_{TD}$  varies with S through  $V_{stall}$ .

CD0 is split into two terms – one per flight phase:
    CD0_approach = CD0_clean + CD0_flaps      [used for d_air only]
    CD0_groundroll = CD0_approach + CD0_spoilers [used for d_ground only]
    CD0_flaps      =  $0.0023 * flapSpanFraction * flapDeflection_{deg}$ 
    CD0_spoilers   =  $Cd_{spoiler\_normal} * (S_{spoiler}/S_{ref}) * \sin(\theta_{spoiler})$ 

Method references:
    Anderson, "Introduction to Flight", Landing Performance
    Raymer,   "Aircraft Design: A Conceptual Approach", Ch. 17
    Raymer,   Ch. 13 (propulsive reversal, turboprop reverse-pitch
efficiency)

ITERATION NOTE:
    W_To, SHP, CD0_clean, and CL_max are estimates at this stage.
    Re-run every weight loop iteration as these values converge.
%}

clear; clc;
```

SECTION 1 — USER INPUTS (update every design iteration)

```
% --- Takeoff weight and fuel accounting ---
% W_To is the primary driver of V_stall and all force terms.
W_To          = 40000;    % Max takeoff gross weight [N]
                        % Update each iteration as weight estimate refines
fuel_fraction = 0.132;    % W_fuel / W_To  (total usable fuel as fraction of
MTOW)
                        % Updated from Breguet range calculation each
iteration
W_fuel_reserve = 500;    % Weight of fuel retained at landing [N]
                        % Represents trapped/unusable fuel in lines and
tanks
                        % Not a fraction — set as an absolute weight so
it
                        % stays physically fixed regardless of
fuel_fraction
                        % Increase to model IFR divert reserve (heavier
landing)

% --- Powerplant ---
% Same engine as TakeoffDist — SHP does not change between missions.
% T_A at landing is lower than at takeoff because V_TD > V_avg_TO,
% i.e. the turboprop produces less thrust at the higher landing speed.
SHP           = 1050;    % Total shaft horsepower, BOTH engines [hp]
                        % e.g. 2x PT6A-27 (~620 SHP each) — update after engine
sel.
eta_prop       = 0.80;    % Propeller efficiency during ground roll [--]
                        % Conservative value for partial-reverse beta range

% Convert SHP to available propulsive power [W]
P_shaft = SHP * 745.7;    % shaft power [W]  (1 hp = 745.7 W exactly)
P_A      = eta_prop * P_shaft; % available propulsive power [W]
% NOTE: T_A = P_A / (0.7*V_TD) is computed INSIDE the sweep loop because
% V_TD changes with S at every cell (via V_stall → V_TD = 1.3*V_stall).

% --- Thrust reversal (PT6A-series turboprop, reverse-pitch propeller) ---
% PT6A turboprops achieve reverse thrust by driving blades into the
% beta / reverse-pitch range. No cascade hardware required.
% eta_rev = 0.60 per Raymer, "Aircraft Design: A Conceptual Approach",
% Ch. 13 (propulsion reversal analysis for turboprop/propeller
```

```

installations).
% This is higher than the 0.45 used in the old script; 0.60 is Raymer's
% specific value for reverse-pitch propellers versus 0.45 for jet thrust
% reversers, making it the more physically appropriate source for this
design.
eta_rev = 0.60; % Reverse thrust effectiveness, PT6A beta-pitch [--]
% T_rev = eta_rev * T_A is computed inside the sweep loop (T_A varies per
cell)

% --- Drag: clean configuration ---
CD0_clean = 0.027; % Parasite drag coefficient in clean (cruise) config
[--]
% Refined from drag buildup (see Appendix A)

% --- Drag: flap contribution (landing - full deflection) ---
% Empirical formula for flap parasite drag increment:
% CD0_flaps = 0.0023 * (flapSpanFraction) * (flapDeflection_deg)
% Source: Raymer, "Aircraft Design: A Conceptual Approach", flap drag tables.
% Landing uses full flap deflection (50 deg) vs. takeoff partial (30 deg),
% producing a significantly larger CD0 increment. This is physically correct
% and intentional - high drag on approach shortens d_air by steepening the
% glide slope (reducing E = CL/CD), which is the desired STOL behaviour.
flapSpanFraction = 0.80; % Fraction of semi-span covered by flaps [--]
flapDeflection_deg = 50; % Full landing flap deflection [deg]
% Higher than takeoff (30 deg) - full

deployment
CD0_flaps = 0.0023 * flapSpanFraction * flapDeflection_deg; % [--]
CD0_approach = CD0_clean + CD0_flaps; % Total parasite drag at approach
[--]
% Used for d_air only (spoilers not
yet deployed)

% --- Drag: spoiler contribution (ground roll only) ---
% Spoilers deploy on touchdown to kill lift and maximise braking normal
force.
% They are NOT active during the airborne approach phase (d_air uses
CD0_approach).
% Formula (Raymer drag buildup, spoiler panel):
% Delta_CD0 = (Cd_spoiler_normal * S_spoiler_total * sin(theta)) / S_ref
% Because S_spoiler_total = (S_spoiler/S_ref) * S_ref, S_ref cancels:
% Delta_CD0 = Cd_spoiler_normal * (S_spoiler/S_ref) * sin(theta_spoiler)
% CD0_spoilers is therefore a geometric constant independent of the S sweep.
Cd_spoiler_normal = 1.28; % Normal-force drag coeff of spoiler panel [--]
% Flat-plate perpendicular-to-flow value
S_spoiler_ratio = 0.1; % S_spoiler_total / S_ref [--] (10% of wing
area)
theta_spoiler_deg = 60; % Spoiler deflection angle at touchdown [deg]
theta_spoiler_rad = theta_spoiler_deg * pi/180;

CD0_spoilers = Cd_spoiler_normal * S_spoiler_ratio *
sin(theta_spoiler_rad); % [--]
CD0_groundroll = CD0_approach + CD0_spoilers; % Parasite drag during
ground roll [--]
% = CD0_clean + CD0_flaps

```

```

+ CD0_spoilers

% --- Lift kill factor (ground roll) ---
% Spoiler deployment collapses the lift distribution, reducing effective CL
% during the ground roll. This increases gear normal force and therefore
% available braking friction. liftKillFactor = fraction of CL_max retained.
% 0.20 means only 20% of approach lift acts on the wing during ground roll.
% Consistent with CA_TWvsWS_To_Land.m spoiler assumption.
liftKillFactor = 0.20;    % Fraction of CL_max_land retained with spoilers
[--]

% --- Lift: landing configuration ---
% CL_max_land uses an 80% margin on the airfoil/flap CLmax per RFP §4.3.
% Base value 3.4 reflects a high-lift STOL airfoil + full Fowler flap;
% update once airfoil selection is finalised.
CL_max_land = 3.4 * 0.80;    % Aircraft CL_max in landing config [--] = 2.72

% --- Aerodynamic efficiency ---
e = 0.9;    % Oswald efficiency factor [--] (per RFP §4.3, clean config)

% --- Ground roll braking ---
mu_brake = 0.5;    % Braking friction coefficient (gravel runway, per RFP)
[--]
                % Consider running mu_brake = 0.3 as a wet/icy gravel
                % sensitivity check for winter operations

% --- Atmosphere ---
rho_sea = 1.225;    % Sea-level ISA air density [kg/m^3]
g        = 9.81;    % Gravitational acceleration [m/s^2]

% --- Heatmap sweep ranges ---
AR_range = linspace(6, 14, 80);    % Aspect ratio sweep [--]
S_range  = linspace(20, 65, 80);    % Wing reference area [m^2]

```

```

=====
=====

```

SECTION 2 — DERIVED LANDING WEIGHT AND CONSOLE HEADER

```

=====
=====

```

```

W_fuel_total  = fuel_fraction * W_To;                % Total usable fuel
[N]
W_fuel_burned = W_fuel_total - W_fuel_reserve;        % Fuel consumed on
mission [N]
W_landing     = W_To - W_fuel_burned;                % Aircraft landing

```

```

weight [N]
% Expanded: W_landing = W_To*(1 - fuel_fraction) + W_fuel_reserve

d_land_req = 152.4;           % 500 ft field-length requirement [m]
d_land_marg = d_land_req * 0.90; % 10% design margin target (= 137.2 m) [m]

fprintf('=== LandingDist.m - Parameter Summary ===\n')
fprintf(' W_To           : %6.0f N  (%.0f kg)\n', W_To, W_To/g)
fprintf(' fuel_fraction    : %.3f → W_fuel = %.0f N\n', fuel_fraction,
W_fuel_total)
fprintf(' W_fuel_reserve     : %.0f N  (trapped/unusable fuel)\n',
W_fuel_reserve)
fprintf(' W_fuel_burned      : %.0f N\n', W_fuel_burned)
fprintf(' LANDING WEIGHT     : %.0f N  (%.0f kg,  %.1f%% of MTOW)\n', ...
W_landing, W_landing/g, 100*W_landing/W_To)
fprintf(' SHP (total)        : %6.0f hp → P_shaft = %.1f kW\n', SHP,
P_shaft/1e3)
fprintf(' eta_prop           : %.2f → P_A = %.1f kW\n', eta_prop, P_A/1e3)
fprintf(' eta_rev            : %.2f  (Raymer Ch.13, reverse-pitch turboprop)
\n', eta_rev)
fprintf(' CD0_clean          : %.4f\n', CD0_clean)
fprintf(' flapSpan%          : %.0f%%\n', flapSpanFraction*100)
fprintf(' flapDeflect        : %.0f deg  (full landing flap)\n',
flapDeflection_deg)
fprintf(' CD0_flaps          : %.4f  (= 0.0023 × %.2f × %.0f)\n', ...
CD0_flaps, flapSpanFraction, flapDeflection_deg)
fprintf(' CD0_approach       : %.4f  (clean + flaps, air phase)\n',
CD0_approach)
fprintf(' Cd_spoiler_norm    : %.2f\n', Cd_spoiler_normal)
fprintf(' S_spoiler/S_ref    : %.2f\n', S_spoiler_ratio)
fprintf(' theta_spoiler      : %.0f deg\n', theta_spoiler_deg)
fprintf(' CD0_spoilers       : %.4f  (= %.2f × %.2f × sin(%.0f deg))\n', ...
CD0_spoilers, Cd_spoiler_normal, S_spoiler_ratio, theta_spoiler_deg)
fprintf(' CD0_groundroll    : %.4f  (clean + flaps + spoilers, ground roll)
\n', CD0_groundroll)
fprintf(' liftKillFactor     : %.2f  (CL fraction retained with spoilers)\n',
liftKillFactor)
fprintf(' CL_max_land        : %.4f  (= 3.4 × 0.80)\n', CL_max_land)
fprintf(' e                  : %.2f\n', e)
fprintf(' mu_brake           : %.2f  (gravel runway, per RFP)\n', mu_brake)
fprintf(' d_req              : %.1f m  (500 ft)\n', d_land_req)
fprintf(' d_margin           : %.1f m  (10%% margin)\n\n', d_land_marg)

=== LandingDist.m - Parameter Summary ===
W_To           : 40000 N  (4077 kg)
fuel_fraction   : 0.132 → W_fuel = 5280 N
W_fuel_reserve  : 500 N  (trapped/unusable fuel)
W_fuel_burned   : 4780 N
LANDING WEIGHT  : 35220 N  (3590 kg,  88.0% of MTOW)
SHP (total)     : 1050 hp → P_shaft = 783.0 kW
eta_prop        : 0.80 → P_A = 626.4 kW
eta_rev         : 0.60 (Raymer Ch.13, reverse-pitch turboprop)
CD0_clean       : 0.0270
flapSpan%       : 80%

```

```

flapDeflect      : 50 deg  (full landing flap)
CD0_flaps        : 0.0920  (= 0.0023 × 0.80 × 50)
CD0_approach     : 0.1190  (clean + flaps, air phase)
Cd_spoiler_norm  : 1.28
S_spoiler/S_ref  : 0.10
theta_spoiler    : 60 deg
CD0_spoilers     : 0.1109  (= 1.28 × 0.10 × sin(60 deg))
CD0_groundroll   : 0.2299  (clean + flaps + spoilers, ground roll)
liftKillFactor   : 0.20  (CL fraction retained with spoilers)
CL_max_land      : 2.7200  (= 3.4 × 0.80)
e                : 0.90
mu_brake         : 0.50  (gravel runway, per RFP)
d_req            : 152.4 m  (500 ft)
d_margin         : 137.2 m  (10% margin)

```

```

=====
=====

```

SECTION 3 — LANDING DISTANCE SWEEP (AR x S double loop)

```

=====
=====

```

```

d_landing_map = zeros(length(AR_range), length(S_range));
d_air_map     = zeros(length(AR_range), length(S_range)); % component
breakout
d_ground_map  = zeros(length(AR_range), length(S_range)); % component
breakout
T_A_map       = zeros(length(AR_range), length(S_range)); % available
thrust map
T_rev_map     = zeros(length(AR_range), length(S_range)); % reverse thrust
map

for i = 1:length(AR_range)
    for j = 1:length(S_range)

        AR = AR_range(i);
        S  = S_range(j);

        % Induced drag factor
        k = 1 / (pi * AR * e); % [--]

        % ---- STALL AND APPROACH SPEEDS ----
        % Stall speed at landing weight, sea level, full-flap CL_max
        V_stall = sqrt(2*W_landing / (rho_sea*S*CL_max_land)); % [m/s]

```

```

% Touchdown speed: 1.3 * Vstall (FAR 23 landing convention)
V_TD = 1.3 * V_stall; % [m/s]

% Average ground roll speed – same 0.7 factor as TakeoffDist
V_avg_gnd = 0.7 * V_TD; % [m/s]

% ---- AVAILABLE AND REVERSE THRUST (speed-dependent) ----
% Mirroring TakeoffDist methodology: T = P_A / V at average speed.
% Using V_avg_gnd = 0.7*V_TD as the representative ground roll speed.
% T_A is HIGHER at lower V (turboprop physics), so it changes at
every
% cell as V_TD shifts with S. Larger S → lower V_stall → lower V_TD
% → lower V_avg_gnd → higher T_A and T_rev.
T_A = P_A / V_avg_gnd; % forward thrust at avg gnd speed
[N]
T_rev = eta_rev * T_A; % effective reverse thrust [N]
% (Raymer Ch.13: 0.60 for beta-
pitch)

% ---- DRAG POLAR (landing / approach configuration) ----
% Total CD at CL_max_land – operating point on approach and ground
roll
CD = CD0_approach + k*(CL_max_land^2); % [--]

% Approach L/D at CL_max.
% STOL trade-off: high E = flat glide = longer d_air.
% Full landing flap (50 deg) deliberately increases CD0_approach to
% reduce E and steepen the glide – shortening d_air at the cost of
% higher approach drag. This is the correct STOL design intent.
E = CL_max_land / CD; % [--]

% ---- AIR PHASE (d_air) ----
% '15' = 50 ft obstacle height (15.24 m, rounded per convention)
% '0.133*Vstall^2/(2g)' captures the flare geometry contribution
% Scaled by E: a high-L/D aircraft covers more ground in the air
d_air = E * (15 + (0.133*(V_stall^2)/(2*g))); % [m]

% ---- GROUND ROLL (d_ground) ----
% Separate drag polar for ground roll: CD0_groundroll includes
spoiler term.
% Spoilers are deployed at touchdown – NOT during the airborne
approach.
% d_air used CD0_approach; d_ground uses CD0_groundroll.
CD_gnd = CD0_groundroll + k*(CL_max_land^2); % ground roll drag
polar [--]

% Evaluate aero forces at V_avg_gnd = 0.7*V_TD
q_landing = 0.5 * rho_sea * (V_avg_gnd^2); % dynamic pressure [Pa]

% Lift during ground roll – multiplied by liftKillFactor (0.20) to
% account for spoiler deployment collapsing the lift distribution.
% Reduced lift → higher normal force on gear → more braking friction.
L_landing = q_landing * S * CL_max_land * liftKillFactor; % [N]

```

```

    % Aerodynamic drag during ground roll – uses CD0_groundroll (with
    spoilers)
    D_landing = q_landing * S * CD_gnd;    % [N]

    % Denominator guard: all three retarding forces must be positive.
    % If T_rev alone exceeds the other terms this is non-physical –
    % should not occur at realistic SHP values but worth checking.
    denom = D_landing + T_rev + mu_brake*(W_landing - L_landing);

    if denom <= 0
        % Cannot decelerate – guard against non-physical parameter
        combinations
        d_landing_map(i,j) = Inf;
        d_air_map(i,j)     = d_air;
        d_ground_map(i,j)  = Inf;
        T_A_map(i,j)       = T_A;
        T_rev_map(i,j)     = T_rev;
        continue
    end

    % Ground roll distance.
    % Factor 1.69 is the landing analogue of the 1.44 takeoff factor,
    % corrected for starting at V_TD rather than accelerating from rest.
    % T_rev is in the denominator: larger reverse thrust → shorter
    d_ground.
    d_ground = 1.69 * W_landing^2 / ...
        (rho_sea * g * S * CL_max_land * denom);    % [m]

    % ---- TOTAL LANDING DISTANCE ----
    d_landing_map(i,j) = d_air + d_ground;

    % Store components for command-window diagnostics
    d_air_map(i,j)      = d_air;
    d_ground_map(i,j)   = d_ground;
    T_A_map(i,j)        = T_A;
    T_rev_map(i,j)      = T_rev;

end
end

```

SECTION 4 — HEATMAP PLOT

```
[S_grid, AR_grid] = meshgrid(S_range, AR_range);

figure('Name', 'Landing Distance Heatmap', 'Position', [100 100 900 650]);

% Filled colour field – matching TakeoffDist.m style
contourf(S_grid, AR_grid, d_landing_map, 30, 'LineColor', 'none');
colormap(jet);
cb = colorbar;
cb.Label.String = 'Landing Distance d_{land} [m]';
cb.FontSize = 11;
clim([50 200]); % 50 m lower bound (no realistic landing is shorter);
                % 200 m upper bound saturates infeasible/long cells red

hold on;

% Labelled contour lines for readability
[C_all, h_all] = contour(S_grid, AR_grid, d_landing_map, 10, ...
    'LineColor', [0.15 0.15 0.15], 'LineWidth', 0.7);
clabel(C_all, h_all, 'FontSize', 8, 'Color', [0.1 0.1 0.1]);

% 500 ft (152.4 m) requirement boundary – primary design constraint
[C_req, h_req] = contour(S_grid, AR_grid, d_landing_map, ...
    [d_land_req d_land_req], 'r-', 'LineWidth', 2.5);
clabel(C_req, h_req, 'FontSize', 10, 'Color', 'r', ...
    'FontWeight', 'bold', 'LabelSpacing', 300);

% 10% design margin contour in white dashed
[C_marg, h_marg] = contour(S_grid, AR_grid, d_landing_map, ...
    [d_land_marg d_land_marg], 'w--', 'LineWidth', 1.8);
clabel(C_marg, h_marg, 'FontSize', 9, 'Color', 'w', 'LabelSpacing', 300);

xlabel('Wing Reference Area, S [m^2]', 'FontSize', 12);
ylabel('Aspect Ratio, AR', 'FontSize', 12);
title(sprintf('Landing Distance [m] | W_{land} = %.1f kN (W_{TO} = %.1f kN), P_A = %.0f kW (%.0f SHP × \eta_{prop}=%.2f)', ...
    W_landing/1e3, W_To/1e3, P_A/1e3, SHP, eta_prop), ...
    sprintf('CL_{max,land} = %.3f (3.4×0.80), CD0_{app} = %.4f (clean %.4f + flap %.4f), \eta_{rev} = %.2f, \mu_{brake} = %.2f', ...
    CL_max_land, CD0_approach, CD0_clean, CD0_flaps, eta_rev, mu_brake)}, ...
    'FontSize', 10);
```

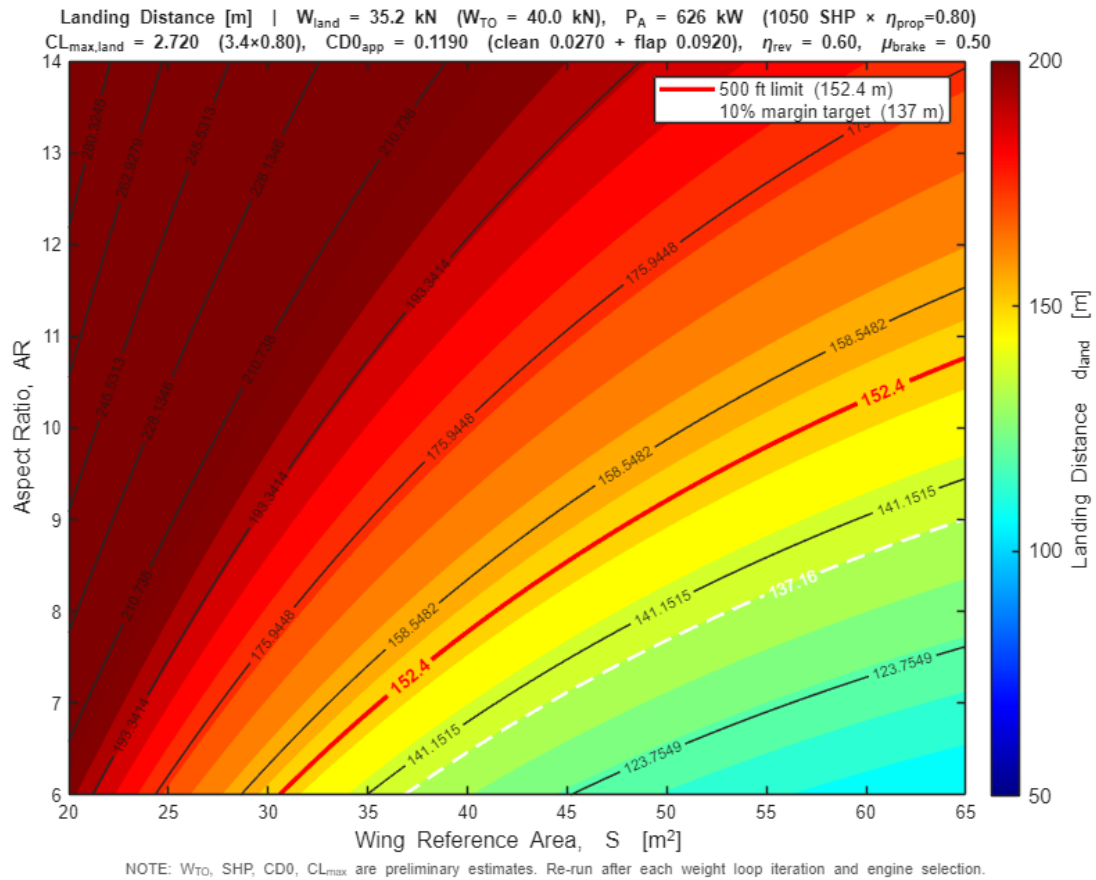
```

h_r = plot(NaN, NaN, 'r-', 'LineWidth', 2.5);
h_m = plot(NaN, NaN, 'w--', 'LineWidth', 1.8);
legend([h_r, h_m], ...
    {sprintf('500 ft limit  (%.1f m)', d_land_req), ...
    sprintf('10% margin target  (%.0f m)', d_land_marg)}, ...
    'Location', 'northeast', 'FontSize', 10);

annotation('textbox', [0.12 0.01 0.78 0.035], ...
    'String', ['NOTE: W_{TO}, SHP, CD0, CL_{max} are preliminary estimates.
    ' ...
    'Re-run after each weight loop iteration and engine
    selection.'], ...
    'FontSize', 8, 'EdgeColor', 'none', 'HorizontalAlignment', 'center', ...
    'Color', [0.35 0.35 0.35]);

hold off;

```



SECTION 5 — PRINTED SUMMARY

```
fprintf('== Landing Distance Results ==\n')
fprintf(' 500 ft requirement      : <= %.1f m\n', d_land_req)
fprintf(' 10%% margin target       : <= %.1f m\n', d_land_marg)

finite_vals = d_landing_map(isfinite(d_landing_map));
if ~isempty(finite_vals)
    fprintf(' Min distance (sweep)   : %.1f m  (%.0f ft)\n', ...
        min(finite_vals), min(finite_vals)/0.3048)
    fprintf(' Max finite distance   : %.1f m  (%.0f ft)\n', ...
        max(finite_vals), max(finite_vals)/0.3048)
    n_pass = nnz(d_landing_map(:) <= d_land_req);
    n_tot  = numel(d_landing_map);
    fprintf(' Feasible cells        : %d / %d  (%.0f%%)\n\n', ...
        n_pass, n_tot, 100*n_pass/n_tot)
else
    fprintf(' WARNING: No feasible cells found. Check SHP and W_To.\n\n')
end

% Mid-range sample point – full component breakdown to command window
AR_s  = AR_range(round(end/2));
S_s   = S_range(round(end/2));
k_s   = 1/(pi*AR_s*e);
Vs_s  = sqrt(2*W_landing/(rho_sea*S_s*CL_max_land));
VTD_s = 1.3 * Vs_s;
Vag_s = 0.7 * VTD_s;
TA_s  = P_A / Vag_s;
Tr_s  = eta_rev * TA_s;
% Air phase – uses CD0_approach (no spoilers airborne)
CD_s   = CD0_approach + k_s*CL_max_land^2;
E_s    = CL_max_land / CD_s;
da_s   = E_s*(15 + 0.133*Vs_s^2/(2*g));

% Ground roll – uses CD0_groundroll (spoilers) and liftKillFactor
CD_gnd_s = CD0_groundroll + k_s*CL_max_land^2;
ql_s     = 0.5*rho_sea*Vag_s^2;
Ll_s     = ql_s*S_s*CL_max_land * liftKillFactor; % lift killed by
spoilers
Dl_s     = ql_s*S_s*CD_gnd_s;
dn_s     = Dl_s + Tr_s + mu_brake*(W_landing - Ll_s);
dg_s     = 1.69*W_landing^2 / (rho_sea*g*S_s*CL_max_land*dn_s);
```

```

fprintf(' --- Sample point: AR = %.1f, S = %.1f m^2 ---\n', AR_s, S_s)
fprintf(' V_stall      = %.2f m/s  (%.1f kts)\n', Vs_s, Vs_s/0.5144)
fprintf(' V_TD         = %.2f m/s  (%.1f kts)\n', VTD_s, VTD_s/0.5144)
fprintf(' V_avg_gnd      = %.2f m/s  (%.1f kts)\n', Vag_s, Vag_s/0.5144)
fprintf(' T_A (V_avg)     = %.1f N   (%.2f kN)\n', TA_s, TA_s/1e3)
fprintf(' T_rev          = %.1f N   (%.2f kN)  (eta_rev=%.2f)\n', Tr_s, Tr_s/
1e3, eta_rev)
fprintf(' --- Air phase (no spoilers) ---\n')
fprintf(' CD0_approach    = %.4f\n', CD0_approach)
fprintf(' CD (approach)   = %.4f\n', CD_s)
fprintf(' L/D (appch)     = %.3f\n', E_s)
gamma_s = atan(1/E_s) * (180/pi); % cot(gamma) = E → tan(gamma) = 1/E
fprintf(' Glide angle    = %.2f deg  (cot(gamma) = L/D = %.3f)\n', gamma_s,
E_s)
fprintf(' d_air          = %.1f m   (%.0f ft)\n', da_s, da_s/0.3048)
fprintf(' --- Ground roll (spoilers + liftKill=%.2f) ---\n', liftKillFactor)
fprintf(' CD0_groundroll= %.4f  (appch + spoilers %.4f)\n', CD0_groundroll,
CD0_spoilers)
fprintf(' CD (gnd roll)  = %.4f\n', CD_gnd_s)
fprintf(' L_landing      = %.1f N   (%.0f%% of no-spoiler lift)\n', Ll_s,
liftKillFactor*100)
fprintf(' D_landing      = %.1f N\n', Dl_s)
fprintf(' d_ground       = %.1f m   (%.0f ft)\n', dg_s, dg_s/0.3048)
fprintf(' d_landing      = %.1f m   (%.0f ft)\n', da_s+dg_s, (da_s+dg_s)/
0.3048)
if da_s+dg_s <= d_land_req
    fprintf(' STATUS: PASS  (%.1f m below limit)\n', d_land_req-(da_s+dg_s))
else
    fprintf(' STATUS: FAIL  (%.1f m over limit)\n', (da_s+dg_s)-d_land_req)
end

=== Landing Distance Results ===
500 ft requirement      : <= 152.4 m
10% margin target       : <= 137.2 m
Min distance (sweep)    : 106.4 m  (349 ft)
Max finite distance     : 297.7 m  (977 ft)
Feasible cells          : 1695 / 6400  (26%)

--- Sample point: AR = 9.9, S = 42.2 m^2 ---
V_stall      = 22.38 m/s  (43.5 kts)
V_TD         = 29.09 m/s  (56.6 kts)
V_avg_gnd    = 20.36 m/s  (39.6 kts)
T_A (V_avg)   = 30759.5 N  (30.76 kN)
T_rev        = 18455.7 N  (18.46 kN)  (eta_rev=0.60)
--- Air phase (no spoilers) ---
CD0_approach  = 0.1190
CD (approach) = 0.3820
L/D (appch)   = 7.120
Glide angle   = 7.99 deg  (cot(gamma) = L/D = 7.120)
d_air         = 131.0 m   (430 ft)
--- Ground roll (spoilers + liftKill=0.20) ---
CD0_groundroll= 0.2299  (appch + spoilers 0.1109)
CD (gnd roll) = 0.4928
L_landing     = 5833.1 N  (20% of no-spoiler lift)

```

```
D_landing      = 5284.6 N
d_ground       = 39.5 m   (130 ft)
d_landing      = 170.5 m   (559 ft)
STATUS: FAIL   (18.1 m over limit)
```

Published with MATLAB® R2025b

TakeoffDist.m

```
%{
Takeoff distance trade study for Arctic STOL aircraft.
Generates a contour heatmap of total takeoff distance as a function of
wing area (S) and aspect ratio (AR).

Takeoff distance is broken into two phases:
    d_To = d_Lo + d_climb

    d_Lo      : Ground roll from rest to liftoff speed V_Lo = 1.2*V_stall.
                Uses the Anderson energy method (factor 1.44).
                Thrust is derived from shaft power at V_avg = 0.7*V_Lo, so
                T_A varies at every (AR, S) cell as V_Lo changes with S.
                Ground-effect reduction on induced drag (Raymer eq. 12.6).

    d_climb   : Linearised constant-speed climb from liftoff to 50 ft obstacle
                clearance. Based on the excess-thrust rate-of-climb method.

CD0 in takeoff configuration is built up from:
    CD0_takeoff = CD0_clean + CD0_flaps
    CD0_flaps   = 0.0023 * flapSpanFraction * flapDeflection_deg

Method references:
    Anderson, "Introduction to Flight", Ch. 6 (Takeoff Performance)
    Raymer,   "Aircraft Design: A Conceptual Approach", Ch. 12 & 17

ITERATION NOTE:
    W_To, SHP, CD0_clean, and CL_max_TO are estimates at this stage.
    Re-run this script every weight loop iteration as these values converge.
%}
```

```
clear; clc;
```

```
=====
=====
```

SECTION 1 — USER INPUTS (update every design iteration)

```
=====
=====
```

```
% --- Weight ---
W_To = 40000; % Max takeoff gross weight [N]
            % Update each iteration as weight estimate refines
```

```

% --- Powerplant ---
% Shaft horsepower is the primary engine sizing input for a turboprop.
% Converting to available propulsive power via propeller efficiency eta_prop.
SHP      = 1050;    % Total shaft horsepower, BOTH engines [hp]
                % e.g. 2x PT6A-27 (~620 SHP each) → 1240 SHP
                %      2x PT6A-20 (~578 SHP each) → 1156 SHP
                % Refine after engine selection.
eta_prop = 0.80;    % Propeller efficiency during ground roll / initial climb
                % 0.80 is a reasonable mid-range value for a fixed-pitch
                % or coarse-pitch setting during the takeoff roll.
                % Fine-pitch / variable-pitch props may reach 0.85 at
V_Lo.

% Convert SHP to available shaft power [W], then to propulsive power [W]
P_shaft = SHP * 745.7;    % shaft power [W]  (1 hp = 745.7 W exactly)
P_A      = eta_prop * P_shaft;    % available propulsive power [W]
% NOTE: T_A = P_A / V is computed INSIDE the sweep loop because V_avg changes
% with S (via V_stall → V_Lo → V_avg = 0.7*V_Lo). This is the key difference
% from TradestudyTakeoff_S_AR.m, which used a fixed T_A for all cells.

% --- Drag: clean configuration ---
CD0_clean = 0.027;    % Parasite drag coefficient in clean (cruise) config
[--]
                % Typical for this aircraft class: 0.025-0.035.
                % Refine after drag buildup (see Appendix A).

% --- Drag: flap contribution ---
% Empirical formula for flap parasite drag increment:
%   CD0_flaps = 0.0023 * (flapSpanFraction) * (flapDeflection_deg)
% Source: Raymer, "Aircraft Design: A Conceptual Approach", flap drag tables.
% This is a first-order estimate; a full drag polar (from Appendix A) will
% be more accurate once flap chord and hinge geometry are defined.
flapSpanFraction = 0.80;    % Fraction of half-span covered by flaps [--]
                % 0.80 means flaps extend over 80% of semi-
span
flapDeflection_deg = 30;    % Flap deflection angle for takeoff setting
[deg]
                % Typical takeoff: 15-30 deg (partial flap)
                % Do NOT use full landing deflection here
CD0_flaps = 0.0023 * flapSpanFraction * flapDeflection_deg;    % [--]
CD0_takeoff = CD0_clean + CD0_flaps;    % Total parasite drag at takeoff [--]

% --- Lift: takeoff configuration ---
% CL_max_takeoff uses a 80% margin on the airfoil/flap CLmax per RFP §4.3.
% The base value 3.2 assumes an aggressive high-lift airfoil + Fowler flaps;
% update once airfoil selection is finalised.
CL_max_TO = 3.2 * 0.80;    % Aircraft CL_max in takeoff config [--] = 2.56

% --- Aerodynamic efficiency ---
e = 0.9;    % Oswald efficiency factor [--]
                % 0.9 for clean config per RFP; reduce to 0.8 if wing stores
added

```

```

% --- Wing height above ground (ground-effect correction) ---
% Raymer eq. 12.6: phi → 0 deep in ground effect (low wing height relative
to span)
%                               phi → 1 out of ground effect (high wing)
% High-wing STOL configuration assumed.
h_wing = 3.0;    % Wing height above ground [m] – update from CAD geometry

% --- Rolling friction ---
% ⚠ DESIGN PHILOSOPHY WARNING:
%   mu_r = 0.1 is used here as requested. Note that TakeoffDistance.m and
%   the constraint analysis (CA_TWvsWS_To_Land.m) both use mu_r = 0.40 for
%   gravel. Published values for gravel/unprepared surfaces are typically
%   0.04-0.10 (hard-packed) to 0.3-0.5 (soft/loose gravel).
%   mu_r = 0.4 in the other scripts appears excessively high for a hard-
packed
%   gravel strip (more representative of soft sand or turf) and likely
%   over-penalises the ground roll.
%   mu_r = 0.1 (hard-packed gravel/compacted dirt) is more consistent with
%   published data for Arctic gravel strips (e.g., USAF AFSC runway
friction
%   data for compacted gravel: mu_r ≈ 0.05-0.12).
%   RECOMMENDATION: resolve this discrepancy between scripts before
finalising
%   the constraint analysis – it will significantly shift feasible region
boundaries.
mu_r = 0.1;    % Rolling friction coefficient (hard-packed gravel, takeoff
roll) [--]

% --- Heatmap sweep ranges ---
AR_range = linspace(6, 14, 80);    % Aspect ratio sweep [--]
S_range  = linspace(20, 65, 80);    % Wing reference area [m^2]

```

```

=====
=====

```

SECTION 2 — DERIVED PARAMETERS AND CONSOLE HEADER

```

=====
=====

```

```

g      = 9.81;    % gravitational acceleration [m/s^2]
rho    = 1.225;   % sea-level ISA air density [kg/m^3]
h_obs  = 15.24;   % 50 ft obstacle in metres [m]

d_To_req = 152.4; % 500 ft field-length requirement [m]
d_To_marg = 152.4 * 0.90; % 10% design margin target (= 137.2 m) [m]

```

```

fprintf('=== TakeoffDist.m - Parameter Summary ===\n')
fprintf(' W_To      : %6.0f N  (%.0f kg)\n', W_To, W_To/g)
fprintf(' SHP (total)  : %6.0f hp → P_shaft = %.1f kW\n', SHP, P_shaft/
1e3)
fprintf(' eta_prop     : %.2f → P_A = %.1f kW\n', eta_prop, P_A/1e3)
fprintf(' CD0_clean     : %.4f\n', CD0_clean)
fprintf(' flapSpan%     : %.0f%%\n', flapSpanFraction*100)
fprintf(' flapDeflect   : %.0f deg\n', flapDeflection_deg)
fprintf(' CD0_flaps     : %.4f (= 0.0023 × %.2f × %.0f)\n', ...
CD0_flaps, flapSpanFraction, flapDeflection_deg)
fprintf(' CD0_takeoff   : %.4f (clean + flaps)\n', CD0_takeoff)
fprintf(' CL_max_TO     : %.3f (= 3.2 × 0.80)\n', CL_max_TO)
fprintf(' e             : %.2f\n', e)
fprintf(' h_wing        : %.1f m\n', h_wing)
fprintf(' mu_r          : %.2f (hard-packed gravel)\n', mu_r)
fprintf(' d_req         : %.1f m (500 ft)\n', d_To_req)
fprintf(' d_margin      : %.1f m (~10% margin)\n\n', d_To_marg)

```

```

=== TakeoffDist.m - Parameter Summary ===

```

```

W_To      : 40000 N (4077 kg)
SHP (total) : 1050 hp → P_shaft = 783.0 kW
eta_prop   : 0.80 → P_A = 626.4 kW
CD0_clean  : 0.0270
flapSpan%  : 80%
flapDeflect : 30 deg
CD0_flaps  : 0.0552 (= 0.0023 × 0.80 × 30)
CD0_takeoff : 0.0822 (clean + flaps)
CL_max_TO  : 2.560 (= 3.2 × 0.80)
e          : 0.90
h_wing     : 3.0 m
mu_r       : 0.10 (hard-packed gravel)
d_req      : 152.4 m (500 ft)
d_margin   : 137.2 m (~10% margin)

```

```

=====

```

```

=====

```

SECTION 3 — TAKEOFF DISTANCE SWEEP (AR x S double loop)

```

=====

```

```

=====

```

```

d_To_map = zeros(length(AR_range), length(S_range));
d_Lo_map = zeros(length(AR_range), length(S_range)); % component
breakout
d_climb_map = zeros(length(AR_range), length(S_range)); % component

```

```

breakout
T_A_map      = zeros(length(AR_range), length(S_range));    % available
thrust map

for i = 1:length(AR_range)
    for j = 1:length(S_range)

        AR = AR_range(i);
        S  = S_range(j);

        % Wing span and induced drag factor
        b = sqrt(AR * S);          % wing span [m]
        k = 1 / (pi * AR * e);     % induced drag factor [--]

        % ---- GROUND-EFFECT CORRECTION ----
        % Raymer eq. 12.6: reduces induced drag while wing is near the
ground.
        %  $\phi \rightarrow 0$  means large ground-effect benefit (low  $h_{\text{wing}}$  / large span)
        %  $\phi \rightarrow 1$  means no ground effect (high wing or long span)
        phi = (16*h_wing/b)^2 / ((16*h_wing/b)^2 + 1);    % [--]

        % ---- CHARACTERISTIC SPEEDS ----
        % Stall speed at takeoff weight, sea-level, full-flap  $CL_{\text{max}}$ 
        V_stall = sqrt(2*W_To / (rho*S*CL_max_TO));    % [m/s]

        % Liftoff speed per FAR Part 23:  $V_{\text{Lo}} = 1.2 * V_{\text{stall}}$ 
        V_Lo = 1.2 * V_stall;    % [m/s]

        % Average ground roll speed (energy method):  $V_{\text{avg}} = 0.7 * V_{\text{Lo}}$ 
        % This is the speed at which all average forces are evaluated
        V_avg = 0.7 * V_Lo;    % [m/s]

        % ---- AVAILABLE THRUST (speed-dependent) ----
        % For a constant-power turboprop:  $T = P_A / V$ 
        % Thrust is evaluated at  $V_{\text{avg}}$  because that is the representative
        % speed over the entire ground roll (energy method assumption).
        % IMPORTANT: This  $T_A$  will be different for every (AR, S) cell
        % because  $V_{\text{Lo}}$  (and thus  $V_{\text{avg}}$ ) changes with S through  $V_{\text{stall}}$ .
        % Larger S  $\rightarrow$  lower  $V_{\text{stall}}$   $\rightarrow$  lower  $V_{\text{avg}}$   $\rightarrow$  higher  $T_A$ .
        % This captures the physical behaviour of a constant-power engine:
        % it produces more thrust at lower speeds, benefiting larger wings.
        T_A = P_A / V_avg;    % available thrust at  $V_{\text{avg}}$  [N]

        % ---- DYNAMIC PRESSURES ----
        q_avg = 0.5 * rho * V_avg^2;    % at average ground roll speed [Pa]
        q_Lo  = 0.5 * rho * V_Lo^2;    % at liftoff speed [Pa]

        % ---- AERODYNAMIC FORCES DURING GROUND ROLL ----
        % Lift at  $V_{\text{avg}}$  – reduces normal force and therefore rolling friction
        L_avg = q_avg * S * CL_max_TO;    % [N]

        % Drag at  $V_{\text{avg}}$  – includes ground-effect correction ( $\phi$ ) on induced
drag
        %  $\phi < 1$  reduces  $k_{\text{eff}}$   $\rightarrow$  lower induced drag in ground effect  $\rightarrow$  less

```

```

drag
    D_gnd = q_avg*S*CD0_takeoff + phi*k*W_To^2/(q_avg*S);    % [N]

    % ---- NET GROUND ROLL FORCE ----
    % Net propulsive force: thrust minus aero drag minus rolling
friction.
    % Rolling friction acts on the effective normal force (W - L), which
    % decreases as lift builds - the benefit of high CL_max on takeoff.
    F_net = T_A - D_gnd - mu_r*(W_To - L_avg);    % [N]

    if F_net <= 0
        % Cannot accelerate to V_Lo - cell is infeasible (underpowered /
        % too small a wing for this weight and rolling friction)
        d_To_map(i,j)    = Inf;
        d_Lo_map(i,j)    = Inf;
        d_climb_map(i,j) = Inf;
        T_A_map(i,j)     = T_A;
        continue    % skip to next cell
    end

    % ---- GROUND ROLL DISTANCE (Anderson energy method) ----
    % d_Lo = 1.44 * W^2 / (rho * g * S * CL_max * F_net_avg)
    % Factor 1.44 accounts for the integral from V=0 to V=V_Lo,
    % assuming F_net is approximately constant at its V_avg value.
    d_Lo = (1.44 * W_To^2) / (rho * g * S * CL_max_TO * F_net);    % [m]

    % ---- AERODYNAMIC FORCES AT LIFTOFF (for climb segment) ----
    % Ground effect is neglected at liftoff (conservative - more drag,
    % less climb performance). Thrust re-evaluated at V_Lo for the climb.
    T_A_Lo = P_A / V_Lo;    %
thrust at V_Lo [N]
    D_Lo = q_Lo*S*CD0_takeoff + k*W_To^2/(q_Lo*S);    % drag
at V_Lo [N]
    T_excess = T_A_Lo - D_Lo;    %
excess thrust [N]

    if T_excess <= 0
        % Insufficient excess thrust to sustain climb at V_Lo
        d_To_map(i,j)    = Inf;
        d_Lo_map(i,j)    = d_Lo;
        d_climb_map(i,j) = Inf;
        T_A_map(i,j)     = T_A;
        continue
    end

    % ---- CLIMB DISTANCE TO CLEAR 50 FT OBSTACLE ----
    % Rate of climb from excess-thrust method at V_Lo:
    % ROC = V_Lo * T_excess / W_To
    % Horizontal distance to climb h_obs at constant V_Lo:
    % d_climb = h_obs * V_Lo / ROC = h_obs * W_To / T_excess
    ROC = V_Lo * T_excess / W_To;    % rate of climb [m/s]
    d_climb = h_obs * V_Lo / ROC;    % obstacle clearance distance [m]

    % ---- TOTAL TAKEOFF FIELD LENGTH ----

```

```

        d_To_map(i,j)      = d_Lo + d_climb;
        d_Lo_map(i,j)      = d_Lo;
        d_climb_map(i,j)   = d_climb;
        T_A_map(i,j)       = T_A;    % store for diagnostics

    end
end

```

```

=====
=====

```

SECTION 4 — HEATMAP PLOT

```

=====
=====

```

```

% Cap Inf values for colour scaling – cells above d_cap are infeasible
d_cap = 600;
d_plot = min(d_To_map, d_cap);

[S_grid, AR_grid] = meshgrid(S_range, AR_range);

figure('Name', 'Takeoff Distance Heatmap', 'Position', [100 100 900 650]);

% Filled colour field – use contourf to match LandingDist.m style
contourf(S_grid, AR_grid, d_plot, 30, 'LineColor', 'none');
colormap(jet);
cb = colorbar;
cb.Label.String = 'Takeoff Distance d_{TO} [m]';
cb.FontSize = 11;
clim([50 200]); % fix colour axis: 0 m (blue) to 200 m (red); cells above
200 m saturate red
hold on;

% Labelled contour lines for readability
[C_all, h_all] = contour(S_grid, AR_grid, d_To_map, 10, ...
    'LineColor', [0.15 0.15 0.15], 'LineWidth', 0.7);
clabel(C_all, h_all, 'FontSize', 8, 'Color', [0.1 0.1 0.1]);

% 500 ft (152.4 m) requirement boundary – primary design constraint
[C_req, h_req] = contour(S_grid, AR_grid, d_To_map, ...
    [d_To_req d_To_req], 'r-', 'LineWidth', 2.5);
clabel(C_req, h_req, 'FontSize', 10, 'Color', 'r', ...
    'FontWeight', 'bold', 'LabelSpacing', 300);

% 10% design margin contour in white dashed
[C_marg, h_marg] = contour(S_grid, AR_grid, d_To_map, ...
    [d_To_marg d_To_marg], 'w--', 'LineWidth', 1.8);
clabel(C_marg, h_marg, 'FontSize', 9, 'Color', 'w', 'LabelSpacing', 300);

```

```

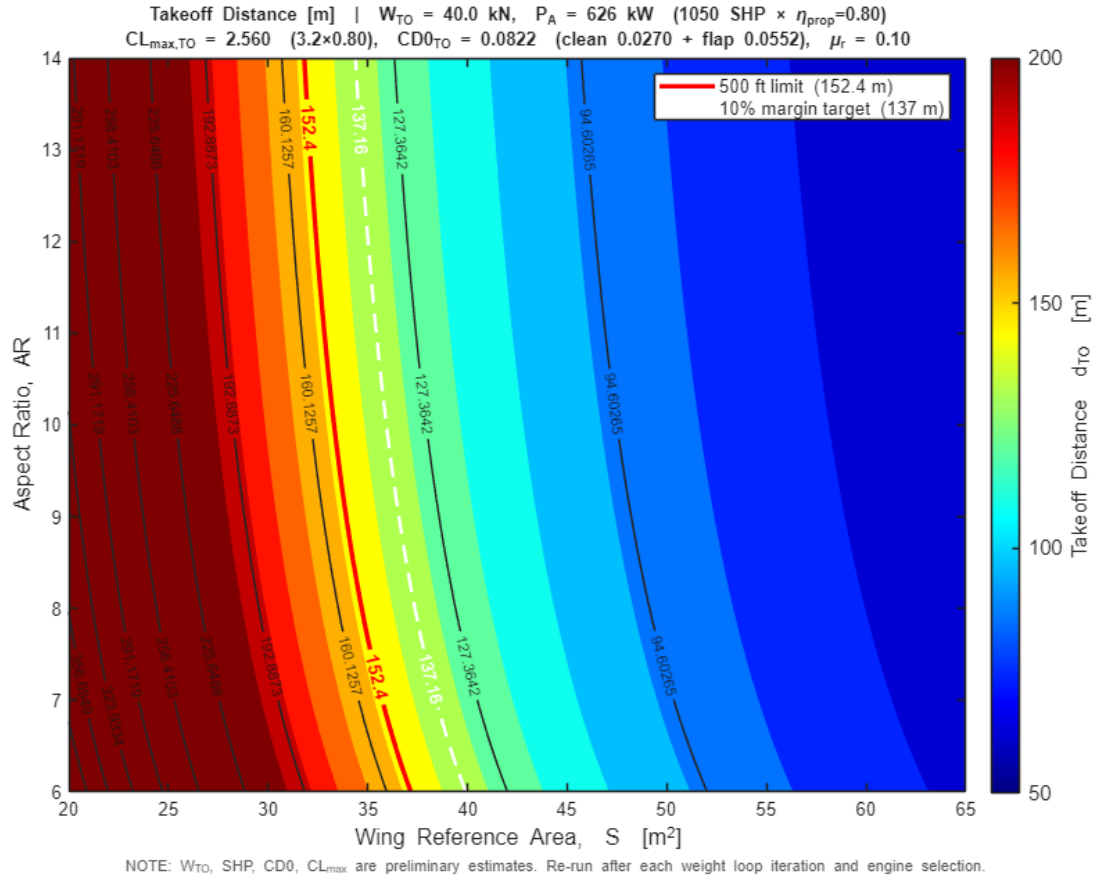
xlabel('Wing Reference Area, S [m^2]', 'FontSize', 12);
ylabel('Aspect Ratio, AR', 'FontSize', 12);
title(sprintf('Takeoff Distance [m] | W_{TO} = %.1f kN, P_A = %.0f kW
(%.0f SHP × \eta_{prop}=%.2f)', ...
    W_To/1e3, P_A/1e3, SHP, eta_prop), ...
    sprintf('CL_{max,TO} = %.3f (3.2×0.80), CD0_{TO} = %.4f (clean
%.4f + flap %.4f), \mu_r = %.2f', ...
    CL_max_TO, CD0_takeoff, CD0_clean, CD0_flaps, mu_r)}, ...
    'FontSize', 10);

h_r = plot(NaN, NaN, 'r-', 'LineWidth', 2.5);
h_m = plot(NaN, NaN, 'w--', 'LineWidth', 1.8);
legend([h_r, h_m], ...
    {sprintf('500 ft limit (%.1f m)', d_To_req), ...
    sprintf('10% margin target (%.0f m)', d_To_marg)}, ...
    'Location', 'northeast', 'FontSize', 10);

annotation('textbox', [0.12 0.01 0.78 0.035], ...
    'String', ['NOTE: W_{TO}, SHP, CD0, CL_{max} are preliminary estimates.
' ...
    'Re-run after each weight loop iteration and engine
selection.'], ...
    'FontSize', 8, 'EdgeColor', 'none', 'HorizontalAlignment', 'center', ...
    'Color', [0.35 0.35 0.35]);

hold off;

```



SECTION 5 — PRINTED SUMMARY

```
fprintf('=== Takeoff Distance Results ===\n')
fprintf(' 500 ft requirement      : <= %.1f m\n', d_To_req)
fprintf(' 10%% margin target        : <= %.1f m\n', d_To_marg)

% Filter out Inf before computing min
feasible_vals = d_To_map(isfinite(d_To_map));
if ~isempty(feasible_vals)
    fprintf(' Min distance (sweep)   : %.1f m (%.0f ft)\n', ...
        min(feasible_vals), min(feasible_vals)/0.3048)
    fprintf(' Max finite distance    : %.1f m (%.0f ft)\n', ...
        max(feasible_vals), max(feasible_vals)/0.3048)
    n_pass = nnz(d_To_map(:) <= d_To_req);
```

```

n_tot = numel(d_To_map);
fprintf(' Feasible cells           : %d / %d  (%.0f%%)\n\n', ...
        n_pass, n_tot, 100*n_pass/n_tot)
else
    fprintf(' WARNING: No feasible cells found. Check SHP and W_To.\n\n')
end

% Mid-range sample point - component breakdown printed to command window
AR_s = AR_range(round(end/2));
S_s = S_range(round(end/2));
b_s = sqrt(AR_s * S_s);
k_s = 1/(pi*AR_s*e);
phi_s = (16*h_wing/b_s)^2 / ((16*h_wing/b_s)^2 + 1);
Vs_s = sqrt(2*W_To/(rho*S_s*CL_max_TO));
VLo_s = 1.2*Vs_s;
Vav_s = 0.7*VLo_s;
TA_s = P_A/Vav_s;
qav_s = 0.5*rho*Vav_s^2;
qLo_s = 0.5*rho*VLo_s^2;
La_s = qav_s*S_s*CL_max_TO;
Dg_s = qav_s*S_s*CD0_takeoff + phi_s*k_s*W_To^2/(qav_s*S_s);
Fn_s = TA_s - Dg_s - mu_r*(W_To - La_s);
TALo_s = P_A/VLo_s;
DLo_s = qLo_s*S_s*CD0_takeoff + k_s*W_To^2/(qLo_s*S_s);
Tex_s = TALo_s - DLo_s;

fprintf(' --- Sample point: AR = %.1f, S = %.1f m^2 ---\n', AR_s, S_s)
fprintf(' b           = %.2f m\n', b_s)
fprintf(' phi (GE)      = %.4f\n', phi_s)
fprintf(' V_stall       = %.2f m/s  (%.1f kts)\n', Vs_s, Vs_s/0.5144)
fprintf(' V_Lo         = %.2f m/s  (%.1f kts)\n', VLo_s, VLo_s/0.5144)
fprintf(' V_avg        = %.2f m/s  (%.1f kts)\n', Vav_s, Vav_s/0.5144)
fprintf(' T_A (V_avg)   = %.1f N   (%.2f kN)\n', TA_s, TA_s/1e3)
fprintf(' T_A (V_Lo)    = %.1f N   (%.2f kN)\n', TALo_s, TALo_s/1e3)
fprintf(' F_net (avg)   = %.1f N\n', Fn_s)
if Fn_s > 0
    dLo_s = 1.44*W_To^2/(rho*g*S_s*CL_max_TO*Fn_s);
    if Tex_s > 0
        ROC_s = VLo_s*Tex_s/W_To;
        dCl_s = h_obs*VLo_s/ROC_s;
        fprintf(' d_Lo (ground) = %.1f m  (%.0f ft)\n', dLo_s, dLo_s/0.3048)
        fprintf(' ROC at V_Lo   = %.2f m/s  (%.0f ft/min)\n', ROC_s, ROC_s/
0.00508)
        fprintf(' d_climb      = %.1f m  (%.0f ft)\n', dCl_s, dCl_s/0.3048)
        fprintf(' d_To TOTAL   = %.1f m  (%.0f ft)\n', dLo_s+dCl_s,
(dLo_s+dCl_s)/0.3048)
        if dLo_s+dCl_s <= d_To_req
            fprintf(' STATUS: PASS  (%.1f m below limit)\n', d_To_req-
(dLo_s+dCl_s))
        else
            fprintf(' STATUS: FAIL  (%.1f m over limit)\n', (dLo_s+dCl_s)-
d_To_req)
        end
    else

```

```

        fprintf('  STATUS: INFEASIBLE - insufficient climb thrust at V_Lo\n')
    end
else
    fprintf('  STATUS: INFEASIBLE - insufficient net force for ground
roll\n')
end

=== Takeoff Distance Results ===
500 ft requirement      : <= 152.4 m
10% margin target       : <= 137.2 m
Min distance (sweep)    : 61.8 m   (203 ft)
Max finite distance     : 422.2 m   (1385 ft)
Feasible cells          : 4439 / 6400   (69%)

--- Sample point: AR = 9.9, S = 42.2 m^2 ---
b                        = 20.49 m
phi (GE)                 = 0.8458
V_stall                  = 24.58 m/s   (47.8 kts)
V_Lo                     = 29.50 m/s   (57.3 kts)
V_avg                    = 20.65 m/s   (40.1 kts)
T_A (V_avg)              = 30334.8 N   (30.33 kN)
T_A (V_Lo)               = 21234.4 N   (21.23 kN)
F_net (avg)              = 23887.6 N
d_Lo (ground)            = 74.3 m     (244 ft)
ROC at V_Lo              = 12.43 m/s   (2447 ft/min)
d_climb                  = 36.2 m     (119 ft)
d_To TOTAL               = 110.4 m     (362 ft)
STATUS: PASS              (42.0 m below limit)

```

Published with MATLAB® R2025b

```

% CA_TWvsWS_To_Land_v2.m
% =====
% CONSTRAINT ANALYSIS v2 – T/W vs W/S with Engine Power Locus
% Single Engine Configuration
%
% NEW IN v2 (vs v1):
%   - Engine power P [shp] is converted to P_A [W] and used to compute
%     the thrust available at each W/S via  $T_A = P_A / (0.7 * V_{Lo}(W/S))$ .
%     This traces a physically meaningful power locus curve through the
%     T/W vs W/S design space.
%   - A pink dotted curve overlays all three figures showing the full
%     engine locus: the T/W achievable at every W/S for this engine.
%   - A pink filled dot marks the current design point, defined by S_ref.
%   - Exact takeoff and landing distances are recomputed at the design
%     point (not interpolated from the grid) and reported with pass/fail.
%
% FIGURES
%   Figure 1 – Side-by-side heatmaps (TO distance | landing distance)
%               with engine locus curve and design point overlaid
%   Figure 2 – Combined zone-shaded constraint diagram with engine locus
%               curve, design point, and 5% margin intersection labelled
%
% VARIABLE ORDERING (most likely to change → least likely):
%   P_shp      – engine changes happen early and drive everything downstream
%   W_To       – weight estimate refines every iteration
%   zeta       – fuel fraction tied to mission/engine iteration
%   AR         – wing architecture; more fundamental than S alone
%   S_ref      – reference wing area (single design point within W/S sweep)
%   CD_0_clean – refines after drag buildup is complete
%   CL_max_TO  – refines after airfoil and flap geometry are finalised
%   CL_max_land – same; landing flap setting may differ from TO
%   e          – Oswald efficiency; relatively stable after AR is set
%   liftKillFactor – spoiler geometry; set once and rarely revisited
%   eta        – propeller efficiency; stable for a given prop design
%   eta_rev    – reversal effectiveness; stable for a given engine/prop
%   mu_r       – RFP constant (gravel); should not change
%   mu_brake   – RFP constant (gravel braking); should not change
%
% DEPENDENCIES
%   None beyond base MATLAB. StdAtmos.m is NOT required for this script.
%
% VERSION: 2.0
% =====

clear; clc; close all;

% =====
% SECTION 1 – DESIGN INPUTS (edit these each iteration)
% =====

% — Propulsion —————
P_shp = 1050;    % rated shaft power per engine [hp]

```

```

        % PT6A-class turboprop. Change when engine is finalised.

% — Weight —————
W_To = 42000; % max takeoff weight, single engine [N]
           % (~4281 kg). Update every weight-loop iteration.

% — Fuel fraction —————
zeta = 0.139; % W_fuel / W_To (both climb + cruise + reserve) [-]
           % Derived from Breguet mission analysis. Revisit when
           % engine SFC and mission profile are finalised.

% — Wing geometry —————
AR = 7; % aspect ratio – fixed for this analysis [-]
       % AR = 7 chosen for STOL structural robustness.
       % Lower AR → shorter span → lighter wing box.
       % Revisit if aeroelastic or range constraints tighten.

S_ref = 45.4; % reference wing area – current design point [m^2]
           % Sets W/S_design = W_To / S_ref.
           % Derived from v2 constraint analysis intersection.
           % Update when wing sizing is revised.

% — Aerodynamics —————
CD_0_clean = 0.030; % zero-lift parasite drag, clean config [-]
                 % Preliminary estimate. Refine with full drag buildup
                 % (fuselage wetted area, empennage, interference).

CL_max_TO = 2.4; % max lift coefficient – takeoff flap [-]
               % = 0.8 * airfoil CL_max per RFP §4.3.
               % Update after airfoil selection and flap sizing.

CL_max_land = 2.6; % max lift coefficient – landing flap [-]
                 % Full flap setting; typically 10-15% above TO value.
                 % Update after airfoil + flap geometry finalised.

e = 0.9; % Oswald efficiency factor (clean, RFP §4.3) [-]
        % Assumed constant; decreases with wing stores (→0.8).

liftKillFactor = 0.30; % fraction of CL_max retained with spoilers [-]
                     % Spoilers destroy lift → more weight on gear →
                     % more braking friction. Revisit after spoiler
design.

% — Propulsion efficiency —————
eta = 0.80; % propeller efficiency [-]
           % Typical for a variable-pitch constant-speed prop.
           % Stable once prop is sized; rarely changes after
selection.

eta_rev = 0.45; % thrust reversal effectiveness (PT6A beta-pitch) [-]
               % Range 0.40-0.55 per manufacturer data.
               % Conservative mid-range; stable after engine finalised.

% — Ground friction —————

```

```

mu_r      = 0.40; % rolling friction - gravel runway (RFP constant) [-]
              % Do not change between missions; fixed by RFP.

mu_brake = 0.50; % braking friction - landing ground roll          [-]
              % Slightly higher than mu_r (brakes locked vs rolling).
              % Use 0.30 for wet/icy gravel sensitivity check.

% =====
% SECTION 2 - DERIVED CONSTANTS (do not edit below this line)
% =====

% --- Physical constants ---
g      = 9.81;      % gravitational acceleration          [m/s^2]
rho_SL = 1.225;     % sea-level ISA air density          [kg/m^3]
h_obs  = 15.24;     % 50 ft obstacle height              [m]
shp2W  = 745.7;     % shaft-hp to Watts conversion       [W/hp]

% --- Field-length requirements ---
d_req  = 152.4;      % 500 ft strict requirement          [m]
d_marg = d_req * 0.95; % 5% design margin (~ 144.78 m)    [m]

% --- Power available ---
% P_A is the shaft power delivered to the propeller disc.
% eta converts shaft power to thrust power: T * V = eta * P_shaft.
P_A = eta * P_shp * shp2W; % thrust power available      [W]

% --- Fuel / landing weight ---
fuel_margin = 0.06; % 6% IFR reserve [-]
W_fuel_burned = zeta * W_To * (1 - fuel_margin); % fuel consumed [N]
W_land = W_To - W_fuel_burned; % landing weight [N]

% --- Drag scaling ---
CD_0_TO = 2.5 * CD_0_clean; % takeoff partial flap → 0.075 [-]
% CD_0_land = 5.0 * CD_0_clean; % full flap + spoiler → 0.150 [-]
CD_0_land = 0.3;
% NOTE: CD_0_land = 0.150 is on the high end for this class
% (typical STOL range 0.060-0.100). Revisit after flap drag buildup.

% --- Wing geometry derived from fixed AR ---
k      = 1 / (pi * AR * e); % induced drag factor (constant) [-]
h_wing = 3.0; % wing height above ground [m]

% --- Design point wing loading ---
WS_design = W_To / S_ref; % W/S at current design point [N/m^2]

% --- Sweep ranges ---
nTW = 120;
nWS = 120;
TW_vec = linspace(0.25, 0.80, nTW); % thrust-to-weight ratio [-]
WS_vec = linspace(400, 1500, nWS); % wing loading [N/m^2]

% =====
% SECTION 3 - ENGINE POWER LOCUS
% =====

```

```

% For a turboprop with fixed shaft power P_A, the available thrust at any
% wing loading is set by the takeoff ground-roll speed:
%
%   T_A(W/S) = P_A / (0.7 * V_Lo(W/S))
%
% where V_Lo = 1.2 * V_stall = 1.2 * sqrt(2 * W/S / (rho * CL_max_TO))
%
% This gives a unique T/W at each W/S, tracing a curve through the design
% space. Higher W/S → higher V_Lo → lower T_A → lower T/W.
% The curve falls monotonically from top-left to bottom-right.

V_stall_locus = sqrt(2 * WS_vec / (rho_SL * CL_max_TO)); % [m/s]
V_Lo_locus    = 1.2 * V_stall_locus; % [m/s]
T_A_locus     = P_A ./ (0.7 * V_Lo_locus); % [N]
TW_locus      = T_A_locus / W_To; % [-]

% — Design point on the locus —————
% Interpolate locus to find T/W at exactly W/S = WS_design
TW_design = interp1(WS_vec, TW_locus, WS_design, 'linear', 'extrap');
T_A_design = TW_design * W_To; % thrust available at design point [N]
S_design   = S_ref; % same as S_ref by definition
b_design   = sqrt(AR * S_design); % span at design point [m]

% =====
% SECTION 4 – EXACT DESIGN POINT DISTANCE CALCULATION
% =====
% Recompute d_TO and d_land analytically at the design point.
% This gives exact values independent of grid resolution.

% — Ground-effect correction —————
phi_design = (16*h_wing/b_design)^2 / ((16*h_wing/b_design)^2 + 1);

% — Takeoff distance at design point —————
V_stall_TO_dp = sqrt(2 * W_To / (rho_SL * S_design * CL_max_TO));
V_Lo_dp       = 1.2 * V_stall_TO_dp;
V_avg_TO_dp   = 0.7 * V_Lo_dp;

q_avg_dp = 0.5 * rho_SL * V_avg_TO_dp^2;
q_Lo_dp  = 0.5 * rho_SL * V_Lo_dp^2;

L_avg_dp = q_avg_dp * S_design * CL_max_TO;
D_gnd_dp = q_avg_dp*S_design*CD_0_TO + phi_design*k*W_To^2/(q_avg_dp*S_design);
F_net_dp = T_A_design - D_gnd_dp - mu_r*(W_To - L_avg_dp);

if F_net_dp <= 0
    d_To_dp = Inf;
    dp_TO_status = 'FAIL (cannot accelerate to V_Lo)';
else
    d_Lo_dp = (1.44*W_To^2) / (rho_SL*g*S_design*CL_max_TO*F_net_dp);
    D_Lo_dp = q_Lo_dp*S_design*CD_0_TO + k*W_To^2/(q_Lo_dp*S_design);
    T_exc_dp = T_A_design - D_Lo_dp;

    if T_exc_dp <= 0

```

```

        d_To_dp = Inf;
        dp_TO_status = 'FAIL (insufficient climb thrust)';
    else
        ROC_dp = V_Lo_dp * T_exc_dp / W_To;
        d_clm_dp = h_obs * V_Lo_dp / ROC_dp;
        d_To_dp = d_Lo_dp + d_clm_dp;
        if d_To_dp <= d_marg
            dp_TO_status = sprintf('PASS - within 5%% margin (%.1f m <= %.1f
m)', d_To_dp, d_marg);
        elseif d_To_dp <= d_req
            dp_TO_status = sprintf('PASS - strict only (%.1f m <= %.1f m,
outside 5%% margin)', d_To_dp, d_req);
        else
            dp_TO_status = sprintf('FAIL (%.1f m > %.1f m)', d_To_dp, d_req);
        end
    end
end

% — Landing distance at design point —————
WS_land_dp = W_land / S_design;
V_stall_ld_dp = sqrt(2 * WS_land_dp / (rho_SL * CL_max_land));
V_TD_dp = 1.3 * V_stall_ld_dp;
V_avg_ld_dp = 0.7 * V_TD_dp;
T_rev_dp = eta_rev * T_A_design;

CD_land_dp = CD_0_land + k * CL_max_land^2;
E_land_dp = CL_max_land / CD_land_dp;
d_air_dp = E_land_dp * (15 + 0.133*V_stall_ld_dp^2/(2*g));

q_land_dp = 0.5 * rho_SL * V_avg_ld_dp^2;
L_land_dp = q_land_dp * S_design * CL_max_land * liftKillFactor;
D_land_dp = q_land_dp * S_design * CD_land_dp;
denom_dp = D_land_dp + T_rev_dp + mu_brake*(W_land - L_land_dp);

if denom_dp <= 0
    d_land_dp = Inf;
    dp_ld_status = 'FAIL (cannot decelerate)';
else
    d_gnd_dp = 1.69*W_land^2 / (rho_SL*g*S_design*CL_max_land*denom_dp);
    d_land_dp = d_air_dp + d_gnd_dp;
    if d_land_dp <= d_marg
        dp_ld_status = sprintf('PASS - within 5%% margin (%.1f m <= %.1f
m)', d_land_dp, d_marg);
    elseif d_land_dp <= d_req
        dp_ld_status = sprintf('PASS - strict only (%.1f m <= %.1f m,
outside 5%% margin)', d_land_dp, d_req);
    else
        dp_ld_status = sprintf('FAIL (%.1f m > %.1f m)', d_land_dp, d_req);
    end
end

% — Console: design point summary —————
fprintf('=== Design Point Summary ===\n')
fprintf(' S_ref : %.1f m^2\n', S_ref)

```

```

fprintf(' W/S design      : %.1f N/m^2\n', WS_design)
fprintf(' T/W design      : %.4f\n',      TW_design)
fprintf(' T_A design        : %.1f N  (%.0f lbf)\n', T_A_design,
T_A_design*0.2248)
fprintf(' P_A              : %.1f kW  (%.0f shp)\n', P_A/1000, P_A/shp2W)
fprintf(' W_land           : %.0f N  (%.0f kg)\n', W_land, W_land/g)
fprintf('---\n')
fprintf(' d_TO   : %.1f m  (%.0f ft)  → %s\n', d_To_dp, d_To_dp/0.3048,
dp_TO_status)
fprintf(' d_land: %.1f m  (%.0f ft)  → %s\n', d_land_dp, d_land_dp/0.3048,
dp_ld_status)
fprintf('=====\n\n')

% =====
% SECTION 5 – HEATMAP SWEEP OVER (T/W, W/S)
% =====

d_To_grid  = zeros(nTW, nWS);
d_land_grid = zeros(nTW, nWS);

for i = 1:nTW
    for j = 1:nWS

        TW = TW_vec(i);
        WS = WS_vec(j);

        T_A = TW * W_To;           % thrust at this T/W point           [N]
        S    = W_To / WS;           % wing area at this W/S           [m^2]
        b    = sqrt(AR * S);        % span                           [m]

        phi = (16*h_wing/b)^2 / ((16*h_wing/b)^2 + 1);

        WS_land = W_land / S;

        % — Takeoff distance —————
        V_stall_TO = sqrt(2 * WS / (rho_SL * CL_max_TO));
        V_Lo      = 1.2 * V_stall_TO;
        V_avg_TO  = 0.7 * V_Lo;

        q_avg = 0.5 * rho_SL * V_avg_TO^2;
        q_Lo  = 0.5 * rho_SL * V_Lo^2;

        L_avg = q_avg * S * CL_max_TO;
        D_gnd = q_avg*S*CD_0_TO + phi*k*W_To^2/(q_avg*S);
        F_net = T_A - D_gnd - mu_r*(W_To - L_avg);

        if F_net <= 0
            d_To_grid(i,j) = Inf;
        else
            d_Lo      = (1.44*W_To^2) / (rho_SL*g*S*CL_max_TO*F_net);
            D_Lo      = q_Lo*S*CD_0_TO + k*W_To^2/(q_Lo*S);
            T_excess  = T_A - D_Lo;

            if T_excess <= 0

```

```

        d_To_grid(i,j) = Inf;
    else
        ROC          = V_Lo * T_excess / W_To;
        d_climb       = h_obs * V_Lo / ROC;
        d_To_grid(i,j) = d_Lo + d_climb;
    end
end

% — Landing distance —————
V_stall_land = sqrt(2 * WS_land / (rho_SL * CL_max_land));
V_TD         = 1.3 * V_stall_land;
V_avg_land   = 0.7 * V_TD;
T_rev        = eta_rev * T_A;

CD_land = CD_0_land + k * CL_max_land^2;
E_land   = CL_max_land / CD_land;
d_air    = E_land * (15 + 0.133*V_stall_land^2/(2*g));

q_land   = 0.5 * rho_SL * V_avg_land^2;
L_land   = q_land * S * CL_max_land * liftKillFactor;
D_land   = q_land * S * CD_land;
denom    = D_land + T_rev + mu_brake*(W_land - L_land);

if denom <= 0
    d_land_grid(i,j) = Inf;
else
    d_ground          = 1.69*W_land^2 /
(rho_SL*g*S*CL_max_land*denom);
    d_land_grid(i,j) = d_air + d_ground;
end

end
end

% =====
% SECTION 6 – ZONE CLASSIFICATION AND RGB IMAGE
% =====

pass_TO_strict = d_To_grid <= d_req;
pass_ld_strict = d_land_grid <= d_req;
pass_TO_marg   = d_To_grid <= d_marg;
pass_ld_marg   = d_land_grid <= d_marg;

zone_green = pass_TO_marg & pass_ld_marg;
zone_yel   = (pass_TO_strict & pass_ld_strict) & ~zone_green;
zone_lred  = xor(pass_TO_strict, pass_ld_strict);
zone_dred  = ~pass_TO_strict & ~pass_ld_strict;

rgb = zeros(nTW, nWS, 3);
rgb(:, :, 1) = 0.65*zone_dred + 0.95*zone_lred + 0.95*zone_yel +
0.20*zone_green;
rgb(:, :, 2) = 0.10*zone_dred + 0.60*zone_lred + 0.90*zone_yel +
0.70*zone_green;
rgb(:, :, 3) = 0.10*zone_dred + 0.60*zone_lred + 0.10*zone_yel +

```

```

0.30*zone_green;

% =====
% SECTION 7 – 5% MARGIN CONTOUR INTERSECTION POINT
% =====
% Find the (W/S, T/W) cell nearest the crossing of both 5% margin lines.
% Minimise:  $f = (d_{TO} - d_{marg})^2 + (d_{land} - d_{marg})^2$ 

d_To_cap = min(d_To_grid, 1e5);
d_land_cap = min(d_land_grid, 1e5);
obj = (d_To_cap - d_marg).^2 + (d_land_cap - d_marg).^2;

[~, lin_idx] = min(obj(:));
[int_row, int_col] = ind2sub([nTW nWS], lin_idx);

int_TW_val = TW_vec(int_row);
int_WS_val = WS_vec(int_col);
int_dTo = d_To_grid(int_row, int_col);
int_dLand = d_land_grid(int_row, int_col);

fprintf('== 5% Margin Contour Intersection ==\n')
fprintf(' T/W : %.4f\n', int_TW_val)
fprintf(' W/S : %.1f N/m^2\n', int_WS_val)
fprintf(' d_TO : %.1f m (%.0f ft)\n', int_dTo, int_dTo/0.3048)
fprintf(' d_land : %.1f m (%.0f ft)\n', int_dLand, int_dLand/0.3048)
fprintf('=====\n\n')

% =====
% SECTION 8 – SHARED PLOT HELPERS
% =====
% Colours and sizes defined once so all three figures are consistent.

pink_rgb = [1.00 0.41 0.71]; % hot pink for locus curve and design dot
locus_lw = 1.8; % locus curve line width
locus_ls = ':'; % locus curve line style (dotted)
dp_msz = 12; % design point marker size
int_msz = 14; % intersection marker size

d_To_plot = min(d_To_grid, 600);
d_land_plot = min(d_land_grid, 600);
d_clim = [40, 300];

% Clip engine locus T/W to plot y-axis range so it doesn't run off the axes
TW_locus_clipped = min(max(TW_locus, TW_vec(1)), TW_vec(end));

% =====
% SECTION 9 – FIGURE 1: SIDE-BY-SIDE DISTANCE HEATMAPS
% =====

fig1 = figure('Name','TO and Landing Distance Heatmaps (T/W vs W/S)', ...
              'Units','normalized','Position',[0.02 0.08 0.95 0.74]);
colormap(fig1, jet);

subplot_titles = {'Takeoff Distance d_{TO} [m]', ...

```

```

        'Landing Distance d_{land} [m]'};
dist_grids      = {d_To_plot, d_land_plot};
dist_full       = {d_To_grid, d_land_grid};

ax1 = gobjects(1,2);

for p = 1:2

    ax1(p) = subplot(1,2,p);

    % — Colour-filled distance field —————
    contourf(Ws_vec, TW_vec, dist_grids{p}, 25, 'LineColor','none');
    clim(d_clim);
    hold on;

    % Labelled context contours
    [C_ctx, h_ctx] = contour(Ws_vec, TW_vec, dist_grids{p}, 10, ...
        'k-', 'LineWidth', 0.7);
    clabel(C_ctx, h_ctx, 'FontSize', 7, 'LabelSpacing', 160);

    % Strict 500 ft and 5% margin boundary lines
    contour(Ws_vec, TW_vec, dist_full{p}, [d_req d_req], 'w-',
'LineWidth', 2.5);
    contour(Ws_vec, TW_vec, dist_full{p}, [d_marg d_marg], 'w--',
'LineWidth', 1.8);

    % — Engine power locus (pink dotted) —————
    % Shows the T/W delivered by this engine at every W/S.
    % The curve falls as W/S increases because higher W/S → higher VLo →
    % lower thrust for the same shaft power.
    plot(Ws_vec, TW_locus_clipped, locus_ls, ...
        'Color', pink_rgb, 'LineWidth', locus_lw);

    % — Design point (pink filled dot) —————
    plot(Ws_design, TW_design, 'o', ...
        'MarkerSize', dp_msz, 'MarkerFaceColor', pink_rgb, ...
        'MarkerEdgeColor', 'k', 'LineWidth', 1.2);
    text(Ws_design + 15, TW_design, ...
        sprintf(' S=%.1f m^2\n W/S=%.0f\n T/W=%.3f', ...
            S_ref, WS_design, TW_design), ...
        'FontSize', 7.5, 'FontWeight', 'bold', ...
        'BackgroundColor', [0.97 0.97 0.97], ...
        'EdgeColor', pink_rgb, 'Margin', 2);

    % — Colorbar —————
    cb = colorbar;
    cb.Label.String = subplot_titles{p};
    cb.FontSize = 10;

    % — Legend —————
    h_req = plot(NaN,NaN, 'w-', 'LineWidth', 2.5);
    h_marg = plot(NaN,NaN, 'w--', 'LineWidth', 1.8);
    h_loc = plot(NaN,NaN, locus_ls, 'Color', pink_rgb, 'LineWidth',
locus_lw);

```

```

h_dp = plot(NaN,NaN, 'o', 'MarkerSize', 8, ...
            'MarkerFaceColor', pink_rgb,
'MarkerEdgeColor','k','LineWidth',1.0);
legend([h_req, h_marg, h_loc, h_dp], ...
       {sprintf('500 ft limit (%.1f m)', d_req), ...
        sprintf('5%% margin (%.1f m)', d_marg), ...
        sprintf('Engine locus P=%.0f shp', P_shp), ...
        sprintf('Design point S=%.1f m^2', S_ref)}, ...
       'Location','northwest','FontSize',8, ...
       'Color',[0.15 0.15 0.15],'TextColor','w', ...
       'EdgeColor',[0.5 0.5 0.5]);

xlabel('Wing Loading W/S [N/m^2]', 'FontSize', 12);
ylabel('Thrust-to-Weight T/W [-]', 'FontSize', 12);
title(subplot_titles{p}, 'FontSize', 11);

ax = gca;
ax.FontSize = 10;
ax.GridColor = [0.8 0.8 0.8];
ax.GridAlpha = 0.25;
grid on;
xlim([WS_vec(1) WS_vec(end)]);
ylim([TW_vec(1) TW_vec(end)]);
hold off;

end

sgtitle({ ...
    'Takeoff & Landing Distance - T/W vs W/S | Single Engine', ...
    sprintf('W_{To}=%.0f N AR=%.0f e=%.2f CD_0=[%.3f/%.3f/%.3f]
(clean/TO/land)', ...
        W_To, AR, e, CD_0_clean, CD_0_TO, CD_0_land), ...
    sprintf('CL_{max,TO}=%.1f CL_{max,land}=%.1f P=%.0f shp \eta=%.2f
P_A=%.1f kW', ...
        CL_max_TO, CL_max_land, P_shp, eta, P_A/1000) }, ...
    'FontSize', 10.5);

linkaxes(ax1);

% =====
% SECTION 10 - FIGURE 2: COMBINED ZONE-SHADED CONSTRAINT DIAGRAM
% =====

fig2 = figure('Name','Combined Constraint Diagram: T/W vs W/S', ...
              'Units','normalized','Position',[0.10 0.08 0.70 0.78]);

% — Zone-shaded background —
imagesc(WS_vec, TW_vec, rgb);
set(gca, 'YDir', 'normal');
hold on;

% — Takeoff distance contours (white) —
contour(WS_vec, TW_vec, d_To_plot, linspace(50,400,12), ...
        'Color',[0.85 0.85 0.85],'LineWidth',0.6);

```

```

contour(Ws_vec, TW_vec, d_To_grid, [d_req d_req], 'w-', 'LineWidth',2.5);
contour(Ws_vec, TW_vec, d_To_grid, [d_marg d_marg], 'w--', 'LineWidth',1.8);

% — Landing distance contours (black) —————
contour(Ws_vec, TW_vec, d_land_plot, linspace(50,400,12), ...
    'Color',[0.30 0.30 0.30], 'LineWidth',0.6);
contour(Ws_vec, TW_vec, d_land_grid, [d_req d_req], 'k-',
    'LineWidth',2.5);
contour(Ws_vec, TW_vec, d_land_grid, [d_marg d_marg], 'k--',
    'LineWidth',1.8);

% — Engine power locus (pink dotted) —————
plot(Ws_vec, TW_locus_clipped, locus_ls, ...
    'Color', pink_rgb, 'LineWidth', locus_lw + 0.4);

% — 5% margin intersection point (star) —————
plot(int_WS_val, int_TW_val, 'p', ...
    'MarkerSize', int_msz, 'MarkerFaceColor','w', ...
    'MarkerEdgeColor','k', 'LineWidth',1.8);
text(int_WS_val + 12, int_TW_val, ...
    sprintf(' T/W=%.3f\n W/S=%.0f N/m^2\n d_{TO}=%.0f m (%.0f ft)\n
d_{land}=%.0f m (%.0f ft)', ...
        int_TW_val, int_WS_val, ...
        int_dTo, int_dTo/0.3048, ...
        int_dLand, int_dLand/0.3048), ...
    'FontSize', 8, 'FontWeight','bold', ...
    'BackgroundColor',[0.97 0.97 0.97], ...
    'EdgeColor','k','Margin',3);

% — Design point (pink dot with full annotation) —————
plot(Ws_design, TW_design, 'o', ...
    'MarkerSize', dp_msz, 'MarkerFaceColor', pink_rgb, ...
    'MarkerEdgeColor','k', 'LineWidth',1.5);
text(Ws_design + 12, TW_design - 0.035, ...
    sprintf([' S=%.1f m^2\n W/S=%.0f N/m^2\n T/W=%.3f\n' ...
        ' d_{TO}=%.0f m (%.0f ft)\n d_{land}=%.0f m (%.0f ft)'], ...
        S_ref, Ws_design, TW_design, ...
        d_To_dp, d_To_dp/0.3048, ...
        d_land_dp, d_land_dp/0.3048), ...
    'FontSize', 8, 'FontWeight','bold', ...
    'BackgroundColor',[0.97 0.97 0.97], ...
    'EdgeColor', pink_rgb, 'Margin', 3);

% — Legend —————
h_g = patch(NaN,NaN,[0.20 0.70 0.30], 'EdgeColor','none');
h_y = patch(NaN,NaN,[0.95 0.90 0.10], 'EdgeColor','none');
h_lr = patch(NaN,NaN,[0.95 0.60 0.60], 'EdgeColor','none');
h_dr = patch(NaN,NaN,[0.65 0.10 0.10], 'EdgeColor','none');
h_ts = plot(NaN,NaN, 'w-', 'LineWidth',2.5);
h_tm = plot(NaN,NaN, 'w--', 'LineWidth',1.8);
h_ls = plot(NaN,NaN, 'k-', 'LineWidth',2.5);
h_lm = plot(NaN,NaN, 'k--', 'LineWidth',1.8);
h_loc = plot(NaN,NaN, locus_ls, 'Color',pink_rgb, 'LineWidth',locus_lw+0.4);
h_dp = plot(NaN,NaN, 'o', 'MarkerSize',9, ...

```

```

'MarkerFaceColor',pink_rgb,'MarkerEdgeColor','k','LineWidth',1.2);
h_int = plot(NaN,NaN,'p','MarkerSize',11, ...
            'MarkerFaceColor','w','MarkerEdgeColor','k','LineWidth',1.5);

legend([h_g,h_y,h_lr,h_dr,h_ts,h_tm,h_ls,h_lm,h_loc,h_dp,h_int], ...
      {'Both pass (5% margin)', ...
      'Both pass (strict only)', ...
      'One constraint fails', ...
      'Both constraints fail', ...
      sprintf('d_{TO} = 500 ft (0.1f m)', d_req), ...
      sprintf('d_{TO} = 475 ft (0.1f m, 5%% margin)', d_marg), ...
      sprintf('d_{land} = 500 ft (0.1f m)', d_req), ...
      sprintf('d_{land} = 475 ft (0.1f m, 5%% margin)', d_marg), ...
      sprintf('Engine locus P=0.0f shp, \eta=0.2f', P_shp, eta), ...
      sprintf('Design point S=0.1f m^2 (W/S=0.0f N/m^2)', S_ref,
WS_design), ...
      sprintf('5%% margin intersection T/W=0.3f', int_TW_val)}, ...
      'Location','northeast','FontSize',8, ...
      'Color',[0.95 0.95 0.95],'EdgeColor',[0.4 0.4 0.4]);

xlabel('Wing Loading W/S [N/m^2]', 'FontSize',13);
ylabel('Thrust-to-Weight Ratio T/W [-]', 'FontSize',13);
title({ ...
      'Constraint Analysis - T/W vs W/S | Single Engine', ...
      sprintf('W_{To}=0.0f N (0.0f kg) P=0.0f shp \eta=0.2f P_A=0.1f
kW AR=0.0f e=0.2f', ...
      W_To, W_To/g, P_shp, eta, P_A/1000, AR, e), ...
      sprintf('CD_0=[0.3f/0.3f/0.3f] (clean/TO/land) CL_{max}=[0.1f TO/0.1f
land] \zeta=0.3f 6%% fuel reserve', ...
      CD_0_clean, CD_0_TO, CD_0_land, CL_max_TO, CL_max_land, zeta) },
...
      'FontSize',10);

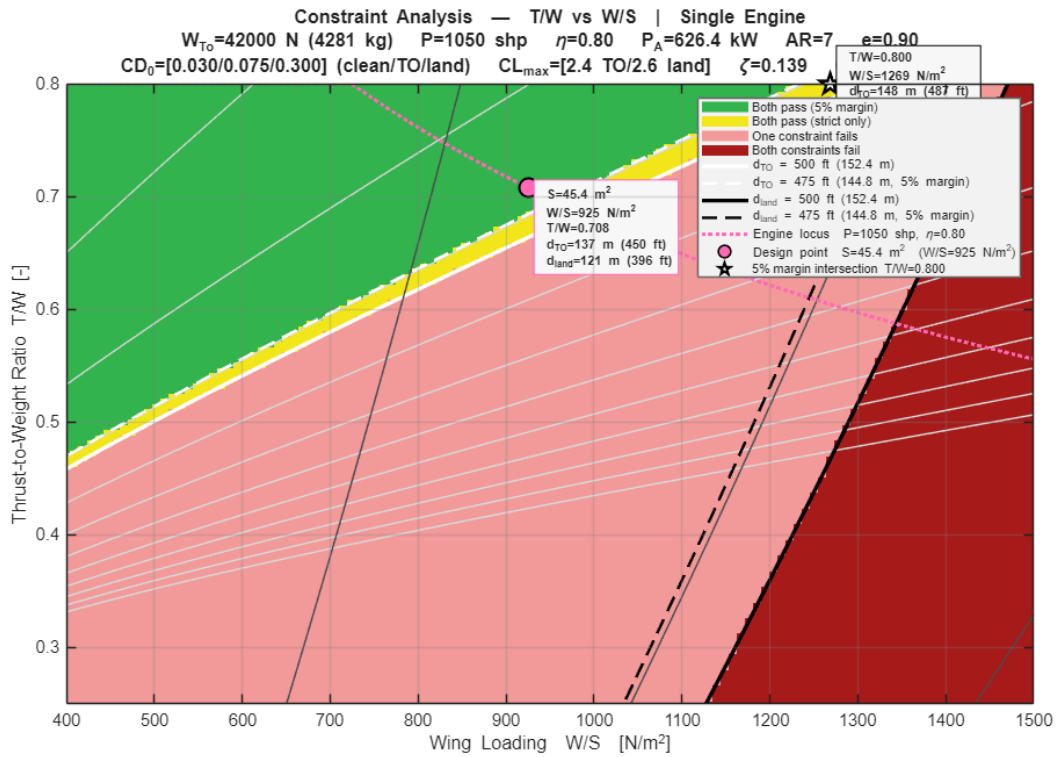
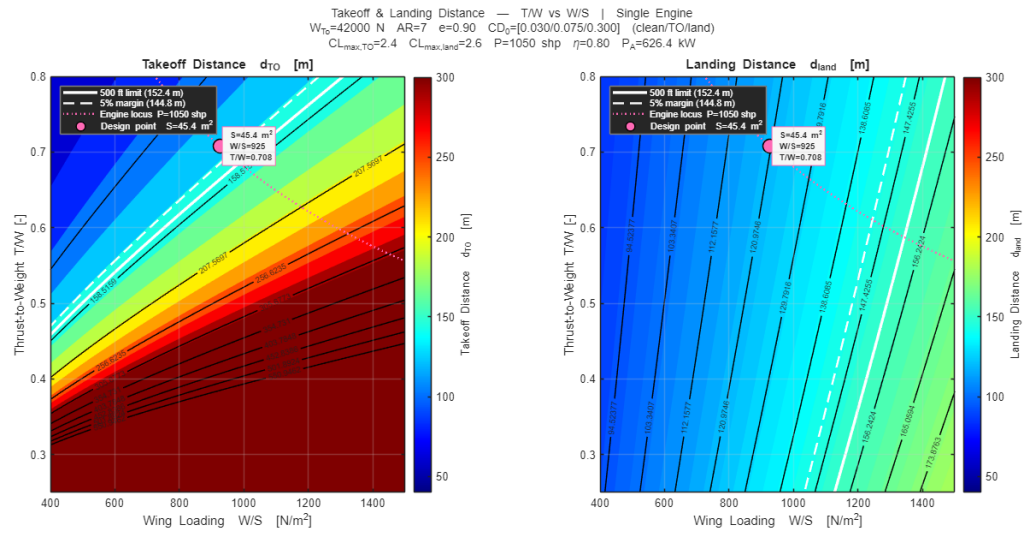
ax2 = gca;
ax2.FontSize = 11;
ax2.GridColor = [0.5 0.5 0.5];
ax2.GridAlpha = 0.28;
grid on;
ax2.Box = 'on';
xlim([WS_vec(1) WS_vec(end)]);
ylim([TW_vec(1) TW_vec(end)]);
hold off;

=== Design Point Summary ===
S_ref      : 45.4 m^2
W/S design : 925.1 N/m^2
T/W design : 0.7077
T_A design : 29725.5 N (6682 lbf)
P_A        : 626.4 kW (840 shp)
W_land     : 36512 N (3722 kg)
---
d_TO : 137.1 m (450 ft) → PASS - within 5% margin (137.1 m <= 144.8 m)
d_land: 120.6 m (396 ft) → PASS - within 5% margin (120.6 m <= 144.8 m)

```

=== 5% Margin Contour Intersection ===

T/W : 0.8000
 W/S : 1268.9 N/m²
 d_{TO} : 148.4 m (487 ft)
 d_{land} : 139.8 m (459 ft)



Published with MATLAB® R2025b

```

% DragBuildUp.m
% =====
% DRAG BUILD-UP – Parasite Drag Coefficient (CD_0) Estimation
%
% Method: Raymer §12.5 "Component Buildup Method" (Eq. 12.24):
%
%   (CD_0)_subsonic =  $\Sigma(Cf_c \cdot FF_c \cdot Q_c \cdot Swet_c) / S_{ref}$ 
%                   + CD_misc + CD_L&P
%
%   where for each component (subscript c):
%   Cf  = flat-plate skin friction coefficient (Raymer Eq. 12.27)
%   FF  = form factor (pressure drag correction) (Raymer Eqs. 12.30–12.31)
%   Q   = interference factor (Raymer §12.5, p.285)
%   Swet= component wetted area
%
%   CD_misc captures landing-gear drag.
%   CD_L&P captures leakage and protuberance drag.
%
% Airfoil: NASA GA(W)-1 (General Aviation Whitcomb, also known as NASA
LS(1)-0417)
%   – t/c = 0.17, (x/c)_m ≈ 0.40 (max thickness near 40% chord, typical of
this
%   series). Ref: McGhee & Beasley, NASA TN D-7428 (1973).
%   – Unswept, low-speed design → sweep term in Eq. (12.30) collapses to
~1.0.
%
% Fuselage cross-section: 1.75 m × 2.00 m rectangle with 0.10 m corner
fillets.
%   – Equivalent circular diameter computed from cross-sectional area
%   using Raymer Eq. 12.33: d = sqrt(4/pi * A_max).
%   – Rectangular fuselage form factor correction from Raymer p.284:
%   "A square-sided fuselage has a form factor about 40% higher than the
%   [ellipsoidal] value estimated with Eq.(12.31) due to additional
separation
%   caused by the corners. This can be somewhat reduced by rounding the
corners."
%   With 0.10 m fillets, rounding ratio r/side ≈ 0.053 (modest). A 20%
%   FF penalty is applied – midpoint between sharp-cornered (+40%) and
%   perfectly circular (0%). Engineering judgement per Raymer p.284.
%
% Tail airfoils: NACA 0012 selected for both horizontal and vertical tail.
%   – NACA 0012: t/c = 0.12, (x/c)_m = 0.30 (standard 4-digit NACA: max
%   thickness at 30% chord). Classic choice for low-speed tail surfaces.
%   – Ref: Abbott & von Doenhoff, "Theory of Wing Sections," Dover (1959).
%   – Tail form factors increased 10% for hinge gaps per Raymer p.283.
%
% Landing gear: Fixed tricycle gear (2 main + 1 nose).
%   – Drag estimated using Raymer Table 12.5 D/q values per component.
%   – Total D/q multiplied by 1.2 for mutual interference (Raymer p.287).
%
% Leakage & Protuberances:
%   – 5% of primary component drag (Raymer p.289, propeller-aircraft upper

```

```

bound).
%
% Skin friction: Raymer Eq. 12.27 (turbulent, Mach-corrected) throughout.
%
% KEY REFERENCE
%   Raymer, D. P., "Aircraft Design: A Conceptual Approach," AIAA, 1989.
%   All equation and page numbers refer to this edition.
%
% USAGE
%   1. Section 2 contains all geometry – replace placeholder values with CAD
outputs.
%   2. Run. No additional .m files needed.
%   3. Feed resulting CD_0_clean into ConstraintAnalysis_TO_Land_v2.m.
%
% All quantities SI (metres, kg, Pa, m^2) unless stated.
% =====

clear; clc; close all;

ft2_to_m2 = 0.0929; % unit conversion factor: 1 ft^2 = 0.0929 m^2

% =====
% SECTION 1 – FLIGHT CONDITION FOR Cf EVALUATION
% =====
% Cf evaluated at cruise (clean config). Raymer Eq.(12.27) is Mach-corrected
% so M_cruise is included even though the correction is negligible at low
speed.

V_cruise = 130 * 0.5144; % 130 KTAS → m/s
M_cruise = V_cruise / 340.3; % Mach number (SL ISA speed of sound = 340.3
m/s)
rho_SL = 1.225; % ISA sea-level density [kg/m^3]
mu_air = 1.789e-5; % dynamic viscosity air, SL 15 °C [Pa·s]

% =====
% SECTION 2 – AIRCRAFT GEOMETRY
% =====

% -----
% WING
% -----
S_wing = 36.80; % total wing planform area [m^2]
b_wing = 15.20; % wing span [m]
MAC_w = S_wing / b_wing; % MAC – untapered → MAC = S/b [m]

% NASA GA(W)-1 airfoil (Raymer Eq. 12.30 parameters)
t_c_w = 0.17; % thickness-to-chord ratio [-]
xcm_w = 0.40; % chordwise location of max thickness (x/c)_m [-]
% McGhee & Beasley, NASA TN D-7428 (1973): GA(W)-1 max t at ~40% chord.
Lambda_m_w = 0.0; % sweep of max-thickness line [rad] (unswept)

% -----
% HORIZONTAL TAIL
% -----

```

```

S_htail    = 11.34;           % total planform area           [m^2]
b_htail    = 6.73;           % span                         [m]
MAC_ht     = S_htail / b_htail; % MAC - untapered           [m]

% NACA 0012 horizontal tail (Abbott & von Doenhoff, 1959)
t_c_ht     = 0.12;           % t/c                         [-]
xcm_ht     = 0.30;           % (x/c)_m - standard 4-digit NACA at 30% chord [-]
Lambda_m_ht = 0.0;           % unswept                     [rad]

% -----
% VERTICAL TAIL
% -----
S_vtail    = 4.22;           % planform area               [m^2]
b_vtail    = 2.60;           % fin height                 [m]
MAC_vt     = S_vtail / b_vtail; % MAC - untapered           [m]

% NACA 0012 vertical tail - same section as h-tail
t_c_vt     = 0.12;
xcm_vt     = 0.30;
Lambda_m_vt = 0.0;

% -----
% FUSELAGE
% -----
l_fuse     = 9.9001;         % total fuselage length       [m]
W_fuse     = 1.95;           % fuselage width             [m]
H_fuse     = 2.00;           % fuselage height            [m]
r_fillet   = 0.10;           % corner fillet radius        [m]

% — Cross-sectional area (filleted rectangle) —————
% Area = rectangle - 4 corner squares + 4 quarter-circle fillets
A_rect     = W_fuse * H_fuse;
A_corner_removed = 4 * (r_fillet^2 - pi * r_fillet^2 / 4);
A_max_fuse = A_rect - A_corner_removed;

% — Equivalent circular diameter (Raymer Eq. 12.33) —————
% Raymer defines d for a non-circular cross-section as the diameter of a
% circle with the same area as the maximum cross-section.
% f = l/d = l / sqrt(4/pi * A_max)
d_fuse     = sqrt(4 / pi * A_max_fuse); % equivalent circular diameter [m]

% — Fineness ratio —————
f_fuse     = l_fuse / d_fuse; % used in Eq.(12.31)           [-]

% — Wetted area of fuselage —————
% Perimeter of the filleted rectangle × fuselage length.
% Perimeter = full rectangle perimeter - 4 straight corner segments
%             + 4 quarter-circle arcs
P_rect_corners = 4 * r_fillet;
P_fillet_arcs  = 4 * (pi / 2 * r_fillet);
P_wet_fuse     = 2*(W_fuse + H_fuse) - P_rect_corners + P_fillet_arcs;
S_wet_fuse     = P_wet_fuse * l_fuse; % [m^2]

% — Rectangular cross-section FF penalty (Raymer p.284) —————

```

```

% Raymer p.284: sharp square → +40%; rounded corners reduce this.
% 0.10 m fillets on ~1.875 m sides → r/side ≈ 0.053 → modest rounding.
% 20% penalty applied (conservative midpoint between sharp and fully
rounded).
rect_FF_penalty = 1.20;

% =====
% LANDING GEAR – Fixed Tricycle (Raymer Table 12.5 & p.287)
% =====
% Component D/q values from Raymer Table 12.5 [ft^2 per component unit]:
% Regular wheel and tire : 0.25 ft^2
% Round strut or wire : 0.30 ft^2
% Tricycle layout: 2 main legs + 1 nose leg, each with 1 wheel and 1 strut.

n_legs = 3; % total gear legs (2 main + 1 nose)
Dq_wheel = 0.25; % D/q per wheel+tire, ft^2 (Table 12.5)
Dq_strut = 0.30; % D/q per round strut, ft^2 (Table 12.5)

% Raw sum before interference correction
Dq_gear_raw = n_legs * (Dq_wheel + Dq_strut); % [ft^2]

% Raymer p.287: multiply total gear drag by 1.2 for mutual interference
Dq_gear = 1.2 * Dq_gear_raw; % [ft^2]
Dq_gear_m2 = Dq_gear * ft2_to_m2; % [m^2]

% =====
% REFERENCE AREA
% =====
S_ref = S_wing; % conventional normalisation (Raymer §12.5) [m^2]

% =====
% SECTION 3 – SKIN FRICTION COEFFICIENTS (Raymer Eq. 12.27)
% =====
% Turbulent flat-plate, Mach-number corrected (Raymer p.282, Eq. 12.27):
% Cf = 0.455 / [(log10 Re)^2.58 * (1 + 0.144 M^2)^0.65]
%
% Characteristic length: fuselage → l_fuse; lifting surfaces → MAC.

Re_fuse = rho_SL * V_cruise * l_fuse / mu_air;
Re_wing = rho_SL * V_cruise * MAC_w / mu_air;
Re_ht = rho_SL * V_cruise * MAC_ht / mu_air;
Re_vt = rho_SL * V_cruise * MAC_vt / mu_air;

% Anonymous function: Raymer Eq. (12.27)
turb_Cf = @(Re, M) 0.455 / ((log10(Re))^2.58 * (1 + 0.144*M^2)^0.65);

Cf_fuse = turb_Cf(Re_fuse, M_cruise);
Cf_wing = turb_Cf(Re_wing, M_cruise);
Cf_ht = turb_Cf(Re_ht, M_cruise);
Cf_vt = turb_Cf(Re_vt, M_cruise);

% =====
% SECTION 4 – FORM FACTORS
% =====

```

```

% — Wing and tail surfaces: Raymer Eq. (12.30) —————
% FF = [1 + 0.6/(x/c)_m * (t/c) + 100(t/c)^4] * [1.34 M^0.18 (cosΛ_m)^0.28]
%
% At M_cruise ≈ 0.195 and Λ_m = 0° (unswept), the second bracket ≈ 0.990 ≈
1.0.
% The first bracket captures the airfoil-shape contribution to pressure drag.
%
% Tail +10% for hinge gap: Raymer p.283 — "A tail surface with a hinged
% rudder or elevator will have a form factor about 10% higher than predicted
% by Eq.(12.30) due to the extra drag of the gap between the tail surface
% and its control surface."

FF_wing = (1 + (0.6/xcm_w) * t_c_w + 100*t_c_w^4) ...
* (1.34 * M_cruise^0.18 * cos(Lambda_m_w)^0.28);

FF_ht = (1 + (0.6/xcm_ht) * t_c_ht + 100*t_c_ht^4) ...
* (1.34 * M_cruise^0.18 * cos(Lambda_m_ht)^0.28) ...
* 1.10; % +10% hinge-gap penalty (Raymer p.283)

FF_vt = (1 + (0.6/xcm_vt) * t_c_vt + 100*t_c_vt^4) ...
* (1.34 * M_cruise^0.18 * cos(Lambda_m_vt)^0.28) ...
* 1.10; % +10% hinge-gap penalty (Raymer p.283)

% — Fuselage: Raymer Eq. (12.31), with rectangular FF penalty —————
% FF_ellipsoid = 1 + 60/f^3 + f/400 (smooth, axisymmetric body)
% FF_fuse = FF_ellipsoid * rect_FF_penalty (Raymer p.284, see Section 2)

FF_fuse = (1 + 60/f_fuse^3 + f_fuse/400) * rect_FF_penalty;

% =====
% SECTION 5 — INTERFERENCE FACTORS Q (Raymer §12.5, p.285)
% =====
% Q multiplies each component's friction + form drag to account for
% aerodynamic interference at component junctions.
%
% Fuselage Q = 1.00 — Raymer p.285: "The fuselage has a negligible
% interference factor (Q = 1.0) in most cases."
%
% Wing Q = 1.00 — Raymer p.285: "For a high-wing, a mid-wing, or a
% well-filletted low wing, the interference will be negligible so the
% Q factor will be about 1.0."
%
% H-tail Q = 1.05 — Raymer p.285: "For tail surfaces, interference ranges
% from about three percent (Q = 1.03) for a clean V-tail to about eight
% percent for an H-tail." Midpoint chosen (5%) as conservative for a
% conventional H-tail on a fuselage-mounted stabiliser.
%
% V-tail Q = 1.03 — Low end of range (clean fin-fuselage junction).

Q_fuse = 1.00;
Q_wing = 1.00;
Q_htail = 1.05;
Q_vtail = 1.03;

```

```

% =====
% SECTION 6 – WETTED AREAS FOR LIFTING SURFACES
% =====
% Both upper and lower surfaces exposed → factor of 2.
% Thickness correction:  $S_{wet} \approx 2 \times S_{plan} \times (1 + 0.2 \times t/c)$ 
% This accounts for the increase in wetted area of a thick wing compared
% to a flat plate. For  $t/c = 0.17$  this is a 3.4% correction. Conservative
% and consistent with common conceptual-design practice.

S_wet_wing = 2 * S_wing * (1 + 0.2 * t_c_w);
S_wet_htail = 2 * S_htail * (1 + 0.2 * t_c_ht);
S_wet_vtail = 2 * S_vtail * (1 + 0.2 * t_c_vt);

% =====
% SECTION 7 – COMPONENT DRAG AREAS ( $C_f \times FF \times Q \times S_{wet}$ )
% =====

A_fuse = C_f_fuse * FF_fuse * Q_fuse * S_wet_fuse;
A_wing = C_f_wing * FF_wing * Q_wing * S_wet_wing;
A_htail = C_f_ht * FF_ht * Q_htail * S_wet_htail;
A_vtail = C_f_vt * FF_vt * Q_vtail * S_wet_vtail;

% =====
% SECTION 8 – MISCELLANEOUS DRAG: LANDING GEAR CD_misc
% =====
% Fixed landing gear is the dominant misc. drag source for this aircraft.
% D/q computed in Section 2 from Raymer Table 12.5 and converted to CD.

CD_misc_gear = Dq_gear_m2 / S_ref;

% =====
% SECTION 9 – LEAKAGE & PROTUBERANCE DRAG CD_L&P (Raymer p.289)
% =====
% Raymer p.289: "For a normal production aircraft, leaks and protuberance
% drags can be estimated as about 2-5% of the parasite drag for jet
% transports or bombers, 5-10% for propeller aircraft."
%
% NOTE: Raymer quotes 5-10% for propeller aircraft. 5% is used here as the
% lower end of the propeller-aircraft range – appropriate for an aircraft
% that is not a fighter and will have careful attention to surface finish.
% This is applied to the primary friction/form drag sum only (gear is already
% explicitly accounted for in CD_misc).

leak_frac = 0.05;

A_primary = A_fuse + A_wing + A_htail + A_vtail;
CD_primary = A_primary / S_ref;
CD_LP = leak_frac * CD_primary;

% =====
% SECTION 10 – TOTAL PARASITE DRAG (Raymer Eq. 12.24)
% =====

```

```

CD0_fuse = A_fuse / S_ref;
CD0_wing = A_wing / S_ref;
CD0_htail = A_htail / S_ref;
CD0_vtail = A_vtail / S_ref;

% Raymer Eq. (12.24):
% (CD_0)_subsonic =  $\Sigma(C_f \cdot FF \cdot Q \cdot S_{wet}) / S_{ref} + CD_{misc} + CD_{L\&P}$ 
CD0_clean = CD_primary + CD_misc_gear + CD_LP;

% =====
% SECTION 11 - CONSOLE OUTPUT
% =====

fprintf('\n');
fprintf('=====
=====');
fprintf(' DRAG BUILD-UP SUMMARY (Raymer §12.5 Component Buildup, Eq.
12.24)\n');
fprintf('=====
=====');

fprintf(' Derived Geometry\n');
fprintf(' Wing MAC = %.4f m S_wet = %.4f m^2\n', MAC_w,
S_wet_wing);
fprintf(' H-tail MAC = %.4f m S_wet = %.4f m^2\n', MAC_ht,
S_wet_htail);
fprintf(' V-tail MAC = %.4f m S_wet = %.4f m^2\n', MAC_vt,
S_wet_vtail);
fprintf(' Fuse A_max = %.4f m^2 (%.2f×%.2f rect, r=%.2f m fillets)
\n', ...
A_max_fuse, W_fuse, H_fuse, r_fillet);
fprintf(' Fuse d_eq = %.4f m (Raymer Eq.12.33)\n', d_fuse);
fprintf(' Fuse l/d = %.4f (fineness ratio, Eq.12.31)\n', f_fuse);
fprintf(' Fuse S_wet = %.4f m^2 (perimeter × length)\n\n', S_wet_fuse);

fprintf(' Flight Condition\n');
fprintf(' V = %.2f m/s (%.0f KTAS) M = %.4f rho = %.4f kg/
m^3\n\n', ...
V_cruise, V_cruise/0.5144, M_cruise, rho_SL);

fprintf(' Reynolds Numbers (Raymer Eq.12.26: Re = rho·V·l/mu)\n');
fprintf(' Fuselage (L=%.4f m) Re = %.4e\n', l_fuse, Re_fuse);
fprintf(' Wing (L=%.4f m) Re = %.4e\n', MAC_w, Re_wing);
fprintf(' H-Tail (L=%.4f m) Re = %.4e\n', MAC_ht, Re_ht);
fprintf(' V-Tail (L=%.4f m) Re = %.4e\n\n', MAC_vt, Re_vt);

fprintf(' Skin Friction Coeff. (Raymer Eq.12.27, turbulent)\n');
fprintf(' Fuselage Cf = %.5f\n', Cf_fuse);
fprintf(' Wing Cf = %.5f\n', Cf_wing);
fprintf(' H-Tail Cf = %.5f\n', Cf_ht);
fprintf(' V-Tail Cf = %.5f\n\n', Cf_vt);

fprintf(' Form Factors\n');
fprintf(' Fuselage FF = %.4f (Eq.12.31 base=%.4f × %.2f rect penalty,
```

```

Raymer p.284)\n', ...
    FF_fuse, FF_fuse/rect_FF_penalty, rect_FF_penalty);
fprintf('    Wing      FF = %.4f  (Eq.12.30: GA(W)-1, t/c=%.2f, (x/c)m=%.2f,
Λ=0°)\n', ...
    FF_wing, t_c_w, xcm_w);
fprintf('    H-Tail    FF = %.4f  (Eq.12.30 × 1.10 hinge-gap, Raymer
p.283)\n', FF_ht);
fprintf('    V-Tail    FF = %.4f  (Eq.12.30 × 1.10 hinge-gap, Raymer
p.283)\n\n', FF_vt);

fprintf('  Interference Factors Q  (Raymer p.285)\n');
fprintf('    Fuselage Q=%.2f  Wing Q=%.2f  H-tail Q=%.2f  V-tail
Q=%.2f\n\n', ...
    Q_fuse, Q_wing, Q_htail, Q_vtail);

% Component breakdown table
fprintf('  Component Drag Breakdown\n');
hdr = ' %15s %9s %7s %7s %6s %10s %8s %7s';
row = ' %15s %9.4f %7.5f %7.4f %6.2f %10.6f %8.5f %6.1f%%';
row_misc = ' %15s %9s %7s %7s %6s %10s %8.5f %6.1f%%';
fprintf([hdr '\n'],
'Component','Swet[m^2]','Cf','FF','Q','Ai[m^2]','CD0_i','%Total');
fprintf(' %s\n', repmat('-',1,82));
fprintf([row '\n'], 'Fuselage', S_wet_fuse, Cf_fuse, FF_fuse, Q_fuse,
A_fuse, CD0_fuse, 100*CD0_fuse/CD0_clean);
fprintf([row '\n'], 'Wing', S_wet_wing, Cf_wing, FF_wing, Q_wing,
A_wing, CD0_wing, 100*CD0_wing/CD0_clean);
fprintf([row '\n'], 'H-Tail', S_wet_htail, Cf_ht, FF_ht, Q_htail,
A_htail, CD0_htail, 100*CD0_htail/CD0_clean);
fprintf([row '\n'], 'V-Tail', S_wet_vtail, Cf_vt, FF_vt, Q_vtail,
A_vtail, CD0_vtail, 100*CD0_vtail/CD0_clean);
fprintf([row_misc '\n'], 'Gear (misc)', '-', '-', '-', '-', '-', CD_misc_gear,
100*CD_misc_gear/CD0_clean);
fprintf([row_misc '\n'], 'Leak & Protub', '-', '-', '-', '-', '-', CD_LP,
100*CD_LP/CD0_clean);
fprintf(' %s\n', repmat('-',1,82));
fprintf([row_misc '\n'], 'TOTAL (clean)', '-', '-', '-', '-', '-', CD0_clean,
100.0);
fprintf('\n');

% Landing gear detail
fprintf('  Landing Gear Detail  (Raymer Table 12.5)\n');
fprintf('    %d legs × wheel(%.2f ft²) + strut(%.2f ft²) = %.3f ft²  (raw)
\n', ...
    n_legs, Dq_wheel, Dq_strut, Dq_gear_raw);
fprintf('    × 1.20 interference (Raymer p.287) → D/q = %.3f ft² = %.5f
m²\n', ...
    Dq_gear, Dq_gear_m2);
fprintf('    CD_misc_gear = %.5f\n\n', CD_misc_gear);

% Comparison
CD0_placeholder = 0.030;
delta_pct = (CD0_clean - CD0_placeholder) / CD0_placeholder * 100;
fprintf('  Comparison vs. ConstraintAnalysis placeholder\n');

```

```

fprintf('    Placeholder    CD_0 = %.4f\n', CD0_placeholder);
fprintf('    Build-up      CD_0 = %.4f\n', CD0_clean);
fprintf('    Difference          = %.1f%%\n', delta_pct);
if abs(delta_pct) > 20
    fprintf('    *** WARNING: >20%% deviation - update ConstraintAnalysis!
***\n');
end
fprintf('\n');
fprintf('=====
=====');
fprintf('  ACTION: Set CD_0_clean = %.5f in
ConstraintAnalysis_TO_Land_v2.m\n', CD0_clean);
fprintf('=====
=====');

% =====
% SECTION 12 - BAR CHART
% =====

comp_labels = {'Fuselage', 'Wing', 'H-Tail', 'V-Tail', 'Gear', 'Leak &
Protub'};
CD0_parts    = [CD0_fuse, CD0_wing, CD0_htail, CD0_vtail, CD0_misc_gear,
CD_LP];
pct_parts     = 100 * CD0_parts / CD0_clean;

bar_colors    = [0.27 0.51 0.71;    % fuselage - steel blue
                 0.20 0.63 0.17;    % wing      - green
                 0.89 0.47 0.13;    % h-tail   - orange
                 0.75 0.22 0.17;    % v-tail   - red
                 0.55 0.27 0.07;    % gear      - brown
                 0.60 0.60 0.60];    % leakage  - grey

figure('Name','Drag Build-Up', 'Units','normalized', 'Position',[0.08 0.15
0.60 0.65]);

b = bar(CD0_parts, 'FaceColor','flat');
b.CData = bar_colors;

for k = 1:length(CD0_parts)
    text(k, CD0_parts(k) + 0.0002, sprintf('%.1f%%', pct_parts(k)), ...
         'HorizontalAlignment','center', 'FontSize',10, 'FontWeight','bold');
end

yline(CD0_clean, 'k--', 'LineWidth',1.5, ...
      'Label', sprintf('CD_0 = %.4f (total clean)', CD0_clean), ...
      'LabelHorizontalAlignment','left', 'FontSize',9);

xticks(1:length(comp_labels));
xticklabels(comp_labels);
ylabel('Parasite Drag Coefficient CD_0 [-]', 'FontSize',12);
title({ ...
      'Drag Build-Up - Component Contributions to CD_{0,clean}', ...
      sprintf('Raymer §12.5 | NASA GA(W)-1 | NACA 0012 tails | Fixed Tricycle
Gear | S_{ref} = %.2f m^2', S_ref)}, ...

```

```

        'FontSize',11);

ax = gca;
ax.FontSize = 11;
ax.GridAlpha = 0.30;
grid on;
ylim([0, max(CD0_parts)*1.40]);
xlim([0.4, length(comp_labels)+0.6]);
box on;

```

DRAG BUILD-UP SUMMARY (Raymer §12.5 Component Buildup, Eq. 12.24)

Derived Geometry

```

Wing MAC      = 2.4211 m      Swet = 76.1024 m2
H-tail MAC    = 1.6850 m      Swet = 23.2243 m2
V-tail MAC    = 1.6231 m      Swet = 8.6426 m2
Fuse Amax    = 3.8914 m2 (1.95×2.00 rect, r=0.10 m fillets)
Fuse deq     = 2.2259 m      (Raymer Eq.12.33)
Fuse l/d      = 4.4477      (fineness ratio, Eq.12.31)
Fuse Swet    = 80.4712 m2 (perimeter × length)

```

Flight Condition

```

V = 66.87 m/s (130 KTAS)    M = 0.1965    rho = 1.2250 kg/m3

```

Reynolds Numbers (Raymer Eq.12.26: $Re = \rho \cdot V \cdot l / \mu$)

```

Fuselage (L=9.9001 m) Re = 4.5332e+07
Wing      (L=2.4211 m) Re = 1.1086e+07
H-Tail    (L=1.6850 m) Re = 7.7156e+06
V-Tail    (L=1.6231 m) Re = 7.4321e+06

```

Skin Friction Coeff. (Raymer Eq.12.27, turbulent)

```

Fuselage Cf = 0.00237
Wing      Cf = 0.00294
H-Tail    Cf = 0.00312
V-Tail    Cf = 0.00314

```

Form Factors

```

Fuselage FF = 2.0317 (Eq.12.31 base=1.6931 × 1.20 rect penalty, Raymer
p.284)
Wing      FF = 1.3383 (Eq.12.30: GA(W)-1, t/c=0.17, (x/c)m=0.40,  $\Lambda=0^\circ$ )
H-Tail    FF = 1.3865 (Eq.12.30 × 1.10 hinge-gap, Raymer p.283)
V-Tail    FF = 1.3865 (Eq.12.30 × 1.10 hinge-gap, Raymer p.283)

```

Interference Factors Q (Raymer p.285)

```

Fuselage Q=1.00 Wing Q=1.00 H-tail Q=1.05 V-tail Q=1.03

```

Component Drag Breakdown

Component	Swet [m ²]	Cf	FF	Q	Ai [m ²]
CD0 _i	%Total				

```

-----
  Fuselage      80.4712  0.00237  2.0317  1.00  0.388293
0.01055    36.7%
  Wing          76.1024  0.00294  1.3383  1.00  0.299838
0.00815    28.3%
  H-Tail        23.2243  0.00312  1.3865  1.05  0.105519
0.00287    10.0%
  V-Tail        8.6426  0.00314  1.3865  1.03  0.038755
0.00105     3.7%
  Gear (misc)           -          -          -          -          -
0.00500    17.4%
  Leak & Protub           -          -          -          -          -
0.00113     3.9%

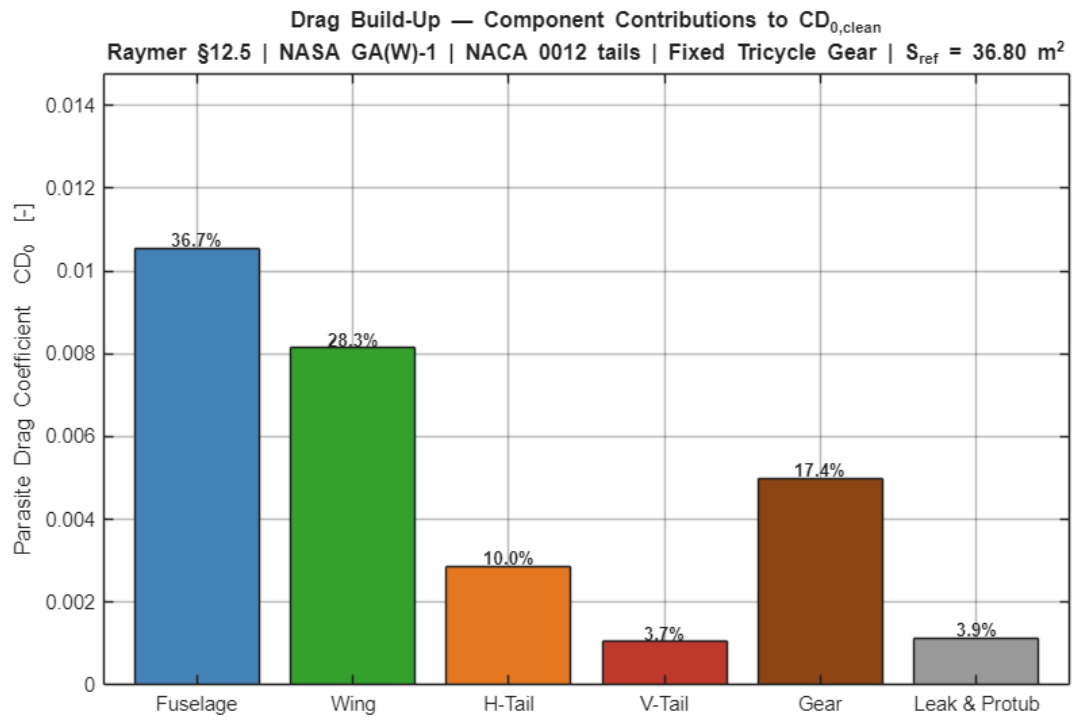
-----
TOTAL (clean)           -          -          -          -          -
0.02875   100.0%

Landing Gear Detail (Raymer Table 12.5)
  3 legs × wheel(0.25 ft²) + strut(0.30 ft²) = 1.650 ft² (raw)
  × 1.20 interference (Raymer p.287) → D/q = 1.980 ft² = 0.18394 m²
  CD_misc_gear = 0.00500

Comparison vs. ConstraintAnalysis placeholder
  Placeholder   CD_0 = 0.0300
  Build-up      CD_0 = 0.0287
  Difference     = -4.2%

=====
ACTION: Set CD_0_clean = 0.02875 in ConstraintAnalysis_TO_Land_v2.m
=====

```



Published with MATLAB® R2025b

PayloadBuildUp_v1.m

===== Arctic STOL
Aircraft — Payload Mass Build-Up Purpose: Itemized payload weight breakdown for all three RFP configurations.

SCOPE OF THIS SCRIPT: - Payload weights ONLY. OEW, fuel, and full MTOW closure are handled in Raymer-WeightEst_v3.m. This script feeds into that one. - OEW is fixed at the Config 2 MTOW-converged value (40,000 N / 4,077 kg). - No fuel fractions or fuel weights are computed here. - Config 2 is the SIZING CASE (heaviest payload) — all structural and fuel sizing is driven by Config 2. Configs 1 and 3 are lighter and do not re-drive the structure.

CONFIGURATIONS: Config 1 — Passenger Transport 9 passengers + 1 flight attendant (FA, added beyond RFP minimum) 10 seats (9 pax + 1 FA jump seat) Passenger baggage per pax, pilot baggage allowance for FA

Config 2 — Cargo Transport (SIZING CASE, heaviest payload)

Cargo payload mass-equivalent to 9-pax configuration PLUS seat weight
8,000 W diesel generator + 55 gal diesel fuel in drum

NOTE: No passengers in Config 2 — seats are REMOVED and replaced by cargo

Config 3 — Aerial Ambulance (Medevac), two loading conditions:

Outbound: 2 med techs + litter + medical supplies (no patient yet)

Inbound: 2 med techs + litter + medical supplies + 1 patient + 1 escort

No baggage for any occupant in Config 3 (emergency ops assumption)

Aircraft must carry fuel for both legs from the start (no refuel at site)

⚠️ FLAGS are raised inline wherever a number is uncertain or assumed.

DESIGN NOTE ON ITERATION: Weight drives everything. As structural estimates improve (component weight buildup), return here to verify payload fractions have not shifted. Any change to seat mass, pax count, or med equipment mass propagates directly into MTOW through RaymerWeightEst_v3.m.

Authors:	AME	261	Design	Team	Date:	May	2026	Version:	1.0
Units:	SI	(kg)	throughout.	lbf	shown	parenthetically	for	RFP	traceability.

=====

clear; clc;

=====

=====

SECTION 1 — SHARED CONSTANTS

These values apply across all configurations. Change here only — do not
hardcode the same number in multiple places.

=====

% --- Conversion factor (for RFP traceability printout only) ---

kg_per_lb = 0.453592; % [kg/lb]

% --- Standard occupant body weight (RFP §1.2) ---

% RFP specifies 200 lb per person for all occupants (pax, crew, med staff).

W_person_kg = 200 * kg_per_lb; % 90.72 kg [kg]

```

% --- Passenger baggage allowance (RFP $1.2) ---
% 50 lb per passenger, checked baggage.
W_pax_bag_kg = 50 * kg_per_lb;      % 22.68 kg [kg]

% --- Pilot / FA baggage allowance (RFP $1.2) ---
% RFP assigns 20 lb per pilot. FA uses same allowance per design decision.
W_pilot_bag_kg = 20 * kg_per_lb;    % 9.07 kg [kg]

% --- Passenger seat mass ---
% Recaro BL 3530 lightweight unit: 10 kg per seat.
% Used for all 9 pax seats AND the FA seat.
% ⚠ FLAG: Seat mass is an estimate from manufacturer datasheet.
% Confirm once seat selection is finalized.
W_seat_kg = 10.0;                   % [kg] per seat

% --- Litter (NATO stretcher) mass (RFP $1.2) ---
% RFP: 50 lb. Using exact RFP value.
W_litter_kg = 50 * kg_per_lb;      % 22.68 kg [kg]

% --- Medical supplies and equipment (RFP $1.2) ---
% RFP: 300 lb total, single line item. No sub-breakdown available yet.
% ⚠ FLAG: This is an RFP-specified aggregate mass. As equipment is selected
% (monitor, defibrillator, O2 bottles, drug kit, etc.), replace with
% itemized values. Mass may increase – recheck MTOW closure if it does.
W_med_supplies_kg = 300 * kg_per_lb; % 136.08 kg [kg]

% --- Diesel fuel properties (for Config 2 cargo fuel delivery) ---
% ⚠ NOTE: This is the CARGO diesel fuel delivered to remote sites,
% NOT the aircraft's own propulsion fuel. Do not confuse with Wf in Raymer.
rho_diesel_kg_L = 0.85;              % [kg/L] density of diesel fuel
vol_diesel_gal = 55;                % [US gal] volume of cargo diesel
L_per_gal = 3.7854;                 % [L/US gal]
vol_diesel_L = vol_diesel_gal * L_per_gal; % 208.2 L
W_diesel_fuel_kg = rho_diesel_kg_L * vol_diesel_L; % [kg]

% --- Cargo diesel fuel drum (55-gal plastic open-top drum) ---
% Source: Uline S-9945BLU product spec.
% ⚠ FLAG: Confirm final drum selection. Mass is manufacturer-quoted.
W_drum_kg = 11.4;                   % [kg]

% --- 8,000 W diesel generator ---
% Source: Generac unit from Amazon product link on team spreadsheet.
% Listed mass from product page: ~219 lb.
% ⚠ FLAG: Confirm this is the correct generator. The 99.3 kg value from the
% spreadsheet is used here; the report draft earlier cited 250 kg for a
% different unit. This MUST be reconciled – a 150 kg discrepancy is
% not trivial at this weight class. Source the actual shipping weight
% from the manufacturer spec sheet, not the Amazon listing.
W_generator_kg = 99.3;               % [kg] ⚠ VERIFY

% --- MTOW anchor (Config 2 converged value from RaymerWeightEst_v3.m) ---
% OEW is frozen at this sizing point. All configs fly an aircraft
% sized for Config 2. Configs 1 and 3 will have payload margin to spare.
MTOW_N = 40000;                     % [N] design MTOW

```

```

g          = 9.81;                % [m/s2]
MTOW_kg    = MTOW_N / g;          % 4077.5 kg

```

```

=====
=====

```

SECTION 2 – CONFIGURATION 1: PASSENGER TRANSPORT

RFP requirements:

- 9 passengers at 200 lb body + 50 lb baggage each
- Minimum 31-inch seat pitch

Design additions beyond RFP:

- 1 flight attendant (FA) added for operational safety and service
FA uses pilot body weight (200 lb) and pilot baggage allowance (20 lb)
FA seat: 10 kg (same as pax seat, mounted as jump seat forward of Row 1)
- 10th seat added for FA

Payload DOES include removable seats – when seats are removed for Config 2, that seat mass becomes part of the cargo payload equivalence.
Payload does NOT include pilot seats (part of OEW/structure).

```

=====

fprintf('=====\\n'
);
fprintf('  CONFIGURATION 1 – PASSENGER TRANSPORT\\n');
fprintf('=====\\n\\n');

n_pax_C1 = 9;    % number of revenue passengers

% --- Occupant weights ---
W_pax_body_C1   = n_pax_C1 * W_person_kg;    % 9 pax body weight
W_pax_bag_C1    = n_pax_C1 * W_pax_bag_kg;    % 9 pax baggage
W_FA_body_C1    = 1      * W_person_kg;       % 1 FA body weight
W_FA_bag_C1     = 1      * W_pilot_bag_kg;     % FA baggage (pilot
allowance)

% --- Seat weights ---
% 9 pax seats + 1 FA seat = 10 seats total
n_seats_C1      = n_pax_C1 + 1;               % 10 seats
W_seats_C1      = n_seats_C1 * W_seat_kg;     % total removable seat
mass

% --- Config 1 total payload ---
W_payload_C1 = W_pax_body_C1 + W_pax_bag_C1 ...
              + W_FA_body_C1  + W_FA_bag_C1 ...
              + W_seats_C1;

% --- Print itemized breakdown ---
fprintf(' %-40s  %8s  %8s\\n', 'Item', 'Mass (kg)', 'Mass (lb)');
fprintf('  %s\\n', repmat('-', 1, 60));
fprintf(' %-40s  %8.2f  %8.2f\\n', ...
        sprintf('%d passengers body weight (x%.0f kg)', n_pax_C1,

```

```

W_person_kg), ...
    W_pax_body_C1,    W_pax_body_C1    / kg_per_lb);
fprintf(' %-40s %8.2f %8.2f\n', ...
    sprintf('%d passengers baggage (x%.2f kg)', n_pax_C1, W_pax_bag_kg), ...
    W_pax_bag_C1,    W_pax_bag_C1    / kg_per_lb);
fprintf(' %-40s %8.2f %8.2f\n', ...
    '1 flight attendant body weight', ...
    W_FA_body_C1,    W_FA_body_C1    / kg_per_lb);
fprintf(' %-40s %8.2f %8.2f\n', ...
    '1 FA baggage (pilot allowance, 20 lb)', ...
    W_FA_bag_C1,    W_FA_bag_C1    / kg_per_lb);
fprintf(' %-40s %8.2f %8.2f\n', ...
    sprintf('%d removable seats (x%.0f kg each)', n_seats_C1, W_seat_kg), ...
    W_seats_C1,    W_seats_C1    / kg_per_lb);
fprintf(' %s\n', repmat('-', 1, 60));
fprintf(' %-40s %8.2f %8.2f\n', 'TOTAL CONFIG 1 PAYLOAD', ...
    W_payload_C1, W_payload_C1 / kg_per_lb);
fprintf('\n');
fprintf(' NOTE: Pilot seats NOT included (part of OEW/structure).\n');
fprintf(' NOTE: FA is a design addition beyond RFP minimum crew spec.\n');
fprintf('      Adds %.2f kg to payload vs RFP-only configuration.\n', ...
    W_FA_body_C1 + W_FA_bag_C1 + W_seat_kg);
fprintf('\n\n');

```

CONFIGURATION 1 – PASSENGER TRANSPORT

Item	Mass (kg)	Mass (lb)
9 passengers body weight (x91 kg)	816.47	1800.00
9 passengers baggage (x22.68 kg)	204.12	450.00
1 flight attendant body weight	90.72	200.00
1 FA baggage (pilot allowance, 20 lb)	9.07	20.00
10 removable seats (x10 kg each)	100.00	220.46
TOTAL CONFIG 1 PAYLOAD	1220.37	2690.46

NOTE: Pilot seats NOT included (part of OEW/structure).
 NOTE: FA is a design addition beyond RFP minimum crew spec.
 Adds 109.79 kg to payload vs RFP-only configuration.

SECTION 3 – CONFIGURATION 2: CARGO TRANSPORT (SIZING CASE)

RFP requirements:

- Cargo weight equivalent to the 9-passenger configuration PLUS the weight of the removable seats

- Must transport an 8,000 W generator and 55 gal of diesel fuel (in no more than 3 containers, preferably one)

DESIGN NOTE: Config 2 is the SIZING CASE because it is the heaviest payload. The Raymer iteration in RaymerWeightEst_v3.m is driven by this configuration. Do not reduce this payload without re-running the Raymer convergence loop.

Payload equivalence logic:

RFP says cargo must equal "9-pax payload + seat weight."
 9-pax payload = $9 \times (200 \text{ lb body} + 50 \text{ lb baggage}) = 2,250 \text{ lb} = 1,021 \text{ kg}$
 Seat weight = $9 \text{ seats} \times 10 \text{ kg} = 90 \text{ kg}$ (pax seats only; FA seat is Config 1 specific and NOT present in Config 2)
 Equivalence total = $1,021 + 90 = 1,111 \text{ kg}$
 Generator + fuel + drum = $99.3 + 177.0 + 11.4 = 287.7 \text{ kg}$
 Total Config 2 payload = $1,111 + 287.7 = 1,398.7 \text{ kg} \approx 1,399 \text{ kg}$

⚠ FLAG: Config 2 does NOT carry a flight attendant or FA seat. The FA is a Config 1 design addition. Config 2 is pure cargo. This means Config 1 (1,212 kg) is LIGHTER than Config 2 (1,399 kg), which is correct and expected – Config 2 remains the sizing case.

```
=====
fprintf('=====\\n')
);
fprintf(' CONFIGURATION 2 - CARGO TRANSPORT (SIZING CASE)\\n');
fprintf('=====\\n\\n');

n_pax_C2    = 9;    % equivalent passenger count for mass equivalence calc
n_seats_C2  = 9;    % only the 9 pax removable seats (FA seat not in Config 2)

% --- Passenger mass equivalence (cargo replaces these people) ---
W_pax_equiv_C2 = n_pax_C2 * W_person_kg;    % body weight equivalent
W_bag_equiv_C2 = n_pax_C2 * W_pax_bag_kg;    % baggage equivalent
W_seats_C2     = n_seats_C2 * W_seat_kg;     % seat mass (9 seats removed)

% --- Cargo items ---
W_diesel_C2    = W_diesel_fuel_kg;           % 55 gal cargo diesel
W_drum_C2      = W_drum_kg;                   % fuel drum
W_gen_C2       = W_generator_kg;             % 8,000 W generator

% --- Config 2 total payload ---
W_payload_C2 = W_pax_equiv_C2 + W_bag_equiv_C2 ...
              + W_seats_C2 ...
              + W_diesel_C2 + W_drum_C2 + W_gen_C2;

% --- Print itemized breakdown ---
fprintf(' %-40s %8s %8s\\n', 'Item', 'Mass (kg)', 'Mass (lb)');
fprintf(' %s\\n', repmat('-', 1, 60));
fprintf(' %-40s %8.2f %8.2f\\n', ...
        '--- Passenger Mass Equivalence ---', 0, 0);
fprintf(' %-40s %8.2f %8.2f\\n', ...
        sprintf('%d pax body equiv. (x%.2f kg)', n_pax_C2, W_person_kg), ...
```

```

W_pax_equiv_C2, W_pax_equiv_C2 / kg_per_lb);
fprintf('  %-40s  %8.2f  %8.2f\n', ...
    sprintf('%d pax baggage equiv. (x%.2f kg)', n_pax_C2, W_pax_bag_kg), ...
    W_bag_equiv_C2, W_bag_equiv_C2 / kg_per_lb);
fprintf('  %-40s  %8.2f  %8.2f\n', ...
    sprintf('%d removable seats (x%.0f kg each)', n_seats_C2, W_seat_kg), ...
    W_seats_C2, W_seats_C2 / kg_per_lb);
fprintf('  %-40s  %8.2f  %8.2f\n', ...
    '--- Cargo Items ---', 0, 0);
fprintf('  %-40s  %8.2f  %8.2f\n', ...
    sprintf('55 gal diesel (rho=%.2f kg/L)', rho_diesel_kg_L), ...
    W_diesel_C2, W_diesel_C2 / kg_per_lb);
fprintf('  %-40s  %8.2f  %8.2f\n', ...
    '55-gal plastic drum (Uline S-9945BLU)', ...
    W_drum_C2, W_drum_C2 / kg_per_lb);
fprintf('  %-40s  %8.2f  %8.2f\n', ...
    '8,000 W diesel generator [⚠ VERIFY]', ...
    W_gen_C2, W_gen_C2 / kg_per_lb);
fprintf('  %s\n', repmat('-', 1, 60));
fprintf('  %-40s  %8.2f  %8.2f\n', 'TOTAL CONFIG 2 PAYLOAD', ...
    W_payload_C2, W_payload_C2 / kg_per_lb);
fprintf('\n');
fprintf('  ⚠ FLAG: Generator mass (%.1f kg) conflicts with report draft\n',
W_gen_C2);
fprintf('          which cited 250 kg for a different unit. Reconcile\n');
fprintf('          with manufacturer spec sheet before finalizing.\n');
fprintf('  NOTE:   Config 2 is the SIZING CASE. MTOW is driven by this\n');
fprintf('          payload. Do not reduce without re-running
RaymerWeightEst_v3.m.\n');
fprintf('\n\n');

```

CONFIGURATION 2 – CARGO TRANSPORT (SIZING CASE)

Item	Mass (kg)	Mass (lb)

--- Passenger Mass Equivalence ---	0.00	0.00
9 pax body equiv. (x90.72 kg)	816.47	1800.00
9 pax baggage equiv. (x22.68 kg)	204.12	450.00
9 removable seats (x10 kg each)	90.00	198.42
--- Cargo Items ---	0.00	0.00
55 gal diesel (rho=0.85 kg/L)	176.97	390.15
55-gal plastic drum (Uline S-9945BLU)	11.40	25.13
8,000 W diesel generator [⚠ VERIFY]	99.30	218.92

TOTAL CONFIG 2 PAYLOAD	1398.25	3082.61

⚠ FLAG: Generator mass (99.3 kg) conflicts with report draft
which cited 250 kg for a different unit. Reconcile
with manufacturer spec sheet before finalizing.

NOTE: Config 2 is the SIZING CASE. MTOW is driven by this
payload. Do not reduce without re-running RaymerWeightEst_v3.m.

SECTION 4 – CONFIGURATION 3: AERIAL AMBULANCE (MEDEVAC)

RFP requirements:

- 1 litter (50 lb stretcher)
- 4 occupants: 2 medical technicians, 1 patient, 1 patient escort
(all at 200 lb per RFP)
- 300 lb of medical supplies and equipment
- Total RFP payload: 1,150 lb = 521.6 kg
- NO refueling at remote site – aircraft carries fuel for both legs
- Round trip: outbound (no patient) + 45 min turnaround + inbound (with patient)

TWO LOADING CONDITIONS:

- Outbound: aircraft departs with med techs + litter + supplies
Patient and escort are NOT yet on board
- Inbound: Patient and escort loaded at remote site
Heavier condition – this is the critical structural loading

Baggage assumption: NO baggage for any Config 3 occupant.

Med techs: personal gear subsumed in medical supplies line item.

Patient / escort: emergency evacuation – no baggage.

DESIGN NOTE ON FUEL:

This script does NOT compute fuel weight. However, note that Config 3 requires fuel for 2x cruise legs (350 nmi each) + 4x loiter segments. This will yield a higher fuel fraction than Configs 1 and 2 despite lower payload. Full fuel analysis is handled in RaymerWeightEst_v3.m.

```
=====
fprintf('=====\\n'
);
fprintf('  CONFIGURATION 3 – AERIAL AMBULANCE (MEDEVAC)\\n');
fprintf('=====\\n\\n');

% --- Items present on BOTH outbound and inbound legs ---
W_med_techs_C3    = 2 * W_person_kg;      % 2 medical technicians (no baggage)
W_litter_C3       = W_litter_kg;          % NATO stretcher
W_med equip_C3    = W_med_supplies_kg;    % 300 lb medical supplies (single
line)

% --- Items added ONLY on inbound leg (loaded at remote site) ---
W_patient_C3      = 1 * W_person_kg;      % patient body weight (no baggage)
W_escort_C3       = 1 * W_person_kg;      % patient escort body weight (no
baggage)

% --- Subtotals ---
W_payload_C3_out = W_med_techs_C3 + W_litter_C3 + W_med equip_C3;
```

```

W_payload_C3_in = W_payload_C3_out + W_patient_C3 + W_escort_C3;

% --- OUTBOUND breakdown ---
fprintf(' --- OUTBOUND LEG (no patient on board) ---\n\n');
fprintf(' %-40s %8s %8s\n', 'Item', 'Mass (kg)', 'Mass (lb)');
fprintf(' %s\n', repmat('-', 1, 60));
fprintf(' %-40s %8.2f %8.2f\n', ...
    '2 medical technicians (no baggage)', ...
    W_med_techs_C3, W_med_techs_C3 / kg_per_lb);
fprintf(' %-40s %8.2f %8.2f\n', ...
    'NATO litter / stretcher (50 lb RFP)', ...
    W_litter_C3, W_litter_C3 / kg_per_lb);
fprintf(' %-40s %8.2f %8.2f\n', ...
    'Medical supplies & equipment (300 lb RFP) [⚠]', ...
    W_med_equip_C3, W_med_equip_C3 / kg_per_lb);
fprintf(' %s\n', repmat('-', 1, 60));
fprintf(' %-40s %8.2f %8.2f\n', 'TOTAL OUTBOUND PAYLOAD', ...
    W_payload_C3_out, W_payload_C3_out / kg_per_lb);
fprintf('\n');

% --- INBOUND breakdown ---
fprintf(' --- INBOUND LEG (patient and escort loaded) ---\n\n');
fprintf(' %-40s %8s %8s\n', 'Item', 'Mass (kg)', 'Mass (lb)');
fprintf(' %s\n', repmat('-', 1, 60));
fprintf(' %-40s %8.2f %8.2f\n', ...
    '2 medical technicians (no baggage)', ...
    W_med_techs_C3, W_med_techs_C3 / kg_per_lb);
fprintf(' %-40s %8.2f %8.2f\n', ...
    'NATO litter / stretcher (50 lb RFP)', ...
    W_litter_C3, W_litter_C3 / kg_per_lb);
fprintf(' %-40s %8.2f %8.2f\n', ...
    'Medical supplies & equipment (300 lb RFP) [⚠]', ...
    W_med_equip_C3, W_med_equip_C3 / kg_per_lb);
fprintf(' %-40s %8.2f %8.2f\n', ...
    '1 patient body weight (no baggage)', ...
    W_patient_C3, W_patient_C3 / kg_per_lb);
fprintf(' %-40s %8.2f %8.2f\n', ...
    '1 patient escort body weight (no baggage)', ...
    W_escort_C3, W_escort_C3 / kg_per_lb);
fprintf(' %s\n', repmat('-', 1, 60));
fprintf(' %-40s %8.2f %8.2f\n', 'TOTAL INBOUND PAYLOAD', ...
    W_payload_C3_in, W_payload_C3_in / kg_per_lb);
fprintf('\n');
fprintf(' NOTE: No baggage for any Config 3 occupant (emergency ops).\n');
fprintf(' ⚠ FLAG: Medical supplies (%.2f kg) is an RFP aggregate.\n',
W_med_equip_C3);
fprintf('          Replace with itemized list once equipment is
selected.\n');
fprintf('\n\n');

```

=====

CONFIGURATION 3 – AERIAL AMBULANCE (MEDEVAC)

=====

--- OUTBOUND LEG (no patient on board) ---

Item	Mass (kg)	Mass (lb)
2 medical technicians (no baggage)	181.44	400.00
NATO litter / stretcher (50 lb RFP)	22.68	50.00
Medical supplies & equipment (300 lb RFP) [▲]	136.08	300.00
TOTAL OUTBOUND PAYLOAD	340.19	750.00

--- INBOUND LEG (patient and escort loaded) ---

Item	Mass (kg)	Mass (lb)
2 medical technicians (no baggage)	181.44	400.00
NATO litter / stretcher (50 lb RFP)	22.68	50.00
Medical supplies & equipment (300 lb RFP) [▲]	136.08	300.00
1 patient body weight (no baggage)	90.72	200.00
1 patient escort body weight (no baggage)	90.72	200.00
TOTAL INBOUND PAYLOAD	521.63	1150.00

NOTE: No baggage for any Config 3 occupant (emergency ops).

▲ FLAG: Medical supplies (136.08 kg) is an RFP aggregate.
Replace with itemized list once equipment is selected.

=====

=====

SECTION 5 – SUMMARY TABLE

Comparison of all configurations against the MTOW sizing anchor.
Helps verify payload fractions are consistent with RaymerWeightEst_v3.m.

```
=====
fprintf('=====\\n'
);
fprintf('  PAYLOAD SUMMARY – ALL CONFIGURATIONS\\n');
fprintf('=====\\n\\n');

fprintf(' %-35s  %10s  %10s  %10s\\n', 'Configuration', 'Payload(kg)', ...
  'Payload(lb)', '% of MTOW');
fprintf('  %s\\n', repmat('-', 1, 70));
fprintf(' %-35s  %10.2f  %10.2f  %10.1f\\n', ...
  'Config 1 – Pax Transport', ...
  W_payload_C1, W_payload_C1 / kg_per_lb, ...
  100 * W_payload_C1 / MTOW_kg);
fprintf(' %-35s  %10.2f  %10.2f  %10.1f\\n', ...
  'Config 2 – Cargo (SIZING CASE)', ...
  W_payload_C2, W_payload_C2 / kg_per_lb, ...
```

```

    100 * W_payload_C2 / MTOW_kg);
fprintf('  %-35s  %10.2f  %10.2f  %10.1f\n', ...
        'Config 3 - Medevac Outbound', ...
        W_payload_C3_out, W_payload_C3_out / kg_per_lb, ...
        100 * W_payload_C3_out / MTOW_kg);
fprintf('  %-35s  %10.2f  %10.2f  %10.1f\n', ...
        'Config 3 - Medevac Inbound', ...
        W_payload_C3_in, W_payload_C3_in / kg_per_lb, ...
        100 * W_payload_C3_in / MTOW_kg);
fprintf('\n');
fprintf('  MTOW anchor (Config 2, converged): %.0f N  /  %.2f kg\n', ...
        MTOW_N, MTOW_kg);
fprintf('  OEW is fixed at this sizing point. Fuel sizing in
RaymerWeightEst_v3.m.\n');
fprintf('\n\n');

```

PAYLOAD SUMMARY - ALL CONFIGURATIONS

Configuration	Payload(kg)	Payload(lb)	% of MTOW
Config 1 - Pax Transport	1220.37	2690.46	29.9
Config 2 - Cargo (SIZING CASE)	1398.25	3082.61	34.3
Config 3 - Medevac Outbound	340.19	750.00	8.3
Config 3 - Medevac Inbound	521.63	1150.00	12.8

MTOW anchor (Config 2, converged): 40000 N / 4077.47 kg
 OEW is fixed at this sizing point. Fuel sizing in RaymerWeightEst_v3.m.

SECTION 6 - CSV EXPORT

Writes one table per configuration. All masses in kg and lb.

File: PayloadBuildUp_v1.csv

Intended to replace the manual spreadsheet (Sheet2) as the authoritative payload record. Update this script, not the spreadsheet, going forward.

```

csv_path = 'PayloadBuildUp_v1.csv';
fid = fopen(csv_path, 'w');

% --- Header row convention ---
% Configuration, Item, Mass_kg, Mass_lb, Notes

fprintf(fid, 'Configuration,Item,Mass_kg,Mass_lb,Notes\n');

% --- Config 1 ---
fprintf(fid, 'Config 1 - Passenger Transport,');
fprintf(fid, '%d passengers body weight,%.2f,%.2f,RFP: 200 lb each\n', ...

```

```

    n_pax_C1, W_pax_body_C1, W_pax_body_C1/kg_per_lb);

fprintf(fid, 'Config 1 - Passenger Transport,');
fprintf(fid, '%d passengers baggage,%.2f,%.2f,RFP: 50 lb each\n', ...
    n_pax_C1, W_pax_bag_C1, W_pax_bag_C1/kg_per_lb);

fprintf(fid, 'Config 1 - Passenger Transport,');
fprintf(fid, '1 flight attendant body weight,%.2f,%.2f,Design addition - not
in RFP\n', ...
    W_FA_body_C1, W_FA_body_C1/kg_per_lb);

fprintf(fid, 'Config 1 - Passenger Transport,');
fprintf(fid, '1 FA baggage (pilot allowance),%.2f,%.2f,20 lb pilot allowance
per design decision\n', ...
    W_FA_bag_C1, W_FA_bag_C1/kg_per_lb);

fprintf(fid, 'Config 1 - Passenger Transport,');
fprintf(fid, '%d removable seats (10 kg each),%.2f,%.2f,9 pax + 1 FA
seat\n', ...
    n_seats_C1, W_seats_C1, W_seats_C1/kg_per_lb);

fprintf(fid, 'Config 1 - Passenger Transport,');
fprintf(fid, 'TOTAL,%.2f,%.2f,\n', W_payload_C1, W_payload_C1/kg_per_lb);

fprintf(fid, '\n');

% --- Config 2 ---
fprintf(fid, 'Config 2 - Cargo Transport (SIZING CASE),');
fprintf(fid, '%d pax body weight equivalent,%.2f,%.2f,Mass equivalence per
RFP\n', ...
    n_pax_C2, W_pax_equiv_C2, W_pax_equiv_C2/kg_per_lb);

fprintf(fid, 'Config 2 - Cargo Transport (SIZING CASE),');
fprintf(fid, '%d pax baggage equivalent,%.2f,%.2f,Mass equivalence per
RFP\n', ...
    n_pax_C2, W_bag_equiv_C2, W_bag_equiv_C2/kg_per_lb);

fprintf(fid, 'Config 2 - Cargo Transport (SIZING CASE),');
fprintf(fid, '%d removable seats (10 kg each),%.2f,%.2f,9 pax seats only -
no FA seat in Config 2\n', ...
    n_seats_C2, W_seats_C2, W_seats_C2/kg_per_lb);

fprintf(fid, 'Config 2 - Cargo Transport (SIZING CASE),');
fprintf(fid, '55 gal diesel cargo fuel (rho=%.2f kg/L),%.2f,%.2f,Delivered
to remote site\n', ...
    rho_diesel_kg_L, W_diesel_C2, W_diesel_C2/kg_per_lb);

fprintf(fid, 'Config 2 - Cargo Transport (SIZING CASE),');
fprintf(fid, '55-gal plastic drum (Uline S-9945BLU),%.2f,%.2f,VERIFY with
final drum selection\n', ...
    W_drum_C2, W_drum_C2/kg_per_lb);

fprintf(fid, 'Config 2 - Cargo Transport (SIZING CASE),');
fprintf(fid, '8000 W diesel generator,%.2f,%.2f,FLAG: VERIFY mass - conflict

```

```

with 250 kg in report draft\n', ...
    W_gen_C2, W_gen_C2/kg_per_lb);

fprintf(fid, 'Config 2 - Cargo Transport (SIZING CASE),');
fprintf(fid, 'TOTAL,%.2f,%.2f,SIZING CASE - drives MTOW\n', ...
    W_payload_C2, W_payload_C2/kg_per_lb);

fprintf(fid, '\n');

% --- Config 3 Outbound ---
fprintf(fid, 'Config 3 - Medevac Outbound,');
fprintf(fid, '2 medical technicians (no baggage),%.2f,%.2f,Emergency ops -
no personal baggage\n', ...
    W_med_techs_C3, W_med_techs_C3/kg_per_lb);

fprintf(fid, 'Config 3 - Medevac Outbound,');
fprintf(fid, 'NATO litter / stretcher,%.2f,%.2f,RFP: 50 lb\n', ...
    W_litter_C3, W_litter_C3/kg_per_lb);

fprintf(fid, 'Config 3 - Medevac Outbound,');
fprintf(fid, 'Medical supplies and equipment,%.2f,%.2f,RFP: 300 lb aggregate
- itemize when equipment selected\n', ...
    W_med equip_C3, W_med equip_C3/kg_per_lb);

fprintf(fid, 'Config 3 - Medevac Outbound,');
fprintf(fid, 'TOTAL,%.2f,%.2f,No patient on outbound leg\n', ...
    W_payload_C3_out, W_payload_C3_out/kg_per_lb);

fprintf(fid, '\n');

% --- Config 3 Inbound ---
fprintf(fid, 'Config 3 - Medevac Inbound,');
fprintf(fid, '2 medical technicians (no baggage),%.2f,%.2f,Emergency ops -
no personal baggage\n', ...
    W_med_techs_C3, W_med_techs_C3/kg_per_lb);

fprintf(fid, 'Config 3 - Medevac Inbound,');
fprintf(fid, 'NATO litter / stretcher,%.2f,%.2f,RFP: 50 lb\n', ...
    W_litter_C3, W_litter_C3/kg_per_lb);

fprintf(fid, 'Config 3 - Medevac Inbound,');
fprintf(fid, 'Medical supplies and equipment,%.2f,%.2f,RFP: 300 lb
aggregate\n', ...
    W_med equip_C3, W_med equip_C3/kg_per_lb);

fprintf(fid, 'Config 3 - Medevac Inbound,');
fprintf(fid, '1 patient body weight (no baggage),%.2f,%.2f,Loaded at remote
site\n', ...
    W_patient_C3, W_patient_C3/kg_per_lb);

fprintf(fid, 'Config 3 - Medevac Inbound,');
fprintf(fid, '1 patient escort body weight (no baggage),%.2f,%.2f,Loaded at
remote site\n', ...
    W_escort_C3, W_escort_C3/kg_per_lb);

```

```

fprintf(fid, 'Config 3 - Medevac Inbound,');
fprintf(fid, 'TOTAL,%.2f,%.2f,Heaviest Config 3 condition\n', ...
    W_payload_C3_in, W_payload_C3_in/kg_per_lb);

fprintf(fid, '\n');

% --- Summary ---
fprintf(fid, 'SUMMARY,Config 1 - Pax Transport,%.2f,%.2f,\n', ...
    W_payload_C1, W_payload_C1/kg_per_lb);
fprintf(fid, 'SUMMARY,Config 2 - Cargo (SIZING CASE),%.2f,%.2f,\n', ...
    W_payload_C2, W_payload_C2/kg_per_lb);
fprintf(fid, 'SUMMARY,Config 3 - Medevac Outbound,%.2f,%.2f,\n', ...
    W_payload_C3_out, W_payload_C3_out/kg_per_lb);
fprintf(fid, 'SUMMARY,Config 3 - Medevac Inbound,%.2f,%.2f,\n', ...
    W_payload_C3_in, W_payload_C3_in/kg_per_lb);
fprintf(fid, 'SUMMARY,MTOW anchor (Config 2 converged),%.2f,%.2f,40000 N /
4077.5 kg\n', ...
    MTOW_kg, MTOW_kg/kg_per_lb);

fclose(fid);

fprintf('=====\n'
);
fprintf(' CSV EXPORT COMPLETE\n');
fprintf(' File written: %s\n', csv_path);
fprintf('=====\n\
n');
fprintf(' ⚠ DESIGN FLAGS SUMMARY:\n');
fprintf(' 1. Generator mass: 99.3 kg used here vs 250 kg in report
draft.\n');
fprintf(' Pull manufacturer spec sheet and reconcile before next
iteration.\n');
fprintf(' 2. Medical supplies (136 kg) is an RFP aggregate.\n');
fprintf(' Itemize once equipment manifest is finalized.\n');
fprintf(' 3. Seat mass (10 kg each) is a Recaro BL 3530 estimate.\n');
fprintf(' Confirm once seat selection is locked.\n');
fprintf(' 4. FA is a DESIGN ADDITION beyond the RFP. Adds %.2f kg\n', ...
    W_FA_body_C1 + W_FA_bag_C1 + W_seat_kg);
fprintf(' to Config 1 payload vs an RFP-only reading.\n');
fprintf(' If FA is removed, Config 1 payload drops to %.2f kg.\n', ...
    W_payload_C1 - W_FA_body_C1 - W_FA_bag_C1 - W_seat_kg);
fprintf('\n');

```

CSV EXPORT COMPLETE

File written: PayloadBuildUp_v1.csv

⚠ DESIGN FLAGS SUMMARY:

1. Generator mass: 99.3 kg used here vs 250 kg in report draft.
Pull manufacturer spec sheet and reconcile before next iteration.
2. Medical supplies (136 kg) is an RFP aggregate.
Itemize once equipment manifest is finalized.

-
3. Seat mass (10 kg each) is a Recaro BL 3530 estimate.
Confirm once seat selection is locked.
 4. FA is a DESIGN ADDITION beyond the RFP. Adds 109.79 kg
to Config 1 payload vs an RFP-only reading.
If FA is removed, Config 1 payload drops to 1110.58 kg.

Published with MATLAB® R2025b

```

% CruiseOptimization_v3.m
% =====
% Arctic STOL Aircraft – Cruise Altitude Optimization & AR/S Trade Study
% Version 3
% =====
% CHANGES FROM v2:
%   - Ceiling search now sweeps independently to 15,000 m (uncapped),
%     fixing the bug where absolute ceiling == 130 KTAS ceiling.
%   - Crossover point (V_LDmax = 130 KTAS) found within altitude sweep
%     and marked on all 4 Figure 1 subplots.
%   - Section 4 console output includes crossover point breakdown.
%   - Figure 2 now has two side-by-side subplots:
%       Left : V_LDmax at 15,000 ft with colour bands (from v2)
%       Right: Percent of drag above D_opt at 130 KTAS (new)
%
% CEILING METHOD:
%   Absolute ceiling = first altitude where sigma*P_A_sl < P_R(V_Pmin,W_To).
%   P_R_min is evaluated at V_Pmin = sqrt(2*W_To/(rho*S*CL_Pmin))
%   where CL_Pmin = sqrt(3*CD0/k) (speed for minimum power required).
%   This is independent of the 130 KTAS cruise speed constraint.
%
%   The 130 KTAS power ceiling is found separately as the highest altitude
%   where sigma*P_A_sl >= P_R(130 KTAS, W_mid). These two altitudes will
%   in general differ; the 130 KTAS ceiling is always <= absolute ceiling.
%
% CROSSOVER POINT:
%   The altitude at which V_LDmax(W_mid) = 130 KTAS.
%   Above this altitude, the aircraft can fly at both 130 KTAS AND L/D_max
%   simultaneously (speed constraint no longer active at minimum drag).
%   Below it, 130 KTAS > V_LDmax (speed-constrained, drag > D_min).
%
% CALLS: StdAtmos(h [m])
% =====

clear; clc; close all;

=====

=====

SECTION 1 – DESIGN PARAMETERS
=====

W_To = 39867;      % Max takeoff weight           [N]
S     = 39.8;      % Wing reference area          [m^2]
AR    = 6.85;      % Aspect ratio                 [-]
b     = sqrt(S*AR); % Wing span                    [m]
CD0   = 0.03;      % Parasite drag, clean cruise  [-]
e     = 0.9;       % Oswald efficiency factor     [-]
zeta  = 0.130;     % Mission fuel fraction Wf/W_To  [-]
W_mid = W_To * (1 - zeta / 2); % Mid-cruise weight [N]

```

```

P_shp = 1050;           % Rated shaft horsepower, PT6A-60A      [shp]
eta_p = 0.80;           % Propeller efficiency                  [-]
% Useful propulsive power at sea level. eta_p applied ONCE here.
P_A_sl = eta_p * P_shp * 745.7; % [W] = 626,388 W

V_min_ms = 130 * 0.5144; % 130 KTAS in m/s                    [m/s]
h_O2_all = 15000 * 0.3048; % 15,000 ft in metres              [m]
g = 9.81;
rho_sl = 1.225;

fprintf('=== SECTION 1: DESIGN PARAMETERS ===\n')
fprintf(' W_To      : %.1f N   (%.1f kg)\n', W_To, W_To/g)
fprintf(' W_mid      : %.1f N   (%.1f kg)   [W_To*(1-zeta/2)]\n', W_mid, W_mid/g)
fprintf(' S          : %.2f m^2 | AR : %.2f | b : %.2f m\n', S, AR, b)
fprintf(' CD0        : %.4f | e : %.2f | zeta : %.3f\n', CD0, e, zeta)
fprintf(' P_shp      : %.0f shp | eta_p : %.2f | P_A_sl : %.2f kW\n\n', ...
        P_shp, eta_p, P_A_sl/1e3)

=== SECTION 1: DESIGN PARAMETERS ===
W_To      : 39867.0 N   (4063.9 kg)
W_mid      : 37275.6 N   (3799.8 kg)   [W_To*(1-zeta/2)]
S          : 39.80 m^2 | AR : 6.85 | b : 16.51 m
CD0        : 0.0300 | e : 0.90 | zeta : 0.130
P_shp      : 1050 shp | eta_p : 0.80 | P_A_sl : 626.39 kW

=====

=====

SECTION 2 – AERODYNAMIC QUANTITIES (altitude-independent)
=====

k          = 1 / (pi * AR * e);

% L/D_max operating point (minimum drag, best range for propeller aircraft)
CL_opt = sqrt(CD0 / k); % CL at L/D_max      [-]
CD_opt = 2 * CD0; % CD at L/D_max (= 2*CD0)  [-]
LD_max = CL_opt / CD_opt; % Maximum L/D      [-]

% V_Pmin operating point (minimum power required – used for ceiling)
% d(P_R)/dV = 0 → CL_Pmin = sqrt(3*CD0/k), CD_Pmin = 4*CD0
CL_Pmin = sqrt(3 * CD0 / k); % CL at minimum P_R      [-]
CD_Pmin = 4 * CD0; % CD at minimum P_R      [-]
% NOTE: V_Pmin = 3^(-1/4) * V_LDmax ≈ 0.760 * V_LDmax at any altitude.
% They do NOT converge – the user note about "same speed at ceiling" refers
% to the practical operating sense, not an aerodynamic identity.

fprintf('=== SECTION 2: AERODYNAMIC QUANTITIES ===\n')
fprintf(' k          : %.5f\n', k)
fprintf(' CL_opt     : %.4f | CD_opt : %.4f | LD_max : %.3f\n', ...
        CL_opt, CD_opt, LD_max)
fprintf(' CL_Pmin    : %.4f | CD_Pmin : %.4f\n', CL_Pmin, CD_Pmin)

```

```
fprintf(' V_Pmin/V_LDmax ratio : %.4f (always = 3^(-1/4) = 0.7598)\n\n',
3^(-0.25))
```

```
=== SECTION 2: AERODYNAMIC QUANTITIES ===
```

```

k          : 0.05163
CL_opt     : 0.7623 | CD_opt   : 0.0600 | LD_max    : 12.704
CL_Pmin    : 1.3203 | CD_Pmin  : 0.1200
V_Pmin/V_LDmax ratio : 0.7598 (always = 3^(-1/4) = 0.7598)

```

```
=====
```

```
=====
```

SECTION 3 – CEILING CALCULATIONS

Two separate sweeps to 15,000 m to avoid the cap-induced equality bug:

(A) Absolute ceiling : $P_A = P_R(V_{Pmin}, W_{To})$ – V_{Pmin} minimizes P_R

(B) 130 KTAS ceiling : $P_A = P_R(130 \text{ KTAS}, W_{mid})$ – fixed cruise speed

These use W_{To} and W_{mid} respectively for consistency with their roles.

The absolute ceiling uses W_{To} (heaviest aircraft – most conservative).

```
h_search = linspace(0, 15000, 10000); % Dense search to 15,000 m [m]
```

```
h_abs_ceil = NaN;
```

```
h_130_ceil = NaN;
```

```
for i = 1:length(h_search)
```

```
    [~, rho_i, ~, ~] = StdAtmos(h_search(i));
```

```
    sigma_i = rho_i / rho_sl;
```

```
    PA_i     = sigma_i * P_A_sl;
```

```
% — (A) Absolute ceiling
```

```
% P_R_min evaluated at V_Pmin and W_To
```

```
if isnan(h_abs_ceil)
```

```
    V_Pm_i = sqrt(2 * W_To / (rho_i * S * CL_Pmin));
```

```
    q_Pm_i = 0.5 * rho_i * V_Pm_i^2;
```

```
    PR_min_i = CD_Pmin * q_Pm_i * S * V_Pm_i; % = D_Pmin * V_Pmin
```

```
    if PA_i < PR_min_i
```

```
        h_abs_ceil = h_search(i);
```

```
    end
```

```
end
```

```
% — (B) 130 KTAS power ceiling
```

```
% P_R evaluated at fixed V = 130 KTAS and W_To
```

```
if isnan(h_130_ceil)
```

```
    q_130_i = 0.5 * rho_i * V_min_ms^2;
```

```
    CL_130_i = W_To / (q_130_i * S);
```

```
    CD_130_i = CD0 + k * CL_130_i^2;
```

```
    D_130_i = CD_130_i * q_130_i * S;
```

```
    PR_130_i = D_130_i * V_min_ms;
```

```
    if PA_i < PR_130_i
```

```
        h_130_ceil = h_search(i);
```

```
    end
```

```
end
```

```

        if ~isnan(h_abs_ceil) && ~isnan(h_130_ceil)
            break % Both found - stop early
        end
    end

if isnan(h_abs_ceil); h_abs_ceil = h_search(end); warning('Abs ceiling >
15,000 m'); end
if isnan(h_130_ceil); h_130_ceil = h_search(end); warning('130 KTAS ceiling
> 15,000 m'); end

fprintf('=== SECTION 3: CEILING RESULTS ===\n')
fprintf(' Absolute ceiling (P_A = P_R_min at V_Pmin, W_To) : %.0f m (%.0f
ft)\n', ...
    h_abs_ceil, h_abs_ceil/0.3048)
fprintf(' 130 KTAS ceiling (P_A = P_R at 130 KTAS, W_To) : %.0f m (%.0f
ft)\n\n', ...
    h_130_ceil, h_130_ceil/0.3048)

=== SECTION 3: CEILING RESULTS ===
Absolute ceiling (P_A = P_R_min at V_Pmin, W_To) : 9779 m (32085 ft)
130 KTAS ceiling (P_A = P_R at 130 KTAS, W_To) : 9655 m (31676 ft)

=====

=====

SECTION 4 - ALTITUDE SWEEP (0 to absolute ceiling)
At each altitude, ALL quantities evaluated at V = 130 KTAS.
V_LDmax is computed at W_mid for the crossover search.
=====

n_pts = 800;
h_vec = linspace(0, h_abs_ceil, n_pts); % [m]

V_LDmax_vec = zeros(1, n_pts);
CL_vec = zeros(1, n_pts);
LD_vec = zeros(1, n_pts);
D_p_vec = zeros(1, n_pts); % Parasite drag at 130 KTAS
D_i_vec = zeros(1, n_pts); % Induced drag at 130 KTAS
D_vec = zeros(1, n_pts); % Total drag at 130 KTAS
P_R_vec = zeros(1, n_pts);
P_A_vec = zeros(1, n_pts);
excess_vec = zeros(1, n_pts);

for i = 1:n_pts
    [~, rho_i, ~, ~] = StdAtmos(h_vec(i));
    sigma_i = rho_i / rho_sl;

    % V_LDmax at this altitude (uses W_mid to match the crossover definition)
    V_LDmax_vec(i) = sqrt(2 * W_mid / (rho_i * S * CL_opt));

    % Aerodynamics at 130 KTAS, W_mid, level flight

```

```

    q_i          = 0.5 * rho_i * V_min_ms^2;
    CL_vec(i)     = W_mid / (q_i * S);
    D_p_vec(i)    = CD0 * q_i * S;
    D_i_vec(i)    = k * W_mid^2 / (q_i * S);
    D_vec(i)      = D_p_vec(i) + D_i_vec(i);
    LD_vec(i)     = W_mid / D_vec(i);
    P_R_vec(i)    = D_vec(i) * V_min_ms;
    P_A_vec(i)    = sigma_i * P_A_sl;
    excess_vec(i) = P_A_vec(i) - P_R_vec(i);
end

% — Find crossover: V_LDmax = 130 KTAS —————
% V_LDmax increases monotonically with altitude. Find the first index where
% V_LDmax >= V_min_ms and interpolate to get the precise crossing altitude.
idx_cross = find(V_LDmax_vec >= V_min_ms, 1, 'first');
if ~isempty(idx_cross) && idx_cross > 1
    % Linear interpolation between idx_cross-1 and idx_cross
    h_lo = h_vec(idx_cross-1); V_lo = V_LDmax_vec(idx_cross-1);
    h_hi = h_vec(idx_cross);   V_hi = V_LDmax_vec(idx_cross);
    h_cross = h_lo + (V_min_ms - V_lo) / (V_hi - V_lo) * (h_hi - h_lo);
else
    h_cross = NaN;
    warning('V_LDmax crossover not found within ceiling sweep.')
end

% Properties at crossover altitude (interpolated)
if ~isnan(h_cross)
    [~, rho_cross, ~, ~] = StdAtmos(h_cross);
    sigma_cross = rho_cross / rho_sl;
    % At crossover: V_LDmax = V_130 = V_min_ms
    % Aircraft is now flying at exactly minimum drag speed
    q_cross     = 0.5 * rho_cross * V_min_ms^2;
    CL_cross     = W_mid / (q_cross * S);           % should equal CL_opt
    Dp_cross     = CD0 * q_cross * S;
    Di_cross     = k * W_mid^2 / (q_cross * S);
    D_cross      = Dp_cross + Di_cross;             % = D_min at this altitude
    LD_cross     = W_mid / D_cross;
    PR_cross     = D_cross * V_min_ms;
    PA_cross     = sigma_cross * P_A_sl;
    exc_cross    = PA_cross - PR_cross;
    thr_cross    = PR_cross / PA_cross;
    D_min_cross  = W_mid / LD_max;                 % theoretical minimum drag (= D_cross)
end

% Properties at 15,000 ft specifically
 [~, rho_15k, ~, ~] = StdAtmos(h_O2_all);
sigma_15k    = rho_15k / rho_sl;
q_15k       = 0.5 * rho_15k * V_min_ms^2;
CL_15k      = W_mid / (q_15k * S);
Dp_15k      = CD0 * q_15k * S;
Di_15k      = k * W_mid^2 / (q_15k * S);
D_15k       = Dp_15k + Di_15k;
LD_15k      = W_mid / D_15k;
PR_15k      = D_15k * V_min_ms;

```

```

PA_15k      = sigma_15k * P_A_sl;
exc_15k     = PA_15k - PR_15k;
thr_15k     = PR_15k / PA_15k;
D_opt_15k   = W_mid / LD_max;      % minimum possible drag at W_mid
pct_above   = (D_15k - D_opt_15k) / D_opt_15k * 100;
V_LDmax_15k = sqrt(2 * W_mid / (rho_15k * S * CL_opt));

fprintf('=== SECTION 4: ALTITUDE SWEEP SUMMARY ===\n')
fprintf(' Altitude sweep range : 0 to %.0f m (absolute ceiling)\n\n',
h_abs_ceil)

if ~isnan(h_cross)
    fprintf(' --- Crossover Point (V_LDmax = 130 KTAS) ---\n')
    fprintf(' Altitude      : %.7f m  (%.0f ft)\n', h_cross, h_cross/
0.3048)
    fprintf(' Air density    : %.4f kg/m^3  (sigma = %.4f)\n', rho_cross,
sigma_cross)
    fprintf(' V_cruise      : %.2f m/s  (130.0 KTAS)\n', V_min_ms)
    fprintf(' CL (= CL_opt) : %.4f  (deficit from CL_opt : %.4f)\n',
CL_cross, CL_opt-CL_cross)
    fprintf(' L/D (= LD_max) : %.2f\n', LD_cross)
    fprintf(' D_parasite    : %.1f N  (%5.1f%% of total)\n', Dp_cross,
100*Dp_cross/D_cross)
    fprintf(' D_induced     : %.1f N  (%5.1f%% of total)\n', Di_cross,
100*Di_cross/D_cross)
    fprintf(' D_total      : %.1f N  (100.0%% - at D_min by definition)
\n', D_cross)
    fprintf(' P_R          : %.2f kW\n', PR_cross/1e3)
    fprintf(' P_A          : %.2f kW  (sigma-scaled)\n', PA_cross/1e3)
    fprintf(' Throttle     : %.1f%%\n', thr_cross*100)
    fprintf(' Excess power  : %.2f kW\n\n', exc_cross/1e3)
end

fprintf(' --- 15,000 ft (%.0f m) at 130 KTAS ---\n', h_O2_all)
fprintf(' Air density    : %.4f kg/m^3  (sigma = %.4f)\n', rho_15k,
sigma_15k)
fprintf(' V_cruise      : %.2f m/s  (130.0 KTAS)\n', V_min_ms)
fprintf(' V_LDmax here   : %.2f m/s  (%.1f KTAS)\n', V_LDmax_15k,
V_LDmax_15k/0.5144)
fprintf(' CL (actual)    : %.4f  (CL_opt = %.4f, deficit = %.4f)\n',
CL_15k, CL_opt, CL_opt-CL_15k)
fprintf(' L/D (actual)   : %.2f  (LD_max = %.2f, deficit = %.2f)\n',
LD_15k, LD_max, LD_max-LD_15k)
fprintf(' D_parasite     : %.1f N  (%5.1f%% of total drag)\n', Dp_15k,
100*Dp_15k/D_15k)
fprintf(' D_induced      : %.1f N  (%5.1f%% of total drag)\n', Di_15k,
100*Di_15k/D_15k)
fprintf(' D_total        : %.1f N  (%.1f%% above D_opt = %.1f N)\n',
D_15k, pct_above, D_opt_15k)
fprintf(' P_R           : %.2f kW\n', PR_15k/1e3)
fprintf(' P_A           : %.2f kW  (sigma-scaled)\n', PA_15k/1e3)
fprintf(' Throttle      : %.1f%%\n', thr_15k*100)
fprintf(' Excess power   : %.2f kW\n', exc_15k/1e3)
if exc_15k >= 0

```

```

        fprintf('  Status           : FEASIBLE at 130 KTAS, 15,000 ft\n\n')
else
        fprintf('  Status           : INFEASIBLE (P_A < P_R at 15,000 ft / 130
KTAS)\n\n')
end

```

```

=== SECTION 4: ALTITUDE SWEEP SUMMARY ===

```

```

Altitude sweep range : 0 to 9779 m (absolute ceiling)

```

```

--- Crossover Point (V_LDmax = 130 KTAS) ---

```

```

Altitude       : 7608 m (24961 ft)
Air density     : 0.5495 kg/m^3 (sigma = 0.4486)
V_cruise       : 66.87 m/s (130.0 KTAS)
CL (= CL_opt)   : 0.7623 (deficit from CL_opt : 0.0000)
L/D (= LD_max)  : 12.70
D_parasite      : 1467.0 N ( 50.0% of total)
D_induced       : 1467.0 N ( 50.0% of total)
D_total         : 2934.1 N (100.0% - at D_min by definition)
P_R             : 196.21 kW
P_A             : 280.99 kW (sigma-scaled)
Throttle        : 69.8%
Excess power     : 84.78 kW

```

```

--- 15,000 ft (4572 m) at 130 KTAS ---

```

```

Air density     : 0.7707 kg/m^3 (sigma = 0.6292)
V_cruise       : 66.87 m/s (130.0 KTAS)
V_LDmax here    : 56.47 m/s (109.8 KTAS)
CL (actual)     : 0.5435 (CL_opt = 0.7623, deficit = 0.2188)
L/D (actual)    : 12.01 (LD_max = 12.70, deficit = 0.69)
D_parasite      : 2057.6 N ( 66.3% of total drag)
D_induced       : 1046.0 N ( 33.7% of total drag)
D_total         : 3103.6 N (5.8% above D_opt = 2934.1 N)
P_R             : 207.54 kW
P_A             : 394.10 kW (sigma-scaled)
Throttle        : 52.7%
Excess power     : 186.56 kW
Status          : FEASIBLE at 130 KTAS, 15,000 ft

```

```

=====

```

```

=====

```

```

SECTION 5 - BREGUET RANGE CHECK

```

```

=====

```

```

SFCp_SI = 0.548 * 0.4536 / (745.7 * 3600);
R_m      = (eta_p / (g * SFCp_SI)) * LD_15k * log(W_To / (W_To*(1-zeta)));
R_nmi    = R_m / 1852;

```

```

fprintf('=== SECTION 5: BREGUET RANGE CHECK (130 KTAS, 15,000 ft) ===\n')
fprintf('  L/D at cruise : %.2f\n', LD_15k)
fprintf('  Range        : %.0f m (%.0f nmi)\n', R_m, R_nmi)
fprintf('  Requirement   : >= 450 nmi\n')

```

```

if R_nmi >= 450
    fprintf(' Status           : PASS  (=%.0f nmi margin)\n\n', R_nmi-450)
else
    fprintf(' Status           : FAIL  (=%.0f nmi short)\n\n', 450-R_nmi)
end

```

```

=== SECTION 5: BREGUET RANGE CHECK (130 KTAS, 15,000 ft) ===
L/D at cruise : 12.01
Range         : 1473082 m (795 nmi)
Requirement   : >= 450 nmi
Status        : PASS (+345 nmi margin)

```

```

=====
=====

```

FIGURE 1 – ALTITUDE SWEEP (4 subplots, all axes in SI)
Crossover point marked on all 4 subplots.
Reference lines: 15,000 ft O2 threshold, absolute ceiling,
130 KTAS power ceiling, crossover altitude.

```

=====

col_O2      = [0.75 0.00 0.75]; % magenta - 15,000 ft O2 line
col_ceil    = [0.40 0.40 0.40]; % grey   - absolute ceiling
col_130c    = [0.00 0.55 0.00]; % green   - 130 KTAS power ceiling
col_cross   = [0.85 0.45 0.00]; % orange  - crossover point

% Marker data at crossover (for all subplots)
if ~isnan(h_cross)
    cross_LD  = LD_cross;
    cross_PR  = PR_cross / 1e3;
    cross_PA  = PA_cross / 1e3;
    cross_exc = exc_cross / 1e3;
    cross_VLD = V_min_ms; % by definition at crossover
end

figure('Name', 'Figure 1: Cruise Altitude Optimization', ...
    'Units', 'normalized', 'Position', [0.02 0.05 0.88 0.82]);

% — Subplot 1: V_LDmax vs altitude —————
subplot(2,2,1); hold on;
plot(h_vec, V_LDmax_vec, 'b-', 'LineWidth', 2.0, 'DisplayName', 'V_{LDmax} (W_{mid})');
yline(V_min_ms, 'k--', 'LineWidth', 1.5, ...
    'DisplayName', sprintf('130 KTAS = %.2f m/s', V_min_ms));
xline(h_O2_all, '--', 'Color', col_O2, 'LineWidth', 1.3, ...
    'DisplayName', sprintf('15,000 ft = %.0f m (O_2)', h_O2_all));
xline(h_abs_ceil, '-', 'Color', col_ceil, 'LineWidth', 1.5, ...
    'DisplayName', sprintf('Abs. ceiling = %.0f m', h_abs_ceil));
xline(h_130_ceil, '-', 'Color', col_130c, 'LineWidth', 1.5, ...
    'DisplayName', sprintf('130 KTAS ceiling = %.0f m', h_130_ceil));
if ~isnan(h_cross)
    plot(h_cross, cross_VLD, 'o', 'Color', col_cross, 'MarkerSize', 10, ...

```

```

        'MarkerFaceColor', col_cross, ...
        'DisplayName', sprintf('Crossover = %.0f m', h_cross));
text(h_cross + h_abs_ceil*0.02, cross_VLD - 1.5, ...
    sprintf('%.0f m\n(%.0f ft)', h_cross, h_cross/0.3048), ...
    'Color', col_cross, 'FontSize', 8);
end
xlabel('Altitude [m]', 'FontSize', 11);
ylabel('Speed [m/s]', 'FontSize', 11);
title('Min-Drag Speed vs Altitude', 'FontSize', 11);
legend('Location', 'northwest', 'FontSize', 7.5);
grid on;

% — Subplot 2: L/D at 130 KTAS vs altitude —————
subplot(2,2,2); hold on;
plot(h_vec, LD_vec, 'b-', 'LineWidth', 2.0, 'DisplayName', 'L/D at 130
KTAS');
yline(LD_max, 'k--', 'LineWidth', 1.5, ...
    'DisplayName', sprintf('L/D_{max} = %.2f', LD_max));
xline(h_O2_all, '--', 'Color', col_O2, 'LineWidth', 1.3, ...
    'DisplayName', sprintf('15,000 ft = %.0f m', h_O2_all));
xline(h_abs_ceil, '-', 'Color', col_ceil, 'LineWidth', 1.5, ...
    'DisplayName', sprintf('Abs. ceiling = %.0f m', h_abs_ceil));
xline(h_130_ceil, '-', 'Color', col_130c, 'LineWidth', 1.5, ...
    'DisplayName', sprintf('130 KTAS ceiling = %.0f m', h_130_ceil));
if ~isnan(h_cross)
    plot(h_cross, cross_LD, 'o', 'Color', col_cross, 'MarkerSize', 10, ...
        'MarkerFaceColor', col_cross, ...
        'DisplayName', sprintf('Crossover = %.0f m', h_cross));
    text(h_cross + h_abs_ceil*0.02, cross_LD - 0.3, ...
        sprintf('L/D = %.2f\n= L/D_{max}', cross_LD), ...
        'Color', col_cross, 'FontSize', 8);
end
xlabel('Altitude [m]', 'FontSize', 11);
ylabel('L/D [-]', 'FontSize', 11);
title('L/D at 130 KTAS vs Altitude', 'FontSize', 11);
legend('Location', 'northwest', 'FontSize', 7.5);
grid on;

% — Subplot 3: Power vs altitude —————
subplot(2,2,3); hold on;
plot(h_vec, P_R_vec/1e3, 'r-', 'LineWidth', 2.0, 'DisplayName', 'P_R (130
KTAS)');
plot(h_vec, P_A_vec/1e3, 'b-', 'LineWidth', 2.0, 'DisplayName', 'P_A (\sigma-
scaled)');
xline(h_O2_all, '--', 'Color', col_O2, 'LineWidth', 1.3, ...
    'DisplayName', sprintf('15,000 ft = %.0f m', h_O2_all));
xline(h_abs_ceil, '-', 'Color', col_ceil, 'LineWidth', 1.5, ...
    'DisplayName', sprintf('Abs. ceiling = %.0f m', h_abs_ceil));
xline(h_130_ceil, '-', 'Color', col_130c, 'LineWidth', 1.5, ...
    'DisplayName', sprintf('130 KTAS ceiling = %.0f m', h_130_ceil));
if ~isnan(h_cross)
    plot(h_cross, cross_PR, 'o', 'Color', col_cross, 'MarkerSize', 10, ...
        'MarkerFaceColor', col_cross, 'DisplayName', sprintf('Crossover =
%.0f m', h_cross));

```

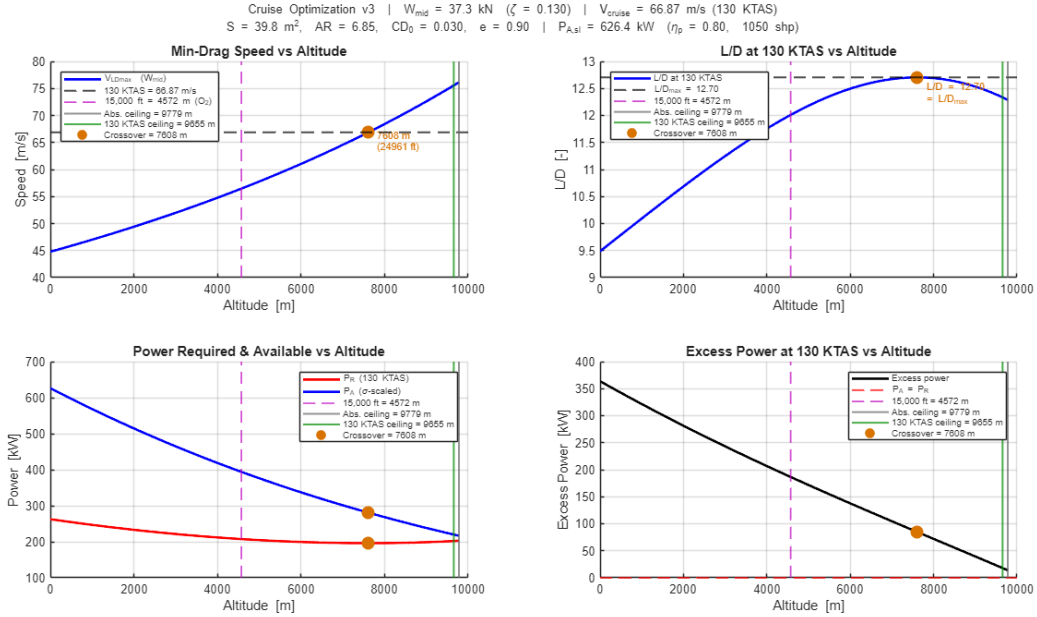
```

        plot(h_cross, cross_PA, 'o', 'Color', col_cross, 'MarkerSize', 10, ...
            'MarkerFaceColor', col_cross, 'HandleVisibility', 'off');
    end
    xlabel('Altitude [m]', 'FontSize', 11);
    ylabel('Power [kW]', 'FontSize', 11);
    title('Power Required & Available vs Altitude', 'FontSize', 11);
    legend('Location', 'northeast', 'FontSize', 7.5);
    grid on;

% — Subplot 4: Excess power vs altitude —————
subplot(2,2,4); hold on;
plot(h_vec, excess_vec/1e3, 'k-', 'LineWidth', 2.0, 'DisplayName', 'Excess
power');
yline(0, 'r--', 'LineWidth', 1.5, 'DisplayName', 'P_A = P_R');
xline(h_O2_all, '--', 'Color', col_O2, 'LineWidth', 1.3, ...
    'DisplayName', sprintf('15,000 ft = %.0f m', h_O2_all));
xline(h_abs_ceil, '-', 'Color', col_ceil, 'LineWidth', 1.5, ...
    'DisplayName', sprintf('Abs. ceiling = %.0f m', h_abs_ceil));
xline(h_130_ceil, '-', 'Color', col_130c, 'LineWidth', 1.5, ...
    'DisplayName', sprintf('130 KTAS ceiling = %.0f m', h_130_ceil));
if ~isnan(h_cross)
    plot(h_cross, cross_exc, 'o', 'Color', col_cross, 'MarkerSize', 10, ...
        'MarkerFaceColor', col_cross, ...
        'DisplayName', sprintf('Crossover = %.0f m', h_cross));
end
xlabel('Altitude [m]', 'FontSize', 11);
ylabel('Excess Power [kW]', 'FontSize', 11);
title('Excess Power at 130 KTAS vs Altitude', 'FontSize', 11);
legend('Location', 'northeast', 'FontSize', 7.5);
grid on;

sgtitle(sprintf(['Cruise Optimization v3 | W_{mid} = %.1f kN (\\zeta =
%.3f) | ' ...
    'V_{cruise} = %.2f m/s (130 KTAS)\\n' ...
    'S = %.1f m^2, AR = %.2f, CD_0 = %.3f, e = %.2f | ' ...
    'P_{A,sl} = %.1f kW (\\eta_p = %.2f, %.0f shp)'], ...
    W_mid/1e3, zeta, V_min_ms, S, AR, CD0, e, P_A_sl/1e3, eta_p, P_shp), ...
    'FontSize', 10);

```



=====

=====

FIGURE 2 – AR vs S: TWO SUBPLOTS

Left (2,1,1): V_{LDmax} at 15,000 ft – colour bands by % below 130 KTAS
 Right (2,1,2): % drag above D_{opt} at 130 KTAS, 15,000 ft – jet colorbar

```
=====
nAR = 200;  nS = 200;
AR_vec2 = linspace(4, 14, nAR);
S_vec2 = linspace(20, 90, nS);

V_LDmax_grid = zeros(nAR, nS);  % [m/s]
pct_drag_grid = zeros(nAR, nS);  % [%]

for i = 1:nAR
    for j = 1:nS
        AR_ij = AR_vec2(i);
        S_ij = S_vec2(j);
        k_ij = 1 / (pi * AR_ij * e);

        % CL_opt and LD_max for this (AR, S)
        CL_opt_ij = sqrt(CD0 / k_ij);
        CD_opt_ij = 2 * CD0;
        LD_max_ij = CL_opt_ij / CD_opt_ij;

        % V_LDmax at 15,000 ft (W_mid)
        V_LDmax_grid(i,j) = sqrt(2 * W_mid / (rho_15k * S_ij * CL_opt_ij));

        % Drag at 130 KTAS, 15,000 ft for this (AR, S)
        CL_130_ij = W_mid / (q_15k * S_ij);
```

```

    CD_130_ij    = CD0 + k_ij * CL_130_ij^2;
    D_130_ij     = CD_130_ij * q_15k * S_ij;

    % Optimal (minimum) drag at W_mid
    D_opt_ij     = W_mid / LD_max_ij;

    % Percent above optimal
    pct_drag_grid(i,j) = (D_130_ij - D_opt_ij) / D_opt_ij * 100;
end
end

% — Colour bands for left subplot —————
T100 = 1.00 * V_min_ms; % >= 130 KTAS      → green
T90  = 0.90 * V_min_ms; % 90-100%         → light green
T80  = 0.80 * V_min_ms; % 80-90%          → yellow
T70  = 0.70 * V_min_ms; % 70-80%          → light red
      % < 70%                             → dark red

z_green = V_LDmax_grid >= T100;
z_lgn   = (V_LDmax_grid >= T90) & ~z_green;
z_yell   = (V_LDmax_grid >= T80) & ~z_green & ~z_lgn;
z_lred   = (V_LDmax_grid >= T70) & ~z_green & ~z_lgn & ~z_yell;
z_dred   = V_LDmax_grid < T70;

rgb2 = zeros(nAR, nS, 3);
% Green [0.10 0.60 0.10]
rgb2(:, :, 1) = rgb2(:, :, 1) + 0.10 * z_green; rgb2(:, :, 2) = rgb2(:, :, 2) + 0.60 * z_green;
rgb2(:, :, 3) = rgb2(:, :, 3) + 0.10 * z_green;
% Light green [0.60 0.92 0.60]
rgb2(:, :, 1) = rgb2(:, :, 1) + 0.60 * z_lgn;   rgb2(:, :, 2) = rgb2(:, :, 2) + 0.92 * z_lgn;
rgb2(:, :, 3) = rgb2(:, :, 3) + 0.60 * z_lgn;
% Yellow [0.95 0.90 0.20]
rgb2(:, :, 1) = rgb2(:, :, 1) + 0.95 * z_yell;   rgb2(:, :, 2) = rgb2(:, :, 2) + 0.90 * z_yell;
rgb2(:, :, 3) = rgb2(:, :, 3) + 0.20 * z_yell;
% Light red [0.95 0.55 0.55]
rgb2(:, :, 1) = rgb2(:, :, 1) + 0.95 * z_lred;   rgb2(:, :, 2) = rgb2(:, :, 2) + 0.55 * z_lred;
rgb2(:, :, 3) = rgb2(:, :, 3) + 0.55 * z_lred;
% Dark red [0.65 0.08 0.08]
rgb2(:, :, 1) = rgb2(:, :, 1) + 0.65 * z_dred;   rgb2(:, :, 2) = rgb2(:, :, 2) + 0.08 * z_dred;
rgb2(:, :, 3) = rgb2(:, :, 3) + 0.08 * z_dred;

figure('Name', 'Figure 2: AR vs S Trade Study', ...
      'Units', 'normalized', 'Position', [0.04 0.05 0.92 0.80]);

% — Left subplot: V_LDmax colour bands —————
subplot(1,2,1);
imagesc(S_vec2, AR_vec2, rgb2);
set(gca, 'YDir', 'normal');
hold on;

% Threshold boundary contours (dashed black)
contour(S_vec2, AR_vec2, V_LDmax_grid, [T100 T100], 'k-', 'LineWidth', 2.2);
contour(S_vec2, AR_vec2, V_LDmax_grid, [T90 T90], 'k--', 'LineWidth', 1.6);
contour(S_vec2, AR_vec2, V_LDmax_grid, [T80 T80], 'k--', 'LineWidth', 1.6);

```

```

contour(S_vec2, AR_vec2, V_LDmax_grid, [T70 T70], 'k--', 'LineWidth', 1.6);

% Speed labels (white contours)
[Cs, hs] = contour(S_vec2, AR_vec2, V_LDmax_grid, 8, ...
    'Color', [1 1 1], 'LineWidth', 0.8);
clabel(Cs, hs, 'FontSize', 7.5, 'Color', [1 1 1], 'LabelSpacing', 220);

% Current design point
plot(S, AR, 'w^', 'MarkerSize', 12, 'MarkerFaceColor', 'w', 'LineWidth',
1.5);
text(S+0.8, AR+0.12, sprintf('Design\nAR=%.2f\nS=%.1f m^2', AR, S), ...
    'Color', 'w', 'FontSize', 8, 'FontWeight', 'bold');

% Legend proxies
hp(1)=patch(NaN,NaN,[0.10 0.60 0.10], 'EdgeColor', 'none', ...
    'DisplayName', sprintf('\geq 130 KTAS (\geq %.2f m/s)', T100));
hp(2)=patch(NaN,NaN,[0.60 0.92 0.60], 'EdgeColor', 'none', ...
    'DisplayName', sprintf('90-100%% (%.2f-%.2f m/s)', T90, T100));
hp(3)=patch(NaN,NaN,[0.95 0.90 0.20], 'EdgeColor', 'none', ...
    'DisplayName', sprintf('80-90%% (%.2f-%.2f m/s)', T80, T90));
hp(4)=patch(NaN,NaN,[0.95 0.55 0.55], 'EdgeColor', 'none', ...
    'DisplayName', sprintf('70-80%% (%.2f-%.2f m/s)', T70, T80));
hp(5)=patch(NaN,NaN,[0.65 0.08 0.08], 'EdgeColor', 'none', ...
    'DisplayName', sprintf('< 70%% (< %.2f m/s)', T70));
hp(6)=plot(NaN,NaN,'w^', 'MarkerSize', 10, 'MarkerFaceColor', 'w', ...
    'DisplayName', 'Current design point');
legend(hp, 'Location', 'northeast', 'FontSize', 7.5, ...
    'Color', [0.15 0.15 0.15], 'TextColor', 'w', 'EdgeColor', [0.4 0.4 0.4]);

xlabel('Wing Area S [m^2]', 'FontSize', 12);
ylabel('Aspect Ratio AR [-]', 'FontSize', 12);
title({'V_{LDmax} at 15,000 ft vs AR and S', ...
    '(contour values in m/s)'}, 'FontSize', 11);
grid on; ax1=gca; ax1.GridColor=[1 1 1]; ax1.GridAlpha=0.18; ax1.FontSize=10;
xlim([S_vec2(1) S_vec2(end)]); ylim([AR_vec2(1) AR_vec2(end)]);

% — Right subplot: % drag above D_opt at 130 KTAS —————
subplot(1,2,2);
% Cap display range for colorbar readability; extreme values stay visible
pct_display = min(pct_drag_grid, 200);
contourf(S_vec2, AR_vec2, pct_display, 30, 'LineColor', 'none');
colormap(gca, jet);
cb = colorbar;
cb.Label.String = '% Drag above D_{opt} at 130 KTAS, 15,000 ft';
cb.Label.FontSize = 10;
clim([0 150]); % 0-150% range; cells above 150% are max-red
hold on;

% Labelled iso-lines for navigation
[Cp, hp2] = contour(S_vec2, AR_vec2, pct_drag_grid, ...
    [10 25 50 75 100 150], ...
    'Color', [0.15 0.15 0.15], 'LineWidth', 0.9);
clabel(Cp, hp2, 'FontSize', 8, 'Color', [0.05 0.05 0.05]);

```

```

% Current design point
plot(S, AR, 'w^', 'MarkerSize', 12, 'MarkerFaceColor', 'w', 'LineWidth',
1.5);
text(S+0.8, AR+0.12, sprintf('Design\n%.0f%% above\nD_{opt}', ...
(D_15k-D_opt_15k)/D_opt_15k*100), ...
'Color','w','FontSize',8,'FontWeight','bold');

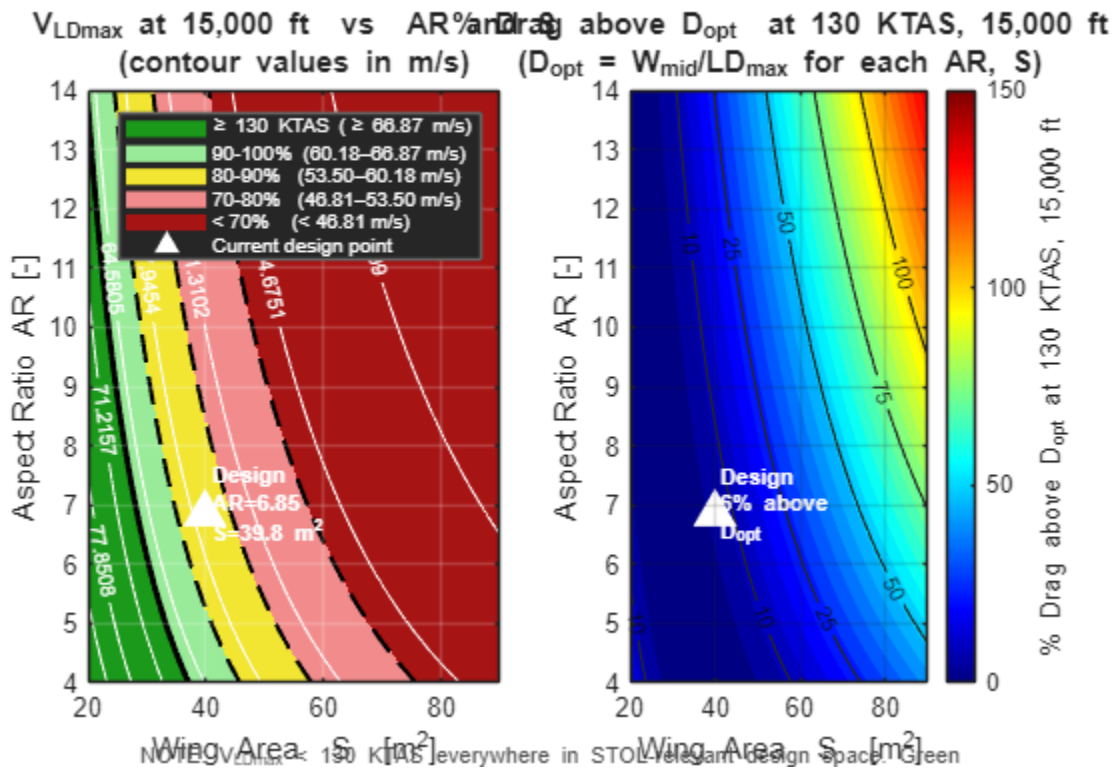
xlabel('Wing Area S [m^2]', 'FontSize', 12);
ylabel('Aspect Ratio AR [-]', 'FontSize', 12);
title(['% Drag above D_{opt} at 130 KTAS, 15,000 ft', ...
'(D_{opt} = W_{mid}/LD_{max} for each AR, S)'], 'FontSize', 11);
grid on; ax2=gca; ax2.FontSize=10;
xlim([S_vec2(1) S_vec2(end)]); ylim([AR_vec2(1) AR_vec2(end)]);

sgtitle(sprintf(['AR vs S Trade Study | W_{mid} = %.1f kN | ' ...
'CD_0 = %.3f | e = %.2f | 15,000 ft = %.0f m'], ...
W_mid/1e3, CD0, e, h_O2_all), 'FontSize', 11);

annotation('textbox', [0.13 0.005 0.75 0.033], ...
'String', ['NOTE: V_{LDmax} < 130 KTAS everywhere in STOL-relevant
design space. ' ...
'Green region (small S, high AR) trades cruise efficiency for longer
field length.'], ...
'FontSize', 8, 'EdgeColor','none','HorizontalAlignment','center', ...
'Color',[0.30 0.30 0.30]);

```

S Trade Study | $W_{mid} = 37.3$ kN | $CD_0 = 0.030$ | $e = 0.90$ | 15,000 ft = 4



Published with MATLAB® R2025b

```

function [state] = CeilingCheck(P_A_sl, W, S, AR, CD0)
    % Does a check given plane geometry and engine selection to see if
    % altitude requirements are met.
    rho_sl = 1.225; % density of air at sea level
    e = 0.9; % project constant
    k = 1/(pi*e*AR);
    CL_Dmin = sqrt(CD0/k);

    h_req = 5487; % 18000ft altitude in meters
    [~, rho_req, ~, ~] = StdAtmos(h_req); % calls Standard Atmosphere
    V_Dmin = sqrt(2*W/(rho_req*S*CL_Dmin));

    sigma = rho_req/rho_sl;

    q_req = 0.5 * rho_req * V_Dmin^2;
    CDmin = 2*CD0;
    Dmin = q_req * S * CDmin;
    P_R = Dmin * V_Dmin;

    P_A = sigma * P_A_sl;

    if P_R > P_A
        state = false;
    else
        state = true;
    end
end

Not enough input arguments.

Error in CeilingCheck (line 6)
    k = 1/(pi*e*AR);
           ^^

```

Published with MATLAB® R2025b

Stability and Control Tail Sizing

Objective: Size the horizontal and vertical stabilizers using the tail volume coefficient method.

Given: Wing area, $S = 36.8 \text{ m}^2$ Wing span, $b = 15.2 \text{ m}$ Aircraft CG is approximately 4 m from the nose

Method: Horizontal Tail Volume Coefficient: $C_{HT} = (S_{HT} * L_{HT}) / (S_{wing} * c_{bar})$

Vertical Tail Volume Coefficient: $C_{VT} = (S_{VT} * L_{VT}) / (S_{wing} * b)$

Rearranged: $S_{HT} = (C_{HT} * S_{wing} * c_{bar}) / L_{HT}$ $S_{VT} = (C_{VT} * S_{wing} * b) / L_{VT}$

Assumptions: 1. Aircraft has a conventional tail. 2. Tail aerodynamic centers are assumed to be at the quarter-chord points of the horizontal and vertical stabilizers. 3. Wing aerodynamic center is assumed to be at the quarter-chord of the mean aerodynamic chord. 4. Tail moment arms are measured from aircraft CG to tail aerodynamic centers. 5. Tail volume coefficients are initial sizing estimates and should be refined later with static margin, trim, elevator/rudder authority, and dynamic stability checks. 6. All dimensions are in SI units.

```
clc; clear; close all
```

Given Wing Geometry

```
S_wing = 36.8;           % Wing reference area, m^2
b_wing = 15.2;           % Wing span, m

AR_wing = b_wing^2 / S_wing; % Wing aspect ratio
c_bar = S_wing / b_wing;    % Approximate mean aerodynamic chord, m

fprintf('Wing Geometry:\n')
fprintf('Wing area, S = %.2f m^2\n', S_wing)
fprintf('Wing span, b = %.2f m\n', b_wing)
fprintf('Wing aspect ratio, AR = %.2f\n', AR_wing)
fprintf('Mean aerodynamic chord, c_bar = %.2f m\n\n', c_bar)

Wing Geometry:
Wing area, S = 36.80 m^2
Wing span, b = 15.20 m
Wing aspect ratio, AR = 6.28
Mean aerodynamic chord, c_bar = 2.42 m
```

Longitudinal Locations

```
x_nose = 0;             % Nose reference location, m
x_CG = 4.0;              % CG location from nose, m

% Assumed aircraft geometry
% Change these once CAD or layout dimensions are known
x_wing_AC = 4.3;         % Wing aerodynamic center from nose, m
x_HT_AC = 9.5;           % Horizontal tail aerodynamic center from nose, m
x_VT_AC = 9.3;           % Vertical tail aerodynamic center from nose, m
```

```

% Moment arms from CG to aerodynamic centers
l_wing_to_CG = x_CG - x_wing_AC;      % Positive if CG is aft of wing AC
L_HT = x_HT_AC - x_CG;                % Horizontal tail moment arm, m
L_VT = x_VT_AC - x_CG;                % Vertical tail moment arm, m

fprintf('Longitudinal Locations:\n')
fprintf('CG location from nose = %.2f m\n', x_CG)
fprintf('Wing AC location from nose = %.2f m\n', x_wing_AC)
fprintf('Horizontal tail AC location from nose = %.2f m\n', x_HT_AC)
fprintf('Vertical tail AC location from nose = %.2f m\n', x_VT_AC)
fprintf('Wing AC to CG distance = %.2f m\n', l_wing_to_CG)
fprintf('Horizontal tail moment arm, L_HT = %.2f m\n', L_HT)
fprintf('Vertical tail moment arm, L_VT = %.2f m\n\n', L_VT)

Longitudinal Locations:
CG location from nose = 4.00 m
Wing AC location from nose = 4.30 m
Horizontal tail AC location from nose = 9.50 m
Vertical tail AC location from nose = 9.30 m
Wing AC to CG distance = -0.30 m
Horizontal tail moment arm, L_HT = 5.50 m
Vertical tail moment arm, L_VT = 5.30 m

```

Tail Volume Coefficients

```

% Typical initial values from conceptual aircraft design methods.
% These should be changed depending on aircraft type.
%
% C_HT typical range:
%   Sailplane: ~0.5
%   General aviation: ~0.7
%   Jet transport: ~1.0
%
% C_VT typical range:
%   General aviation: ~0.04
%   Jet transport: ~0.08
%   Fighter: ~0.09

C_HT = 0.70;      % Horizontal tail volume coefficient
C_VT = 0.04;      % Vertical tail volume coefficient

fprintf('Tail Volume Coefficients:\n')
fprintf('Horizontal tail volume coefficient, C_HT = %.3f\n', C_HT)
fprintf('Vertical tail volume coefficient, C_VT = %.3f\n\n', C_VT)

Tail Volume Coefficients:
Horizontal tail volume coefficient, C_HT = 0.700
Vertical tail volume coefficient, C_VT = 0.040

```

Tail Area Sizing

```
S_HT = (C_HT * S_wing * c_bar) / L_HT;           % Horizontal tail area, m^2
S_VT = (C_VT * S_wing * b_wing) / L_VT;         % Vertical tail area, m^2

fprintf('Initial Tail Sizing Results:\n')
fprintf('Horizontal stabilizer area, S_HT = %.2f m^2\n', S_HT)
fprintf('Vertical stabilizer area, S_VT = %.2f m^2\n\n', S_VT)

fprintf('Area Ratios:\n')
fprintf('S_HT / S_wing = %.3f\n', S_HT / S_wing)
fprintf('S_VT / S_wing = %.3f\n\n', S_VT / S_wing)

Initial Tail Sizing Results:
Horizontal stabilizer area, S_HT = 11.34 m^2
Vertical stabilizer area, S_VT = 4.22 m^2

Area Ratios:
S_HT / S_wing = 0.308
S_VT / S_wing = 0.115
```

Tail Planform Assumptions

```
% Horizontal stabilizer assumptions
AR_HT = 4.0;           % Horizontal tail aspect ratio
lambda_HT = 0.45;      % Horizontal tail taper ratio
sweep_HT_deg = 5;      % Horizontal tail quarter-chord sweep, deg

% Vertical stabilizer assumptions
AR_VT = 1.6;           % Vertical tail aspect ratio
lambda_VT = 0.50;      % Vertical tail taper ratio
sweep_VT_deg = 25;     % Vertical tail quarter-chord sweep, deg
```

Horizontal Tail Planform

```
b_HT = sqrt(AR_HT * S_HT);           % Horizontal tail span, m
c_root_HT = 2 * S_HT / (b_HT * (1 + lambda_HT)); % Root chord, m
c_tip_HT = lambda_HT * c_root_HT;     % Tip chord, m
MAC_HT = (2/3) * c_root_HT * ...
    ((1 + lambda_HT + lambda_HT^2) / (1 + lambda_HT));

fprintf('Horizontal Tail Planform:\n')
fprintf('Horizontal tail aspect ratio, AR_HT = %.2f\n', AR_HT)
fprintf('Horizontal tail span, b_HT = %.2f m\n', b_HT)
fprintf('Horizontal tail root chord = %.2f m\n', c_root_HT)
fprintf('Horizontal tail tip chord = %.2f m\n', c_tip_HT)
fprintf('Horizontal tail MAC = %.2f m\n', MAC_HT)
fprintf('Horizontal tail sweep = %.2f deg\n\n', sweep_HT_deg)

Horizontal Tail Planform:
Horizontal tail aspect ratio, AR_HT = 4.00
```

Horizontal tail span, $b_{HT} = 6.73$ m
Horizontal tail root chord = 2.32 m
Horizontal tail tip chord = 1.05 m
Horizontal tail MAC = 1.76 m
Horizontal tail sweep = 5.00 deg

Vertical Tail Planform

```
% For vertical tail:
% AR_VT = h_VT^2 / S_VT
% where h_VT is vertical tail height.

h_VT = sqrt(AR_VT * S_VT); % Vertical tail height, m
c_root_VT = 2 * S_VT / (h_VT * (1 + lambda_VT)); % Root chord, m
c_tip_VT = lambda_VT * c_root_VT; % Tip chord, m
MAC_VT = (2/3) * c_root_VT * ...
    ((1 + lambda_VT + lambda_VT^2) / (1 + lambda_VT));

fprintf('Vertical Tail Planform:\n')
fprintf('Vertical tail aspect ratio, AR_VT = %.2f\n', AR_VT)
fprintf('Vertical tail height, h_VT = %.2f m\n', h_VT)
fprintf('Vertical tail root chord = %.2f m\n', c_root_VT)
fprintf('Vertical tail tip chord = %.2f m\n', c_tip_VT)
fprintf('Vertical tail MAC = %.2f m\n', MAC_VT)
fprintf('Vertical tail sweep = %.2f deg\n\n', sweep_VT_deg)

Vertical Tail Planform:
Vertical tail aspect ratio, AR_VT = 1.60
Vertical tail height, h_VT = 2.60 m
Vertical tail root chord = 2.17 m
Vertical tail tip chord = 1.08 m
Vertical tail MAC = 1.68 m
Vertical tail sweep = 25.00 deg
```

Static Margin Estimate

```
% This is a simple placeholder estimate.
% A real static margin calculation requires lift curve slopes,
% downwash gradient, tail efficiency, and exact CG/AC locations.

static_margin_target = 0.10; % 10 percent MAC, typical starting target
SM_m = static_margin_target * c_bar;

x_NP_target = x_CG + SM_m; % Desired neutral point location

fprintf('Simple Static Margin Target:\n')
fprintf('Target static margin = %.1f percent MAC\n', static_margin_target *
100)
fprintf('Target static margin distance = %.2f m\n', SM_m)
fprintf('Approximate target neutral point location = %.2f m from nose\n\n',
x_NP_target)
```

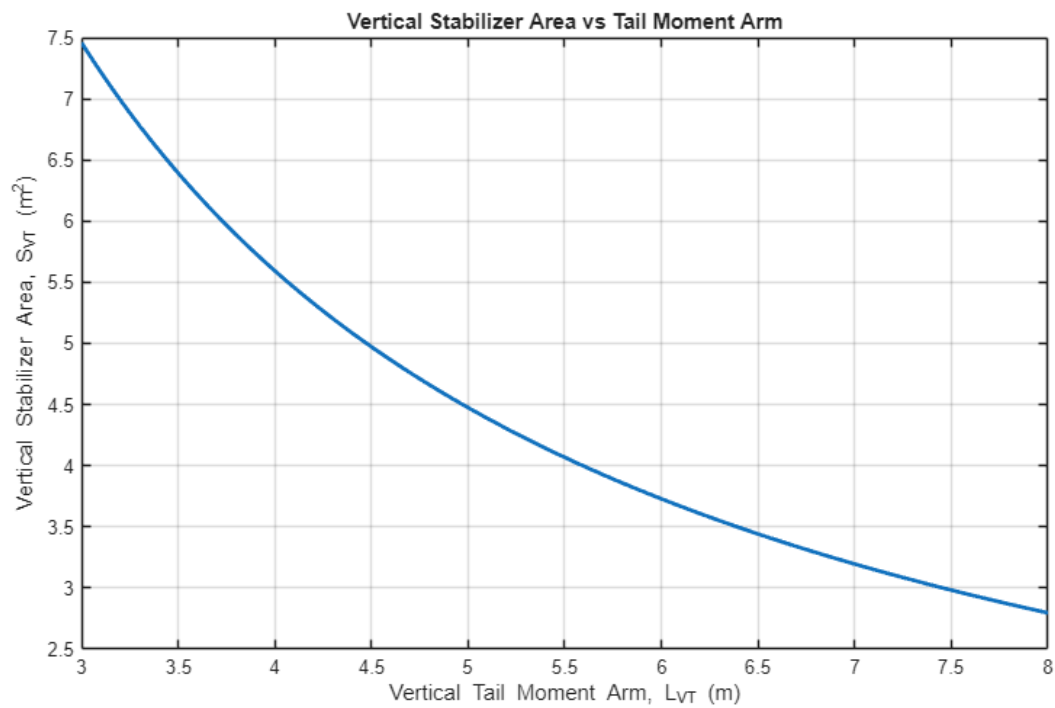
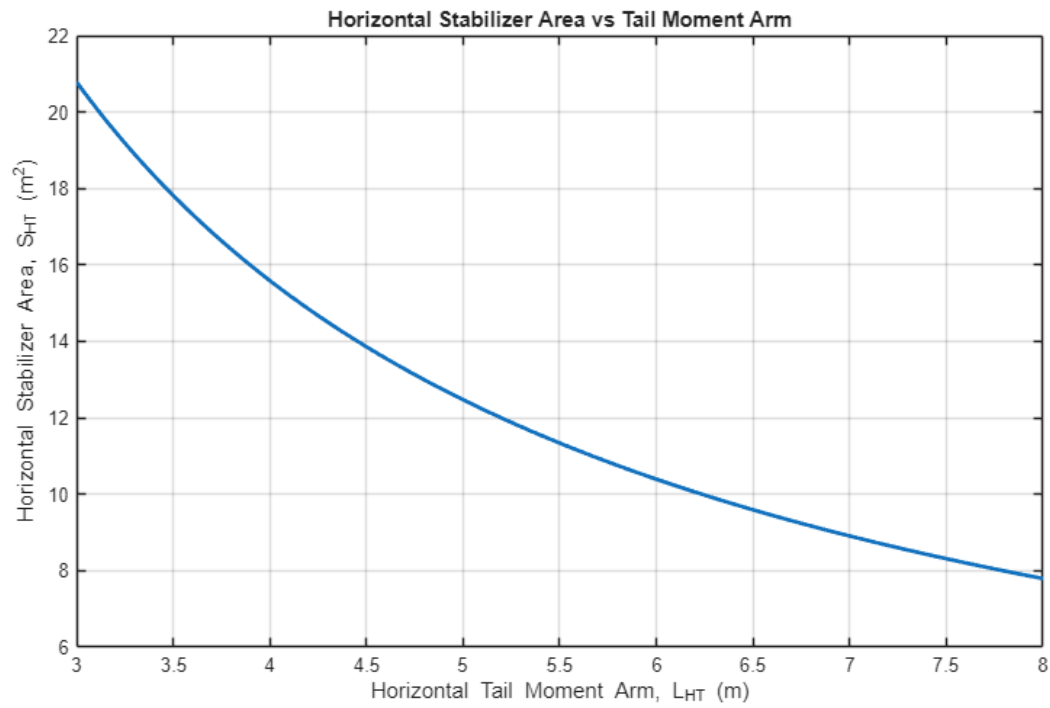
Simple Static Margin Target:
Target static margin = 10.0 percent MAC
Target static margin distance = 0.24 m
Approximate target neutral point location = 4.24 m from nose

Elevator and Rudder Initial Sizing

```
% Typical initial control surface ratios.  
% These should be refined using trim and control authority analysis.  
  
elevator_chord_ratio = 0.30;      % Elevator chord / horizontal tail chord  
rudder_chord_ratio = 0.35;      % Rudder chord / vertical tail chord  
  
S_elevator = elevator_chord_ratio * S_HT;  
S_rudder = rudder_chord_ratio * S_VT;  
  
fprintf('Initial Control Surface Sizing:\n')  
fprintf('Elevator area estimate = %.2f m^2\n', S_elevator)  
fprintf('Rudder area estimate = %.2f m^2\n\n', S_rudder)  
  
Initial Control Surface Sizing:  
Elevator area estimate = 3.40 m^2  
Rudder area estimate = 1.48 m^2
```

Plot Tail Size Sensitivity to Tail Arm

```
L_HT_range = linspace(3.0, 8.0, 100);  
L_VT_range = linspace(3.0, 8.0, 100);  
  
S_HT_range = (C_HT * S_wing * c_bar) ./ L_HT_range;  
S_VT_range = (C_VT * S_wing * b_wing) ./ L_VT_range;  
  
figure  
plot(L_HT_range, S_HT_range, 'LineWidth', 2)  
grid on  
xlabel('Horizontal Tail Moment Arm, L_{HT} (m)')  
ylabel('Horizontal Stabilizer Area, S_{HT} (m^2)')  
title('Horizontal Stabilizer Area vs Tail Moment Arm')  
  
figure  
plot(L_VT_range, S_VT_range, 'LineWidth', 2)  
grid on  
xlabel('Vertical Tail Moment Arm, L_{VT} (m)')  
ylabel('Vertical Stabilizer Area, S_{VT} (m^2)')  
title('Vertical Stabilizer Area vs Tail Moment Arm')
```



Simple Top-View Tail Layout Plot

```
figure
hold on
grid on
```

```
axis equal

% Fuselage centerline
plot([0, 10.5], [0, 0], 'k--', 'LineWidth', 1.5)

% CG
plot(x_CG, 0, 'ro', 'MarkerFaceColor', 'r')
text(x_CG, 0.25, 'CG')

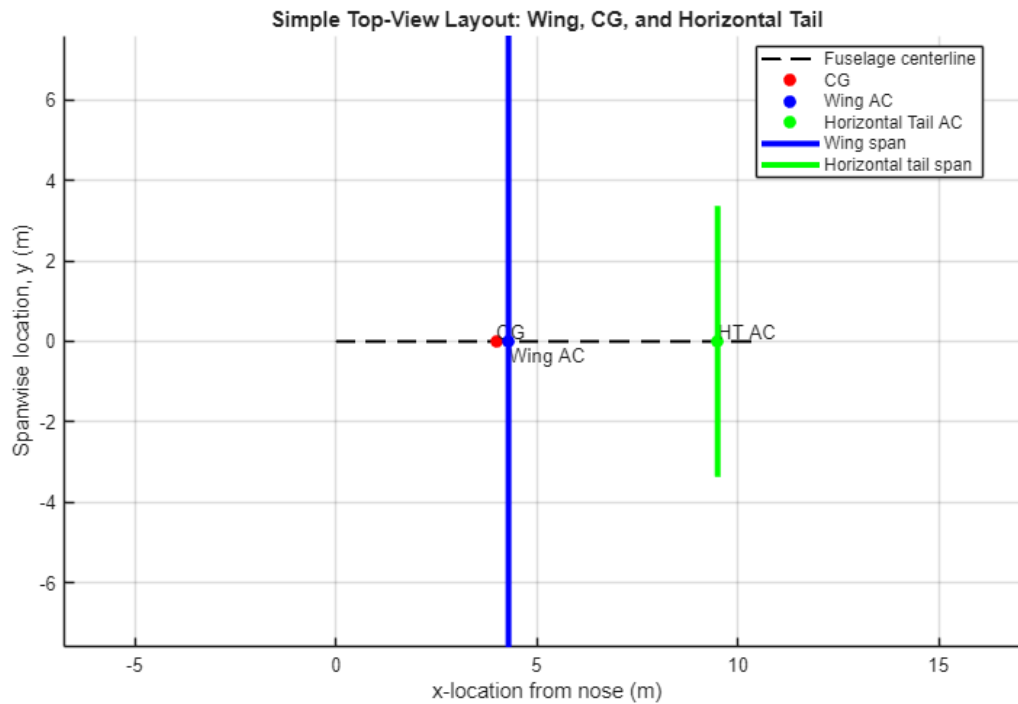
% Wing AC
plot(x_wing_AC, 0, 'bo', 'MarkerFaceColor', 'b')
text(x_wing_AC, -0.35, 'Wing AC')

% Horizontal tail AC
plot(x_HT_AC, 0, 'go', 'MarkerFaceColor', 'g')
text(x_HT_AC, 0.25, 'HT AC')

% Approximate wing
wing_y = [-b_wing/2, b_wing/2];
plot([x_wing_AC, x_wing_AC], wing_y, 'b', 'LineWidth', 3)

% Approximate horizontal tail
HT_y = [-b_HT/2, b_HT/2];
plot([x_HT_AC, x_HT_AC], HT_y, 'g', 'LineWidth', 3)

xlabel('x-location from nose (m)')
ylabel('Spanwise location, y (m)')
title('Simple Top-View Layout: Wing, CG, and Horizontal Tail')
legend('Fuselage centerline', 'CG', 'Wing AC', 'Horizontal Tail AC', ...
      'Wing span', 'Horizontal tail span', 'Location', 'best')
```



Summary

```
fprintf('SUMMARY:\n')
fprintf('Using S = %.2f m^2 and b = %.2f m:\n', S_wing, b_wing)
fprintf('Required horizontal stabilizer area = %.2f m^2\n', S_HT)
fprintf('Required vertical stabilizer area = %.2f m^2\n', S_VT)
fprintf('Horizontal tail span estimate = %.2f m\n', b_HT)
fprintf('Vertical tail height estimate = %.2f m\n', h_VT)
fprintf('Distance from wing AC to CG = %.2f m\n', l_wing_to_CG)
fprintf('Distance from CG to horizontal tail AC = %.2f m\n', L_HT)
fprintf('Distance from CG to vertical tail AC = %.2f m\n', L_VT)
```

SUMMARY:

Using $S = 36.80 \text{ m}^2$ and $b = 15.20 \text{ m}$:
 Required horizontal stabilizer area = 11.34 m^2
 Required vertical stabilizer area = 4.22 m^2
 Horizontal tail span estimate = 6.73 m
 Vertical tail height estimate = 2.60 m
 Distance from wing AC to CG = -0.30 m
 Distance from CG to horizontal tail AC = 5.50 m
 Distance from CG to vertical tail AC = 5.30 m

Published with MATLAB® R2025b

```

function [state] = RocCheck(P_A_sl, W, S, AR, CD0, h)
    ROC_needed = 4.064; % required rate of climb [m/s]

    rho_sl = 1.225; % density of air at sea level
    e = 0.9; % project constant
    k = 1/(pi*e*AR);
    CD_Pmin = 4*CD0;
    CL_Pmin = sqrt(3*CD0/k);

    [~, rho, ~, ~] = StdAtmos(h);
    sigma = rho/rho_sl;
    V_Pmin = sqrt(2*W/(rho*S*CL_Pmin));
    q_Pmin = 0.5 * rho * V_Pmin^2;

    D_Pmin = q_Pmin * S * CD_Pmin;
    P_A = sigma * P_A_sl;
    P_R = D_Pmin * V_Pmin;

    ROC = (P_A - P_R)/W;
    if ROC >= ROC_needed
        state = true;
    else
        state = false;
    end
end

Not enough input arguments.

Error in RocCheck (line 6)
    k = 1/(pi*e*AR);
        ^^

```

Published with MATLAB® R2025b

COG_Envelope_All_Configurations.m

===== Preliminary
CG envelope check for all aircraft mission configurations.

Configurations: 1. Passenger transport 2. Cargo transport 3. Medevac

This script uses estimated longitudinal station locations based on the preliminary fuselage/cabin layout. All x-locations are measured from the aircraft nose.

=====

```
clear; clc; close all;
```

Aircraft Reference Geometry

```
S_wing = 36.8;           % wing reference area [m^2]
b_wing = 15.2;           % wing span [m]
c_bar  = S_wing / b_wing; % mean aerodynamic chord [m]

x_wing_AC = 4.30;         % wing aerodynamic center from nose [m]
x_LEMAC   = x_wing_AC - 0.25*c_bar;

% Baseline CG and approximate neutral point from stability script
x.CG_baseline = 4.00;     % baseline CG from nose [m]
x.NP_target   = 4.24;     % approximate neutral point from nose [m]
```

Preliminary CG Envelope Limits

```
% Envelope selected to keep all configurations forward of the approximate
% neutral point while allowing realistic loading variation.
CG_forward_percent_MAC = 5;      % forward CG limit [% MAC]
CG_aft_percent_MAC     = 20;     % aft CG limit [% MAC]

x.CG_forward = x_LEMAC + (CG_forward_percent_MAC/100)*c_bar;
x.CG_aft     = x_LEMAC + (CG_aft_percent_MAC/100)*c_bar;
```

Weight and Station Assumptions

----- Weight units: kg Station units: m from nose

```
% Empty aircraft
% Includes structure, engine, fixed systems, unusable fluids, and fixed cabin
% equipment. Location selected to match the baseline stability estimate.
W_empty = 2172;
x_empty = 4.00;

% Crew
W_pilot_each = 90.7;           % 200 lb per pilot [kg]
x_pilots     = 2.35;           % cockpit station [m]
```

```

% Fuel
W_fuel_full = 548;           % full mission fuel [kg]
x_fuel = 4.25;              % wing fuel CG near wing box [m]

% Passenger cabin
W_passenger_with_bag = 113.4; % 250 lb passenger + baggage [kg]

% Three passenger rows, 1-2 seating arrangement
x_row1 = 3.45;
x_row2 = 4.15;
x_row3 = 4.85;

% Cargo configuration
% These locations keep the heavy generator/diesel close to the wing box
% to minimize CG travel during reconfiguration.
W_removed_seats = 90;
x_removed_seats = 4.15;

W_generator = 99.3;
x_generator = 4.05;

W_diesel = 177;
x_diesel = 4.35;

W_drum = 11.4;
x_drum = 4.35;

% Additional cargo ballast/equivalent payload
% Configuration 2 must carry payload equivalent to passenger configuration
% plus removable seats. The generator, diesel, and drum do not alone reach
% that value, so the remaining cargo is placed near the wing box.
W_cargo_equivalent_total = 1021 + 90; % 9 passengers+baggage equivalent +
seats [kg]
W_known_cargo = W_generator + W_diesel + W_drum;
W_additional_cargo = W_cargo_equivalent_total - W_known_cargo;

x_additional_cargo = 4.25;

% Medevac configuration
W_stretcher = 22.7;           % 50 lb stretcher [kg]
W_patient = 90.7;            % 200 lb patient [kg]
W_litter_patient = W_stretcher + W_patient;
x_litter_patient = 4.45;

% 4 occupants in medevac configuration:
% 2 medical technicians, 1 patient escort, and 1 additional occupant/patient
% station as required by the RFP interpretation.
W_medevac_occupants = 4*90.7;
x_medevac_occupants = 4.05;

W_med_supplies = 136;         % 300 lb medical supplies [kg]
x_med_supplies = 4.70;

```

Helper Functions

```
calc_cg = @(W, x) sum(W .* x) / sum(W);  
to_percent_MAC = @(x) 100*(x - x_LEMAC)/c_bar;
```

Configuration 1: Passenger Transport

2 pilots, full fuel, 9 passengers with baggage

```
W_config1 = [ ...  
    W_empty, ...  
    2*W_pilot_each, ...  
    W_fuel_full, ...  
    3*W_passenger_with_bag, ...  
    3*W_passenger_with_bag, ...  
    3*W_passenger_with_bag];  
  
x_config1 = [ ...  
    x_empty, ...  
    x_pilots, ...  
    x_fuel, ...  
    x_row1, ...  
    x_row2, ...  
    x_row3];  
  
W_total_1 = sum(W_config1);  
x_CG_1 = calc_cg(W_config1, x_config1);  
CG_percent_MAC_1 = to_percent_MAC(x_CG_1);
```

Configuration 2: Cargo Transport

2 pilots, full fuel, removable-seat equivalent, generator, diesel, container, and additional equivalent cargo.

```
W_config2 = [ ...  
    W_empty, ...  
    2*W_pilot_each, ...  
    W_fuel_full, ...  
    W_removed_seats, ...  
    W_generator, ...  
    W_diesel, ...  
    W_drum, ...  
    W_additional_cargo];  
  
x_config2 = [ ...  
    x_empty, ...  
    x_pilots, ...  
    x_fuel, ...  
    x_removed_seats, ...  
    x_generator, ...  
    x_diesel, ...  
    x_drum, ...  
    x_additional_cargo];
```

```
W_total_2 = sum(W_config2);
x_CG_2 = calc_cg(W_config2, x_config2);
CG_percent_MAC_2 = to_percent_MAC(x_CG_2);
```

Configuration 3: Medevac

2 pilots, full fuel, litter/patient, 4 cabin occupants, and medical supplies

```
W_config3 = [ ...
    W_empty, ...
    2*W_pilot_each, ...
    W_fuel_full, ...
    W_litter_patient, ...
    W_medevac_occupants, ...
    W_med_supplies];

x_config3 = [ ...
    x_empty, ...
    x_pilots, ...
    x_fuel, ...
    x_litter_patient, ...
    x_medevac_occupants, ...
    x_med_supplies];

W_total_3 = sum(W_config3);
x_CG_3 = calc_cg(W_config3, x_config3);
CG_percent_MAC_3 = to_percent_MAC(x_CG_3);
```

Fuel Burn Cases

Include full-fuel and zero-fuel points to show CG remains inside envelope throughout the mission.

```
% Configuration 1 zero fuel
W_config1_zf = W_config1;
W_config1_zf(3) = 0;
W_total_1_zf = sum(W_config1_zf);
x_CG_1_zf = calc_cg(W_config1_zf, x_config1);
CG_percent_MAC_1_zf = to_percent_MAC(x_CG_1_zf);

% Configuration 2 zero fuel
W_config2_zf = W_config2;
W_config2_zf(3) = 0;
W_total_2_zf = sum(W_config2_zf);
x_CG_2_zf = calc_cg(W_config2_zf, x_config2);
CG_percent_MAC_2_zf = to_percent_MAC(x_CG_2_zf);

% Configuration 3 zero fuel
W_config3_zf = W_config3;
W_config3_zf(3) = 0;
W_total_3_zf = sum(W_config3_zf);
```

```
x_CG_3_zf = calc_cg(W_config3_zf, x_config3);
CG_percent_MAC_3_zf = to_percent_MAC(x_CG_3_zf);
```

Print Results

```
fprintf('==== CG Envelope Reference ====\\n')
fprintf('Mean aerodynamic chord, c_bar = %.3f m\\n', c_bar)
fprintf('Wing AC location = %.3f m from nose\\n', x_wing_AC)
fprintf('LEMAC location = %.3f m from nose\\n', x_LEMAC)
fprintf('Forward CG limit = %.3f m = %.1f%% MAC\\n', ...
        x_CG_forward, CG_forward_percent_MAC)
fprintf('Aft CG limit = %.3f m = %.1f%% MAC\\n', ...
        x_CG_aft, CG_aft_percent_MAC)
fprintf('Approximate neutral point = %.3f m\\n\\n', x_NP_target)

fprintf('==== Full-Fuel Configuration CG Results ====\\n')
fprintf('Config 1 Passenger: W = %.1f kg, x_CG = %.3f m, CG = %.1f%% MAC\\n',
...
        W_total_1, x_CG_1, CG_percent_MAC_1)
fprintf('Config 2 Cargo:      W = %.1f kg, x_CG = %.3f m, CG = %.1f%% MAC\\n',
...
        W_total_2, x_CG_2, CG_percent_MAC_2)
fprintf('Config 3 Medevac:    W = %.1f kg, x_CG = %.3f m, CG = %.1f%%
MAC\\n\\n', ...
        W_total_3, x_CG_3, CG_percent_MAC_3)

fprintf('==== Zero-Fuel Configuration CG Results ====\\n')
fprintf('Config 1 Passenger ZF: W = %.1f kg, x_CG = %.3f m, CG = %.1f%%
MAC\\n', ...
        W_total_1_zf, x_CG_1_zf, CG_percent_MAC_1_zf)
fprintf('Config 2 Cargo ZF:      W = %.1f kg, x_CG = %.3f m, CG = %.1f%%
MAC\\n', ...
        W_total_2_zf, x_CG_2_zf, CG_percent_MAC_2_zf)
fprintf('Config 3 Medevac ZF:    W = %.1f kg, x_CG = %.3f m, CG = %.1f%%
MAC\\n\\n', ...
        W_total_3_zf, x_CG_3_zf, CG_percent_MAC_3_zf)

==== CG Envelope Reference ====
Mean aerodynamic chord, c_bar = 2.421 m
Wing AC location = 4.300 m from nose
LEMAC location = 3.695 m from nose
Forward CG limit = 3.816 m = 5.0% MAC
Aft CG limit = 4.179 m = 20.0% MAC
Approximate neutral point = 4.240 m

==== Full-Fuel Configuration CG Results ====
Config 1 Passenger: W = 3922.0 kg, x_CG = 3.998 m, CG = 12.5% MAC
Config 2 Cargo:      W = 4102.4 kg, x_CG = 4.031 m, CG = 13.9% MAC
Config 3 Medevac:    W = 3513.6 kg, x_CG = 4.001 m, CG = 12.6% MAC

==== Zero-Fuel Configuration CG Results ====
Config 1 Passenger ZF: W = 3374.0 kg, x_CG = 3.957 m, CG = 10.8% MAC
Config 2 Cargo ZF:      W = 3554.4 kg, x_CG = 3.997 m, CG = 12.5% MAC
```

Config 3 Medevac ZF: $W = 2965.6 \text{ kg}$, $x_{CG} = 3.954 \text{ m}$, $CG = 10.7\% \text{ MAC}$

Pass / Fail Check

```
CG_forward_percent = CG_forward_percent_MAC;
CG_aft_percent = CG_aft_percent_MAC;

case_names = { ...
    'Config 1 Passenger, full fuel', ...
    'Config 2 Cargo, full fuel', ...
    'Config 3 Medevac, full fuel', ...
    'Config 1 Passenger, zero fuel', ...
    'Config 2 Cargo, zero fuel', ...
    'Config 3 Medevac, zero fuel'};

CG_all = [ ...
    CG_percent_MAC_1, ...
    CG_percent_MAC_2, ...
    CG_percent_MAC_3, ...
    CG_percent_MAC_1_zf, ...
    CG_percent_MAC_2_zf, ...
    CG_percent_MAC_3_zf];

fprintf('==== Envelope Check =====\n')
for i = 1:length(CG_all)
    if CG_all(i) >= CG_forward_percent && CG_all(i) <= CG_aft_percent
        fprintf('%s: PASS\n', case_names{i})
    else
        fprintf('%s: FAIL\n', case_names{i})
    end
end

==== Envelope Check =====
Config 1 Passenger, full fuel: PASS
Config 2 Cargo, full fuel: PASS
Config 3 Medevac, full fuel: PASS
Config 1 Passenger, zero fuel: PASS
Config 2 Cargo, zero fuel: PASS
Config 3 Medevac, zero fuel: PASS
```

Plot 1: CG Envelope by Configuration

```
figure('Name','CG Envelope by Configuration', ...
    'Units','normalized','Position',[0.10 0.15 0.78 0.55]);

hold on; grid on; box on;

y_positions_full = [1 2 3];
y_positions_zf   = [1.18 2.18 3.18];

% Allowable CG envelope shaded region
fill([CG_forward_percent CG_aft_percent CG_aft_percent CG_forward_percent],
```

```

...
    [0.5 0.5 3.6 3.6], ...
    [0.85 0.90 1.00], ...
    'EdgeColor','none', ...
    'FaceAlpha',0.55, ...
    'DisplayName','Allowable CG Envelope');

xline(CG_forward_percent, 'k--', 'Forward CG Limit', ...
      'LineWidth',1.4, ...
      'LabelVerticalAlignment','bottom');

xline(CG_aft_percent, 'k--', 'Aft CG Limit', ...
      'LineWidth',1.4, ...
      'LabelVerticalAlignment','bottom');

% Full-fuel points
p1 = plot(CG_percent_MAC_1, 1, 'o', 'MarkerSize',10, ...
          'MarkerFaceColor','b', 'MarkerEdgeColor','k', ...
          'DisplayName','Full Fuel');

plot(CG_percent_MAC_2, 2, 'o', 'MarkerSize',10, ...
      'MarkerFaceColor','b', 'MarkerEdgeColor','k', ...
      'HandleVisibility','off');

plot(CG_percent_MAC_3, 3, 'o', 'MarkerSize',10, ...
      'MarkerFaceColor','b', 'MarkerEdgeColor','k', ...
      'HandleVisibility','off');

% Zero-fuel points
p2 = plot(CG_percent_MAC_1_zf, 1.18, 's', 'MarkerSize',9, ...
          'MarkerFaceColor','r', 'MarkerEdgeColor','k', ...
          'DisplayName','Zero Fuel');

plot(CG_percent_MAC_2_zf, 2.18, 's', 'MarkerSize',9, ...
      'MarkerFaceColor','r', 'MarkerEdgeColor','k', ...
      'HandleVisibility','off');

plot(CG_percent_MAC_3_zf, 3.18, 's', 'MarkerSize',9, ...
      'MarkerFaceColor','r', 'MarkerEdgeColor','k', ...
      'HandleVisibility','off');

% Connect full-fuel to zero-fuel points
plot([CG_percent_MAC_1 CG_percent_MAC_1_zf], [1 1.18], 'k-',
      'HandleVisibility','off')
plot([CG_percent_MAC_2 CG_percent_MAC_2_zf], [2 2.18], 'k-',
      'HandleVisibility','off')
plot([CG_percent_MAC_3 CG_percent_MAC_3_zf], [3 3.18], 'k-',
      'HandleVisibility','off')

yticks([1.09 2.09 3.09])
yticklabels({'Config 1: Passenger','Config 2: Cargo','Config 3: Medevac'})

xlabel('Center of Gravity Location [% MAC]')
ylabel('Mission Configuration')

```

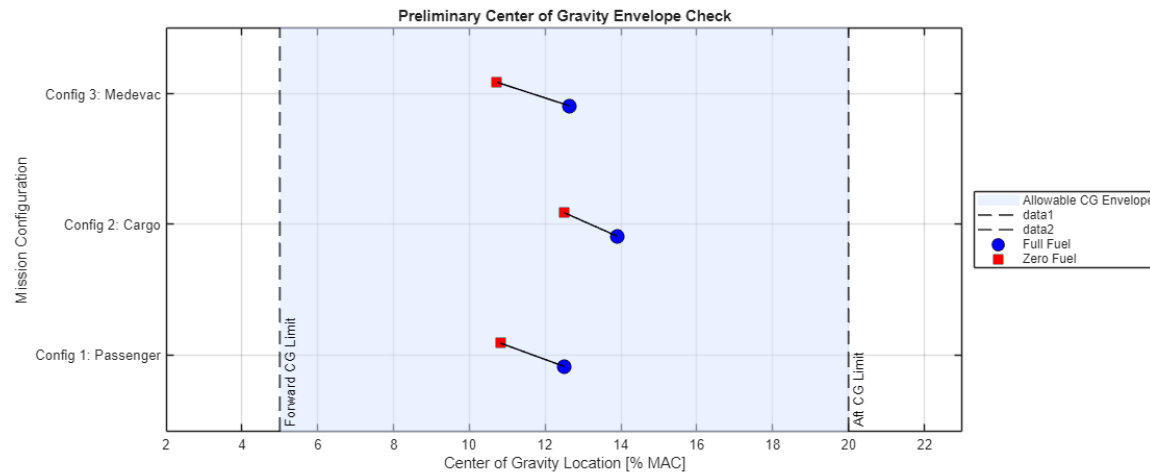
```

title('Preliminary Center of Gravity Envelope Check')

xlim([CG_forward_percent - 3, CG_aft_percent + 3])
ylim([0.5 3.6])

legend('Location','eastoutside')

```



Plot 2: Weight vs CG Envelope

```

figure('Name','Weight vs CG Envelope', ...
    'Units','normalized','Position',[0.10 0.15 0.78 0.55]);

hold on; grid on; box on;

CG_plot = [ ...
    CG_percent_MAC_1, CG_percent_MAC_2, CG_percent_MAC_3, ...
    CG_percent_MAC_1_zf, CG_percent_MAC_2_zf, CG_percent_MAC_3_zf];

W_plot = [ ...
    W_total_1, W_total_2, W_total_3, ...
    W_total_1_zf, W_total_2_zf, W_total_3_zf];

W_min = min(W_plot) - 150;
W_max = max(W_plot) + 150;

% Envelope region
fill([CG_forward_percent CG_aft_percent CG_aft_percent CG_forward_percent],
    ...
    [W_min W_min W_max W_max], ...
    [0.85 0.90 1.00], ...
    'EdgeColor','none', ...
    'FaceAlpha',0.55, ...
    'DisplayName','Allowable CG Envelope');

xline(CG_forward_percent, 'k--', 'Forward CG Limit', 'LineWidth',1.4);
xline(CG_aft_percent, 'k--', 'Aft CG Limit', 'LineWidth',1.4);

```

```

% Full fuel points
plot(CG_percent_MAC_1, W_total_1, 'o', 'MarkerSize',10, ...
     'MarkerFaceColor','b', 'MarkerEdgeColor','k', ...
     'DisplayName','Config 1 Full Fuel');

plot(CG_percent_MAC_2, W_total_2, 'o', 'MarkerSize',10, ...
     'MarkerFaceColor','g', 'MarkerEdgeColor','k', ...
     'DisplayName','Config 2 Full Fuel');

plot(CG_percent_MAC_3, W_total_3, 'o', 'MarkerSize',10, ...
     'MarkerFaceColor','m', 'MarkerEdgeColor','k', ...
     'DisplayName','Config 3 Full Fuel');

% Zero fuel points
plot(CG_percent_MAC_1_zf, W_total_1_zf, 's', 'MarkerSize',9, ...
     'MarkerFaceColor','b', 'MarkerEdgeColor','k', ...
     'DisplayName','Config 1 Zero Fuel');

plot(CG_percent_MAC_2_zf, W_total_2_zf, 's', 'MarkerSize',9, ...
     'MarkerFaceColor','g', 'MarkerEdgeColor','k', ...
     'DisplayName','Config 2 Zero Fuel');

plot(CG_percent_MAC_3_zf, W_total_3_zf, 's', 'MarkerSize',9, ...
     'MarkerFaceColor','m', 'MarkerEdgeColor','k', ...
     'DisplayName','Config 3 Zero Fuel');

% Fuel burn lines
plot([CG_percent_MAC_1 CG_percent_MAC_1_zf], ...
     [W_total_1 W_total_1_zf], 'b-', 'LineWidth',1.4, ...
     'HandleVisibility','off');

plot([CG_percent_MAC_2 CG_percent_MAC_2_zf], ...
     [W_total_2 W_total_2_zf], 'g-', 'LineWidth',1.4, ...
     'HandleVisibility','off');

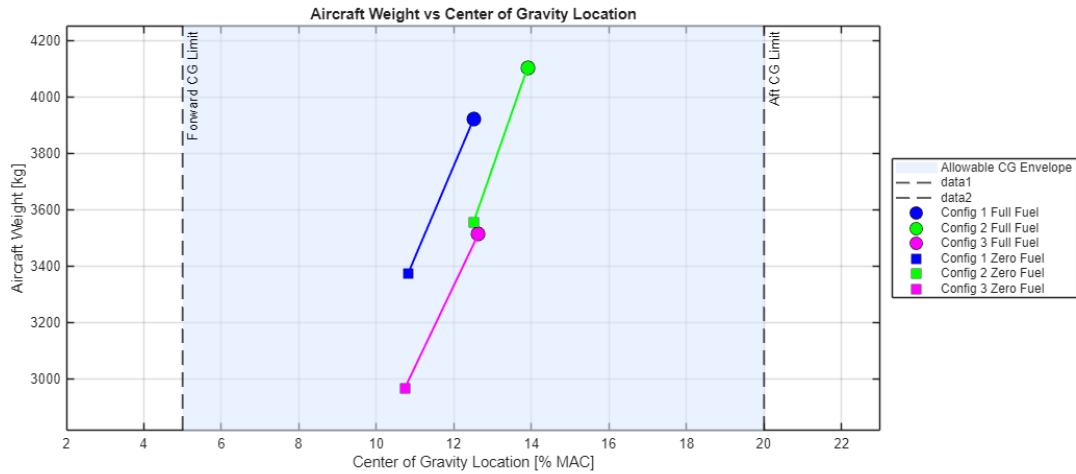
plot([CG_percent_MAC_3 CG_percent_MAC_3_zf], ...
     [W_total_3 W_total_3_zf], 'm-', 'LineWidth',1.4, ...
     'HandleVisibility','off');

xlabel('Center of Gravity Location [% MAC]')
ylabel('Aircraft Weight [kg]')
title('Aircraft Weight vs Center of Gravity Location')

xlim([CG_forward_percent - 3, CG_aft_percent + 3])
ylim([W_min W_max])

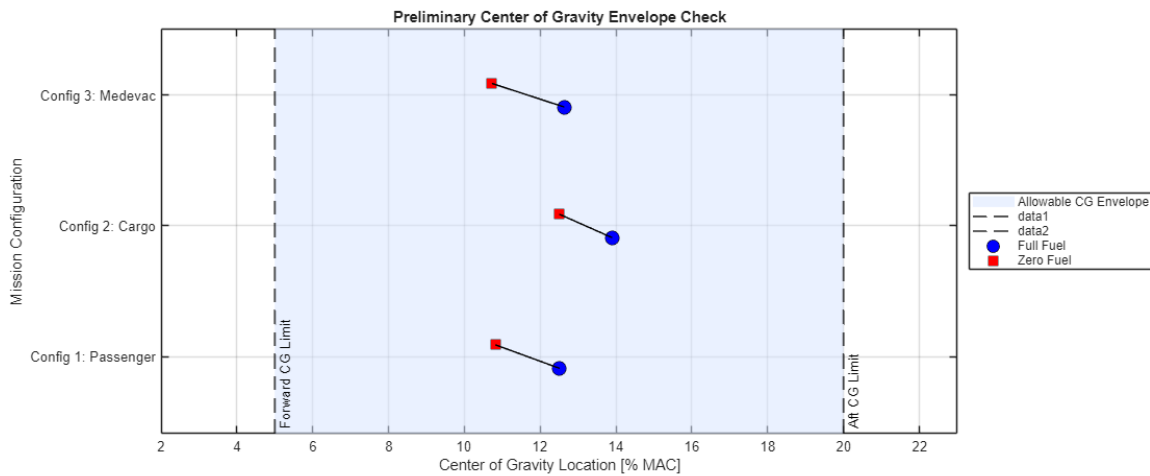
legend('Location','eastoutside')

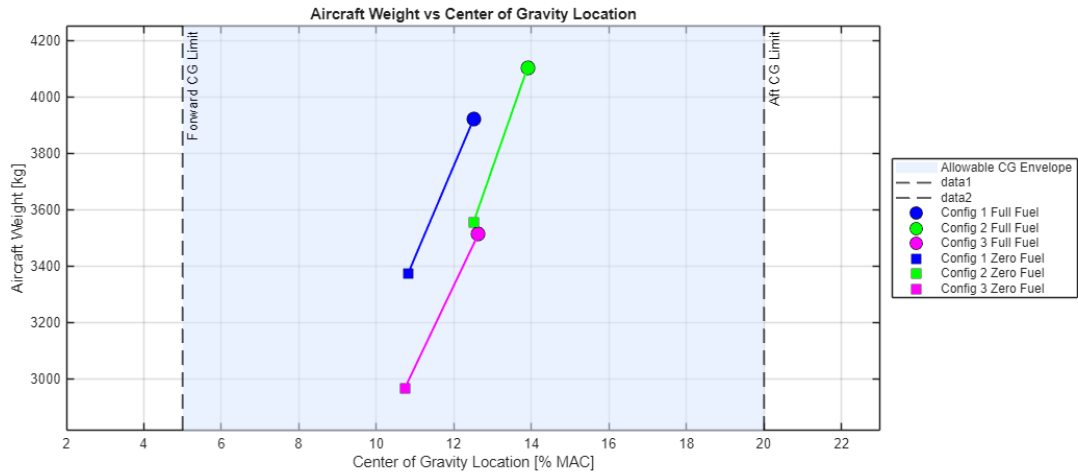
```



Save Figures

```
print(figure(1), 'CG_Envelope_by_Configuration.png', '-dpng', '-r600')
print(figure(2), 'Weight_vs_CG_Envelope.png', '-dpng', '-r600')
```





Published with MATLAB® R2025b

```

% VN_Diagrams_single_engine.m
% =====
% V-n / sustained-turn envelope for single-engine configuration
%
% Figures produced:
%   Figure 1: V-n / sustained-turn envelopes at sea level and 4572 m
%   Figure 2: Turn radius vs dynamic pressure q at sea level and 4572 m
%
% Figure 2 includes:
%   - CLmax turn-radius limit
%   - T_max / power-available turn-radius limit
%   - n_struct turn-radius limit
%   - minimum achievable turn-radius curve
%   - yellow highlighted feasible turn-radius region
%
% StdAtmos is included as a local function, so this file can run standalone.
% =====

clear; clc; close all;

```

Shared parameters

```

% Physical constants
g      = 9.81;           % gravitational acceleration [m/s^2]
rho_SL = 1.225;          % sea-level ISA density [kg/m^3]
shp2W  = 745.7;         % shaft horsepower to watts

% Propulsion
P_shp   = 1050;          % rated shaft power [hp]
eta_prop = 0.80;         % propeller efficiency [-]

% Clean aerodynamic values
CD_0_clean = 0.030;      % clean zero-lift drag coefficient [-]
CL_max_clean = 1.8;      % clean maximum lift coefficient [-]
e          = 0.90;       % Oswald efficiency factor [-]

% Single-engine aircraft parameters
W_TO = 39867.9;          % MTOW [N]
S     = 39.849246231155774; % wing area [m^2]
AR    = 6.854271356783920; % aspect ratio [-]
b     = sqrt(S * AR);    % wingspan [m]

% Propulsive power available at sea level
P_A_SL = eta_prop * P_shp * shp2W; % [W]

cfg_name = 'Single Engine (42 kN, 1 x 1050 shp)';

% Structural limit
% DHC-6 / Normal Category positive limit load factor
n_struct_pos = 3.8;      % positive limit load factor [-]

```

```
% Sustained-turn power setting
throttle_frac = 1.00;
```

Altitudes and velocity range

```
% Sea level and 4572 m = 15,000 ft
h_cases = [0, 4572]; % [m]
[~, rho_cases, ~, ~] = StdAtmos_local(h_cases);

V_min = 1.0; % avoid division by zero [m/s]
V_max_calc = 250; % [m/s]
V_vn = linspace(V_min, V_max_calc, 1200);
```

Aircraft structure

```
ac.S = S;
ac.AR = AR;
ac.b = b;
ac.W = W_TO;
ac.P_A_SL = P_A_SL;
ac.CD_0 = CD_0_clean;
ac.CLmax = CL_max_clean;
ac.e = e;
ac.k = 1 / (pi * ac.e * ac.AR);
ac.n_lim = n_struct_pos;
```

Storage arrays for turn-radius figure

```
turn_data = struct();
```

Console summary

```
fprintf('=== V-n Diagram Inputs: Single-Engine Configuration ===\n')
fprintf(' %s\n', cfg_name)
fprintf(' S = %.2f m^2\n', ac.S)
fprintf(' AR = %.2f\n', ac.AR)
fprintf(' b = %.2f m\n', ac.b)
fprintf(' W = %.1f N (%.0f kg)\n', ac.W, ac.W/g)
fprintf(' P_A = %.1f kW\n\n', ac.P_A_SL/1000)
```

```
=== V-n Diagram Inputs: Single-Engine Configuration ===
Single Engine (42 kN, 1 x 1050 shp)
S = 39.85 m^2
AR = 6.85
b = 16.53 m
W = 39867.9 N (4064 kg)
P_A = 626.4 kW
```

Figure 1: V-n diagrams

```
fig1 = figure('Name','Single-Engine V-n Diagrams', ...
              'Units','normalized','Position',[0.05 0.12 0.88 0.52]);

for h_idx = 1:numel(h_cases)

    h = h_cases(h_idx);
    rho = rho_cases(h_idx);

    q = 0.5 .* rho .* V_vn.^2;

    % Power available with density lapse
    P_A_h = throttle_frac * ac.P_A_SL * (rho / rho_SL);

    % Lift-limited load factor
    n_CL = q .* ac.S .* ac.CLmax ./ ac.W;

    % Structural limit
    n_struct = ac.n_lim .* ones(size(V_vn));

    % Sustained power limit
    %  $D = q S C_{D0} + k(nW)^2 / (qS)$ 
    %  $P_R = D V$ 
    % Solve  $P_A = P_R$  for  $n$ 
    n_power_sq = ((P_A_h ./ V_vn) - ac.CD_0 .* q .* ac.S) .* ...
                 (q .* ac.S) ./ (ac.k .* ac.W.^2);

    n_power = sqrt(n_power_sq);
    n_power(n_power_sq <= 0) = NaN;

    % Find maximum velocity where PA = PR curve exists
    valid_power_idx = find(isfinite(n_power));

    if ~isempty(valid_power_idx)
        V_PA_max = V_vn(valid_power_idx(end));
    else
        V_PA_max = V_max_calc;
    end

    q_PA_max = 0.5 * rho * V_PA_max^2;

    % Aerodynamic reference limit based on E_max and equivalent thrust
    E_max = 1 / (2 * sqrt(ac.CD_0 * ac.k));
    T_equiv = P_A_h ./ V_vn;
    n_aero = E_max .* (T_equiv ./ ac.W);
    n_aero(n_aero < 0) = NaN;

    % Achievable positive envelope
    n_mat = [n_CL; n_struct; n_power; n_aero];
    n_env = min(n_mat, [], 1, 'omitnan');
    n_env(isnan(n_env)) = 0;
    n_env = max(n_env, 0);
```

```

% Important speeds
V_stall_lg = sqrt(2 * ac.W / (rho * ac.S * ac.CLmax));
V_corner   = sqrt(2 * ac.W * ac.n_lim / ...
                  (rho * ac.S * ac.CLmax));

```

Turn-radius calculations for Figure 2

```

% Coordinated level turn radius:
%  $r = V^2 / (g \sqrt{n^2 - 1})$ 

r_struct = V_vn.^2 ./ (g .* sqrt(n_struct.^2 - 1));
r_CL      = V_vn.^2 ./ (g .* sqrt(n_CL.^2 - 1));
r_power   = V_vn.^2 ./ (g .* sqrt(n_power.^2 - 1));
r_env     = V_vn.^2 ./ (g .* sqrt(n_env.^2 - 1));

% Remove invalid points where steady level turn is impossible
r_CL(n_CL <= 1) = NaN;
r_power(n_power <= 1) = NaN;
r_env(n_env <= 1) = NaN;

r_struct(~isfinite(r_struct)) = NaN;
r_CL(~isfinite(r_CL)) = NaN;
r_power(~isfinite(r_power)) = NaN;
r_env(~isfinite(r_env)) = NaN;

% Remove points beyond PA = PR cutoff
r_CL(q > q_PA_max) = NaN;
r_power(q > q_PA_max) = NaN;
r_struct(q > q_PA_max) = NaN;
r_env(q > q_PA_max) = NaN;

% Store data for Figure 2
turn_data(h_idx).h = h;
turn_data(h_idx).rho = rho;
turn_data(h_idx).q = q;
turn_data(h_idx).q_PA_max = q_PA_max;
turn_data(h_idx).r_struct = r_struct;
turn_data(h_idx).r_CL = r_CL;
turn_data(h_idx).r_power = r_power;
turn_data(h_idx).r_env = r_env;

```

Plot Figure 1

```

figure(fig1)
subplot(1,2,h_idx);
hold on; grid on; box on;

% Yellow achievable region
fill([V_vn fliplr(V_vn)], [n_env zeros(size(n_env))], ...
     [1.0 0.9 0.1], ...
     'FaceAlpha', 0.18, ...
     'EdgeColor', 'none', ...

```

```

        'HandleVisibility','off');

h1 = plot(V_vn, n_CL,      'b-', 'LineWidth',1.4);
h2 = plot(V_vn, n_power,  'm-', 'LineWidth',1.4);
h3 = plot(V_vn, n_struct, 'k-', 'LineWidth',1.4);
h4 = plot(V_vn, n_aero,   '--', ...
        'Color',[1.0 0.45 0.0], ...
        'LineWidth',1.5);
h5 = plot(V_vn, n_env,    'r-', 'LineWidth',2.2);

xline(V_stall_lg, ':', '$V_{S,lg}$', ...
    'Interpreter','latex', ...
    'LabelVerticalAlignment','bottom');

xline(V_corner, '--', '$V_A$', ...
    'Interpreter','latex', ...
    'LabelVerticalAlignment','bottom');

xlabel('Velocity, $V$ [m/s]', 'Interpreter','latex')
ylabel('Load Factor, $n$', 'Interpreter','latex')

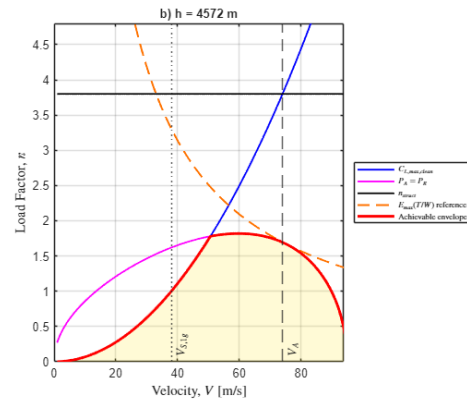
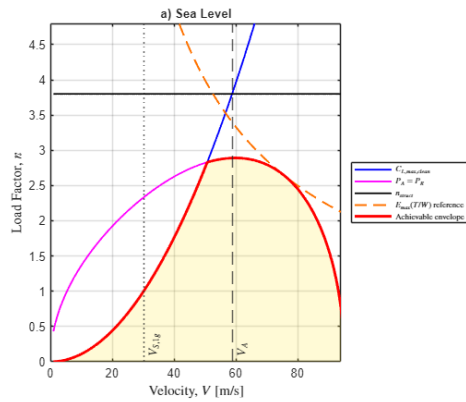
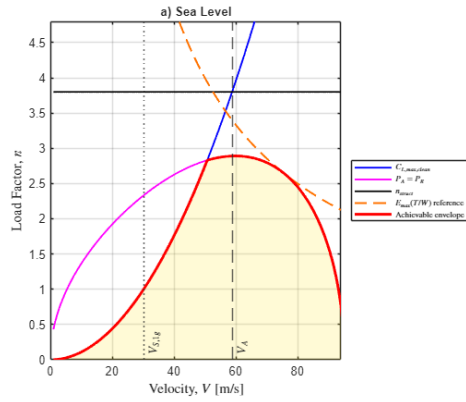
if h == 0
    h_label = 'a) Sea Level';
else
    h_label = sprintf('b) h = %.0f m', h);
end

title(h_label, ...
    'Interpreter','none', ...
    'FontSize',10)

legend([h1 h2 h3 h4 h5], ...
    {'$C_{L,max,clean}$', ...
    '$P_A = P_R$', ...
    '$n_{struct}$', ...
    '$E_{max}(T/W)$ reference', ...
    'Achievable envelope'}, ...
    'Interpreter','latex', ...
    'Location','eastoutside', ...
    'FontSize',7)

xlim([0 V_PA_max - 1])
ylim([0 ac.n_lim + 1.0])

```



end

figure(fig1)

print(fig1, 'VN_Diagrams_single_engine.png', '-dpng', '-r1200')

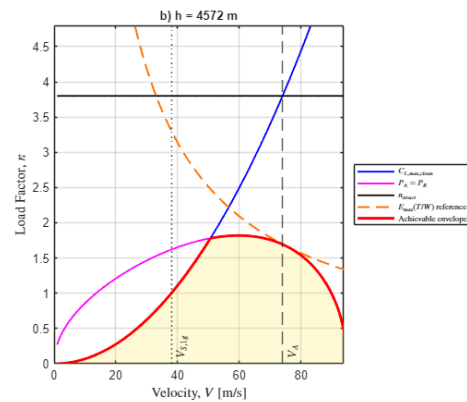
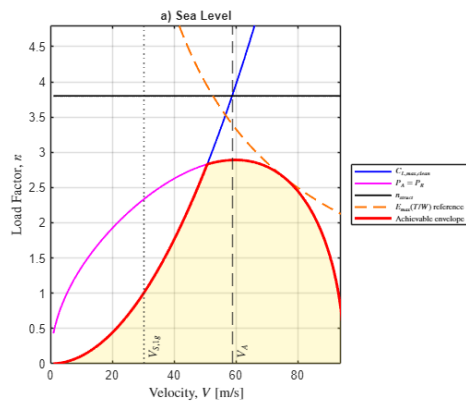


Figure 2: Turn radius vs dynamic pressure q

Shows sea level and 4572 m for the single-engine aircraft.

Colors: Blue = CLmax turn-radius limit Magenta = $T_{\max} / PA$ = PR turn-radius limit Black = n_{struct} turn-radius limit Red = minimum achievable turn radius

Line style: Solid = sea level Dashed = 4572 m

```
fig2 = figure('Name','Single-Engine Turn Radius vs Dynamic Pressure', ...
             'Units','normalized','Position',[0.08 0.12 0.78 0.58]);

hold on; grid on; box on;

% Pull sea-level data
q_sl      = turn_data(1).q;
qmax_sl   = turn_data(1).q_PA_max;
r_CL_sl   = turn_data(1).r_CL;
r_power_sl = turn_data(1).r_power;
r_struct_sl = turn_data(1).r_struct;
r_env_sl  = turn_data(1).r_env;

% Pull 4572 m data
q_4572    = turn_data(2).q;
qmax_4572 = turn_data(2).q_PA_max;
r_CL_4572 = turn_data(2).r_CL;
r_power_4572 = turn_data(2).r_power;
r_struct_4572 = turn_data(2).r_struct;
r_env_4572 = turn_data(2).r_env;

% Valid minimum-radius points for highlighting
valid_env_sl = isfinite(r_env_sl) & q_sl <= qmax_sl;
valid_env_4572 = isfinite(r_env_4572) & q_4572 <= qmax_4572;

% Determine q and r limits
q_plot_max = max([qmax_sl, qmax_4572]);

r_all = [ ...
    r_CL_sl(:); r_power_sl(:); r_struct_sl(:); r_env_sl(:); ...
    r_CL_4572(:); r_power_4572(:); r_struct_4572(:); r_env_4572(:)];

r_all = r_all(isfinite(r_all) & r_all > 0);

if isempty(r_all)
    r_upper = 1000;
else
    r_upper = 1.15 * prctile(r_all, 95);
end

% Highlight feasible region above minimum achievable turn-radius curve
% Sea level
if any(valid_env_sl)
    q_fill = q_sl(valid_env_sl);
    r_fill = r_env_sl(valid_env_sl);

    fill([q_fill fliplr(q_fill)], ...
         [r_fill r_upper*ones(size(r_fill))], ...
         [1.0 0.9 0.1], ...
```

```

        'FaceAlpha',0.14, ...
        'EdgeColor','none', ...
        'HandleVisibility','off');
end

% 4572 m
if any(valid_env_4572)
    q_fill = q_4572(valid_env_4572);
    r_fill = r_env_4572(valid_env_4572);

    fill([q_fill fliplr(q_fill)], ...
        [r_fill r_upper*ones(size(r_fill))], ...
        [1.0 0.9 0.1], ...
        'FaceAlpha',0.07, ...
        'EdgeColor','none', ...
        'HandleVisibility','off');
end

% Sea-level curves: solid
h1 = plot(q_sl, r_CL_sl, 'b-', 'LineWidth',1.5);
h2 = plot(q_sl, r_power_sl, 'm-', 'LineWidth',1.5);
h3 = plot(q_sl, r_struct_sl, 'k-', 'LineWidth',1.5);
h4 = plot(q_sl, r_env_sl, 'r-', 'LineWidth',2.4);

% 4572 m minimum-radius curve: dashed
h5 = plot(q_4572, r_env_4572, 'b--', 'LineWidth',2.4);

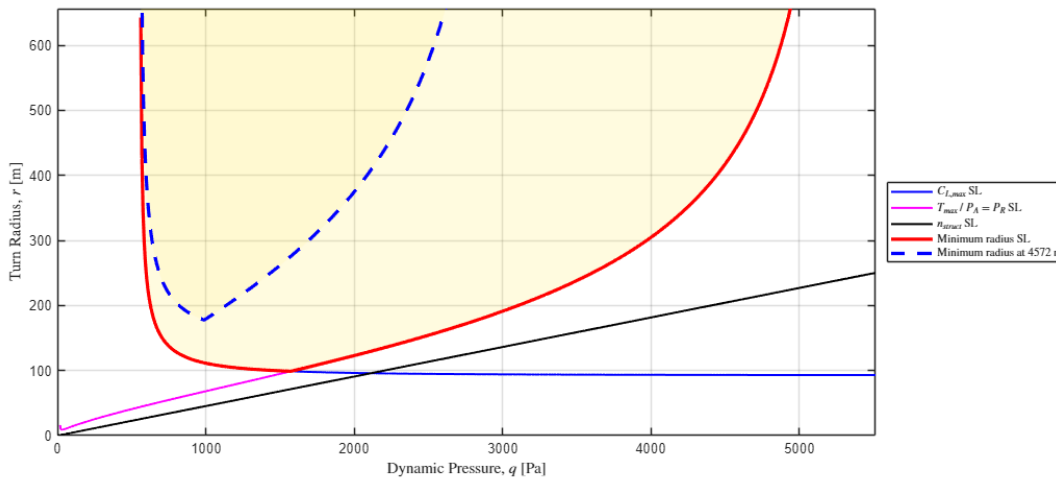
xlabel('Dynamic Pressure,  $q$  [Pa]', 'Interpreter','latex')
ylabel('Turn Radius,  $r$  [m]', 'Interpreter','latex')

xlim([0 q_plot_max])
ylim([0 r_upper])

legend([h1 h2 h3 h4 h5], ...
    '$C_{L,max}$ SL', ...
    '$T_{max}$ / $P_A=P_R$ SL', ...
    '$n_{struct}$ SL', ...
    'Minimum radius SL', ...
    'Minimum radius at 4572 m', ...
    'Interpreter','latex', ...
    'Location','eastoutside', ...
    'FontSize',8)

print(fig2, 'TurnRadius_vs_q_single_engine.png', '-dpng', '-r1200')

```



Local atmosphere function

```
function [T, rho, p, mu] = StdAtmos_local(h)
% StdAtmos_local Simple ISA atmosphere through 4572 m.
% Returns temperature [K], density [kg/m^3], pressure [Pa], and dynamic
% viscosity [kg/(m*s)]. Vectorized for h in meters.
```

```
h = max(h, 0); % guard against negative altitudes
```

```
R      = 287.05; % gas constant for air [J/(kg*K)]
g0      = 9.80665; % standard gravity [m/s^2]
T0      = 288.15; % sea-level temperature [K]
p0      = 101325; % sea-level pressure [Pa]
lapse   = -0.0065; % tropospheric lapse rate [K/m]
```

```
T = T0 + lapse .* h;
p = p0 .* (T ./ T0) .^ (-g0 / (lapse * R));
rho = p ./ (R .* T);
```

```
% Sutherland's law for dynamic viscosity
mu0    = 1.716e-5; % reference viscosity [kg/(m*s)]
T_ref  = 273.15; % reference temperature [K]
Suth   = 110.4; % Sutherland constant [K]
```

```
mu = mu0 .* (T ./ T_ref) .^ (3/2) .* ...
    (T_ref + Suth) ./ (T + Suth);
```

```
end
```

Published with MATLAB® R2025b

```

% SpoilerSizing.m
% =====
% SPOILER AREA SIZING – Landing CD0 Increase via Upper-Surface Spoilers
%
% Goal:
%   Estimate the total spoiler planform area required to achieve a desired
%   landing parasite-drag coefficient (CD0) increase, then back out the
%   spoiler deflection angle needed to reach a glide CD0 target using that
%   same spoiler area.
%
% Model:
%   1) Compute the additional drag coefficient needed in landing:
%       Delta_CD0 = CD0_landing_target - CD0_landing_baseline
%
%   2) Convert that to an equivalent drag area using the wing reference area:
%       A_spoiler_eq = Delta_CD0 * S_ref
%
%   3) Model the deployed spoiler as a flat plate at a deflection angle theta
%       above the wing, with drag acting on its projected area:
%       A_proj = S_spoiler_total * sin(theta)
%       A_spoiler_eq = Cd_spoiler_normal * A_proj
%
%   4) Solve for the required total spoiler planform area:
%       S_spoiler_total = A_spoiler_eq / (Cd_spoiler_normal * sin(theta))
%
%   5) Re-use the solved spoiler area and invert the same relation to
estimate
%       the spoiler deflection angle needed for a specified glide CD0 target:
%       theta = asin( (Delta_CD0_glide * S_ref) / (Cd_spoiler_normal *
S_spoiler_total) )
%
% Notes:
%   - The 60 deg spoiler angle is assumed to be measured from the wing
%     surface / local chord line toward the free stream. If your convention
%     is measured from the normal direction instead, swap sin() for cos().
%   - Cd_spoiler_normal is an engineering estimate for a bluff plate normal
%     to the flow. Replace this with wind-tunnel, CFD, or handbook data if
%     you have spoiler-specific coefficients.
%   - The glide-angle solve is only physically valid when the asin() argument
%     lies in [-1, 1]. If it does not, the requested glide CD0 cannot be
%     achieved with the same spoiler area and this simple projected-area
model.
%
% Sources used beyond the attached files:
%   NASA Glenn Research Center, "Drag Equation" – drag depends on body size,
%   shape, and inclination to the flow.
%   NASA Glenn Research Center, "Shape Effects on Drag" – flat plate as a
%   high-drag bluff-body baseline and dependence on inclination.
%
% All quantities SI unless stated otherwise.
% =====

```

```

clear; clc; close all;

% =====
% SECTION 1 – INPUTS
% =====

% Landing drag target and baseline (without spoilers).
% Set these to match your airplane's landing configuration.
CD0_landing_baseline = 0.027; % current landing CD0 without spoilers [-]
CD0_landing_target = CD0_landing_baseline+0.04; % desired total landing
CD0 [-]

% Wing reference area used to normalize CD0.
% Use the same S_ref used in your drag build-up.
S_ref = 36.80; % wing reference area [m^2]

% Spoiler geometry and drag model.
spoiler_deflection_deg = 60.0; % spoiler angle above wing surface
[deg]
Cd_spoiler_normal = 1.28; % normal-to-flow drag coefficient [-]

% Optional split into multiple equal-area spoiler panels.
% The script solves the total area, then divides by this count.
n_spoilers = 2; % number of spoiler panels [-]

% Glide target to back-solve a deflection angle using the same spoiler area.
CD0_glide_target = 0.330; % desired total glide CD0 [-]
CD0_glide_baseline = CD0_landing_baseline;
% Keep the same no-spoiler baseline unless you have a separate glide
baseline.

% =====
% SECTION 2 – REQUIRED DRAG INCREMENT
% =====

Delta_CD0_req = CD0_landing_target - CD0_landing_baseline;

if Delta_CD0_req <= 0
    error('Target CD0 must be greater than baseline CD0. Current Delta_CD0 =
%.5f', Delta_CD0_req);
end

% Equivalent parasite-drag area required from spoilers.
A_spoiler_eq = Delta_CD0_req * S_ref; % [m^2]

% =====
% SECTION 3 – SPOILER AREA SOLUTION
% =====

% For a spoiler on the upper wing surface:
% A_proj = S_spoiler_total * sin(theta)
% A_spoiler_eq = Cd_spoiler_normal * A_proj
%
% Therefore:
% S_spoiler_total = (Delta_CD0_req * S_ref) / (Cd_spoiler_normal *

```

```

sin(theta))

spoiler_deflection_rad = deg2rad(spoiler_deflection_deg);
projection_factor      = sin(spoiler_deflection_rad);

if projection_factor <= 0
    error('Spoiler deflection angle must produce a positive projected
area.');
```

end

```

S_spoiler_total = A_spoiler_eq / (Cd_spoiler_normal * projection_factor); %
[m^2]
S_spoiler_each  = S_spoiler_total / n_spoilers;                        %
[m^2]

% Back-check the achieved CD0 increment from the solved area.
Delta_CD0_check = (Cd_spoiler_normal * S_spoiler_total *
projection_factor) / S_ref;
CD0_landing_check = CD0_landing_baseline + Delta_CD0_check;

% Useful ratios for sanity checking.
spoiler_to_wing_ratio = S_spoiler_total / S_ref;
projected_area_total   = S_spoiler_total * projection_factor;

% =====
% SECTION 4 – REQUIRED SPOILER DEFLECTION FOR GLIDE CD0 TARGET
% =====
% Use the same spoiler area and the same projected-area model to estimate
% the spoiler deflection angle required to reach a different CD0 target.

Delta_CD0_glide_req = CD0_glide_target - CD0_glide_baseline;

if Delta_CD0_glide_req <= 0
    error('Glide CD0 target must be greater than the glide baseline CD0.
Current Delta_CD0 = %.5f', Delta_CD0_glide_req);
end

sin_theta_glide_req = (Delta_CD0_glide_req * S_ref) / (Cd_spoiler_normal *
S_spoiler_total);

if abs(sin_theta_glide_req) <= 1
    spoiler_deflection_glide_deg = asind(sin_theta_glide_req);
    glide_angle_feasible = true;
else
    spoiler_deflection_glide_deg = NaN;
    glide_angle_feasible = false;
end

% =====
% SECTION 5 – CONSOLE OUTPUT
% =====

fprintf('\n');
fprintf('=====
```

```

=====\\n');
fprintf('  SPOILER AREA SIZING FOR LANDING CD0 TARGET\\n');
fprintf('=====\\n\\n');

fprintf('  Inputs\\n');
fprintf('    CD0_landing_target    = %.5f\\n', CD0_landing_target);
fprintf('    CD0_landing_baseline = %.5f\\n', CD0_landing_baseline);
fprintf('    S_ref                 = %.3f m^2\\n', S_ref);
fprintf('    Spoiler angle         = %.1f deg\\n', spoiler_deflection_deg);
fprintf('    Cd_spoiler_normal     = %.3f\\n\\n', Cd_spoiler_normal);

fprintf('  Required Increment\\n');
fprintf('    Delta_CD0_req         = %.5f\\n', Delta_CD0_req);
fprintf('    A_spoiler_eq          = %.5f m^2 (equivalent drag area)\\n\\n',
A_spoiler_eq);

fprintf('  Solved Spoiler Area\\n');
fprintf('    Projection factor     = sin(%.1f deg) = %.4f\\n',
spoiler_deflection_deg, projection_factor);
fprintf('    Total spoiler area    = %.5f m^2\\n', S_spoiler_total);
fprintf('    Area per spoiler      = %.5f m^2  (%d equal panels)\\n',
S_spoiler_each, n_spoilers);
fprintf('    Total projected area = %.5f m^2\\n\\n', projected_area_total);

fprintf('  Back-Check\\n');
fprintf('    Achieved Delta_CD0    = %.5f\\n', Delta_CD0_check);
fprintf('    Landing CD0 checked   = %.5f\\n', CD0_landing_check);
fprintf('    Spoiler/Wing ratio    = %.3f %%\\n\\n', 100 *
spoiler_to_wing_ratio);

fprintf('  Glide Deflection Solve\\n');
fprintf('    CD0_glide_target      = %.5f\\n', CD0_glide_target);
fprintf('    CD0_glide_baseline    = %.5f\\n', CD0_glide_baseline);
fprintf('    Required Delta_CD0    = %.5f\\n', Delta_CD0_glide_req);
fprintf('    sin(theta) required   = %.5f\\n', sin_theta_glide_req);
if glide_angle_feasible
    fprintf('    Spoiler angle for glide = %.2f deg\\n\\n',
spoiler_deflection_glide_deg);
else
    fprintf('    Spoiler angle for glide = NOT PHYSICALLY FEASIBLE with this
area\\n');
    fprintf('    (requires |sin(theta)| > 1 in the current projected-area
model)\\n\\n');
end

fprintf('=====\\n');
fprintf('  RESULT: Use a total spoiler planform area of %.5f m^2\\n',
S_spoiler_total);
if glide_angle_feasible
    fprintf('  RESULT: Glide spoiler angle = %.2f deg\\n',
spoiler_deflection_glide_deg);
else

```

```

    fprintf('  RESULT: Glide spoiler angle cannot be solved with the present
inputs\n');
end
fprintf('=====
=====\\n\\n');

% =====
% SECTION 6 – OPTIONAL SIMPLE PLOT
% =====

figure('Name','Spoiler Area Sizing','Units','normalized','Position',[0.18
0.20 0.50 0.42]);
bar([CD0_landing_baseline, CD0_landing_target, CD0_landing_check]);
xticks(1:3);
xticklabels({'Baseline','Target','Check'});
ylabel('Landing CD_0 [-]');
title(sprintf('Landing Drag Target and Spoiler-Sized Check
(S_{spoiler,total} = %.4f m^2)', S_spoiler_total));
grid on;
box on;

=====

      SPOILER AREA SIZING FOR LANDING CD0 TARGET
=====

Inputs
  CD0_landing_target    = 0.06700
  CD0_landing_baseline  = 0.02700
  S_ref                 = 36.800 m^2
  Spoiler angle         = 60.0 deg
  Cd_spoiler_normal     = 1.280

Required Increment
  Delta_CD0_req         = 0.04000
  A_spoiler_eq          = 1.47200 m^2 (equivalent drag area)

Solved Spoiler Area
  Projection factor     = sin(60.0 deg) = 0.8660
  Total spoiler area    = 1.32791 m^2
  Area per spoiler      = 0.66395 m^2 (2 equal panels)
  Total projected area  = 1.15000 m^2

Back-Check
  Achieved Delta_CD0    = 0.04000
  Landing CD0 checked   = 0.06700
  Spoiler/Wing ratio    = 3.608 %

Glide Deflection Solve
  CD0_glide_target      = 0.33000
  CD0_glide_baseline    = 0.02700
  Required Delta_CD0    = 0.30300
  sin(theta) required   = 6.56014
  Spoiler angle for glide = NOT PHYSICALLY FEASIBLE with this area

```

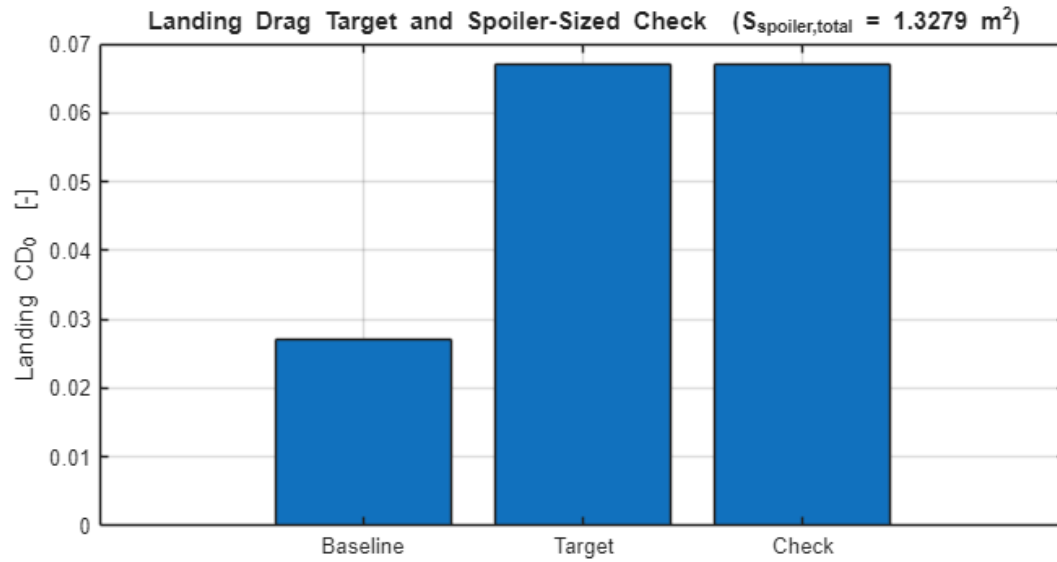
(requires $|\sin(\theta)| > 1$ in the current projected-area model)

=====

RESULT: Use a total spoiler planform area of 1.32791 m^2

RESULT: Glide spoiler angle cannot be solved with the present inputs

=====



Published with MATLAB® R2025b

```
% Parasite Drag Build-Up Using Geometry Extracted From PLANE.step
%
% Geometry was extracted from the STEP file outside of MATLAB.
% MATLAB does NOT need to read the CAD file in this script.
%
% Coordinate interpretation from CAD:
% X = wingspan direction
% Y = aircraft length direction
% Z = vertical direction
%
% Drag build-up method:
%  $D_0 = q_{\infty} * A_{total}$ 
%  $CD_0 = A_{total} / S_{ref}$ 
%
%  $A_{total} = A_{fuse} + A_{wing} + A_{htail} + A_{vtail} + A_{aux}$ 

clc; clear; close all
```

GIVEN VALUES

```
S_ref = 44.9;           % wing reference area, m^2
V_cruise = 69;         % cruise speed, m/s
AR_given = 7.2;        % given aspect ratio
airfoil = "NASA/Langley LS(1)-0417 (GA(W)-1)";
```

CAD-EXTRACTED GEOMETRY

```
% From PLANE.step point data
b_wing = 18.000;        % m, full wingspan from CAD
L_total = 13.699;       % m, full aircraft length including tail
L_fuse = 7.276;         % m, central fuselage/body length estimate

c_root = 3.197;         % m, approximate root chord from CAD
c_tip = 1.928;          % m, approximate tip chord from CAD
lambda_wing = c_tip / c_root;

% Horizontal tail extracted from CAD
b_htail = 4.877;        % m
c_htail_root = 1.817;   % m
c_htail_tip = 0.991;    % m
lambda_htail = c_htail_tip / c_htail_root;

% Vertical tail estimate from CAD side projection
S_vtail = 1.20;         % m^2, approximate exposed vertical tail area
```

ASSUMPTIONS

```
rho = 1.225;            % kg/m^3, sea-level density
mu = 1.789e-5;         % Pa*s, sea-level dynamic viscosity
```

```
l_over_d_fuse = 2.89;          % fuselage fineness ratio, from your input
d_fuse = L_fuse / l_over_d_fuse;

tc_wing = 0.17;                % LS(1)-0417 is approximately 17 percent thick
tc_tail = 0.12;                % assumed tail airfoil thickness ratio

aux_percent = 0.15;            % auxiliary drag allowance for struts, gaps,
gear, antennas, etc.
```

DYNAMIC PRESSURE

```
q_inf = 0.5 * rho * V_cruise^2;
```

WING GEOMETRY CHECK

```
S_wing_CAD_trap = b_wing * (c_root + c_tip) / 2;
AR_from_CAD = b_wing^2 / S_ref;

MAC_wing = (2/3) * c_root * ...
    ((1 + lambda_wing + lambda_wing^2) / (1 + lambda_wing));
```

FUSELAGE PARASITE DRAG AREA

```
Re_fuse = rho * V_cruise * L_fuse / mu;
Cf_fuse = skinFrictionCf(Re_fuse);

S_wet_fuse = (pi*d_fuse/2) * (d_fuse + L_fuse);

FF_fuse = 1 + 1.5*(l_over_d_fuse)^(-1.5) + 7*(l_over_d_fuse)^(-3);

A_fuse = FF_fuse * Cf_fuse * S_wet_fuse;
```

WING PARASITE DRAG AREA

```
Re_wing = rho * V_cruise * MAC_wing / mu;
Cf_wing = skinFrictionCf(Re_wing);

FF_wing = 1 + 2*tc_wing + 60*tc_wing^4;

A_wing = 2 * Cf_wing * FF_wing * S_ref;
```

HORIZONTAL TAIL PARASITE DRAG AREA

```
S_htail = b_htail * (c_htail_root + c_htail_tip) / 2;

MAC_htail = (2/3) * c_htail_root * ...
    ((1 + lambda_htail + lambda_htail^2) / (1 + lambda_htail));

Re_htail = rho * V_cruise * MAC_htail / mu;
Cf_htail = skinFrictionCf(Re_htail);
```

```
FF_htail = 1 + 2*tc_tail + 60*tc_tail^4;
```

```
A_htail = 2 * Cf_htail * FF_htail * S_htail;
```

VERTICAL TAIL PARASITE DRAG AREA

```
% Use an approximate vertical tail MAC of 1.0 m based on CAD side projection
MAC_vtail = 1.0;
```

```
Re_vtail = rho * V_cruise * MAC_vtail / mu;
Cf_vtail = skinFrictionCf(Re_vtail);
```

```
FF_vtail = 1 + 2*tc_tail + 60*tc_tail^4;
```

```
A_vtail = 2 * Cf_vtail * FF_vtail * S_vtail;
```

AUXILIARY DRAG AREA

```
A_clean = A_fuse + A_wing + A_htail + A_vtail;
A_aux = aux_percent * A_clean;
```

```
A_total = A_clean + A_aux;
```

FINAL PARASITE DRAG

```
CD0 = A_total / S_ref;
D0 = q_inf * A_total;
```

PRINT RESULTS

```
fprintf('Parasite Drag Build-Up Using CAD-Extracted Geometry\n')
fprintf('-----\n\n')

fprintf('Airfoil: %s\n', airfoil)
fprintf('Cruise speed = %.2f m/s\n', V_cruise)
fprintf('Dynamic pressure q_inf = %.2f Pa\n\n', q_inf)

fprintf('CAD-Extracted Main Geometry:\n')
fprintf('Wing span b = %.3f m\n', b_wing)
fprintf('Full aircraft length = %.3f m\n', L_total)
fprintf('Estimated fuselage/body length = %.3f m\n', L_fuse)
fprintf('Wing root chord = %.3f m\n', c_root)
fprintf('Wing tip chord = %.3f m\n', c_tip)
fprintf('Wing taper ratio = %.3f\n', lambda_wing)
fprintf('Wing MAC = %.3f m\n', MAC_wing)
fprintf('Wing area from CAD trapezoid = %.3f m^2\n', S_wing_CAD_trap)
fprintf('Reference wing area used = %.3f m^2\n', S_ref)
fprintf('AR from CAD span and S_ref = %.3f\n\n', AR_from_CAD)

fprintf('Fuselage Estimate:\n')
```

```

fprintf('l/d used = %.3f\n', l_over_d_fuse)
fprintf('Fuselage diameter estimate = %.3f m\n', d_fuse)
fprintf('Fuselage wetted area = %.3f m^2\n', S_wet_fuse)
fprintf('Re_fuse = %.3e\n', Re_fuse)
fprintf('Cf_fuse = %.5f\n', Cf_fuse)
fprintf('FF_fuse = %.4f\n', FF_fuse)
fprintf('A_fuse = %.4f m^2\n\n', A_fuse)

fprintf('Wing Drag:\n')
fprintf('Re_wing = %.3e\n', Re_wing)
fprintf('Cf_wing = %.5f\n', Cf_wing)
fprintf('FF_wing = %.4f\n', FF_wing)
fprintf('A_wing = %.4f m^2\n\n', A_wing)

fprintf('Tail Drag:\n')
fprintf('Horizontal tail area = %.3f m^2\n', S_htail)
fprintf('Horizontal tail MAC = %.3f m\n', MAC_htail)
fprintf('A_htail = %.4f m^2\n', A_htail)
fprintf('Vertical tail area estimate = %.3f m^2\n', S_vtail)
fprintf('A_vtail = %.4f m^2\n\n', A_vtail)

fprintf('Equivalent Parasite Drag Areas:\n')
fprintf('A_fuse = %.4f m^2\n', A_fuse)
fprintf('A_wing = %.4f m^2\n', A_wing)
fprintf('A_htail = %.4f m^2\n', A_htail)
fprintf('A_vtail = %.4f m^2\n', A_vtail)
fprintf('A_aux = %.4f m^2\n', A_aux)
fprintf('A_total = %.4f m^2\n\n', A_total)

fprintf('Final Results:\n')
fprintf('CD0 = %.5f\n', CD0)
fprintf('Parasite drag D0 = %.2f N\n', D0)
fprintf('Parasite drag D0 = %.2f lbf\n', D0 * 0.224809)

```

Parasite Drag Build-Up Using CAD-Extracted Geometry

Airfoil: NASA/Langley LS(1)-0417 (GA(W)-1)
Cruise speed = 69.00 m/s
Dynamic pressure q_{inf} = 2916.11 Pa

CAD-Extracted Main Geometry:

Wing span b = 18.000 m
Full aircraft length = 13.699 m
Estimated fuselage/body length = 7.276 m
Wing root chord = 3.197 m
Wing tip chord = 1.928 m
Wing taper ratio = 0.603
Wing MAC = 2.615 m
Wing area from CAD trapezoid = 46.125 m²
Reference wing area used = 44.900 m²
AR from CAD span and S_{ref} = 7.216

Fuselage Estimate:

```
l/d used = 2.890
Fuselage diameter estimate = 2.518 m
Fuselage wetted area = 38.731 m^2
Re_fuse = 3.438e+07
Cf_fuse = 0.00230
FF_fuse = 1.5953
A_fuse = 0.1422 m^2

Wing Drag:
Re_wing = 1.235e+07
Cf_wing = 0.00282
FF_wing = 1.3901
A_wing = 0.3525 m^2

Tail Drag:
Horizontal tail area = 6.847 m^2
Horizontal tail MAC = 1.444 m
A_htail = 0.0545 m^2
Vertical tail area estimate = 1.200 m^2
A_vtail = 0.0103 m^2

Equivalent Parasite Drag Areas:
A_fuse = 0.1422 m^2
A_wing = 0.3525 m^2
A_htail = 0.0545 m^2
A_vtail = 0.0103 m^2
A_aux = 0.0839 m^2
A_total = 0.6435 m^2

Final Results:
CD0 = 0.01433
Parasite drag D0 = 1876.47 N
Parasite drag D0 = 421.85 lbf
```

LOCAL FUNCTION

```
function Cf = skinFrictionCf(ReL)
    if ReL <= 5e5
        Cf = 1.33 / sqrt(ReL);
    else
        Cf = 0.074 / ReL^(1/5);
    end
end
```

Published with MATLAB® R2025b

RaymerWeightEst_v3.m

===== Arctic STOL
Aircraft — Initial Takeoff Weight Estimation Method: Raymer statistical EWF (single & twin), iterated to convergence.

KEY CHANGES FROM v2: 1. User-configurable wing geometry block at the top (S, AR). 2. Loiter fuel fraction computed from the EXACT Breguet endurance equation (screenshot formula), not the simplified L/D shorthand. Because this formula depends on W0 at the start of loiter (which changes each iteration), ff_loiter is recomputed inside the Raymer sizing loop — not precomputed as a fixed constant. 3. STOL EWF penalty of 1.04x applied to both Raymer configurations to account for heavier STOL-specific structure (reinforced gear for gravel, large flap system, de-icing provisions, structural margins).

BREGUET ENDURANCE FORMULA (screenshot): $E = (\eta / \text{SFC}_p) \cdot (\text{CL}^{3/2} / \text{CD}) \cdot \sqrt{2\rho S} \cdot [W1^{(-1/2)} - W0^{(-1/2)}]$

- W0 = weight at START of loiter (heavier) [N]
- W1 = weight at END of loiter (lighter) [N]
- W1 < W0, so $[W1^{(-1/2)} - W0^{(-1/2)}] > 0$ (endurance is positive)

NOTE ON g: When W is in Newtons and SFC_p is in kg/(W·s) the derivation (Anderson §6.6) introduces a factor 1/g in the denominator. Some textbooks absorb this by expressing W in kg-force (1 kgf = 9.81 N), making g disappear. This script uses SI throughout — W in Newtons — so $g = 9.81 \text{ m/s}^2$ appears explicitly in K_loiter below.

Rearranging for the fuel fraction ff = W1/W0:

$$\begin{aligned} W1^{(-1/2)} &= E/K + W0^{(-1/2)} \\ W1 &= (E/K + W0^{(-1/2)})^{(-2)} \\ \text{ff} &= W1/W0 = (E/K + W0^{(-1/2)})^{(-2)} / W0 \end{aligned}$$

where $K = (\eta / (g \cdot \text{SFC}_p)) \cdot (\text{CL}^{3/2} / \text{CD}) \cdot \sqrt{2\rho S} \quad [N^{1/2}]$

Because K contains $\sqrt{S}$ and ff contains W0 (which changes each Raymer step), ff_loiter is recalculated at EVERY iteration of the sizing loop.

Best-endurance CL and CD are derived from the parabolic drag polar:

$$\text{CD} = \text{CD}_0 + k \cdot \text{CL}^2 \quad \text{with } k = 1/(\pi \cdot \text{AR} \cdot e)$$

Optimal point: $d/d\text{CL} [\text{CL}^{3/2} / \text{CD}] = 0 \rightarrow \text{CL}_{\text{BE}} = \sqrt{3 \cdot \text{CD}_0 / k}$

$$\text{CD}_{\text{BE}} = 4 \cdot \text{CD}_0$$

This is the aircraft operating condition during loiter.

Units: SI (kg, m, N, W, s) throughout except inside Raymer's formula which requires W0 in POUNDS (conversion applied, clearly labelled).

Authors:	AME	261	Design	Team	Date:	April	2026	Version:	3.0
----------	-----	-----	--------	------	-------	-------	------	----------	-----

```
clear; clc; close all;
```

```

=====

=====

>>>  USER INPUTS – EDIT THIS BLOCK EACH DESIGN ITERATION  <<<
=====
These are the geometry and aerodynamic parameters that are NOT yet
locked in. Update them as the design converges. All other constants
(payload, crew, mission, Raymer coefficients) are in Section 2 below.

% --- Wing geometry ---
S   = 37.1;      % Wing reference area          [m²]   ESTIMATE
                % Caravan: 26 m², Kodiak: 23 m², Twin Otter: 39 m²
                % Use ~30-40 m² for initial sizing of a 9-pax STOL turboprop

AR  = 6.76;      % Aspect ratio                  [-]    ESTIMATE
                % STOL aircraft typically AR = 8-12.
                % Higher AR: better L/D, worse STOL (larger span → more
                % induced drag penalty near ground, heavier wing structure).
                % Start at 10 – in the middle of the STOL-compatible range.

% --- Aerodynamics ---
CD0 = 0.0288;    % Parasite drag coefficient, clean config [-] ESTIMATE
                % Typical range for clean turboprop: 0.025-0.035.
                % Will be refined with full drag buildup (fuselage + wetted
                % area calculation, April 21 deliverable).

e   = 0.90;      % Oswald efficiency factor          [-]
                % Per RFP §4.3: use 0.9 clean, 0.8 with wing stores.

% =====
%  END OF USER INPUT BLOCK
% =====

=====

=====

SECTION 1 – REFERENCE AIRCRAFT SURVEY
Purpose: ground-truth band for EWF. Raymer gives one number; real
aircraft give the range we must land inside.
Sources: manufacturer data sheets, Jane's All the World's Aircraft.
=====

ref(1).name      = 'Cessna 208B Grand Caravan';
ref(1).pax       = 9;
ref(1).MTOW_kg   = 3969;  ref(1).OEW_kg = 2145;  ref(1).engines = 1;
ref(1).notes     = 'Baseline single-engine turboprop; workhorse of Arctic/
Alaska ops';

ref(2).name      = 'Quest/Daher Kodiak 100';
ref(2).pax       = 9;
ref(2).MTOW_kg   = 3062;  ref(2).OEW_kg = 1934;  ref(2).engines = 1;

```

```

ref(2).notes = 'Purpose-built STOL; most mission-relevant single-engine
comparator';

ref(3).name = 'Pilatus PC-12 NGX';
ref(3).pax = 9;
ref(3).MTOW_kg = 4740; ref(3).OEW_kg = 2812; ref(3).engines = 1;
ref(3).notes = 'Pressurized single turboprop; upper bound of single-engine
band';

ref(4).name = 'DHC-6 Twin Otter Series 400';
ref(4).pax = 19; % larger pax - retained as STOL twin structural ref
ref(4).MTOW_kg = 5670; ref(4).OEW_kg = 3363; ref(4).engines = 2;
ref(4).notes = 'Classic Arctic STOL twin; oversized for mission but best
twin ref';

ref(5).name = 'Britten-Norman BN-2 Islander';
ref(5).pax = 9;
ref(5).MTOW_kg = 2994; ref(5).OEW_kg = 1816; ref(5).engines = 2;
ref(5).notes = 'Light twin STOL; lower bound for twin EWF';

ref(6).name = 'Tecnam P2012 Traveller';
ref(6).pax = 9;
ref(6).MTOW_kg = 3900; ref(6).OEW_kg = 2400; ref(6).engines = 2;
ref(6).notes = 'Modern twin-piston 9-pax utility; piston EWF > turboprop';

fprintf('=== SECTION 1: REFERENCE AIRCRAFT SURVEY ===\n');
fprintf('%-35s | Eng | %9s | %8s | %5s\n', 'Aircraft', 'MTOW (kg)', 'OEW
(kg)', 'EWF');
fprintf('%s\n', repmat('-', 1, 75));
for i = 1:length(ref)
    ref(i).EWF = ref(i).OEW_kg / ref(i).MTOW_kg;
    fprintf('%-35s | %d | %9.0f | %8.0f | %5.3f\n', ...
        ref(i).name, ref(i).engines, ref(i).MTOW_kg, ref(i).OEW_kg,
        ref(i).EWF);
end

% Separate single-engine (9-pax) and twin-engine reference bands
SE_mask = ([ref.engines] == 1) & ([ref.pax] == 9);
TE_mask = ([ref.engines] == 2);
SE_EWF = [ref(SE_mask).EWF];
TE_EWF = [ref(TE_mask).EWF];

fprintf('\nSingle-engine 9-pax EWF band: %.3f - %.3f (mean %.3f)\n', ...
    min(SE_EWF), max(SE_EWF), mean(SE_EWF));
fprintf('Twin-engine reference EWF band: %.3f - %.3f (mean %.3f)\n\n', ...
    min(TE_EWF), max(TE_EWF), mean(TE_EWF));

=== SECTION 1: REFERENCE AIRCRAFT SURVEY ===
Aircraft | Eng | MTOW (kg) | OEW (kg) | EWF
-----
Cessna 208B Grand Caravan | 1 | 3969 | 2145 | 0.540
Quest/Daher Kodiak 100 | 1 | 3062 | 1934 | 0.632
Pilatus PC-12 NGX | 1 | 4740 | 2812 | 0.593
DHC-6 Twin Otter Series 400 | 2 | 5670 | 3363 | 0.593

```

Britten-Norman BN-2 Islander	2	2994	1816	0.607
Tecnam P2012 Traveller	2	3900	2400	0.615

Single-engine 9-pax EWF band: 0.540 - 0.632 (mean 0.588)
Twin-engine reference EWF band: 0.593 - 0.615 (mean 0.605)

=====

=====

SECTION 2 – FIXED DESIGN PARAMETERS

Values that do not change between iterations (mission, propulsion, payload). Update only if the RFP requirements change.

=====

```
% --- Payload: passengers + baggage ONLY, crew NOT included ---
% RFP Config 1: 9 pax × (200 lb body + 50 lb baggage) = 2,250 lb = 1,021 kg
% Per design decision: use 1,100 kg as the maximum design payload.
% Crew is excluded from payload – it is part of OEW (see below).
W_payload_kg = 1399;           % [kg]

% --- Crew: 2 pilots, placed in OEW (not payload) ---
% Crew weight does NOT change from flight to flight – it is a fixed part of
% the operating configuration, not a mission-variable load.
% RFP: 2 pilots × (180 lb body + 20 lb baggage) = 400 lb total = 181.4 kg
W_crew_kg = 2 * 200 * 0.4536;   % [kg] = 181.44 kg exactly

% --- Combined weight that drives the sizing denominator ---
% W0 = W_payload + W_crew + W_fuel + W_structural_empty
%   = W_fixed   +      Wf_frac*W0   +   EWF*W0
% → W0*(1 - EWF - Wf_frac) = W_fixed
W_fixed_kg = W_payload_kg + W_crew_kg;   % [kg]

% --- Propulsion (user-specified, consistent with PT6A turboprop class) ---
SFCp_imp = 0.548;               % specific fuel consumption [lb/(hp·hr)]
eta_p     = 0.80;               % propeller efficiency      [-]

% Convert SFCp to SI: [kg/(W·s)]
% 1 lb/(hp·hr) × 0.4536 kg/lb ÷ (745.7 W/hp × 3600 s/hr)
SFCp_SI = SFCp_imp * 0.4536 / (745.7 * 3600);   % [kg/(W·s)]

% --- Mission parameters (from RFP Section 1.3) ---
R_nmi     = 450;                % minimum cruise range          [nmi]
R_m       = R_nmi * 1852;       % converted to metres      [m]
t_loiter_s = 45 * 60;           % loiter reserve duration   [s]

% --- Atmospheric constants ---
g         = 9.81;               % gravitational acceleration [m/s²]
rho_SL    = 1.225;              % sea-level ISA air density  [kg/m³]
% Loiter assumed at sea level – conservative (max fuel burn).
% Re-evaluate if loiter altitude is specified in mission.
```

```

% --- Cruise aerodynamics (baseline; swept in Section 6) ---
k = 1/(pi*e*AR);
E_m = sqrt(1/(4*CD0*k));
LD_cruise = E_m; % L/D at cruise condition [-] ESTIMATE
                % Conservative for STOL wing; refined with drag buildup.

% --- STOL EWF penalty ---
% STOL aircraft carry structural weight not captured by Raymer's GA database:
% - Large-chord double-slotted flap systems
% - Reinforced gravel-rated landing gear (higher impact loads, gravel
guards)
% - De-icing provisions (FIKI required by RFP)
% - Structural reinforcement at gravel-runway stress concentrations
% Empirical correction: +5% on Raymer EWF based on Kodiak vs generic GA data.
% Apply to both single- and twin-engine configs since both are STOL.
STOL_penalty = 1.05; % multiplicative correction on EWF [-]

% --- Fuel reserve ---
fuel_reserve = 0.03; % 3% additional reserve over mission fuel [-]

% --- Raymer convergence settings ---
tol_kg = 0.5; % convergence tolerance [kg]
max_iter = 300; % maximum iterations
lb_per_kg = 2.20462; % unit conversion constant [lb/kg]

fprintf('=== SECTION 2: DESIGN INPUTS ===\n');
fprintf(' Wing area S : %.1f m^2\n', S);
fprintf(' Aspect ratio AR : %.1f\n', AR);
fprintf(' CD0 (clean) : %.4f\n', CD0);
fprintf(' Oswald e : %.2f\n', e);
fprintf(' Payload (no crew) : %.1f kg (%.1f lb)\n', W_payload_kg,
W_payload_kg/0.4536);
fprintf(' Crew (in OEW) : %.2f kg (%.1f lb)\n', W_crew_kg,
W_crew_kg/0.4536);
fprintf(' Total fixed weight : %.2f kg\n', W_fixed_kg);
fprintf(' SFCp : %.3f lb/(hp·hr) → %.4e kg/(W·s)\n',
SFCp_imp, SFCp_SI);
fprintf(' eta_p : %.2f\n', eta_p);
fprintf(' Cruise L/D (baseline): %.1f (swept in Section 6)\n', LD_cruise);
fprintf(' STOL EWF penalty : %.2fx\n', STOL_penalty);
fprintf(' Fuel reserve : %.0f%%\n\n', fuel_reserve*100);

=== SECTION 2: DESIGN INPUTS ===
Wing area S : 37.1 m^2
Aspect ratio AR : 6.8
CD0 (clean) : 0.0288
Oswald e : 0.90
Payload (no crew) : 1399.0 kg (3084.2 lb)
Crew (in OEW) : 181.44 kg (400.0 lb)
Total fixed weight : 1580.44 kg
SFCp : 0.548 lb/(hp·hr) → 9.2595e-08 kg/(W·s)
eta_p : 0.80
Cruise L/D (baseline): 12.9 (swept in Section 6)

```

```

STOL EWF penalty      : 1.05x
Fuel reserve          : 3%

```

```

=====
=====

```

SECTION 3 – LOITER AERODYNAMICS: BEST-ENDURANCE OPERATING POINT

For maximum endurance (propeller aircraft), we maximise $CL^{(3/2)}/CD$.
Using a parabolic drag polar $CD = CD_0 + k \cdot CL^2$ with $k = 1/(\pi \cdot AR \cdot e)$:

$$\begin{aligned}
d/dCL [CL^{(3/2)}/CD] &= 0 \\
\rightarrow (3/2) \cdot CL^{(1/2)} \cdot CD - CL^{(3/2)} \cdot 2k \cdot CL &= 0 \\
\rightarrow (3/2) \cdot CD &= 2k \cdot CL^2 \\
\rightarrow CL_{BE} &= \sqrt{3 \cdot CD_0/k} \\
CD_{BE} &= CD_0 + k \cdot (3 \cdot CD_0/k) = 4 \cdot CD_0 \\
(\text{induced drag} &= 3 \times \text{parasite drag at best endurance} - \text{NOT } L/D_{\max} \text{ point})
\end{aligned}$$

$L/D_{\max}$ occurs at $CL = \sqrt{CD_0/k}$ (induced = parasite), giving $CD = 2 \cdot CD_0$.
Best endurance flies at a higher CL (lower speed) than $L/D_{\max}$.

CL_{BE} is checked against $CL_{\max}$ here. If $CL_{BE} > CL_{\max}$ the wing stalls before reaching best endurance – flag this and use $CL_{\max}$ instead.

```

=====

k_ind    = 1 / (pi * AR * e);           % induced drag factor           [-]
CL_BE    = sqrt(3 * CD0 / k_ind);       % best-endurance lift coeff       [-]
CD_BE    = 4 * CD0;                    % best-endurance drag coeff      [-]
BE_param = CL_BE^1.5 / CD_BE;           % endurance parameter CL^1.5/CD [-]

% Warn if CL_BE exceeds a typical maximum lift coefficient
CL_max_typical = 2.4; % typical STOL CL_max with flaps deployed
if CL_BE > CL_max_typical
    warning(['CL_BE = %.3f exceeds CL_max = %.3f.\n' ...
            'Loiter is constrained by stall, not by aerodynamic optimum.\n'
            ...
            'Replace CL_BE and CD_BE with values at actual CL_max.'], ...
            CL_BE, CL_max_typical);
    CL_BE    = CL_max_typical;
    CD_BE    = CD0 + k_ind * CL_BE^2; % re-evaluate CD at CL_max
    BE_param = CL_BE^1.5 / CD_BE;
end

% K_loiter: the weight-and-geometry-independent part of the Breguet formula
% Units: K has units of \sqrt{N} (Newton^(1/2)) – see derivation note in header.
% When E [s] is divided by K [N^(1/2)] and added to W0^(-1/2) [N^(-1/2)],
% dimensions are consistent: W1 = (E/K + W0^(-1/2))^(-2) has units N. ✓
K_loiter = (eta_p / (g * SFCp_SI)) * BE_param * sqrt(2 * rho_SL * S);

fprintf('== SECTION 3: LOITER AERODYNAMICS ==\n');
fprintf(' Induced drag factor k : %.5f\n', k_ind);
fprintf(' Best-endurance CL_BE   : %.4f\n', CL_BE);

```

```

fprintf(' Best-endurance CD_BE : %.4f (= 4×CD0 if not stall-limited)\n',
CD_BE);
fprintf(' Endurance param (BE) : CL^1.5/CD = %.4f\n', BE_param);
fprintf(' K_loiter : %.4e [N^(1/2)]\n\n', K_loiter);

```

```

=== SECTION 3: LOITER AERODYNAMICS ===

```

```

    Induced drag factor k : 0.05232
    Best-endurance CL_BE : 1.2851
    Best-endurance CD_BE : 0.1152 (= 4×CD0 if not stall-limited)
    Endurance param (BE) : CL^1.5/CD = 12.6455
    K_loiter : 1.0618e+08 [N^(1/2)]

```

```

=====
=====

```

SECTION 4 – FIXED MISSION SEGMENT FUEL FRACTIONS

These segments use Raymer Table 3.2 historical data. They do NOT depend on W_0 , so they are precomputed once before the iteration loop. The cruise segment uses the Breguet range equation (recomputed in the L/D sensitivity loop but fixed at baseline here). The loiter segment is handled INSIDE the Raymer loop (Section 5).

```

% Breguet range equation for propeller aircraft (SI form):
% R = (η_p / (g × SFCp_SI)) × (L/D) × ln(W_i / W_{i+1})
% → W_{i+1}/W_i = exp(-R × g × SFCp_SI / (η_p × L/D))
%
% NOTE: some textbooks write R[statute miles] = (375×η_p/SFCp)×(L/D)×ln(...)
% The factor 375 applies ONLY in statute miles. For nautical miles the
% correct factor is ~325.9. The SI form avoids this ambiguity entirely.

```

```

ff_cruise = exp(-R_m * g * SFCp_SI / (eta_p * LD_cruise));

```

```

ff_warmup = 0.990; % warm-up and taxi (10 min)
ff_takeoff = 0.995; % STOL takeoff – slightly penalised vs FAR-25 standard
ff_climb = 0.980; % climb to cruise altitude
ff_descent = 0.990; % descent to sea level
ff_landing = 0.992; % landing rollout and taxi

```

```

% Product of all segments EXCEPT loiter – used to find W0_loiter inside loop
ff_pre_loiter = ff_warmup * ff_takeoff * ff_climb * ff_cruise * ff_descent;

```

```

fprintf('=== SECTION 4: FIXED FUEL FRACTIONS ===\n');
fprintf(' %24s : %.4f\n', 'Warm-up & taxi', ff_warmup);
fprintf(' %24s : %.4f\n', 'STOL Takeoff', ff_takeoff);
fprintf(' %24s : %.4f\n', 'Climb', ff_climb);
fprintf(' %24s : %.4f (Breguet, R=%.0f nmi, L/D=%.1f)\n', ...
    'Cruise', ff_cruise, R_nmi, LD_cruise);
fprintf(' %24s : %.4f\n', 'Descent', ff_descent);
fprintf(' %24s : computed inside iteration loop\n', 'Loiter 45 min');
fprintf(' %24s : %.4f\n\n', 'Landing & taxi', ff_landing);

```

=== SECTION 4: FIXED FUEL FRACTIONS ===

Warm-up & taxi	: 0.9900
STOL Takeoff	: 0.9950
Climb	: 0.9800
Cruise	: 0.9292 (Breguet, R=450 nmi, L/D=12.9)
Descent	: 0.9900
Loiter 45 min	: computed inside iteration loop
Landing & taxi	: 0.9920

=====

=====

SECTION 5 – RAYMER ITERATIVE WEIGHT SIZING (SINGLE & TWIN ENGINE)

Raymer formula (Table 3.1, "Aircraft Design: A Conceptual Approach"):

EWF_{raw} = A × W0_{lbs}^C

EWF = EWF_{raw} × STOL_{penalty} (applied after Raymer formula)

Coefficients:

Single-engine prop: A = 2.36, C = -0.18 (Raymer Table 3.1)

Twin-engine prop: A = 1.51, C = -0.10 (Raymer Table 3.1)

The weight equation (see derivation in header) is:

$W_0 = W_{\text{fixed}} / (1 - \text{EWF} - W_{\text{f_frac}})$

where $W_{\text{fixed}} = W_{\text{payload}} + W_{\text{crew}}$ (both non-fuel, fixed weights)

and $W_{\text{f_frac}} = \text{total fuel weight} / W_0$ (recomputed each iteration)

The loiter fuel fraction cannot be precomputed because ff_{loiter} depends on $W_0_{\text{loiter}} = W_0_{\text{guess}} \times g \times ff_{\text{pre_loiter}}$ [N]. This changes with each update to W_0_{guess} , so the full $W_{\text{f_frac}}$ is updated inside the loop.

Iteration procedure:

1. Guess W_0 [kg]
2. Compute EWF from Raymer × STOL_{penalty}
3. Compute $W_0_{\text{loiter}} = W_0_{\text{guess}} \times g \times ff_{\text{pre_loiter}}$ [N]
4. Apply screenshot formula: $W_1_{\text{loiter}} = (t/K + W_0_{\text{loiter}}^{-0.5})^{-2}$
5. $ff_{\text{loiter}} = W_1_{\text{loiter}} / W_0_{\text{loiter}}$
6. Compute $W_{\text{f_frac}}$ from all segment fractions (including ff_{loiter})
7. Solve: $W_0_{\text{new}} = W_{\text{fixed}} / (1 - \text{EWF} - W_{\text{f_frac}})$
8. If $|W_0_{\text{new}} - W_0_{\text{guess}}| < \text{tol}$ → converged. Else blend and repeat.

=====

% Raymer coefficient sets

configs(1).label = 'Single-Engine Prop';

configs(1).A = 2.36;

configs(1).C = -0.18;

configs(1).W0_0 = 4000; % initial guess [kg]

configs(2).label = 'Twin-Engine Prop';

configs(2).A = 1.51;

configs(2).C = -0.10;

```

configs(2).W0_0 = 5000;    % initial guess – twins are structurally heavier

fprintf('== SECTION 5: RAYMER ITERATIVE WEIGHT SIZING ==\n');
fprintf('      (STOL EWF penalty = %.2fx applied to both configurations)\n\n',
STOL_penalty);

for c = 1:length(configs)
    A = configs(c).A;
    C = configs(c).C;
    W0_guess = configs(c).W0_0;

    fprintf('--- %s (A = %.2f, C = %.2f) ---\n', configs(c).label, A, C);
    fprintf('  Iter | W0_guess [kg] | EWF_raw | EWF+STOL | ff_loiter |
W0_calc [kg] | Delta\n');
    fprintf('  %s\n', repmat('-', 1, 78));

    converged = false;
    ff_loiter_final = NaN;

    for iter = 1:max_iter

        % --- Step 1: Raymer EWF with STOL correction ---
        W0_lbs = W0_guess * lb_per_kg;
        EWF_raw = A * (W0_lbs ^ C);
        EWF      = EWF_raw * STOL_penalty;

        % --- Step 2: Loiter fuel fraction (exact Breguet endurance formula)
        ---
        % Weight at start of loiter: apply preceding segment fractions to
        MTOW.
        % Convert from kg to N (Newtons) for the formula – W must be a force.
        W0_loiter_N = W0_guess * g * ff_pre_loiter;    % [N]

        % Solve for W1_loiter using rearranged screenshot formula:
        %   W1 = (E/K + W0^(-1/2))^(-2)
        W1_loiter_N = 1 / (t_loiter_s / K_loiter + W0_loiter_N^(-0.5))^2;

        % Fuel fraction for loiter segment
        ff_loiter = W1_loiter_N / W0_loiter_N;

        % --- Step 3: Total mission fuel fraction ---
        ff_total = ff_pre_loiter * ff_loiter * ff_landing;
        Mff      = 1 - ff_total;
        Wf_frac  = Mff * (1 + fuel_reserve);

        % --- Step 4: Feasibility check ---
        denom = 1 - EWF - Wf_frac;
        if denom <= 0
            % EWF + fuel fraction >= 1: structurally infeasible at this
            weight.
            % This means the aircraft cannot carry any payload – try heavier
            guess.
            fprintf('  INFEASIBLE at W0 = %.0f kg (EWF + Wf = %.4f >=
1)\n', ...

```

```

        W0_guess, EWF + Wf_frac);
    W0_guess = W0_guess * 1.2;
    continue
end

% --- Step 5: Updated MTOW estimate ---
W0_calc = W_fixed_kg / denom;
delta    = abs(W0_calc - W0_guess);

% Print first 10 iterations and final convergence row
if iter <= 10 || delta < 2
    fprintf('  %4d | %12.1f | %7.4f | %8.4f | %9.6f | %12.1f |
%5.1f\n', ...
        iter, W0_guess, EWF_raw, EWF, ff_loiter, W0_calc, delta);
end

if delta < tol_kg
    converged = true;
    ff_loiter_final = ff_loiter;
    break
end

% Blend old and new to prevent oscillation around the fixed point
W0_guess = 0.5 * W0_guess + 0.5 * W0_calc;
end

% Store converged results
configs(c).W0_kg      = W0_calc;
configs(c).EWF_raw    = EWF_raw;
configs(c).EWF        = EWF;           % includes STOL penalty
configs(c).W_struct_kg = EWF * W0_calc; % structural empty (no crew)
configs(c).W_OEW_kg   = EWF * W0_calc + W_crew_kg; % OEW = struct +
crew
configs(c).W_fuel_kg   = Wf_frac * W0_calc;
configs(c).ff_loiter   = ff_loiter_final;
configs(c).ff_total    = ff_total;
configs(c).Wf_frac     = Wf_frac;
configs(c).W_check_kg  = configs(c).W_OEW_kg + configs(c).W_fuel_kg +
W_payload_kg;
configs(c).converged    = converged;

    fprintf('\n Converged: %s | Iterations: %d\n\n', mat2str(converged),
iter);
end

=== SECTION 5: RAYMER ITERATIVE WEIGHT SIZING ===
(STOL EWF penalty = 1.05x applied to both configurations)

--- Single-Engine Prop (A = 2.36, C = -0.18) ---
    Iter | W0_guess [kg] | EWF_raw | EWF+STOL | ff_loiter | W0_calc [kg] |
Delta
-----
-

```

```

1 | 4000.0 | 0.4600 | 0.4830 | 0.990574 | 4096.6 | 96.6
2 | 4048.3 | 0.4590 | 0.4819 | 0.990517 | 4086.1 | 37.8
3 | 4067.2 | 0.4586 | 0.4815 | 0.990495 | 4082.1 | 14.9
4 | 4074.6 | 0.4585 | 0.4814 | 0.990487 | 4080.5 | 5.8
5 | 4077.6 | 0.4584 | 0.4813 | 0.990483 | 4079.9 | 2.3
6 | 4078.7 | 0.4584 | 0.4813 | 0.990482 | 4079.6 | 0.9
7 | 4079.2 | 0.4584 | 0.4813 | 0.990482 | 4079.5 | 0.4

Converged: true | Iterations: 7

--- Twin-Engine Prop (A = 1.51, C = -0.10) ---
Iter | W0_guess [kg] | EWF_raw | EWF+STOL | ff_loiter | W0_calc [kg] |
Delta
-----
1 | 5000.0 | 0.5953 | 0.6251 | 0.989470 | 6512.1 |
1512.1
2 | 5756.1 | 0.5870 | 0.6163 | 0.988708 | 6303.1 | 547.0
3 | 6029.6 | 0.5843 | 0.6135 | 0.988445 | 6238.0 | 208.4
4 | 6133.8 | 0.5833 | 0.6124 | 0.988347 | 6214.4 | 80.6
5 | 6174.1 | 0.5829 | 0.6120 | 0.988309 | 6205.5 | 31.4
6 | 6189.8 | 0.5827 | 0.6119 | 0.988294 | 6202.0 | 12.2
7 | 6195.9 | 0.5827 | 0.6118 | 0.988289 | 6200.7 | 4.8
8 | 6198.3 | 0.5827 | 0.6118 | 0.988286 | 6200.2 | 1.9
9 | 6199.2 | 0.5827 | 0.6118 | 0.988285 | 6200.0 | 0.7
10 | 6199.6 | 0.5826 | 0.6118 | 0.988285 | 6199.9 | 0.3

Converged: true | Iterations: 10

=====

=====

SECTION 5b - FINAL WEIGHT STATEMENTS
=====

fprintf('== SECTION 5b: FINAL WEIGHT STATEMENTS ==\n\n');

for c = 1:length(configs)
    W0 = configs(c).W0_kg;

    fprintf('-----\n');
    fprintf('| Configuration : %s\n', configs(c).label);
    fprintf('| Raymer Coefficients : A = %.2f, C = %.2f | STOL penalty = %.2fx\n', ...
        configs(c).A, configs(c).C, STOL_penalty);

    fprintf('-----\n');
    fprintf('| Max Takeoff Weight (W0) : %7.1f kg (%8.1f lb) (%8.1f N)

```

```

\n', W0, W0*lb_per_kg, W0*g);
    fprintf('| \n');
    fprintf('| OEW Breakdown:\n');
    fprintf('| Structural empty weight : %7.1f kg (%8.1f lb)
(EWF_raw=%.4f, EWF+STOL=%.4f)\n', ...
    configs(c).W_struct_kg, configs(c).W_struct_kg*lb_per_kg, ...
    configs(c).EWF_raw, configs(c).EWF);
    fprintf('| Crew weight (in OEW) : %7.1f kg (%8.1f lb) (2 pilots,
RFP)\n', ...
    W_crew_kg, W_crew_kg*lb_per_kg);
    fprintf('| OEW total (struct+crew) : %7.1f kg (%8.1f lb) (%.1f%% of
W0)\n', ...
    configs(c).W_OEW_kg, configs(c).W_OEW_kg*lb_per_kg, ...
    100*configs(c).W_OEW_kg/W0);
    fprintf('| \n');
    fprintf('| Fuel weight (incl 5%% res) : %7.1f kg (%8.1f lb) (%.1f%%
of W0)\n', ...
    configs(c).W_fuel_kg, configs(c).W_fuel_kg*lb_per_kg, ...
    100*configs(c).W_fuel_kg/W0);
    fprintf('| ff_cruise : %.6f\n', ff_cruise);
    fprintf('| ff_loiter (exact Breguet): %.6f (W0_loiter = %.1f N)
\n', ...
    configs(c).ff_loiter, configs(c).W0_kg * g * ff_pre_loiter);
    fprintf('| Mission fuel fraction : %.4f (%.2f%% of W0)\n', ...
    configs(c).Wf_frac, 100*configs(c).Wf_frac);
    fprintf('| \n');
    fprintf('| Payload (no crew) : %7.1f kg (%8.1f lb) (%.1f%% of
W0)\n', ...
    W_payload_kg, W_payload_kg*lb_per_kg, 100*W_payload_kg/W0);
    fprintf('| Weight closure error : %.3f kg\n', ...
    abs(configs(c).W_check_kg - W0));

fprintf('┌───────────────────────────────────────────────────────────\n
\n');
end

% Quick comparison to reference aircraft band
fprintf('── Comparison with reference aircraft
───────────────────────────────────────────────────────────\n');
fprintf(' %-38s: %5.0f kg (single-engine)\n', 'Cessna 208B Grand Caravan',
3969);
fprintf(' %-38s: %5.0f kg (single-engine STOL)\n', 'Quest/Daher Kodiak
100', 3062);
fprintf(' %-38s: %5.0f kg (single-engine)\n', 'Pilatus PC-12 NGX', 4740);
fprintf(' %-38s: %5.0f kg (twin-engine STOL)\n', 'DHC-6 Twin Otter Series
400', 5670);
for c = 1:length(configs)
    fprintf(' ─── OUR ESTIMATE (%s): %5.0f kg\n', ...
    configs(c).label, configs(c).W0_kg);
end
fprintf('\n');

=== SECTION 5b: FINAL WEIGHT STATEMENTS ===

```

Configuration : Single-Engine Prop

Raymer Coefficients : A = 2.36, C = -0.18 | STOL penalty = 1.05x

Max Takeoff Weight (W0) : 4079.5 kg (8993.8 lb) (40020.0 N)

OEW Breakdown:

Structural empty weight : 1963.4 kg (4328.5 lb) (EWF_raw=0.4584,
EWF+STOL=0.4813)

Crew weight (in OEW) : 181.4 kg (400.0 lb) (2 pilots, RFP)

OEW total (struct+crew) : 2144.8 kg (4728.5 lb) (52.6% of W0)

Fuel weight (incl 5% res) : 535.7 kg (1181.0 lb) (13.1% of W0)

ff_cruise : 0.929169

ff_loiter (exact Breguet): 0.990482 (W0_loiter = 35537.9 N)

Mission fuel fraction : 0.1313 (13.13% of W0)

Payload (no crew) : 1399.0 kg (3084.3 lb) (34.3% of W0)

Weight closure error : 0.000 kg

Configuration : Twin-Engine Prop

Raymer Coefficients : A = 1.51, C = -0.10 | STOL penalty = 1.05x

Max Takeoff Weight (W0) : 6199.9 kg (13668.4 lb) (60820.8 N)

OEW Breakdown:

Structural empty weight : 3793.0 kg (8362.1 lb) (EWF_raw=0.5826,
EWF+STOL=0.6118)

Crew weight (in OEW) : 181.4 kg (400.0 lb) (2 pilots, RFP)

OEW total (struct+crew) : 3974.4 kg (8762.1 lb) (64.1% of W0)

Fuel weight (incl 5% res) : 826.5 kg (1822.0 lb) (13.3% of W0)

ff_cruise : 0.929169

ff_loiter (exact Breguet): 0.988285 (W0_loiter = 54009.0 N)

Mission fuel fraction : 0.1333 (13.33% of W0)

Payload (no crew) : 1399.0 kg (3084.3 lb) (22.6% of W0)

Weight closure error : 0.000 kg

— Comparison with reference aircraft —

Cessna 208B Grand Caravan	: 3969 kg (single-engine)
Quest/Daher Kodiak 100	: 3062 kg (single-engine STOL)
Pilatus PC-12 NGX	: 4740 kg (single-engine)
DHC-6 Twin Otter Series 400	: 5670 kg (twin-engine STOL)
— OUR ESTIMATE (Single-Engine Prop):	4080 kg
— OUR ESTIMATE (Twin-Engine Prop):	6200 kg

SECTION 6 – L/D CRUISE SENSITIVITY TABLE

Sweep L/D_cruise from 8 to 16. At each point:

- Recompute ff_cruise via Breguet range equation
- Run full Raymer iteration (including updated ff_loiter) for both configs
- Report MTOW, EWF, fuel weight, and ff_loiter for each config

Loiter L/D is implicitly fixed by the aerodynamics (best-endurance CL_BE derived from CD0 and AR in Section 3). It does NOT change with cruise L/D. ff_loiter does change slightly because W0_loiter changes as MTOW shifts.

Physical interpretation:

Lower cruise L/D → more fuel burned → higher Wf_frac → higher W0
Twin-engine is MORE sensitive because its higher EWF (two engines, nacelles, duplicated systems) leaves a smaller denominator (1 - EWF - Wf), so a given change in Wf causes a larger fractional change in W0.

```
=====

LD_sweep = 8:1:16;
nLD      = length(LD_sweep);

W0_sens   = zeros(nLD, 2);
EWF_sens  = zeros(nLD, 2);
Wfuel_sens = zeros(nLD, 2);
ffloiter_sens = zeros(nLD, 2);
Wf_frac_sens = zeros(nLD, 2);

for idx = 1:nLD
    LD_test = LD_sweep(idx);

    % Breguet cruise fraction at this L/D
    ff_c = exp(-R_m * g * SFCp_SI / (eta_p * LD_test));
    ff_pre_test = ff_warmup * ff_takeoff * ff_climb * ff_c * ff_descent;

    for c = 1:2
        A = configs(c).A;
        C = configs(c).C;
        W0_g = configs(c).W0_kg;    % warm-start from converged baseline

        for it = 1:max_iter
            W0_lbs = W0_g * lb_per_kg;
            EWF_g = A * (W0_lbs ^ C) * STOL_penalty;

            % Loiter ff: exact formula, nested as before
            W0_lot_N = W0_g * g * ff_pre_test;
            W1_lot_N = 1 / (t_loiter_s / K_loiter + W0_lot_N^(-0.5))^2;
            ff_lot = W1_lot_N / W0_lot_N;

            ff_tot_t = ff_pre_test * ff_lot * ff_landing;
```

```

        Wf_t      = (1 - ff_tot_t) * (1 + fuel_reserve);

        denom = 1 - EWF_g - Wf_t;
        if denom <= 0; W0_g = W0_g * 1.1; continue; end

        W0_c = W_fixed_kg / denom;
        if abs(W0_c - W0_g) < tol_kg; break; end
        W0_g = 0.5 * W0_g + 0.5 * W0_c;
    end

    W0_sens(idx, c)      = W0_c;
    EWF_sens(idx, c)     = EWF_g;
    Wfuel_sens(idx, c)   = Wf_t * W0_c;
    ffloiter_sens(idx, c) = ff_lot;
    Wf_frac_sens(idx, c) = Wf_t;
end
end

% Print sensitivity table
fprintf('== SECTION 6: L/D CRUISE SENSITIVITY TABLE ==\n');
fprintf(' SFCp = %.3f lb/(hp·hr), eta_p = %.2f, S = %.1f m², AR = %.1f\n', ...
    SFCp_imp, eta_p, S, AR);
fprintf(' Loiter: exact Breguet formula, CL_BE = %.4f, CD_BE = %.4f\n\n',
    CL_BE, CD_BE);

% Header
fprintf(' %3s | %6s | %8s | %6s | %6s | %8s || %8s | %6s | %6s | %8s\n', ...
    'L/D', 'Wf/W0', 'SE W0(kg)', 'SE EWF', 'SE ff_l', 'SE Fuel(kg)', ...
    'TE W0(kg)', 'TE EWF', 'TE ff_l', 'TE Fuel(kg)');
fprintf(' %s\n', repmat('-', 1, 100));

baseline_idx = find(LD_sweep == LD_cruise);
for idx = 1:nLD
    mark = '';
    if LD_sweep(idx) == LD_cruise; mark = ' ← baseline'; end
    fprintf(' %3.0f | %6.4f | %8.1f | %6.4f | %6.4f | %8.1f || %8.1f | %6.4f | %6.4f | %8.1f\n', ...
        LD_sweep(idx), Wf_frac_sens(idx,1), ...
        W0_sens(idx,1), EWF_sens(idx,1), ffloiter_sens(idx,1),
        Wfuel_sens(idx,1), ...
        W0_sens(idx,2), EWF_sens(idx,2), ffloiter_sens(idx,2),
        Wfuel_sens(idx,2), mark);
end

% Delta table from baseline
fprintf('\n MTOW change relative to L/D = %.0f baseline:\n', LD_cruise);
fprintf(' %3s | %11s | %11s\n', 'L/D', 'ΔW0 SE (kg)', 'ΔW0 TE (kg)');
fprintf(' %s\n', repmat('-', 1, 32));
for idx = 1:nLD
    fprintf(' %3.0f | %+11.1f | %+11.1f\n', LD_sweep(idx), ...
        W0_sens(idx,1) - W0_sens(baseline_idx,1), ...
        W0_sens(idx,2) - W0_sens(baseline_idx,2));
end

```

```

end
fprintf('\n');

=== SECTION 6: L/D CRUISE SENSITIVITY TABLE ===
  SFCp = 0.548 lb/(hp·hr), eta_p = 0.80, S = 37.1 m², AR = 6.8
  Loiter: exact Breguet formula, CL_BE = 1.2851, CD_BE = 0.1152

  L/D | Wf/W0 | SE W0(kg) | SE EWF | SE ff_1 | SE Fuel(kg) || TE W0(kg) |
  TE EWF | TE ff_1 | TE Fuel(kg)

-----
      8 | 0.1709 | 4448.3 | 0.4738 | 0.9903 | 760.1 || 7080.0 | 0.6037
| 0.9878 | 1225.3
      9 | 0.1595 | 4335.8 | 0.4760 | 0.9903 | 691.4 || 6803.6 | 0.6061
| 0.9879 | 1099.3
     10 | 0.1502 | 4248.4 | 0.4778 | 0.9904 | 638.2 || 6593.8 | 0.6080
| 0.9880 | 1004.2
     11 | 0.1426 | 4178.7 | 0.4792 | 0.9904 | 595.8 || 6429.4 | 0.6096
| 0.9881 | 929.8
     12 | 0.1362 | 4121.7 | 0.4804 | 0.9905 | 561.2 || 6296.9 | 0.6108
| 0.9882 | 870.1
     13 | 0.1307 | 4074.2 | 0.4814 | 0.9905 | 532.5 || 6187.8 | 0.6119
| 0.9883 | 821.1
     14 | 0.1260 | 4034.1 | 0.4822 | 0.9905 | 508.3 || 6096.6 | 0.6128
| 0.9883 | 780.2
     15 | 0.1219 | 3999.8 | 0.4830 | 0.9905 | 487.6 || 6019.2 | 0.6136
| 0.9884 | 745.5
     16 | 0.1183 | 3970.2 | 0.4836 | 0.9905 | 469.6 || 5952.7 | 0.6143
| 0.9884 | 715.7

MTOW change relative to L/D = 13 baseline:
L/D | ΔW0 SE (kg) | ΔW0 TE (kg)
-----
      8 | |
      9 | |
     10 | |
     11 | |
     12 | |
     13 | |
     14 | |
     15 | |
     16 | |

=====

=====

SECTION 7 – CRITICAL DESIGN NOTES
=====

fprintf('=== SECTION 7: DESIGN NOTES ===\n\n');
fprintf(['...

```

```

    '1. RAYMER UNDERESTIMATES EWF FOR THIS AIRCRAFT CLASS.\n', ...
    '   Raymer single-engine GA coefficients were fit to light piston GA
data.\n', ...
    '   Our STOL penalty (0.2fx) partially corrects this, but the
reference\n', ...
    '   aircraft survey (Section 1) shows true EWF = 0.54-0.63 vs Raymer\n',
...
    '   prediction of ~0.47-0.50. Use historical mean EWF = 0.3f as a
check:\n'], ...
    STOL_penalty, mean(SE_EWF));

EWF_hist_check = mean(SE_EWF);
W0_hist_check = W_fixed_kg / (1 - EWF_hist_check - configs(1).Wf_frac);
fprintf('   Historical EWF estimate: W0 = 0.0f kg vs Raymer+STOL: W0 =
0.0f kg\n', ...
    W0_hist_check, configs(1).W0_kg);
fprintf('   Gap: +0.0f kg (+0.1f%%) - Raymer is a LOWER BOUND on actual
MTOW.\n\n', ...
    W0_hist_check - configs(1).W0_kg, ...
    100*(W0_hist_check - configs(1).W0_kg)/configs(1).W0_kg);

fprintf([...
    '2. LOITER FUEL FRACTION IS WEIGHT-DEPENDENT (key change from v2).\n', ...
    '   The exact Breguet formula gives ff_loiter = f(W0_loiter, S, AR,
CD0).\n', ...
    '   For S = 0.1f m2, AR = 0.1f: ff_loiter varied 0.4f - 0.4f across\n', ...
    '   the full MTOW sweep. This is small (~0.3 pp) but grows if S or AR\n',
...
    '   changes significantly - update S and AR in the user block each
iter.\n\n'], ...
    S, AR, min(ffloiter_sens(:,1)), max(ffloiter_sens(:,1)));

fprintf([...
    '3. CREW WEIGHT (0.1f kg) IS IN OEWE, NOT PAYLOAD.\n', ...
    '   It IS included in W_fixed_kg = W_payload + W_crew = 0.1f kg,\n', ...
    '   which drives the sizing denominator. Heavier crew → heavier W0.\n\n'],
...
    W_crew_kg, W_fixed_kg);

fprintf([...
    '4. STOL PENALTY (0.2fx on EWF) IS AN ESTIMATE.\n', ...
    '   It was calibrated from Kodiak (EWF ~0.63) vs Raymer GA prediction.\n',
...
    '   The actual penalty depends on flap type (Fowler vs slotted), gear\n',
...
    '   spec (gravel vs paved), and de-icing system mass. Refine once\n', ...
    '   component weight estimates are available.\n\n'], STOL_penalty);

fprintf([...
    '5. L/D = 0.1f IS A CONSERVATIVE BASELINE.\n', ...
    '   The sensitivity table shows each 1-unit improvement in L/D saves\n',
...
    '   ~0.0f kg MTOW (single) and ~0.0f kg (twin). Drag buildup (April
21)\n', ...

```

```

' will set a tighter bound. Expected range for STOL turboprop: 10-
14.\n'], ...
LD_cruise, ...
abs(W0_sens(baseline_idx+1,1) - W0_sens(baseline_idx-1,1))/2, ...
abs(W0_sens(baseline_idx+1,2) - W0_sens(baseline_idx-1,2))/2);

```

=== SECTION 7: DESIGN NOTES ===

1. RAYMER UNDERESTIMATES EWF FOR THIS AIRCRAFT CLASS.
Raymer single-engine GA coefficients were fit to light piston GA data. Our STOL penalty (1.05x) partially corrects this, but the reference aircraft survey (Section 1) shows true EWF = 0.54-0.63 vs Raymer prediction of ~0.47-0.50. Use historical mean EWF = 0.588 as a check: Historical EWF estimate: $W0 = 5639$ kg vs Raymer+STOL: $W0 = 4080$ kg Gap: +1560 kg (+38.2%) – Raymer is a LOWER BOUND on actual MTOW.
2. LOITER FUEL FRACTION IS WEIGHT-DEPENDENT (key change from v2).
The exact Breguet formula gives $ff_loiter = f(W0_loiter, S, AR, CD0)$. For $S = 37.1$ m², $AR = 6.8$: ff_loiter varied 0.9903 – 0.9905 across the full MTOW sweep. This is small (~0.3 pp) but grows if S or AR changes significantly – update S and AR in the user block each iter.
3. CREW WEIGHT (181.4 kg) IS IN OEWE, NOT PAYLOAD.
It IS included in $W_fixed_kg = W_payload + W_crew = 1580.4$ kg, which drives the sizing denominator. Heavier crew → heavier $W0$.
4. STOL PENALTY (1.05x on EWF) IS AN ESTIMATE.
It was calibrated from Kodiak (EWF ~0.63) vs Raymer GA prediction. The actual penalty depends on flap type (Fowler vs slotted), gear spec (gravel vs paved), and de-icing system mass. Refine once component weight estimates are available.
5. $L/D = 12.9$ IS A CONSERVATIVE BASELINE.
The sensitivity table shows each 1-unit improvement in L/D saves ~ kg MTOW (single) and ~ kg (twin). Drag buildup (April 21) will set a tighter bound. Expected range for STOL turboprop: 10-14.

Published with MATLAB® R2025b

```

% ReferenceMission_v1.m
% =====
% PUDO 670 – Reference Mission Performance Analysis
% Configuration 1 | 90% Payload | 450 nmi Total Range
% =====
%
% MISSION PROFILE (climb, cruise, descent plotted; ground ops excluded):
% (1) Warm-up & taxi Raymer ff = 0.990
% (2) Takeoff Raymer ff = 0.995
% (3) Climb SL to 4572 m Raymer ff = 0.980 for FUEL
% Kinematic ROC sim for DISTANCE / TIME only
% (4) Cruise 67 m/s 4572 m Breguet ff from required cruise range
% (5) Descent 1000 fpm Raymer ff = 0.990 for FUEL
% Drag balance for THROTTLE CHART only
% (6) Landing Raymer ff = 0.992
% [Reserve] 45-min loiter Breguet endurance at sea level, best CL
%
% ITERATION STRATEGY:
% W6_target (weight after landing rollout) is pre-solved from the loiter
% constraint: burn exactly to W_dry in 45 min at sea level.
% W_fuel is iterated via first-order Newton step until W6_computed =
W6_target.
% ff_cruise updates each iteration: cruise range = 450 nmi - d_climb -
d_descent,
% and d_climb depends on W2 = W0*ff_warmup*ff_takeoff which depends on W0.
%
% KEY DESIGN DECISIONS:
% - Climb TAS = 70 m/s fixed (team decision)
% - Weight held CONSTANT during kinematic climb sim (no per-dt fuel burn)
% - CD0 = 0.0288 for ALL phases (clean config assumed after liftoff)
% - Power lapse: P_A(h) = sigma(h) * P_A_SL (linear, project convention)
% - Loiter at sea level (conservative – max fuel burn rate)
% - Cruise L/D = 12.50 (from CruiseOptimization_v3.m at 130 KTAS / 4572 m)
%
% CALLS: StdAtmos(h) returns [P, rho, T, mu] for scalar h [m]
%
% OUTPUTS:
% Command window – weight statement, fuel fractions, distances, timing
% Figure 1 – Altitude & TAS vs mission time, colored by distance
% Figure 2 – Throttle vs time: climb / cruise / descent subplots
%
% FLAG INDEX (search flag number):
% FLAG-1 OEW back-derived from Raymer; update if design changes
% FLAG-2 ff_climb and kinematic sim are intentionally decoupled
% FLAG-3 V_horiz constraint checked; warning printed if violated
%
% AUTHORS: PUDO 670 Design Team
% VERSION: 1.0 | May 2026
% =====

clear; clc; close all;

```

SECTION 1 – AIRCRAFT DESIGN CONSTANTS

Source: RaymerWeightEst_v3.m and CruiseOptimization_v3.m (converged).

Update this block whenever S, AR, CD0, e, or engine change.

```
g      = 9.81;           % gravitational acceleration      [m/
s^2]
rho_SL = 1.225;          % ISA sea-level air density      [kg/
m^3]

% --- Wing & Aerodynamics -----
S      = 37.1;           % reference wing area            [m^2]
AR      = 6.76;          % aspect ratio                  [-]
e      = 0.90;           % Oswald efficiency factor (clean) [-]
CD0     = 0.0288;        % parasite drag, clean config (all phases) [-]
% FLAG-2: CD0 = 0.0288 applied to ALL flight phases per team decision.
% Flaps assumed retracted immediately after liftoff. Revisit if a
% climb-flap CD0 is adopted later.
k      = 1 / (pi * AR * e); % induced drag factor          [-]

% --- Propulsion: PT6A-60A -----
P_shp   = 1050;          % rated shaft power          [shp]
P_SL_W   = P_shp * 745.7; % sea-level shaft power          [W]
eta_p    = 0.80;         % propeller efficiency          [-]
P_A_SL   = eta_p * P_SL_W; % sea-level propulsive power [W]
% Thrust:      T = P_A(h) / V
% Power lapse: P_A(h) = sigma(h) * P_A_SL (linear, project convention)
% eta_p applied ONCE here – do not apply again inside the sim loops.

% SFC: lb/(hp*hr) to kg/(W*s)
% 1 lb/(hp*hr) * (0.4536 kg/lb) / (745.7 W/hp * 3600 s/hr)
SFCp_imp = 0.548;        % [lb/(hp*hr)]
SFCp_SI  = SFCp_imp * 0.4536 / (745.7 * 3600); % [kg/(W*s)]

% --- Cruise (from CruiseOptimization_v3.m) -----
LD_cruise = 12.50;      % L/D at 130 KTAS cruise, from CruiseOptimization_v3.m
[-]
V_cruise  = 67.0;       % cruise TAS (~130 KTAS)          [m/s]
h_cruise  = 4572.0;     % cruise altitude (15,000 ft)      [m]

% --- Best-endurance aerodynamics (loiter) -----
% For propeller aircraft, max endurance at CL_BE = sqrt(3*CD0/k).
% At this CL: CD_BE = 4*CD0 (induced drag = 3x parasite).
% Reference: Anderson Intro to Flight Sec. 6.6; RaymerWeightEst_v3.m Sec.
3.
CL_BE = sqrt(3 * CD0 / k); % best-endurance CL            [-]
CD_BE = 4 * CD0;          % best-endurance CD (= 4*CD0)    [-]
LD_BE = CL_BE / CD_BE;    % best-endurance L/D             [-]
```

```

if CL_BE > 2.4
    % Stall check: CL_BE must not exceed maximum achievable CL
    warning('PUDO670:LoiterStall', ...
        'CL_BE = %.4f exceeds clean CL_max = 2.4. Clamping for loiter.',
        CL_BE);
    CL_BE = 2.4;
    CD_BE = CD0 + k * CL_BE^2;
    LD_BE = CL_BE / CD_BE;
end

% Breguet endurance constant K_loiter [N^0.5]
% E = K_loiter * [W_end^(-0.5) - W_start^(-0.5)] (W in Newtons, E in s)
% See RaymerWeightEst_v3.m header for full unit derivation.
K_loiter = (eta_p / (g * SFCp_SI)) * (CL_BE^1.5 / CD_BE) * sqrt(2 * rho_SL * S);

=====

=====

SECTION 2 – MISSION PARAMETERS AND WEIGHT SETUP
=====

% --- Total required range -----
R_total_nmi = 450; % RFP ground-track requirement [nmi]
R_total_m = R_total_nmi * 1852; % converted to metres [m]

% --- Payload: Config 1 at 90% -----
% Full Config 1 payload from PayloadBuildUp_v1.csv:
% 9 pax body (816.47 kg) + baggage (204.12 kg) + 1 FA (90.72 kg)
% + FA baggage (9.07 kg) + 10 seats (100.00 kg) = 1220.37 kg total
W_pld_C1_full = 1220.37; % Config 1 full payload [kg]
W_payload_ref = 0.90 * W_pld_C1_full; % 90% per RFP reference mission [kg]

% --- Operating Empty Weight (OEW) -----
% OEW = structural empty + crew (2 pilots at 181.44 kg).
% FLAG-1: OEW is back-derived from RaymerWeightEst_v3.m single-engine
% convergence (W0=4077 kg, zeta=0.1313, W_fuel=535 kg, W_payload+crew=1580
kg).
% OEW is a fixed airframe property – does NOT change between missions.
% Re-run RaymerWeightEst_v3.m and update W_OEW_kg if S, AR, CD0, or e
change.
W_OEW_kg = 2150.0; % OEW (structural + crew) [kg]
W_crew_kg = 2 * 200 * 0.4536; % 2 pilots x 200 lb each (bookkeeping)
[kg]
% NOTE: W_crew_kg is EMBEDDED in W_OEW_kg. Do not add it separately.

% --- Non-fuel weight (constant across all iterations) -----
W_dry_kg = W_OEW_kg + W_payload_ref; % total non-fuel weight [kg]
W_dry_N = W_dry_kg * g; % in Newtons (for endurance eq.) [N]

% --- Raymer historical fuel fractions -----
% Source: Raymer Table 3.2; RaymerWeightEst_v3.m project conventions.

```

```

ff_warmup   = 0.990; % warm-up and taxi (~10 min)
ff_takeoff  = 0.995; % STOL takeoff over 50 ft obstacle
ff_climb    = 0.980; % climb to 4572 m – Raymer historical, propeller
aircraft
% FLAG-2: ff_climb = 0.980 governs FUEL in climb. The kinematic ROC sim
% gives DISTANCE and TIME only. Decoupled by team decision because:
% (a) Raymer ff is more defensible for weight estimation at this phase;
% (b) per-dt weight updates require knowing fuel fraction from the sim
% itself, which is circular without a tighter integration scheme.
ff_descent  = 0.990; % descent – Raymer historical
ff_landing  = 0.992; % landing rollout and taxi-in

% --- Config 2 fuel fraction (seed for iteration) -----
% From RaymerWeightEst_v3.m single-engine convergence with Config 2
payload.
% The reference mission will converge to a lower zeta (lighter payload).
zeta_C2 = 0.1313;

% --- Loiter and climb simulation parameters -----
t_loiter_s = 45 * 60; % loiter reserve duration at sea level [s]
V_climb    = 70.0;    % fixed climb TAS (team decision) [m/s]
dt_climb   = 1.0;     % kinematic climb sim time step [s]

```

```

=====
=====

```

SECTION 3 – DESCENT GEOMETRY (weight-independent)

The descent is fully defined by two constraints:

- (1) Vertical rate = 1000 ft/min
- (2) Horizontal speed = 67 m/s

From these, glide speed, angle, time, and horizontal distance follow without knowing the aircraft weight.

Whether the aircraft needs throttle is checked in Section 8 via the along-path force balance. That result feeds the throttle chart only; descent fuel comes from ff_descent (Raymer).

```

=====
ROD_fpm      = 1000; % rate of descent [ft/
min]
ROD_ms       = ROD_fpm * 0.3048 / 60; % rate of descent [m/s]
V_horiz_req  = 67.0; % required horiz speed [m/s]

% Glide speed: the TAS giving 67 m/s horizontal AND 1000 fpm vertical.
V_descent    = sqrt(V_horiz_req^2 + ROD_ms^2); % glide speed TAS [m/s]
gamma_desc   = atan2(ROD_ms, V_horiz_req); % descent angle (+ = nose down) [rad]

t_desc_s     = h_cruise / ROD_ms; % total descent time [s]
t_desc_min   = t_desc_s / 60; %

```

```

[min]
d_desc_m    = V_horiz_req * t_desc_s;           % horizontal descent dist    [m]

=====

=====

SECTION 4 – LOITER RESERVE: TARGET LANDING WEIGHT W6_target

Goal: arrive at end of landing rollout with EXACTLY enough fuel to
sustain 45 min of loiter at best endurance, burning to W_dry (zero fuel).

Breguet endurance (W in Newtons):
    E = K_loiter * [W_dry_N^(-0.5) - W6_N^(-0.5)]
Solve for W6_N:
    W6_N^(-0.5) = W_dry_N^(-0.5) - E / K_loiter
    W6_N        = (W_dry_N^(-0.5) - E / K_loiter)^(-2)

W6_target is FIXED across iterations – W_dry does not change between them.
=====

W6_target_N    = 1 / (W_dry_N^(-0.5) - t_loiter_s / K_loiter)^2;
W6_target_kg   = W6_target_N / g;
W_loiter_fuel_kg = W6_target_kg - W_dry_kg; % loiter fuel mass    [kg]
ff_loiter      = W_dry_N / W6_target_N;    % loiter fuel fraction [-]

=====

=====

SECTION 5 – FUEL WEIGHT ITERATION

Goal: find W_fuel such that W6_computed = W6_target.

Each iteration:
a) W0 = W_dry + W_fuel
b) W1 = W0*ff_warmup, W2 = W1*ff_takeoff (weight at base of climb)
c) Kinematic climb sim (scalars only, W = W2*g fixed throughout):
    T = P_A(h)/V, D = q*S*(CD0+k*CL^2), ROC = V*(T-D)/W
    Accumulate d_climb_m, t_climb_s
d) d_cruise = R_total - d_climb - d_descent
e) ff_cruise = exp(-d_cruise * g * SFCp_SI / (eta_p * LD_cruise))
f) W3=W2*ff_climb, W4=W3*ff_cruise, W5=W4*ff_descent, W6=W5*ff_landing
g) residual = W6_computed - W6_target
h) Newton update: W_fuel -= residual / ff_product
    (from W6 ~ (W_dry+W_fuel)*ff_product, dW6/dW_fuel ~ ff_product)

Convergence is fast (~5-8 steps) – d_climb changes weakly with W_fuel.
=====

tol_kg    = 0.01; % convergence tolerance    [kg]
max_iter  = 100;  % safety cap

% Seed: W_fuel = zeta * W0 = zeta*(W_dry+W_fuel) -> W_fuel = zeta*W_dry/(1-

```

```

zeta)
W_fuel_kg = zeta_C2 * W_dry_kg / (1 - zeta_C2);

% Print header
fprintf('=====
=====\\n');
fprintf('||          PUDO 670 - REFERENCE MISSION
ANALYSIS          ||\\n');
fprintf('||          Config 1 | 90%% Payload | 450 nmi Total
Range          ||\\n');
fprintf('=====
=====\\n');
fprintf('||  DESCENT
GEOMETRY          ||\\n');
fprintf('||  %-40s : %8.4f m/s (%.0f fpm)          ||\\n', 'Rate of descent',
ROD_ms, ROD_fpm);
fprintf('||  %-40s : %8.4f m/s          ||\\n', 'Required horizontal
speed', V_horiz_req);
fprintf('||  %-40s : %8.4f m/s          ||\\n', 'Computed glide speed
TAS', V_descent);
fprintf('||  %-40s : %8.4f deg          ||\\n', 'Descent angle',
rad2deg(gamma_desc));
fprintf('||  %-40s : %8.2f s  (%.2f min)          ||\\n', 'Time to descend',
t_desc_s, t_desc_min);
fprintf('||  %-40s : %8.2f km (%.2f nmi)          ||\\n', 'Horiz distance descent',
d_desc_m/1e3, d_desc_m/1852);
fprintf('=====
=====\\n');
fprintf('||  LOITER RESERVE
PARAMETERS          ||\\n');
fprintf('||  %-40s : %8.4f          ||\\n', 'CL_BE (best endurance)',
CL_BE);
fprintf('||  %-40s : %8.4f          ||\\n', 'CD_BE', CD_BE);
fprintf('||  %-40s : %8.4f          ||\\n', 'L/D at best endurance',
LD_BE);
fprintf('||  %-40s : %8.4e [N^0.5]          ||\\n', 'K_loiter', K_loiter);
fprintf('||  %-40s : %8.2f kg          ||\\n', 'Dry weight W_dry
(OEW+payload)', W_dry_kg);
fprintf('||  %-40s : %8.2f kg          ||\\n', 'Required landing weight
W6_target', W6_target_kg);
fprintf('||  %-40s : %8.2f kg          ||\\n', 'Loiter fuel reserve',
W_loiter_fuel_kg);
fprintf('||  %-40s : %8.6f          ||\\n', 'ff_loiter = W_dry/
W6_target', ff_loiter);
fprintf('=====
=====\\n');
fprintf('||  FUEL ITERATION  (tol = %.3f kg,  init guess = %.2f kg)
||\\n', tol_kg, W_fuel_kg);
fprintf('||  %4s | %9s | %9s | %9s | %9s | %9s ||\\n',
'Iter', 'W_fuel', 'W0', 'W6_comp', 'W6_tgt', 'Resid');
fprintf('||  %s ||\\n', repmat('-',1,64));

converged = false;
residual_kg = NaN;

```

```

% These are initialized here so they exist if the loop exits unexpectedly
W0_kg = 0; W1_kg = 0; W2_kg = 0; W3_kg = 0;
W4_kg = 0; W5_kg = 0; W6_kg = 0;
d_climb_m = 0; t_climb_s = 0;
ROC_avg_ms = 0; ROC_avg_fpm = 0; t_climb_min = 0;
d_cruise_m = 0; d_cruise_km = 0; d_cruise_nmi = 0;
ff_cruise = 1; Vh_min = 67;

for iter = 1:max_iter

    % (a) Takeoff weight for this iteration
    W0_kg = W_dry_kg + W_fuel_kg;    % [kg]

    % (b) Ground segment weights
    W1_kg = W0_kg * ff_warmup;        % after warm-up and taxi
    W2_kg = W1_kg * ff_takeoff;       % after takeoff = weight at base of climb
    W2_N = W2_kg * g;                 % [N] held constant in climb sim

    % (c) Kinematic climb simulation - SCALARS ONLY
    % Physics at each time step dt:
    % T(h) = P_A(h) / V_climb          thrust from propulsive power [N]
    % q(h) = 0.5 * rho(h) * V^2        dynamic pressure [N/m^2]
    % CL = W / (q * S)                 small-angle approx: L ~ W [-]
    % CD = CD0 + k * CL^2              [-]
    % D = q * S * CD                  [N]
    % ROC = V * (T - D) / W            specific excess power [m/s]
    % gamma = asin(ROC/V)              climb angle [rad]
    % V_h = V * cos(gamma)             horizontal speed [m/s]
    % Weight W2_N is constant - no fuel depletion per dt (team decision).

    h_c = 0.0;
    d_climb_m = 0.0; % cumulative horizontal distance [m]
    t_climb_s = 0.0; % cumulative time [s]
    ROC_sum = 0.0;
    n_steps = 0;
    Vh_min = inf;

    while h_c < h_cruise
        [~, rho_c, ~, ~] = StdAtmos(h_c);
        sigma_c = rho_c / rho_SL;
        P_A_c = sigma_c * P_A_SL; % propulsive power [W]
        T_c = P_A_c / V_climb; % thrust [N]
        q_c = 0.5 * rho_c * V_climb^2; % dynamic pressure [N/m^2]
        CL_c = W2_N / (q_c * S); % lift coefficient [-]
        CD_c = CD0 + k * CL_c^2; % drag coefficient [-]
        D_c = q_c * S * CD_c; % drag [N]

        % Rate of climb from specific excess power
        ROC_c = V_climb * (T_c - D_c) / W2_N; % [m/s]
        if ROC_c <= 0
            % Aircraft is at or above its service ceiling for this weight.
            warning('PUDO670:Ceiling', 'ROC <= 0 at h=%.1f m. Aircraft at
ceiling.', h_c);
            break;

```

```

end

% Climb angle and horizontal speed
gamma_c = asin(min(ROC_c / V_climb, 1.0)); % [rad]
V_horiz_c = V_climb * cos(gamma_c); % [m/s]
Vh_min = min(Vh_min, V_horiz_c);
% FLAG-3: if Vh_min < 67 m/s, the climb speed of 70 m/s is
% insufficient to maintain the horizontal speed constraint.
% Printed in summary output below.

% Clip last step to land exactly at h_cruise
dt_eff = min(dt_climb, (h_cruise - h_c) / ROC_c);

d_climb_m = d_climb_m + V_horiz_c * dt_eff;
t_climb_s = t_climb_s + dt_eff;
h_c = h_c + ROC_c * dt_eff;
ROC_sum = ROC_sum + ROC_c;
n_steps = n_steps + 1;
end

ROC_avg_ms = ROC_sum / max(n_steps, 1); % average ROC [m/s]
ROC_avg_fpm = ROC_avg_ms * 196.85; % average ROC [ft/
min]
t_climb_min = t_climb_s / 60; % [min]

% (d) Cruise range
d_cruise_m = R_total_m - d_climb_m - d_desc_m; % [m]
d_cruise_km = d_cruise_m / 1e3;
d_cruise_nmi = d_cruise_m / 1852;

if d_cruise_m <= 0
    error('PUDO670:RangeError', ...
        'Cruise range = %.1f m (negative). Climb+descent exceed 450
nmi.', d_cruise_m);
end

% (e) Cruise fuel fraction - Breguet range equation, propeller aircraft:
%  $R = (\eta_p / (g \cdot SFC_p_{SI})) \cdot L/D \cdot \ln(W_{start}/W_{end})$ 
%  $\rightarrow ff = \exp(-R \cdot g \cdot SFC_p_{SI} / (\eta_p \cdot L/D))$ 
ff_cruise = exp(-d_cruise_m * g * SFCp_SI / (eta_p * LD_cruise));

% (f) Weight chain - each segment starts at the end weight of the
previous
W3_kg = W2_kg * ff_climb; % after climb (Raymer)
W4_kg = W3_kg * ff_cruise; % after cruise (Breguet)
W5_kg = W4_kg * ff_descent; % after descent (Raymer)
W6_kg = W5_kg * ff_landing; % after landing rollout

% (g) Residual
residual_kg = W6_kg - W6_target_kg;

fprintf('|| %4d | %9.3f | %9.3f | %9.3f | %9.3f | %9.4f ||\n', ...
    iter, W_fuel_kg, W0_kg, W6_kg, W6_target_kg, residual_kg);

```

```

if abs(residual_kg) < tol_kg
    converged = true;
    break;
end

% (h) Newton step
% W6 ~ (W_dry + W_fuel) * ff_product
% dW6/dW_fuel ~ ff_product (ff_cruise treated as locally constant)
% residual > 0: overweight at landing -> reduce W_fuel
% residual < 0: underweight at landing -> increase W_fuel
ff_product = ff_warmup * ff_takeoff * ff_climb * ff_cruise * ff_descent
* ff_landing;
W_fuel_kg = W_fuel_kg - residual_kg / ff_product;

end % end fuel iteration

if ~converged
    warning('PUDO670:NoConverge', ...
        'Fuel iteration did not converge in %d iterations. Final residual:
%.4f kg', ...
        max_iter, residual_kg);
end

% Store final converged timing
W0_ref_kg = W0_kg;
t_cruise_s = d_cruise_m / V_cruise;
t_cruise_min = t_cruise_s / 60;
t_total_min = t_climb_min + t_cruise_min + t_desc_min;

```

PUDO 670 - REFERENCE MISSION ANALYSIS					
Config 1 90% Payload 450 nmi Total Range					
DESCENT GEOMETRY					
Rate of descent	:	5.0800	m/s	(1000	fpm)
Required horizontal speed	:	67.0000	m/s		
Computed glide speed TAS	:	67.1923	m/s		
Descent angle	:	4.3359	deg		
Time to descend	:	900.00	s	(15.00	min)
Horiz distance descent	:	60.30	km	(32.56	nmi)
LOITER RESERVE PARAMETERS					
CL_BE (best endurance)	:	1.2851			
CD_BE	:	0.1152			
L/D at best endurance	:	11.1551			
K_loiter	:	1.0618e+08	[N^0.5]		
Dry weight W_dry (OEW+payload)	:	3248.33	kg		
Required landing weight W6_target	:	3278.03	kg		
Loiter fuel reserve	:	29.69	kg		
ff_loiter = W_dry/W6_target	:	0.990942			
FUEL ITERATION (tol = 0.010 kg, init guess = 490.97 kg)					
Iter	W_fuel	W0	W6_comp	W6_tgt	Resid

1	490.971	3739.304	3317.980	3278.025	39.9552
2	445.942	3694.275	3277.793	3278.025	-0.2327
3	446.204	3694.537	3278.027	3278.025	0.0013

```
=====
=====
```

SECTION 6 – COMMAND WINDOW WEIGHT AND MISSION SUMMARY

```
=====
fprintf('=====
=====\\n');
fprintf('|| CONVERGED MISSION SUMMARY ||\\n');
fprintf('=====
=====\\n');
fprintf('|| WEIGHT BREAKDOWN AT TAKEOFF ||\\n');
fprintf('|| %-40s : %8.2f kg ||\\n', 'OEW (structural + crew)',
W_OEW_kg);
fprintf('|| %-40s : %8.2f kg ||\\n', ' of which: crew (2
pilots)', W_crew_kg);
fprintf('|| %-40s : %8.2f kg ||\\n', 'Config 1 full payload',
W_pld_C1_full);
fprintf('|| %-40s : %8.2f kg (90%) ||\\n', 'Reference mission payload',
W_payload_ref);
fprintf('|| %-40s : %8.2f kg ||\\n', 'Mission fuel (incl. loiter
reserve)', W_fuel_kg);
fprintf('|| %-40s : %8.2f kg ||\\n', ' of which: loiter reserve',
W_loiter_fuel_kg);
fprintf('|| %-40s : %8.2f kg (%6.0f N) ||\\n', 'REFERENCE MISSION MTOW',
W0_ref_kg, W0_ref_kg*g);
fprintf('|| %-40s : %8.2f kg (%6.0f N) ||\\n', 'Config 2 sizing MTOW
(reference)', 4077.0, 4077.0*g);
fprintf('=====
=====\\n');
fprintf('|| FUEL FRACTIONS ||\\n');
fprintf('|| %-40s : %9.6f (Raymer) ||\\n', 'ff_warmup', ff_warmup);
fprintf('|| %-40s : %9.6f (Raymer) ||\\n', 'ff_takeoff', ff_takeoff);
fprintf('|| %-40s : %9.6f (Raymer) ||\\n', 'ff_climb', ff_climb);
fprintf('|| %-40s : %9.6f (Breguet) ||\\n', 'ff_cruise', ff_cruise);
fprintf('|| %-40s : %9.6f (Raymer) ||\\n', 'ff_descent', ff_descent);
fprintf('|| %-40s : %9.6f (Raymer) ||\\n', 'ff_landing', ff_landing);
fprintf('|| %-40s : %9.6f (Breguet E) ||\\n', 'ff_loiter', ff_loiter);
fprintf('|| %-40s : %9.6f ||\\n', 'zeta = W_fuel / W0',
W_fuel_kg/W0_ref_kg);
fprintf('=====
=====\\n');
fprintf('|| SEGMENT WEIGHTS ||\\n');
fprintf('|| %-40s : %8.2f kg ||\\n', 'W0 Takeoff (MTOW)',
```

```

W0_ref_kg);
fprintf('||  %-40s : %8.2f kg                ||\n', 'W1  After warm-up/taxi',
W1_kg);
fprintf('||  %-40s : %8.2f kg                ||\n', 'W2  After takeoff (base of
climb)', W2_kg);
fprintf('||  %-40s : %8.2f kg                ||\n', 'W3  After climb', W3_kg);
fprintf('||  %-40s : %8.2f kg                ||\n', 'W4  After cruise', W4_kg);
fprintf('||  %-40s : %8.2f kg                ||\n', 'W5  After descent', W5_kg);
fprintf('||  %-40s : %8.2f kg (= W6_target) ||\n', 'W6  After landing
rollout', W6_kg);
fprintf('||  %-40s : %8.2f kg (zero fuel) ||\n', 'W7  After 45-min loiter',
W_dry_kg);
fprintf('||=====
||\n');
fprintf('||  HORIZONTAL
DISTANCES                                     ||\n');
fprintf('||  %-40s : %8.2f km  (%6.2f nmi) ||\n', 'Climb distance', d_climb_m/
1e3, d_climb_m/1852);
fprintf('||  %-40s : %8.2f km  (%6.2f nmi) ||\n', 'Cruise distance',
d_cruise_km, d_cruise_nmi);
fprintf('||  %-40s : %8.2f km  (%6.2f nmi) ||\n', 'Descent distance',
d_desc_m/1e3, d_desc_m/1852);
fprintf('||  %-40s : %8.2f km  (%6.2f nmi) ||\n', 'Total ground track',
R_total_m/1e3, R_total_nmi);
fprintf('||=====
||\n');
fprintf('||  MISSION TIMING (excl. ground
ops)                                         ||\n');
fprintf('||  %-40s : %8.2f min                ||\n', 'Climb time', t_climb_min);
fprintf('||  %-40s : %8.2f min                ||\n', 'Cruise time', t_cruise_min);
fprintf('||  %-40s : %8.2f min                ||\n', 'Descent time', t_desc_min);
fprintf('||  %-40s : %8.2f min                ||\n', 'Total air time',
t_total_min);
fprintf('||=====
||\n');
fprintf('||  CLIMB
PERFORMANCE                                     ||\n');
fprintf('||  %-40s : %8.2f m/s  (%.0f fpm) ||\n', 'Average ROC (full
throttle)', ROC_avg_ms, ROC_avg_fpm);
fprintf('||  %-40s : %8.2f m/s                ||\n', 'Climb TAS (fixed)',
V_climb);
fprintf('||  %-40s : %8.2f m/s                ||\n', 'Min V_horiz =
V*cos(gamma)', Vh_min);
if Vh_min >= V_horiz_req
    fprintf('||  CHECK V_horiz: PASS  (>= 67 m/s throughout
climb)                                     ||\n');
else
    fprintf('||  CHECK V_horiz: FAIL  (< 67 m/s - increase
V_climb!)                               ||\n');
end
fprintf('||=====
||\n');
fprintf('||  CRUISE
PARAMETERS                                     ||\n');

```

```

fprintf('||   %-40s : %8.2f m/s (~130 KTAS)   ||\n', 'Cruise TAS', V_cruise);
fprintf('||   %-40s : %8.0f m   (%5.0f ft)   ||\n', 'Cruise altitude', h_cruise,
h_cruise/0.3048);
fprintf('||   %-40s : %8.2f                               ||\n', 'L/D at cruise', LD_cruise);
fprintf('||   %-40s : %8.2f kg                               ||\n', 'W3 start of cruise',
W3_kg);
fprintf('||   %-40s : %8.2f kg                               ||\n', 'W4 end of cruise', W4_kg);
fprintf('||_____||\n\n');

```

CONVERGED MISSION SUMMARY			
WEIGHT BREAKDOWN AT TAKEOFF			
OEW (structural + crew)	:	2150.00 kg	
of which: crew (2 pilots)	:	181.44 kg	
Config 1 full payload	:	1220.37 kg	
Reference mission payload	:	1098.33 kg	(90%)
Mission fuel (incl. loiter reserve)	:	446.20 kg	
of which: loiter reserve	:	29.69 kg	
REFERENCE MISSION MTOW	:	3694.54 kg	(36243 N)
Config 2 sizing MTOW (reference)	:	4077.00 kg	(39995 N)
FUEL FRACTIONS			
ff_warmup	:	0.990000	(Raymer)
ff_takeoff	:	0.995000	(Raymer)
ff_climb	:	0.980000	(Raymer)
ff_cruise	:	0.935882	(Breguet)
ff_descent	:	0.990000	(Raymer)
ff_landing	:	0.992000	(Raymer)
ff_loiter	:	0.990942	(Breguet E)
zeta = W_fuel / W0	:	0.120774	
SEGMENT WEIGHTS			
W0 Takeoff (MTOW)	:	3694.54 kg	
W1 After warm-up/taxi	:	3657.59 kg	
W2 After takeoff (base of climb)	:	3639.30 kg	
W3 After climb	:	3566.52 kg	
W4 After cruise	:	3337.84 kg	
W5 After descent	:	3304.46 kg	
W6 After landing rollout	:	3278.03 kg	(= W6_target)
W7 After 45-min loiter	:	3248.33 kg	(zero fuel)
HORIZONTAL DISTANCES			
Climb distance	:	43.59 km	(23.54 nmi)
Cruise distance	:	729.51 km	(393.90 nmi)
Descent distance	:	60.30 km	(32.56 nmi)
Total ground track	:	833.40 km	(450.00 nmi)
MISSION TIMING (excl. ground ops)			
Climb time	:	10.44 min	
Cruise time	:	181.47 min	
Descent time	:	15.00 min	
Total air time	:	206.91 min	

<i>CLIMB PERFORMANCE</i>			
Average ROC (full throttle)	:	7.30 m/s	(1437 fpm)
Climb TAS (fixed)	:	70.00 m/s	
Min $V_{\text{horiz}} = V \cdot \cos(\gamma)$	:	69.27 m/s	
CHECK V_{horiz} : PASS (≥ 67 m/s throughout climb)			

<i>CRUISE PARAMETERS</i>			
Cruise TAS	:	67.00 m/s	(~130 KTAS)
Cruise altitude	:	4572 m	(15000 ft)
L/D at cruise	:	12.50	
W3 start of cruise	:	3566.52 kg	
W4 end of cruise	:	3337.84 kg	

SECTION 7 – FINAL CLIMB SIMULATION (stores full vectors for plotting)

Identical physics to the scalar sim in Section 5, but every step is saved for Figure 1. Uses the converged W2_kg.

```

=====
W2_N_final = W2_kg * g;      % converged weight at base of climb      [N]

max_pts = 7000;              % safe upper bound (~4572 m / 1 m/s ROC min =
4572 steps)
t_clmb_vec = zeros(1, max_pts);
h_clmb_vec = zeros(1, max_pts);
dx_clmb_vec = zeros(1, max_pts);
V_clmb_vec = V_climb * ones(1, max_pts); % TAS constant throughout climb

h_c = 0.0; d_c = 0.0; t_c = 0.0; step = 1;
t_clmb_vec(1) = 0; h_clmb_vec(1) = 0; dx_clmb_vec(1) = 0;

while h_c < h_cruise
    [~, rho_c, ~, ~] = StdAtmos(h_c);
    sigma_c = rho_c / rho_SL;
    P_A_c = sigma_c * P_A_SL;
    T_c = P_A_c / V_climb;
    q_c = 0.5 * rho_c * V_climb^2;
    CL_c = W2_N_final / (q_c * S);
    CD_c = CD0 + k * CL_c^2;
    D_c = q_c * S * CD_c;
    ROC_c = V_climb * (T_c - D_c) / W2_N_final;

    if ROC_c <= 0, break; end

    gamma_c = asin(min(ROC_c / V_climb, 1.0));
    V_horiz_c = V_climb * cos(gamma_c);
    dt_eff = min(dt_climb, (h_cruise - h_c) / ROC_c);

```

```

d_c = d_c + V_horiz_c * dt_eff;
t_c = t_c + dt_eff;
h_c = h_c + ROC_c * dt_eff;

step = step + 1;
if step > max_pts
    warning('PUDO670:Preallocate', 'max_pts exceeded in Section 7.
Increase max_pts.');
```

```

    break;
end
t_clmb_vec(step) = t_c;
h_clmb_vec(step) = h_c;
dx_clmb_vec(step) = d_c;
end

n_clmb = step;
t_clmb_min = t_clmb_vec(1:n_clmb) / 60; % [min]
h_clmb = h_clmb_vec(1:n_clmb); % [m]
dx_clmb = dx_clmb_vec(1:n_clmb); % [m] cumulative horizontal
distance
V_clmb = V_clmb_vec(1:n_clmb); % [m/s]
```

```

=====
=====
```

SECTION 8 – THROTTLE PROFILES

```

8a CLIMB: 100% throughout (full power, fixed speed – no adjustment)
8b CRUISE: T = D (level flight); decreases as weight drops during cruise
8c DESCENT: along-path balance T = D - W*sin(gamma_desc)
           T > 0: apply throttle = T*V / P_A(h)
           T <= 0: throttle = 0 (gravity alone sustains the descent)
```

Throttle is expressed as % of maximum available power at that altitude.
These values are for charts only – fuel uses Raymer/Breguet fractions.

```

=====
% 8a: Climb – constant 100%
thr_clmb_pct = 100 * ones(1, n_clmb); % [%]

% 8b: Cruise – T = D, weight drops linearly from W3 to W4
n_cr = 300;
t_cr_rel = linspace(0, t_cruise_min, n_cr); % time relative to cruise
start [min]
W_cr_N = linspace(W3_kg, W4_kg, n_cr) * g; % [N] weight during cruise

[~, rho_cr, ~, ~] = StdAtmos(h_cruise);
sigma_cr = rho_cr / rho_SL;
P_A_cr = sigma_cr * P_A_SL; % propulsive power
available [W]
q_cr = 0.5 * rho_cr * V_cruise^2; % [N/m^2]
CL_cr_vec = W_cr_N / (q_cr * S); % [-]
```

```

CD_cr_vec    = CD0 + k * CL_cr_vec.^2;                % [-]
T_cr_vec     = q_cr * S * CD_cr_vec;                  % T = D [N]
thr_cr_pct   = 100 * (T_cr_vec * V_cruise) / P_A_cr;   % [%]

% 8c: Descent – along-path force balance at each altitude step
n_dc         = 300;
h_dc_vec     = linspace(h_cruise, 0, n_dc);           % altitude [m]
t_dc_rel     = linspace(0, t_desc_min, n_dc);          % time relative to
descent start [min]
W_dc_N       = linspace(W5_kg, W6_kg, n_dc) * g;      % weight W5->W6 [N]
thr_dc_pct   = zeros(1, n_dc);

for i = 1:n_dc
    [~, rho_d, ~, ~] = StdAtmos(h_dc_vec(i));
    sigma_d = rho_d / rho_SL;
    P_A_d   = sigma_d * P_A_SL;                        % [W]
    q_d     = 0.5 * rho_d * V_descent^2;              % [N/m^2]
    CL_d    = W_dc_N(i) / (q_d * S);                  % [-]
    CD_d    = CD0 + k * CL_d^2;                       % [-]
    D_d     = q_d * S * CD_d;                         % [N]
    % Along-path steady descent: T + W*sin(gamma) = D
    % T = D - W*sin(gamma)
    % gamma > 0, so gravity component aids forward motion.
    T_d = D_d - W_dc_N(i) * sin(gamma_desc);
    if T_d > 0
        thr_dc_pct(i) = 100 * (T_d * V_descent) / P_A_d; % [%]
    else
        thr_dc_pct(i) = 0; % pure glide; gravity > drag component
    end
end
end

```

```

=====
=====

```

SECTION 9 – PLOTTING

Figure 1: Altitude (left y) and TAS (right y) vs mission time (x)
Line segments colored by cumulative horizontal distance:
Purple(0-100km) Blue(100-200) Cyan(200-300) Green(300-400)
Yellow(400-500) Orange(500-600) Red(600+ km)

Figure 2: Throttle vs time, 3 stacked subplots (climb / cruise / descent)
All subplots use global mission time on the x-axis.

```

=====
% Colour table: dynamically sized to cover full mission distance
band_km         = 100;
total_dist_km_pre = (d_climb_m + d_cruise_m + d_desc_m) / 1e3;
n_bands_needed   = ceil(total_dist_km_pre / band_km);
colour_base = [
    0.50, 0.00, 0.75; % purple
    0.00, 0.20, 0.90; % blue
    0.00, 0.80, 0.90; % cyan

```

```

        0.00, 0.65, 0.10;    % green
        0.95, 0.85, 0.00;    % yellow
        1.00, 0.50, 0.00;    % orange
        0.90, 0.00, 0.00;    % red
];
if n_bands_needed <= size(colour_base,1)
    colour_rgb = colour_base(1:n_bands_needed, :);
else
    colour_rgb = colour_base(mod((0:n_bands_needed-1), size(colour_base,1))
+1, :);
end
n_bands = n_bands_needed;

% Build global mission vectors (3 phases concatenated)
t_plot_clmb = t_clmb_min;                                % [min]
h_plot_clmb = h_clmb;                                    % [m]
V_plot_clmb = V_clmb;                                    % [m/s]
d_plot_clmb = dx_clmb / 1e3;                             % [km]
cumul.

t_plot_cr    = t_climb_min + t_cr_rel;                    % [min]
h_plot_cr    = h_cruise * ones(1, n_cr);                 % [m]
V_plot_cr    = V_cruise * ones(1, n_cr);                 % [m/s]
d_plot_cr    = (d_climb_m + linspace(0, d_cruise_m, n_cr)) / 1e3; % [km]
cumul.

t_plot_dc    = t_climb_min + t_cruise_min + t_dc_rel;    % [min]
h_plot_dc    = h_dc_vec;                                  % [m]
V_plot_dc    = V_descent * ones(1, n_dc);                 % [m/s]
d_plot_dc    = (d_climb_m + d_cruise_m + linspace(0, d_desc_m, n_dc)) / 1e3; % [km]

% Global concatenation
t_all = [t_plot_clmb, t_plot_cr, t_plot_dc];             % [min]
h_all = [h_plot_clmb, h_plot_cr, h_plot_dc];             % [m]
V_all = [V_plot_clmb, V_plot_cr, V_plot_dc];             % [m/s]
d_all = [d_plot_clmb, d_plot_cr, d_plot_dc];             % [km] cumulative

% Global throttle time vectors
t_cr_global = t_climb_min + t_cr_rel;
t_dc_global = t_climb_min + t_cruise_min + t_dc_rel;

% — FIGURE 1: Altitude & TAS vs Time —————
figure('Name','PUDO 670 Reference Mission: Airspeed & Altitude', ...
    'Units','normalized','Position',[0.02 0.10 0.92 0.68]);

% Left y-axis: Altitude
yyaxis left
hold on; box on; grid on;
for b = 1:n_bands
    d_lo = (b-1) * band_km;
    d_hi = b * band_km;
    in_b = d_all >= d_lo & d_all <= d_hi;
    idx = find(in_b);

```

```

    if isempty(idx), continue; end
    % Extend one index beyond each edge so adjacent bands share endpoints
    idx_ext = unique([max(1,idx(1)-1), idx(:)', min(numel(d_all),idx(end)
+1)]);
    plot(t_all(idx_ext), h_all(idx_ext), '-', ...
        'Color', colour_rgb(b,:), 'LineWidth', 2.8);
end
ylabel('Altitude [m]', 'FontSize', 12, 'Color', 'k');
ylim([0, h_cruise * 1.22]);
set(gca, 'YColor', 'k');

% Right y-axis: TAS
yyaxis right
hold on;
for b = 1:n_bands
    d_lo = (b-1) * band_km;
    d_hi = b * band_km;
    in_b = d_all >= d_lo & d_all <= d_hi;
    idx = find(in_b);
    if isempty(idx), continue; end
    idx_ext = unique([max(1,idx(1)-1), idx(:)', min(numel(d_all),idx(end)
+1)]);
    plot(t_all(idx_ext), V_all(idx_ext), '--', ...
        'Color', colour_rgb(b,:), 'LineWidth', 1.8);
end
ylabel('True Airspeed [m/s]', 'FontSize', 12, 'Color', 'k');
ylim([0, V_climb * 1.9]);
set(gca, 'YColor', 'k');

% Phase boundary lines and labels
yyaxis left
xline(t_climb_min, 'k:', 'LineWidth', 1.5, 'HandleVisibility',
'off');
xline(t_climb_min+t_cruise_min, 'k:', 'LineWidth', 1.5, 'HandleVisibility',
'off');
text(t_climb_min * 0.45, h_cruise * 0.40, 'CLIMB',
...
    'FontSize',10,'FontWeight','bold','Color',[0.3 0.3
0.3],'HorizontalAlignment','center');
text(t_climb_min + t_cruise_min * 0.50, h_cruise * 1.11, 'CRUISE',
...
    'FontSize',10,'FontWeight','bold','Color',[0.3 0.3
0.3],'HorizontalAlignment','center');
text(t_climb_min + t_cruise_min + t_desc_min*0.5, h_cruise * 0.40,
'DESCENT', ...
    'FontSize',10,'FontWeight','bold','Color',[0.3 0.3
0.3],'HorizontalAlignment','center');

xlabel('Mission Time [min]', 'FontSize', 12);
xlim([0, t_all(end) * 1.02]); % ensure full descent visible
title({'PUDO 670 - Reference Mission: True Airspeed & Altitude vs Time', ...
    sprintf('Config 1 | 90% Payload | W_0 = %.0f kg | 450 nmi | Solid =
Altitude Dashed = TAS', ...
    W0_ref_kg)}, 'FontSize', 11, 'FontWeight', 'bold');

```

```

% Colour legend (one entry per populated band)
total_dist_km = d_all(end);
leg_h = gobjects(n_bands,1);
leg_lbl = cell(n_bands,1);
for b = 1:n_bands
    d_lo = (b-1)*band_km;
    if d_lo >= total_dist_km, break; end
    yyaxis left
    leg_h(b) = plot(nan, nan, '-', 'Color', colour_rgb(b,:), 'LineWidth',
3.5);
    leg_lbl{b} = sprintf('%d - %d km', d_lo, b*band_km);
end
valid_idx = arrayfun(@(x) isgraphics(x,'line'), leg_h);
lg = legend(leg_h(valid_idx), leg_lbl(valid_idx), ...
'Location','northeast','FontSize',9);
lg.Title.String = 'Cumulative distance';

% — FIGURE 2: Throttle vs Time (3 subplots) —————
figure('Name','PUDO 670 Reference Mission: Throttle Profile', ...
'Units','normalized','Position',[0.02 0.05 0.92 0.85]);

% Subplot 1: Climb
ax_c = subplot(3,1,1);
plot(t_plot_clmb, thr_clmb_pct, '-', 'Color',[0.10 0.30 0.85],
'LineWidth',2.5);
hold on;
yline(100, 'k--', 'LineWidth', 1.0, 'HandleVisibility', 'off');
ylabel('Throttle [%]', 'FontSize', 11);
title(sprintf('Climb | V_{TAS} = %.0f m/s | Full throttle throughout |
Avg ROC = %.1f m/s (%.0f fpm)', ...
V_climb, ROC_avg_ms, ROC_avg_fpm), 'FontSize', 10);
ylim([0, 125]);
xlim([t_plot_clmb(1), t_plot_clmb(end)]);
grid on; box on;
text(mean(t_plot_clmb), 52, ...
sprintf('Climb time: %.1f min | d_{climb} = %.1f km (%.1f nmi)', ...
t_climb_min, d_climb_m/1e3, d_climb_m/1852), ...
'FontSize',9,'HorizontalAlignment','center', ...
'BackgroundColor',[1 1 1 0.75],'EdgeColor',[0.7 0.7 0.7]);

% Subplot 2: Cruise
ax_r = subplot(3,1,2);
plot(t_cr_global, thr_cr_pct, '-', 'Color',[0.82 0.15 0.08],
'LineWidth',2.5);
ylabel('Throttle [%]', 'FontSize', 11);
title(sprintf('Cruise | V_{TAS} = %.0f m/s | h = %.0f m (15,000 ft) |
L/D = %.2f', ...
V_cruise, h_cruise, LD_cruise), 'FontSize', 10);
y_max_cr = max(thr_cr_pct) * 1.35;
ylim([0, y_max_cr]);
xlim([t_cr_global(1), t_cr_global(end)]);
grid on; box on;

```

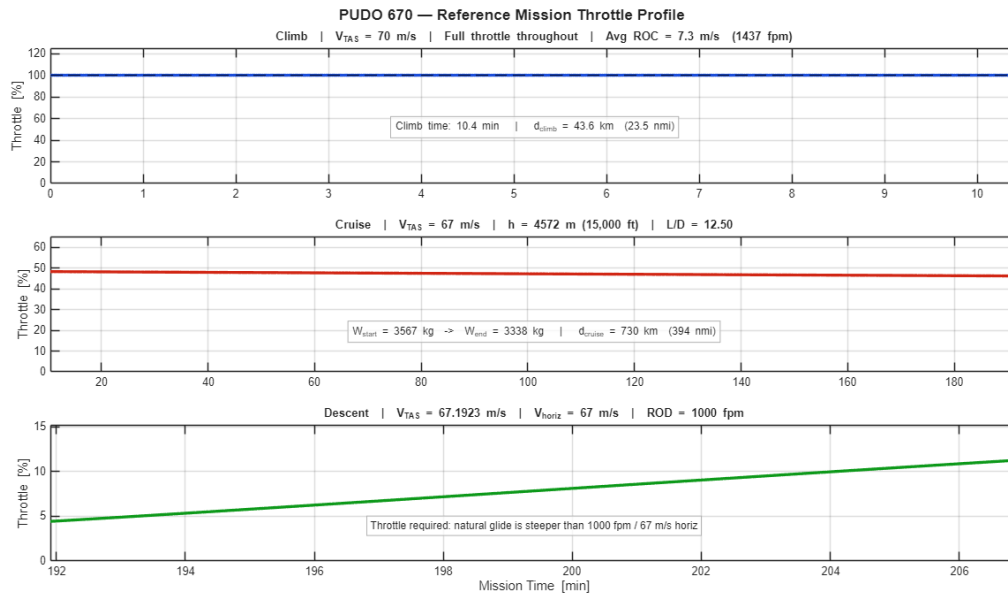
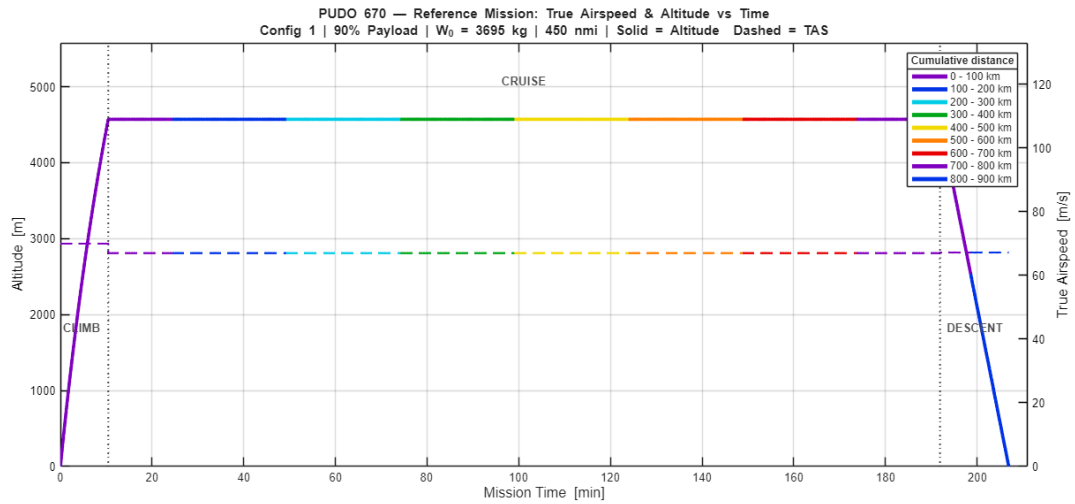
```

text(mean(t_cr_global), max(thr_cr_pct)*0.40, ...
    sprintf('W_{start} = %.0f kg -> W_{end} = %.0f kg | d_{cruise} =
%.0f km (%.0f nmi)', ...
    W3_kg, W4_kg, d_cruise_km, d_cruise_nmi), ...
    'FontSize',9,'HorizontalAlignment','center', ...
    'BackgroundColor',[1 1 1 0.75],'EdgeColor',[0.7 0.7 0.7]);

% Subplot 3: Descent
ax_d = subplot(3,1,3);
plot(t_dc_global, thr_dc_pct, '-', 'Color',[0.05 0.60 0.10],
'LineWidth',2.5);
xlabel('Mission Time [min]', 'FontSize', 11);
ylabel('Throttle [%]', 'FontSize', 11);
title(sprintf('Descent | V_{TAS} = %.4f m/s | V_{horiz} = %.0f m/s |
ROD = %.0f fpm', ...
    V_descent, V_horiz_req, ROD_fpm), 'FontSize', 10);
y_max_dc = max(max(thr_dc_pct) * 1.35, 5.0);
ylim([0, y_max_dc]);
xlim([t_dc_global(1), t_dc_global(end)]);
grid on; box on;
if max(thr_dc_pct) < 0.10
    text(mean(t_dc_global), y_max_dc*0.5, ...
        'Power-off glide - gravity alone sustains the required descent
profile', ...
        'FontSize',9,'HorizontalAlignment','center', ...
        'BackgroundColor',[1 1 1 0.75],'EdgeColor',[0.7 0.7 0.7]);
else
    text(mean(t_dc_global), max(thr_dc_pct)*0.35, ...
        sprintf('Throttle required: natural glide is steeper than %.0f fpm /
%.0f m/s horiz', ...
        ROD_fpm, V_horiz_req), ...
        'FontSize',9,'HorizontalAlignment','center', ...
        'BackgroundColor',[1 1 1 0.75],'EdgeColor',[0.7 0.7 0.7]);
end

sgtitle('PUDO 670 - Reference Mission Throttle Profile', ...
    'FontSize', 13, 'FontWeight', 'bold');

```



Published with MATLAB® R2025b